

Self-organizing Nervous Systems for Robot Swarms

W. Zhu^{1,*}, S. Oguz^{1,3,*}, M.K. Heinrich^{1,†,*}, M. Allwright¹,
M. Wahby¹, A. Lyhne Christensen², E. Garone³, M. Dorigo^{1,†}

¹IRIDIA, Université Libre de Bruxelles, Brussels, Belgium

²SDU UAS Center, MIMI, University of Southern Denmark, Odense, Denmark

³SAAS, Université Libre de Bruxelles, Brussels, Belgium

[†]To whom correspondence should be addressed; E-mails:

mary.katherine.heinrich@ulb.be, mdorigo@ulb.ac.be.

*These authors contributed equally to this work and share first authorship.

Abstract

The system architecture controlling a group of robots is generally set before deployment and can be either centralized or decentralized. This dichotomy is highly constraining, because decentralized systems are typically fully self-organized and therefore difficult to design analytically, whereas centralized systems have single points of failure and limited scalability. To address this dichotomy, we present the Self-organizing Nervous System (SoNS), a novel robot swarm architecture based on self-organized hierarchy. The SoNS approach enables robots to autonomously establish, maintain, and reconfigure dynamic multi-level system architectures. For example, a robot swarm consisting of n independent robots could transform into a single n -robot SoNS and then into several independent smaller SoNSs, where each SoNS uses a temporary and dynamic hierarchy. Leveraging the SoNS approach, we show that sensing, actuation, and decision-making can be coordinated in a locally centralized way, without sacrificing the benefits of scalability, flexibility, and fault tolerance, for which swarm robotics is usually studied. In several proof-of-concept robot missions—including binary decision-making and search-and-rescue—we demonstrate that the capabilities of the SoNS approach greatly advance the state of the art in swarm robotics. The missions are conducted with a real heterogeneous aerial-ground robot swarm, using a custom-developed quadrotor platform. We also demonstrate the scalability of the SoNS approach in swarms of up to 250 robots in a physics-based simulator, and demonstrate several types of system fault tolerance in simulation and reality.

1 Introduction

In the last two decades, swarm robotics research has demonstrated that it is possible to coordinate a large group of autonomous robots without any central coordinating entity. Elegant collective solutions have been developed for a broad scope of tasks, such as decision making [1], navigation and transport [2, 3], construction [4], and bio-hybrid interaction [5, 6]. By using strictly self-organized control within flat (single-level and fully decentralized) system architectures, swarm robotics behaviors have leveraged redundancy and parallelism to consistently achieve the hallmark advantages of a robot swarm—scalability, flexibility, the absence of single points of failure, and some degree of inherent fault tolerance. These characteristics are prohibitively difficult to obtain in fully centralized systems.

Despite the significant progress and the advantages of self-organized flat systems, the swarm behaviors being developed in abstract laboratory experiments are persistently slow to be adopted in real applications [7, 8]. This slow adoption rate can be attributed to the fact that, although there are significant advantages to self-organized control using flat system architectures, there are also inherent limitations. One crucial limitation is due to swarm behaviors occurring at the macroscale, but arising from self-organization among robots programmed at the microscale. Swarm behaviors are therefore difficult or impossible to design analytically, and fully self-organized swarms can take an undesirably long time to complete a task or converge on a decision. Even experienced researchers in the field often conduct a long trial-and-error design process in order to develop an incrementally novel behavior, and after several distinct behaviors have been developed, it is also not trivial to combine them. Furthermore, if an environment occupied by a swarm changes, especially to conditions that were not explicitly forecast, it can be difficult to predict how this will influence the swarm’s collective behavior. As a consequence of design difficulty and slow development processes, there are a great many tasks that we currently do not know how to perform using strictly self-organized control in flat systems.

The key limitations seen in flat self-organization are not prevalent in centralized systems: it is much more straightforward to design and combine centralized behaviors than self-organized ones. Using centralized systems, we already know how to execute many sophisticated behaviors that we currently do not know how to accomplish with many robots in a strictly self-organized way, such as SLAM [9] or optimization of online task scheduling or vehicle routing [10]. However, bottlenecks and single points of failure are unavoidable in strictly centralized systems, bringing inherent scalability and fault tolerance limitations that are typically absent in self-organized behaviors. In this paper, we propose that key impediments to rapid progress in swarm robotics can be overcome by partially integrating centralized control into an otherwise self-organized system through the introduction of a dynamic multi-level architecture—in other words, a self-organized hierarchy.

Self-organized hierarchy has been identified as a key unsolved challenge for the future of swarm robotics [7, 8]. Hierarchy can offer swarm robotics easier and faster behavior design and management as well as more flexibility when combining behaviors. However, not just any hierarchy is suitable. To still behave like a swarm and get the oft-cited benefits of scalability, flexibility, and fault tolerance for which swarm robotics is generally studied, the hierarchy cannot be imposed from the outside and must also be controllable. The members must autonomously establish an ad-hoc dynamic hierarchy among themselves and be able to comprehensively manage it in a self-organized way.

To undertake this challenge, we present Self-organizing Nervous Systems (SoNS) for robot swarms. SoNS is a novel robot swarm architecture that uses self-organized hierarchy to allow dynamically determined “brain” robots to coordinate sensing, actuation, and decision making in temporarily centralized sub-swarms, without sacrificing the scalability, flexibility, and fault tolerance of self-organization. The SoNS approach allows a completely decentralized system in which subsets of robots can self-organize into temporary locally centralized dynamic control structures when needed.

1.1 Self-organizing Nervous System (SoNS) concept

In the SoNS concept, robots autonomously organize themselves into dynamic multi-level system architectures using ad-hoc remote bidirectional connections. The result is a swarm composed of a number of reconfigurable Self-organizing Nervous Systems—SoNSs, see Fig. 1 and Movie 1. In each SoNS instance, each robot has chosen to temporarily grant explicit supervisory powers to a robot in the level above it, culminating in a single “brain” robot that acts impermanently as a coordinating entity.

The process by which robots establish and maintain explicit bidirectional connections in SoNSs is entirely self-organized, using exclusively local communication. Therefore, the basic structure of a SoNS includes not only the topology of the remote connections, but also the relative robot positions required to maintain those connections using local communication and sensing.

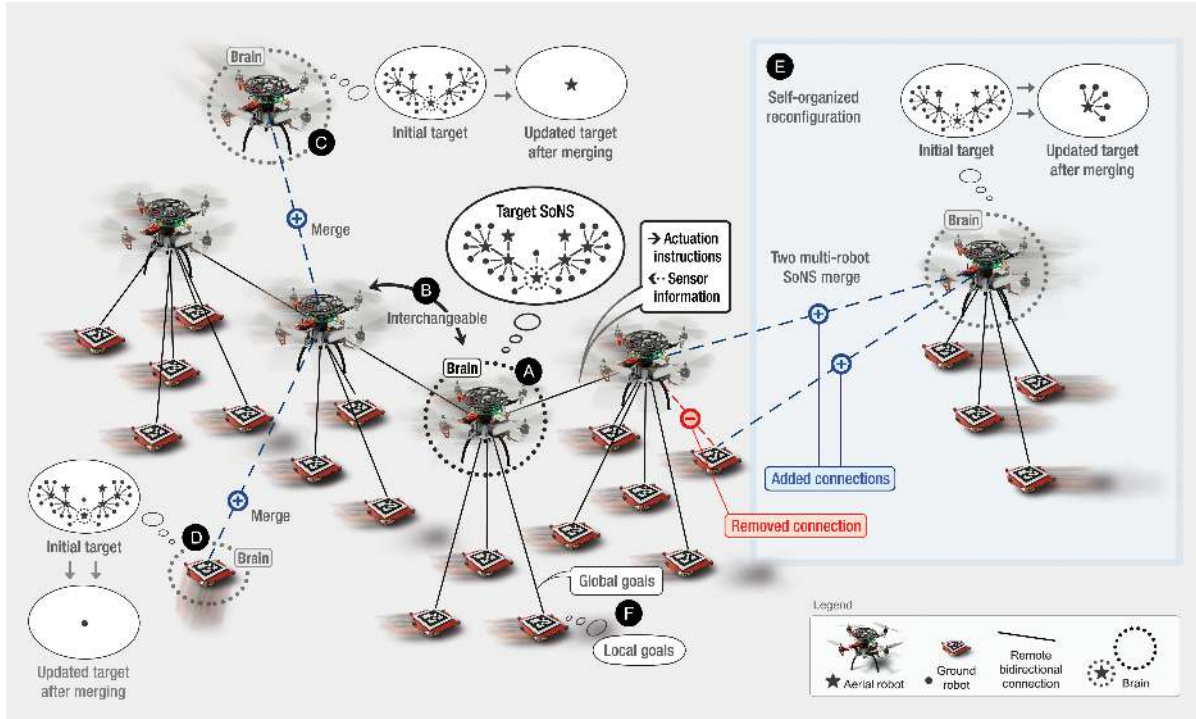


Fig. 1: The Self-organizing Nervous System (SoNS) concept: robots self-organize dynamic multi-level system architectures using exclusively local communication. (A) In a SoNS, each robot chooses to temporarily grant explicit supervisory powers to a robot in the level above it, culminating in a single “brain” robot that acts impermanently as a SoNS-wide coordinating entity. The brain robot does not communicate with all members directly—rather, at each bidirectional connection, the robot at a lower level in the architecture sends sensor information upstream, and the robot at a higher level sends actuation instructions downstream. The brain tries to establish its target SoNS and manages the global goals of the SoNS. (B) Any robot at any level of hierarchy can be interchangeable with another robot—even the brain. (C,D) Each robot initializes as the brain of its own single-robot SoNS, with a map of its target SoNS and, potentially, mission-dependent global goals. If it encounters another SoNS, it can choose to accept recruitment and merge with it, thereby abdicating its “brain” status. (E) The process by which robots establish and maintain connections is entirely self-organized. Therefore, the topology can be reconfigured on demand by removing and adding connections, and multi-robot SoNS can split and merge as needed. Here, we see a five-robot SoNS (right) that merges with a 20-robot SoNS (left). (F) The topology of bidirectional connections is used to grant supervisory powers and send global goals downstream, but robots can negotiate the inter-level control distributions on the fly, adapting the degree of centralization or decentralization in the decision-making processes of the SoNS.

Movie 1 : Explanation video of the Self-organizing Nervous System (SoNS) concept.

(Note: To play embedded video, open the PDF in Adobe Acrobat.)

Each connection, including its relative robot positions, is managed independently by the two robots sharing the connection. Therefore, the self-organized aspects of the SoNS architecture are scalable. For instance, a new connection does not become any more difficult to establish if a SoNS recruits more members. Also, every robot is replaceable—even the brain. If any robot fails or is lost, it can be replaced automatically by another robot in its SoNS or by a new robot recruited from the surroundings. If no extra robots are available, a SoNS can reconfigure utilizing the robots it already has.

In SoNSs, unlike in centralized systems with root nodes, a brain robot does not communicate with all its members directly. Rather, at each bidirectional connection, the robot at a lower level in the architecture sends sensor information upstream, and the robot at a higher level sends actuation instructions downstream. Members of one SoNS can thereby act seamlessly as a single virtual body, despite only communicating with their direct neighbors.

In a SoNS, the inter-level control distribution of the system—that is, the degree of centralization or decentralization of decision-making—is dynamic and thus not statically determined by the topology of the bidirectional connections. The topology is indeed used to grant supervisory powers and send global goals downstream. However, the robots can continuously negotiate the inter-level control distributions on the fly, adapting the degree of centralization or decentralization of decision-making to the requirements of the task being performed. For instance, to

balance global and local goals, a robot might receive downstream instructions to move in a certain direction, but might temporarily negotiate a different inter-level control distribution while reacting locally to a small obstacle.

The system architecture and behavior of SoNSs are also dynamic and can be fully reorganized by the brain on demand (see Fig. 1 and Movie 1). This can include changes to the topology of connections, the relative positions used to maintain those connections, the global actuation goals such as a target motion trajectory, and even the number of separate SoNSs acting autonomously in the same environment. Indeed, the self-organization process by which SoNSs are built and managed also allows them to split and merge as needed. For instance, according to the number and sizes of SoNSs suitable for a certain task, the robots in a shared environment could start as many single-robot SoNSs that interact in a fully decentralized way, then merge into one shared multi-level SoNS to interact hierarchically, then split into several multi-robot SoNSs that can interact with each other as independent systems. In short, the SoNS approach is highly flexible and allows robot systems to fully self-reconfigure their architectures on demand.

1.2 Related work

In existing approaches, the overall architecture of a multi-robot system is normally set before deployment. In other words, the communication structure (the organization of communication links and the system levels they can span), control distribution (such as fully decentralized or fully centralized), and behavior structure of the system (for example, which sources of sensor information can influence which actions) are predefined, and the robots coordinate within this static architecture. Traditionally, swarm robotics has used self-organized control with strict heterarchy (a network or system of unranked elements) which is seen, for example, in aerial robot flocking [11]. This trend is not surprising, as the swarm robotics field was originally biologically inspired [7] and many biological systems with unranked members, seen in social organisms [12, 13, 14], have inspired artificial swarm intelligence [15]. However, despite the development of many advanced behaviors (for example, [11]), heterarchical swarm robotics approaches have rarely been used in real-world applications. To enable wider real-world use, the field is expected to develop more elaborate behavior design and coordination approaches [7, 8], such as automatic design [16], behavioral heterogeneity [17], and self-organizing hierarchy.

Many existing swarm robotics approaches have incorporated mechanisms relevant to hierarchy or leadership. For instance, behavioral heterogeneity can result in the implicit leadership of some individuals that are more informed than most of their peers [18, 19], send more messages when having higher information quality [20, 1], have greater behavioral persistence [21], or are partially remotely controlled by human operators [22, 23]. However, this behavioral heterogeneity in swarms has normally been implemented with unranked members, i.e., in a de-

centralized and single-level system. By contrast, double-level systems have been used when robot swarms have incorporated an explicit leader, e.g., for disseminating information to other robots via one-hop broadcast [24] or multi-hop broadcast over an ad-hoc structure [25]. However, the explicit leadership allocation in these examples is static after deployment and is usually defined manually.

Robot systems with explicit leadership and multiple leader-follower pairs have also been explored, especially for the task of flocking. However, these systems either use predefined and static leaders [26, 27, 28] or have unstructured relationships between pairs. In the latter type, interactions are one-way and therefore can be interpreted as a hierarchy of many temporary leader-follower pairs, for instance in flocking [29, 30, 31] or self-assembly [32], but any ranking among these pairs is emergent, not explicitly controlled. Therefore, these systems have the advantage of intrinsic flexibility—robots can be displaced unexpectedly and can immediately form new connections and be accommodated in their new positions—but have no mechanism for configuring or reconfiguring the organization of the hierarchy. Similarly, many sophisticated multi-drone navigation approaches that are partly centralized and partly decentralized have been developed, notably the recent trajectory planner for drone swarms by Zhou et al. [33]. These approaches provide high-performing solutions for collective navigation. However, they use fixed collaboration structures, because configuring and reconfiguring swarm architectures is not the focus of these studies.

In short, no existing approach has provided a comprehensive way to self-organize dynamic and highly reconfigurable multi-level system architectures in a robot swarm. The SoNS approach is designed to address this gap.

For our development of SoNS, we have taken inspiration from our ‘mergeable nervous systems’ (MNS) [34], an approach for physically-connected robots. We have incorporated some of its high-level ideas into our SoNS concept and robot swarm architecture and reference this inspiration by retaining the term ‘nervous systems.’ We have also conducted some preliminary simulation-only studies on extending MNS ideas to other contexts [35, 36, 37, 38, 39]. In this paper, we present for the first time our novel SoNS concept and robot swarm architecture and provide a thorough SoNS proof-of-concept using real robots.

1.3 Novel features of SoNS

The primary contribution of our SoNS robot swarm architecture is that a robot system can integrate the manageability advantages of centralized systems without sacrificing the scalability, flexibility, and fault-tolerance advantages of self-organized systems. This contribution is founded on four novel features for robot swarms (see Fig. 1): self-organized controllable hierarchy, interchangeable leadership (i.e, interchangeability of the brain), explicit inter-system

reconfiguration, and reconfigurable swarm behavior structures. Together, these features enable robot swarms to self-organize dynamic multi-level system architectures, including their communication structures, control distributions, and system behaviors.

Self-organized controllable hierarchy. The SoNS approach allows a robot swarm to self-organize a dynamic hierarchical communication structure that (i) is built and maintained using exclusively local communication, (ii) is not imposed from the outside, and (iii) is comprehensively controllable (that is, the SoNS-wide multi-level structure can move from any initial state to any desired state in its configuration space of directed acyclic graphs). In other words, a SoNS hierarchy can be explicitly defined and redefined by the brain and the desired changes occur through robots configuring and reconfiguring their local connections in a self-organized way. Self-organized controllable hierarchy has been shown in physically-connected robots but is novel in robot swarms, which so far have shown only emergent hierarchy (for example, [18, 19]), not controllable hierarchy. The remote connections of robot swarms bring significantly different requirements than physical connections. The physical locations and topology constrain each other less under remote connections than under physical connections, which provides much more flexibility in how a SoNS can be organized, but also adds the challenge that physical location and topology must both be actively (and sometimes separately) maintained.

Interchangeable leadership. In a SoNS, all robots occupy an explicitly defined position in a hierarchy, but any robot, at any level of hierarchy, can be interchanged autonomously and on demand. This interchange is self-organized by the robots using only local communication. This means that, if the brain fails, the SoNS self-organizes to automatically and immediately substitute it with the nearest robot, which continues to specify the same SoNS structure and mission goals as the previous leader. This is a novel feature for robot swarms with explicit leaders, which so far have used static (and sometimes manually defined) leadership. It is also a departure from many other types of multi-robot systems, which often use indiscriminate followers or groups of followers, but have not yet developed approaches in which the position of every robot in an explicit control hierarchy is defined by a self-organized process using strictly local communication.

Explicit inter-system reconfiguration. The SoNS approach allows reconfiguration between multiple SoNS—that is, several SoNSs can split and merge themselves in a self-organized way that is coordinated by the brains of the SoNSs and uses exclusively local communication, without losing the existing sub-structures that could be retained. For example, several independent SoNSs could agree to merge simultaneously, and the robots would reorganize themselves around the new shared brain, retaining sub-sections of the old structures when possible. Robot systems in the literature have shown splitting and merging of sub-swarms or sub-teams, but with the non-leader members being unranked (for example, [40, 41]), so no reconfiguration of explicit sub-system architectures was demonstrated.

Reconfigurable swarm behavior structures. The inter-level control distribution and system behaviors within a SoNS (for example, the global structure defining which information sources influence which actions) can be negotiated and explicitly reconfigured without breaking the system architecture. Reconfiguration can occur (i) locally and temporarily to balance conflicting global and local goals; (ii) SoNS-wide for the purpose of global sensing, actuation, and decision-making goals set by the brain; but also (iii) locally for internal re-organization of a SoNS (for example, robots automatically redistributing themselves to compensate for a failed robot). Based on this capacity for internal re-organization, if the needed changes to behavior are too substantial to be managed by inter-level negotiation, an entirely new SoNS architecture can be initialized by the brain and self-organized by the robots. No existing work has presented a robot swarm architecture with these explicit reconfiguration capabilities.

2 Results

We demonstrate the capabilities of the SoNS approach (see Movie 2) in experiments using real heterogeneous aerial-ground swarms consisting of standard differential drive e-puck robots [42, 43] and our custom-developed S-drone quadrotors [44]. To also demonstrate the capabilities in swarms larger than our real robot arena allows, we run experiments in a simulator that is cross-verified against the behavior of the real robots. Within each experiment, all robots run an identical SoNS program and operate fully autonomously—without any global positioning system, remote control station, or off-board sensing. The robots use vision-based relative positioning and are allowed to communicate wirelessly only if one robot is in the other’s field of view. Actuation in the experiments is confined to motion. The open-source software used in all experiments (both with real robots and in simulation) and all experiment data are available in online repositories. In addition to the experiments, we also provide theoretical analyses to show that the convergence and stability of the SoNS architecture are guaranteed.

2.1 Robot missions

We conduct five proof-of-concept robot missions that together demonstrate the key capabilities and novel features of the SoNS approach.

The first novel feature, self-organized controllable hierarchy, is shown in all missions: the first mission (Sec. 2.1.1) shows the process of establishing the self-organized hierarchy, the second mission (Sec. 2.1.2) shows that the self-organized hierarchy can be maintained despite external disturbances, and the reconfigurations in the rest of the missions show that the self-organized hierarchy is comprehensively controllable.

The novel features of interchangeable leadership and explicit inter-system reconfiguration

are both demonstrated in the mission on splitting and merging of systems (Sec. 2.1.5), as robots reconfigure into different sets of SoNSs and also reconfigure their leadership allocations during the splits and merges. (Note that interchangeable leadership is also demonstrated in the later fault-tolerance experiments, see Sec. 2.3).

The last novel feature of reconfigurable swarm behavior structures is shown in all four missions after the first (Secs. 2.1.2-2.1.5): reconfiguration is shown (i) locally and temporarily in the mission on balancing global and local goals (Sec. 2.1.2); (ii) SoNS-wide in the missions on global sensing and actuation and binary decision-making (Secs. 2.1.3 and 2.1.4); and (iii) locally for internal re-organization during the last three missions (Secs. 2.1.3-2.1.5).

Together, the missions demonstrate the ability to self-organize dynamic multi-level system architectures, including their communication structures, inter-level control distributions, and system behaviors.

Movie 2 : Summary video of key results.

(Note: To play embedded video, open the PDF in Adobe Acrobat.)

For each proof-of-concept mission, we report at least five trials with real robots (up to 12 robots) and 50 trials in simulation (with up to 75 robots), with a maximum run time of 15 minutes (constrained by the battery capacity of the quadrotor platform). The goals and scope of possible behaviors for each mission are designed offline. We give the mission schematics (see Figs. 2-6), show that the qualitative goals of the mission were achieved, and assess the results

in terms of actuation error, where the actuation error is the position tracking error E (Eq. 1, Sec. 4.2, Materials and Methods) with respect to the lower bound B (Eq. 2, Sec. 4.2, Materials and Methods). The actuation error is used here as a comprehensive metric because it encapsulates the other types of error that can occur (except for incomplete mission goals). Any errors in sensing and decision-making will cause large fluctuations in the actuation instructions sent in the system, and therefore cause significant increases in the actuation error.

2.1.1 Establishing self-organized hierarchy

The first proof-of-concept mission demonstrates the establishment of SoNSs (see Fig. 2): independent robots in arbitrary start positions need to self-organize into a single hierarchical system with the target multi-level communication structure, including the correct topology and correct relative positions. In the two mission variants, robots start on the ground either (i) with some of the robots clustered tightly (and therefore resulting in robot–robot interference) in varying arbitrary locations, or (ii) with all robots scattered in varying arbitrary positions throughout the arena. All robots run identical programs and begin as independent single-robot SoNSs, of which they are the brain by default. SoNSs try to recruit each other and change their relative positions until the mission goal is complete. In order to complete these missions in the tightly constrained arena, the SoNSs must perform many inter-system merge operations simultaneously, while also continuously re-organizing their internal structures by redistributing themselves (see example flowcharts in Fig. 2A).

In all experiments, the robots complete the mission: they converge on one SoNS and establish the correct topology and relative positions for the target communication topology (see example trial in Fig. 2B and Movie S1 in the supplementary materials). The progression of a typical experiment can be seen in the example trial (see Fig. 2C), in which the robots converge to the correct topology and reach a low steady-state actuation error. All trials reach a low steady-state error, with the clustered start variant displaying slightly higher average error than the scattered start variant (see Figs. 2D,E).

2.1.2 Balancing global and local goals

The next proof-of-concept demonstration is an obstacle-avoidance mission (see Fig. 3), in which robots in a SoNS must negotiate the inter-level control distribution on the fly (i.e., adapting the degree of centralization or decentralization in the decision-making of the SoNS). The SoNS needs to balance the global goal of an overall motion trajectory chosen by the brain with the local goals of ground robots circumventing small obstacles. Robots begin as members of a single SoNS and the brain begins with a straight trajectory in a given direction. The robots navigate through an unknown field of obstacles while the brain follows its straight trajectory,

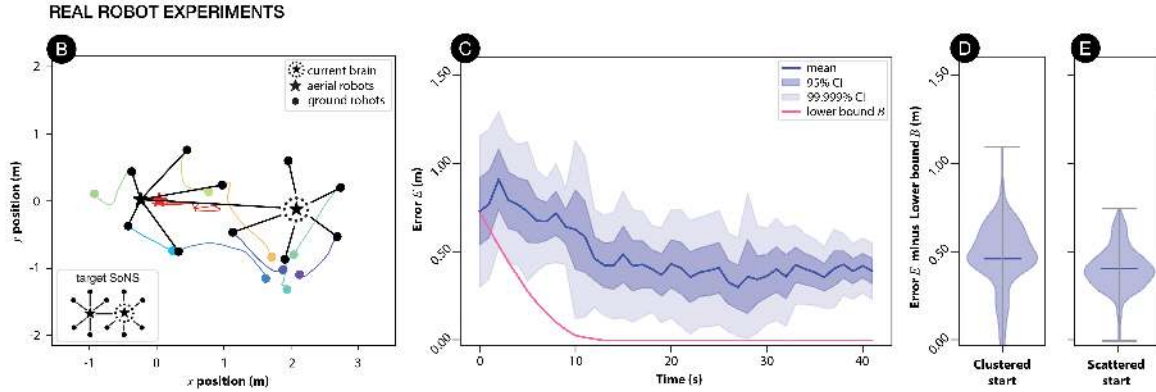
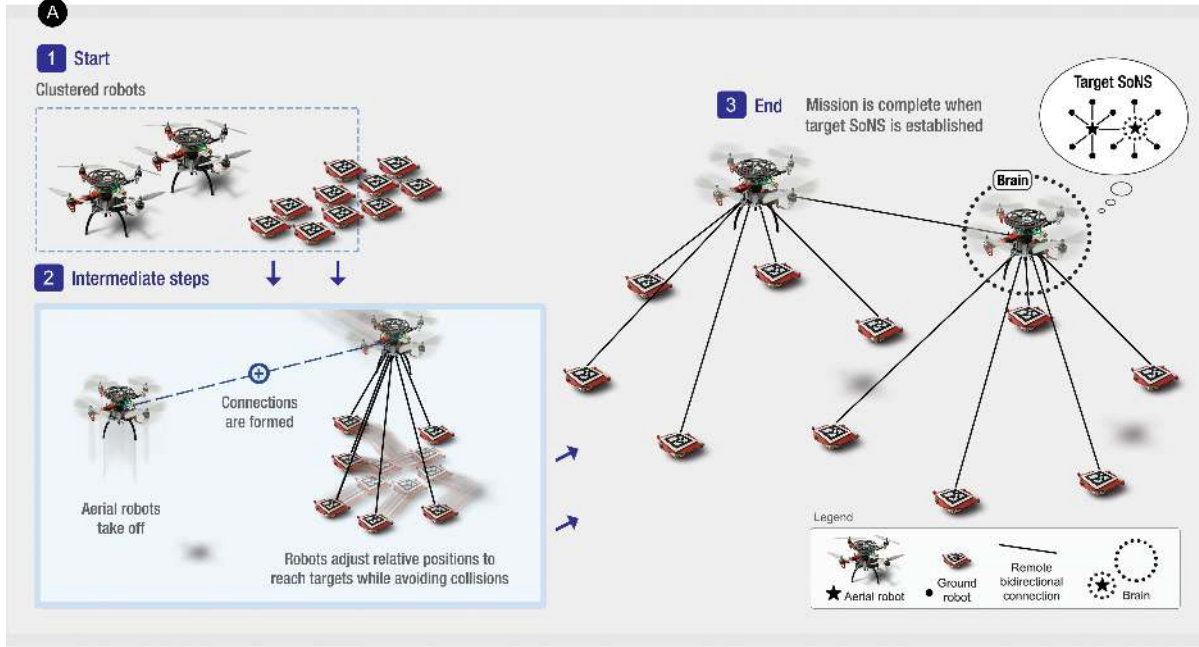


Fig. 2: Establishing self-organized hierarchy. In this experiment, independent robots in arbitrary starting positions need to self-organize into a single hierarchical system with the target multi-level communication structure and target relative positions—i.e., into the target SoNS. **(A)** Mission schematic: (1) Starting on the ground, some robots are positioned in tight clusters, which increases the possibility for interference between robots. (In a second variant, not pictured here, all robots start in scattered positions throughout the arena.) (2) To self-organize into the target SoNS, first the aerial robots take off and all robots begin searching for peers. Then, robots start forming connections, merging their respective SoNSs, and reallocating themselves into positions that match the target SoNS. During this process, robots continually adjust their relative positions while coordinating locally to avoid collisions, until (3) the target SoNS is complete. **(B-E)** Results of real robot experiments. **(B)** Trajectories of robots over time, with the initial and final positions indicated in color and in black, respectively, and **(C)** actuation error E (mean and confidence interval per robot over time, see Eq. 1, with lower bound B , see Eq. 2, plotted for reference) in a real example trial (shown in Movie S1 in the supplementary materials). **(D,E)** Violin plots of the actuation error minus the lower bound $E - B$ (mean and confidence interval per robot per second) in all real experiment trials, for both mission variants: **(D)** clustered start, six trials, and **(E)** scattered start, five trials. In this and following figures, the violin plots show that the given example trial is not an outlier and is indicative of the overall results of the respective mission. (For more detailed experiment results, including simulation experiments, see Sec. S1 of the supplementary materials.)

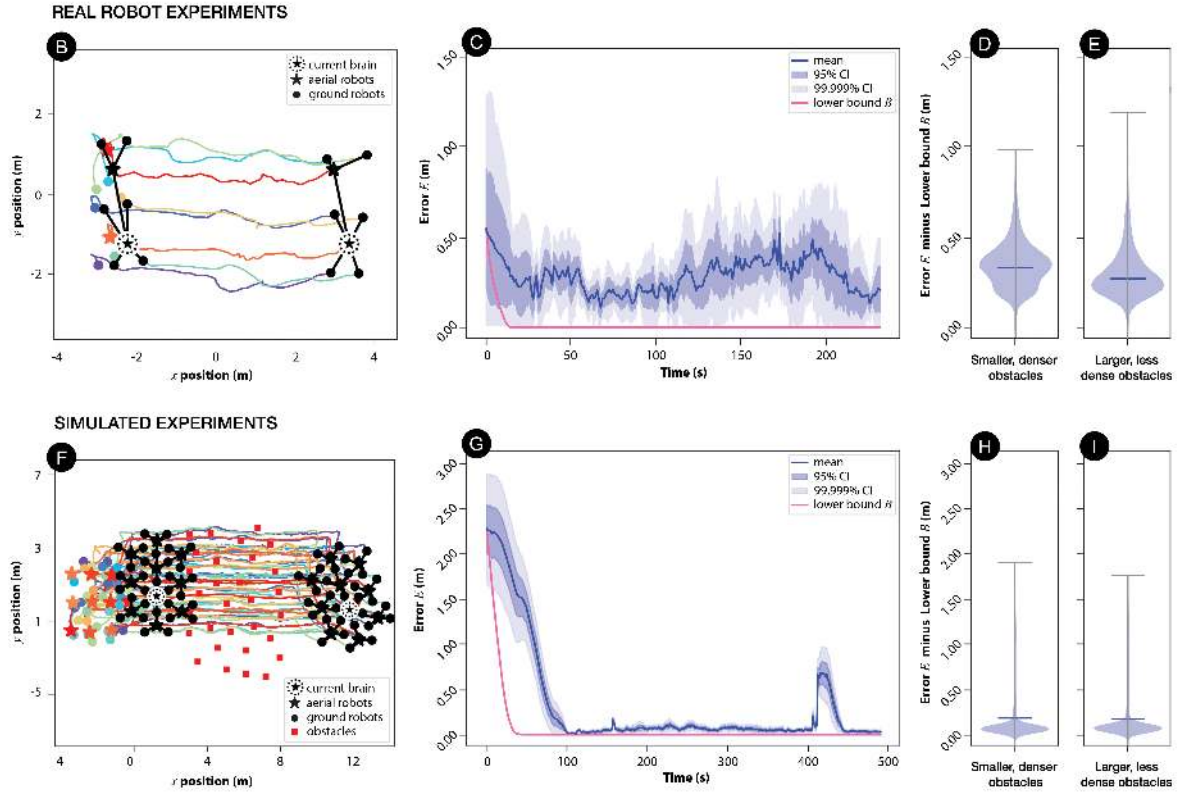
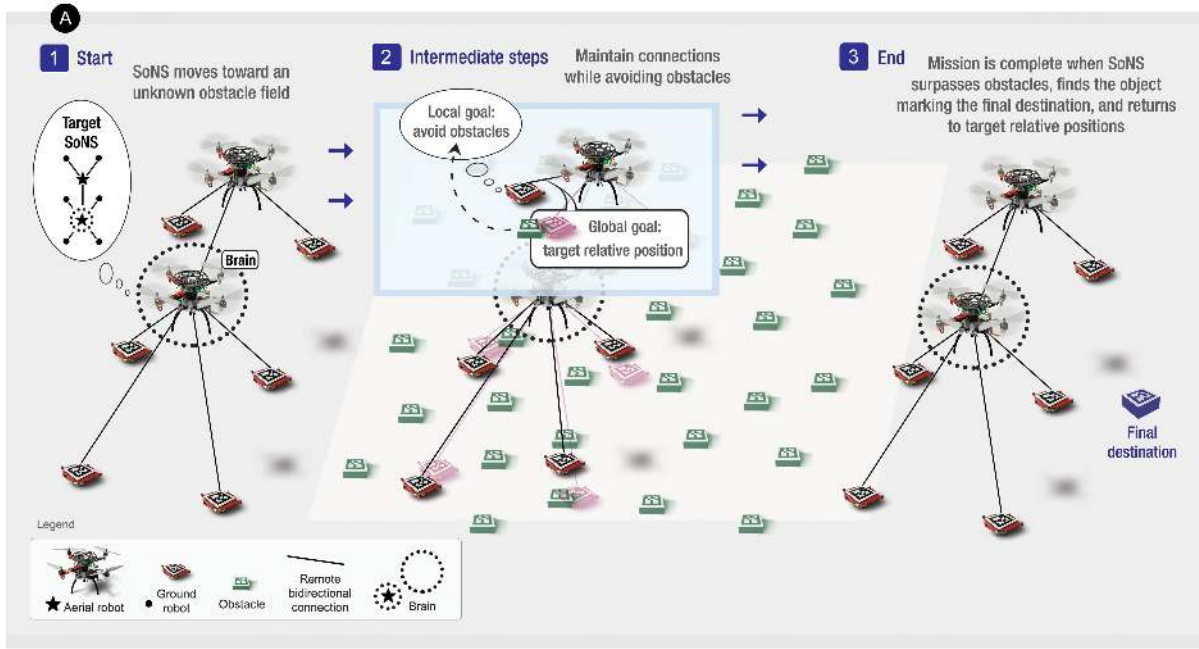


Fig. 3: Balancing global and local goals. In this experiment, robots need to navigate a field of obstacles scattered in one portion of the arena, while maintaining the bidirectional connections of the target SoNS. **(A)** Mission schematic: (1) Robots begin as members of a single SoNS and begin moving across an environment with an unknown field of small, dense obstacles, searching for an object that marks the final destination. (In a second variant, not pictured here, the obstacles are larger and less dense.) (2) Robots move through the obstacle field, collaboratively balancing global and local goals at each bidirectional link, to avoid obstacles while still keeping the SoNS together, until (3) the SoNS surpasses the obstacle field and senses the final destination object. **(B-E)** Results of the real experiments. **(B)** Trajectories of robots over time, with the initial and final SoNS indicated in black (on the left and on the right, respectively) and **(C)** actuation error E (mean and confidence interval per robot over time, see Eq. 1, with lower bound B , see Eq. 2, plotted for reference) in a real example trial (shown in Movie S2 in the supplementary materials). **(D,E)** Violin plots of the actuation error minus the lower bound $E - B$ (mean and confidence interval per robot per second) in all real experiment trials, for both mission variants: **(D)** smaller and denser obstacles, five trials, and **(E)** larger and less dense obstacles, five trials. **(F-I)** Results of the simulated experiments, given in the same format: **(F)** trajectories and **(G)** actuation error E in a simulated example trial, with **(H,I)** violin plots of the actuation error minus the lower bound $E - B$ in all simulated trials, 50 trials per variant. (For more detailed experiment results, see Sec. S1 of the supplementary materials.)

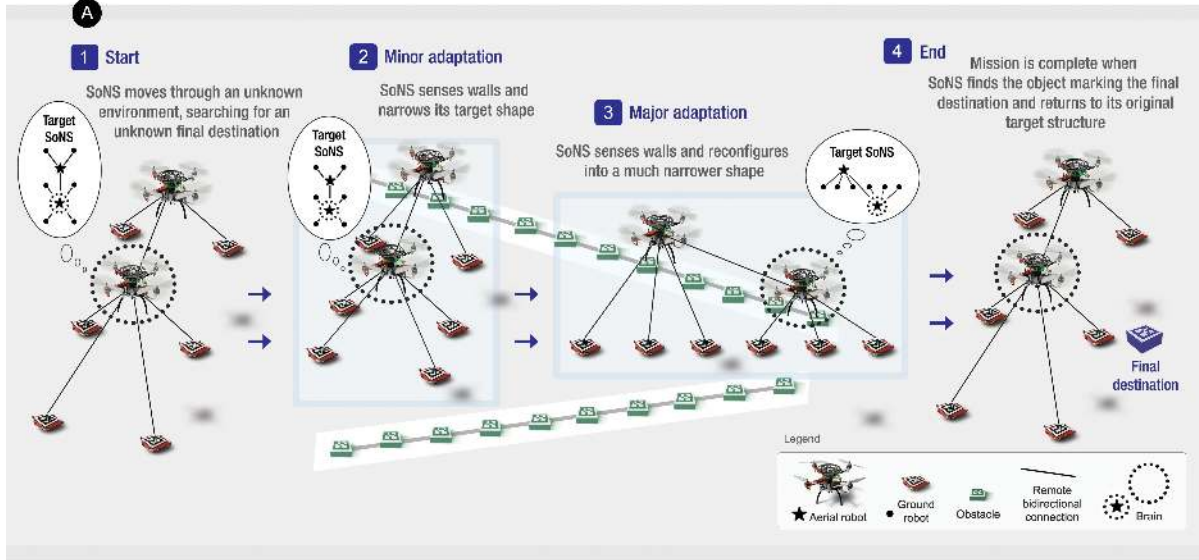
until reaching an object that marks the final destination (at an unknown position). The goal of these missions is for the robots firstly to maintain the target topology of the communication structure without any connection breaks, despite the disturbances from the environment caused by the presence of obstacles, and secondly to recover the target relative positions once the obstacle field has been surpassed. In this mission, the SoNS maintains a consistent communication structure and overall system behavior throughout, and allows the inter-level control distribution along each bidirectional connection to be adjusted as needed.

The obstacles are scattered in one portion of the arena and their positions and types are not known by any of the robots beforehand. In the two mission variants, either (i) obstacles are larger than the ground robots, so that circumventing the obstacles is challenging, or (ii) obstacles are roughly the same size as the ground robots and are positioned more densely, so that navigating through the gaps is challenging (this variant is shown in Fig. 3A).

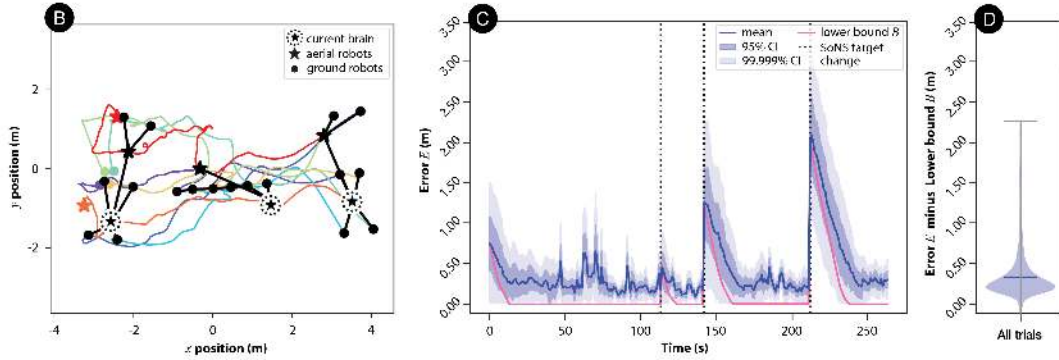
In all trials, the robots successfully complete the mission goals and the actuation error returns to its pre-obstacles level (see final SoNS positions in Figs. 3B,F; see final error levels after 230 s in trials with real robots in Fig. 3C and after 450 s in simulation in Fig. 3G). The progression of a typical real experiment can be seen in the example trial in Fig. 3C: the actuation error drops as the robots reach their initial target relative positions (approx. 0 to 30 s), then rises when the SoNS begins to encounter obstacles (at approx. 30 s), remains unsteady and continues rising as the SoNS passes through the obstacle field (approx. 30 to 190 s), and starts to decline when the ground robots start to surpass the last obstacles (at approx. 190 s); then, once all robots have exited the obstacle field, the SoNS re-converges to its target relative positions. The progression of a typical simulated experiment can be seen in the example trial in Fig. 3G: the actuation error drops to almost zero as the robots reach their initial target relative positions (at approx. 100 s), then rises slightly and becomes slightly unsteady as the SoNS passes through the obstacle field (approx. 150 to 400 s); after the SoNS exits the obstacle field, the robots at the front start to sense the final destination object and the SoNS adjusts its target trajectory to move towards it, causing the error to increase (approx. 410 to 440 s) until the adjustment is complete and the SoNS returns to negligible steady-state error (at approx. 440 s). All trials reach a low steady-state error, in both reality and simulation, with the smaller, denser obstacles variant displaying slightly higher average error than the larger, less dense variant (see Figs. 2D,E,H,I).

2.1.3 Collective sensing and actuation

The next proof-of-concept setup is a sweeping mission (see Fig. 4), in which a SoNS needs to navigate an environment while collectively sensing and reacting to unknown conditions. Like in the previous mission, the robots begin as members of one SoNS and the brain begins with a straight trajectory in a given direction. The goal of the mission is to move straight through a passage while keeping the SoNS shape as wide as can be fit, between walls composed of



REAL ROBOT EXPERIMENTS



SIMULATED EXPERIMENTS

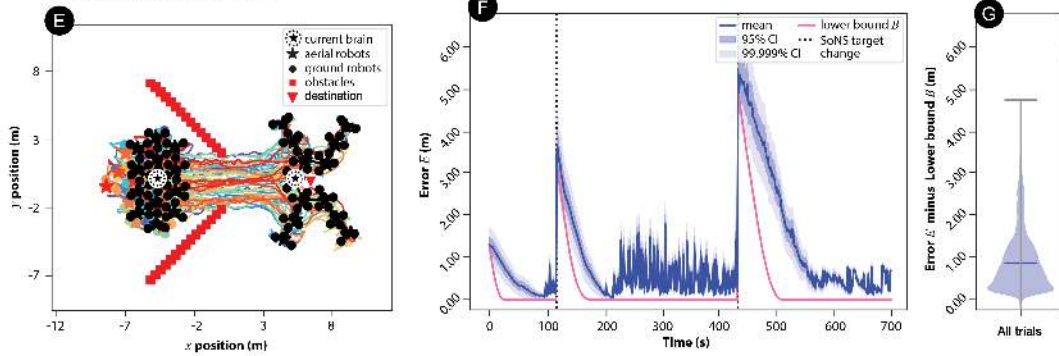


Fig. 4: Collective sensing and actuation. In this experiment, robots need to sweep the environment while collectively reacting to the width of the passage. **(A)** Mission schematic: (1) Robots that have established their target SoNS begin moving across an environment, searching for an object that marks the final destination. (2) Robots sense obstacles that form the walls of a passage and start to adapt by narrowing the shape of the target SoNS. (3) Robots sense the walls narrowing and then adapt further by reconfiguring into a different target SoNS that has an even narrower shape. (4) Robots sense that there are no longer walls constraining them, return to their original target SoNS, and stop when they sense the final destination object, completing the mission. **(B-D)** Results of the real experiments. **(B)** Trajectories of robots over time, with the initial, an example intermediate, and the final SoNS indicated in black (from left to right, respectively) and **(C)** actuation error E (mean and confidence interval per robot over time, see Eq. 1, with lower bound B , see Eq. 2, plotted for reference) in a real example trial (shown in Movie S3 in the supplementary materials). **(D)** Violin plot of the actuation error minus the lower bound $E - B$ (mean and confidence interval per robot per second) in all five real experiment trials. **(E-G)** Results of the simulated experiments, given in the same format: **(E)** trajectories and **(F)** actuation error E in a simulated example trial, with **(G)** a violin plot of the actuation error minus the lower bound $E - B$ in all 50 simulated trials. (For more detailed experiment results, see Sec. S1 of the supplementary materials.)

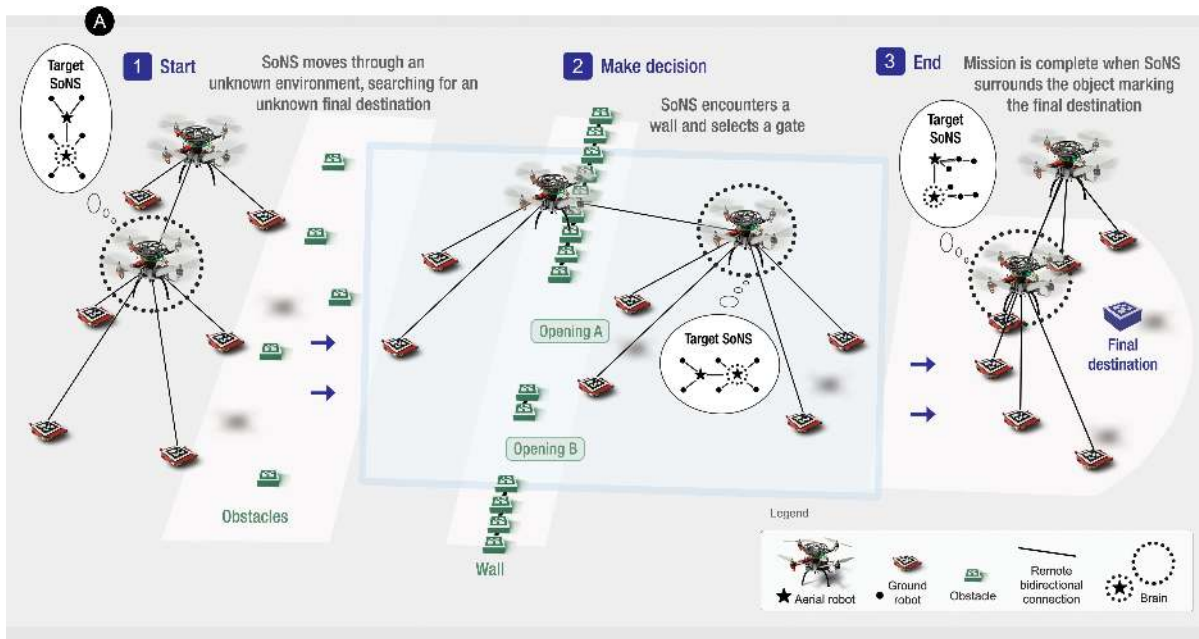
obstacles that enclose a passage of unknown width, until finding an object that marks the final destination. To accomplish this, the brain needs to fuse collective sensor information, determine the current width of the passage, and update the SoNS’s target communication topology and relative positions as needed. When these updates occur, the robots in the SoNS need to collectively reorganize, maintaining portions of the old structure when possible. In this mission, the SoNS reconfigures its communication topology as needed, while maintaining a consistent overall system behavior and consistent inter-level control distributions at each connection.

The environment is set up so that the walls get narrower as the SoNS progresses (see Fig. 4A). At first, the SoNS might be able to fit through the passage by simply narrowing the shape of its robot formation (i.e., the topology remains the same but the relative positions change). Later, the passage becomes so narrow that a full re-organization into a linear formation is required. Once the SoNS exits the narrow passage, it has the space to re-organize into a wider formation near the final destination object.

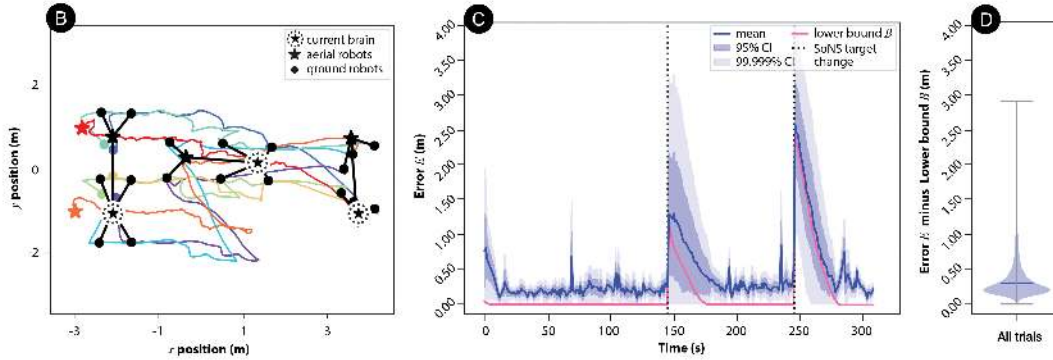
In all trials, the robots complete the mission goals. The actuation error starts low, spiking each time the brain initializes a new target structure, then declining gradually until the next re-organization (see Fig. 4C,F). Between re-organizations, many small spikes occur, as robots at the edge of the formation collide with the walls but determine the disruption can be managed locally and adjust their positions accordingly. The largest spikes occur when both the target relative positions and the target topology change. This can be seen in the example trial with real robots (see Fig. 4C): compare the first change (at approx. 115 s), which involves only positions, to the second (at approx. 140 s) and third (at approx. 215 s) target changes, which involve both topology and position. In the simulated example trial, the same progression can be observed, but with only two changes to the target SoNS (see Fig. 4F). Finally, after the SoNS re-organizes and reaches the final destination object, it returns to low steady-state error.

2.1.4 Binary decision making

The next proof-of-concept setup is a reactive path planning mission (see Fig. 5), in which a SoNS must make a binary choice between possible paths and update its trajectory accordingly, while continuously reconfiguring itself to navigate an unknown environment and search for an object that marks the final destination (see Fig. 5A). The robots begin the mission as members of one SoNS and the brain begins with a straight trajectory in a given direction. To start, the SoNS begins sweeping the environment in as wide a formation as can fit in the environment, because the position of the final destination object is unknown. When it encounters a field of small randomly scattered obstacles, the SoNS needs to negotiate its inter-level control distribution while continuing to sweep. When it encounters a wall, the SoNS must collectively detect two openings located at varying positions in the wall and choose the one that is larger, then update its path and reorganize itself to fit through the larger opening. The decision-making process is self-



REAL ROBOT EXPERIMENTS



SIMULATED EXPERIMENTS

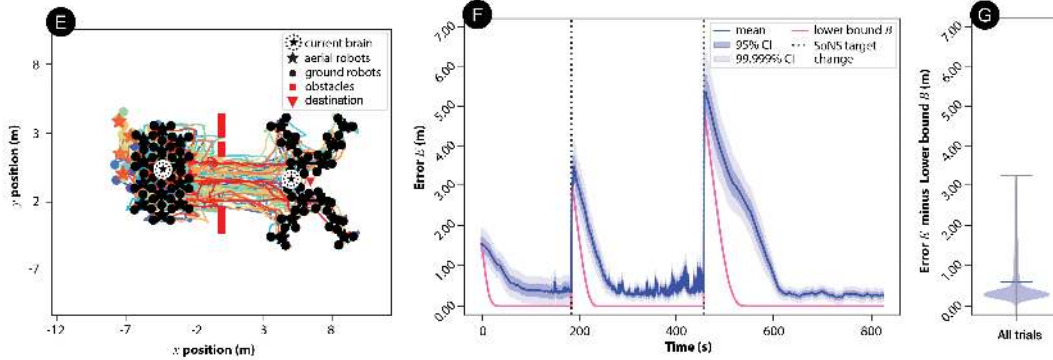


Fig. 5: Binary decision making. In this experiment, robots need to sweep the environment to find an object that marks the final destination, making a binary choice between two possible paths. **(A)** Mission schematic: (1) Robots that have established their target SoNS begin moving across an environment, searching for the final destination object. (2) After surpassing an obstacle field, robots sense a wall, collaboratively choose the largest opening, and adjust the path of the SoNS to pass through the selected opening. (3) Robots sense that there are no longer walls constraining them, then sense the final destination object and change their target SoNS to surround it, completing the mission. **(B-D)** Results of the real experiments. **(B)** Trajectories of robots over time, with the initial, an example intermediate, and the final SoNS indicated in black (from left to right, respectively) and **(C)** actuation error E (mean and confidence interval per robot over time, see Eq. 1, with lower bound B , see Eq. 2, plotted for reference) in a real example trial (shown in Movie S4 in the supplementary materials). **(D)** Violin plot of the actuation error minus the lower bound $E - B$ (mean and confidence interval per robot per second) in all five real experiment trials. **(E-G)** Results of the simulated experiments, given in the same format: **(E)** trajectories and **(F)** actuation error E in a simulated example trial, with **(G)** a violin plot of the actuation error minus the lower bound $E - B$ in all 50 simulated trials. (For more detailed experiment results, see Sec. S1 of the supplementary materials.)

organized: robots that sense openings vote on them according to size and reach a consensus, with the first robot to propose the selected opening then becoming the new brain. Once the SoNS passes through the opening, it must find the final destination object and reorganize to encircle it. The mission goal is to reach the final destination object by choosing the correct path between binary options with different qualities, without suffering any connection breaks from environmental disturbances in the meantime. In various portions of this mission, the SoNS adjusts its communication structure, inter-level control distributions, and overall system behavior as needed.

In all trials, the robots successfully complete the mission goals: the SoNS senses both openings and chooses the larger one, then reaches the final destination object, without any breaks in the system architecture (see Fig. 5B,E). The progression of a typical experiment can be seen in the example trials (see Fig. 5C,F): due to the complexity of the mission, the actuation error experiences many spikes. However, after these spikes, the robots converge to a topology encircling the final destination object, with low steady-state error. All trials reach a low steady-state error, with the average error with real robots being slightly lower than in simulation (see Figs. 5D,G).

2.1.5 Splitting and merging systems

The last proof-of-concept setup with real robots is a search-and-rescue mission (see Fig. 6), in which SoNSs split or merge in order to reunite with the missing robots. At initiation, one or more single-robot SoNSs are isolated somewhere in the environment and wait there to be found by a rescue team. There is also a primary multi-robot SoNS somewhere in the environment. This SoNS notices that it is missing a robot and starts a rescue mission to find one (see Fig. 6A). The primary SoNS does not know the direction of the missing robot(s), so the brain issues instructions to split into two independent SoNS with new mission goals. The SoNS associated with the original brain ('home' SoNS) will stay in place and wait, while the newly split 'rescuer' SoNS will explore. The rescuer SoNS follows found landmarks along a non-convex barrier until it finds and merges with the isolated robot(s). It then guides each found robot out of the barriers, and returns to and merges with the home SoNS by backtracking along the landmarks. The goal of these missions is for all robots in the environment to eventually merge into a single shared SoNS. In this mission, adjusting the inter-level control distribution at each connection would not be sufficient for the task requirements, so the SoNSs completely reconfigure their communication structures and system behaviors to complete the mission. In an alternative mission variant (see Sec. S1 of the supplementary materials), the primary SoNS knows the direction of its missing robot, but needs to physically push an obstruction out of the way, then merge with the missing robot and guide it out of a convex barrier. (For results of simulation experiments, see Sec. S1 of the supplementary materials.)

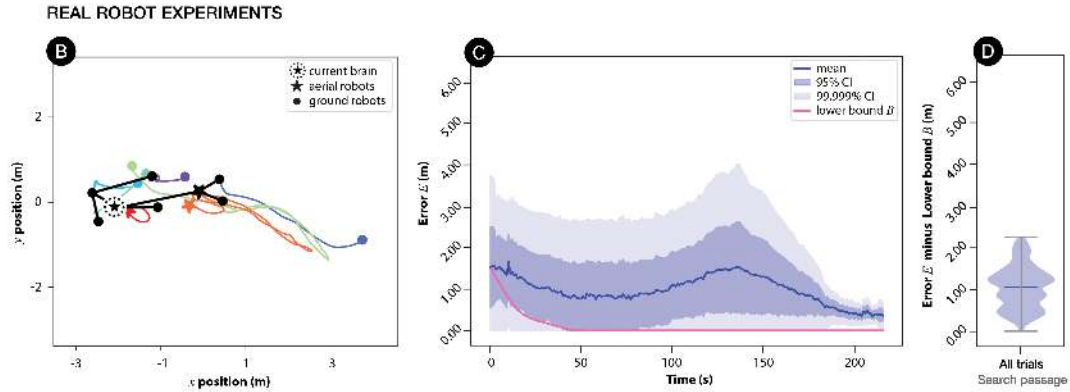
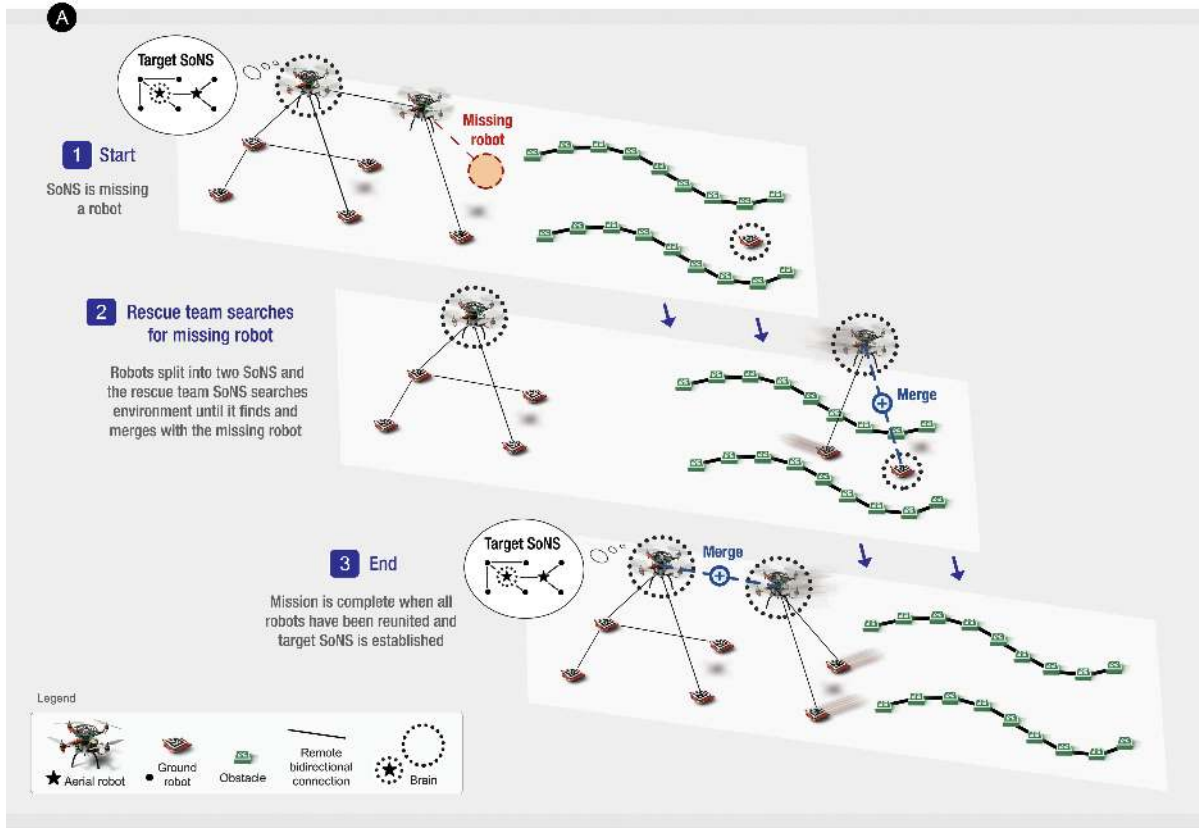


Fig. 6: Splitting and merging systems. In this experiment, robots need to conduct multi-SoNS search-and-rescue missions to find missing robot(s) in a passage between two walls composed of obstacles and reunite into a single SoNS. (In a second variant, not pictured here, but included in Sec. S1 of the supplementary materials, the robots need to collaborate to push an obstruction out of their way.) **(A)** Mission schematic: (1) When robots start, they are in a SoNS that is missing a member. (2) The brain instructs one of the robots to split from it and temporarily form its own multi-robot SoNS as a rescue team. The rescue team SoNS searches the environment until it finds the missing robot in its own single-robot SoNS and merges with it. (3) The merged rescue team SoNS then returns to the location where it initially split off and re-merges with the remaining SoNS, so that all robots are reunited. **(B-D)** Results of real robot experiments. **(B)** Trajectories of robots over time, with the initial and final positions indicated in color and in black, respectively, and **(C)** actuation error E (mean and confidence interval per robot over time, see Eq. 1, with lower bound B , see Eq. 2, plotted for reference) in a real example trial (shown in Movie S5 in the supplementary materials). **(D)** Violin plot of the actuation error minus the lower bound $E - B$ (mean and confidence interval per robot per second) in all five trials. (For more detailed experiment results, including the second variant and the simulation experiments, see Sec. S1 of the supplementary materials.)

In all trials, the robots successfully complete the mission by splitting and then merging (see Fig. 6B,D): the multi-robot SoNS(s) are successfully reunited with the isolated single-robot SoNS(s). The actuation error rises during the period of re-organizations, but all robots re-merge into one SoNS and the system returns to a low steady-state error (see Fig. 6C,D).

2.2 Scalability

We demonstrate the scalability of the SoNS architecture in swarms of up to 250 robots (200 ground robots, 50 aerial robots). We use the binary decision-making mission setup (see Sec. 2.1.4) with four different system sizes, up to 125 robots (50 trials per system size), and the establishment mission setup (see Sec. 2.1.1) with 50 different system sizes, up to 250 robots (30 trials per system size). Given that the real arena size is limited, we run the scalability experiments only in simulation (for cross-verification between the simulator and reality, see Sec. S2 in the supplementary materials). We also disregard the battery capacity of the quadrotor platform in these experiments.

The results of the binary decision-making experiments show that the robots complete all parts of the mission successfully (see example trial with 125 robots in Fig. 7A,B and Movie S6 in the supplementary materials; for more detailed experiment results, see Sec. S1 of the supplementary materials). These results provide a proof of concept that the demonstrated capabilities of the SoNS approach do not break down in larger swarms (up to 125 robots): the SoNS can balance global and local goals, collectively sense and react to an environment, make a collective binary decision, and reconfigure when needed, all without breaking the system architecture.

In the establishment mission experiments, we aim to test the scalability limits of the SoNS architecture under the current software implementation. We run experiments with system size $n = \{5, 10, 15, \dots, 250\}$ robots (30 trials for each system size) and a maximum experiment time of $t = 500$ s for $n \leq 125$ robots and $t = 4n + 4(n - 125)$ s for $125 < n \leq 250$ robots (see dashed line in Fig. 7F). In system sizes of $n \leq 125$ robots, all trials converge before the maximum experiment time. In systems of $125 < n < 220$ robots, one or two trials per system size do not converge before the maximum time (approx. 5% of trials, on average). In systems of $220 \leq n \leq 250$ robots, approx. 20% of trials do not converge before the maximum time. In trials that converge (shown in Fig. 7F), the mean convergence time rises superlinearly, with the rate of change increasing most noticeably after system sizes of 150 robots. We therefore consider the performance of the current SoNS implementation to be fully reliable in system sizes up to 125 robots, to be somewhat stable until 220 robots, and to degrade substantially in larger systems. See Sec. 3.1 for a discussion of actuation error and convergence times in swarms of 220 robots and larger.

The results of the establishment mission trials that converged before the maximum time

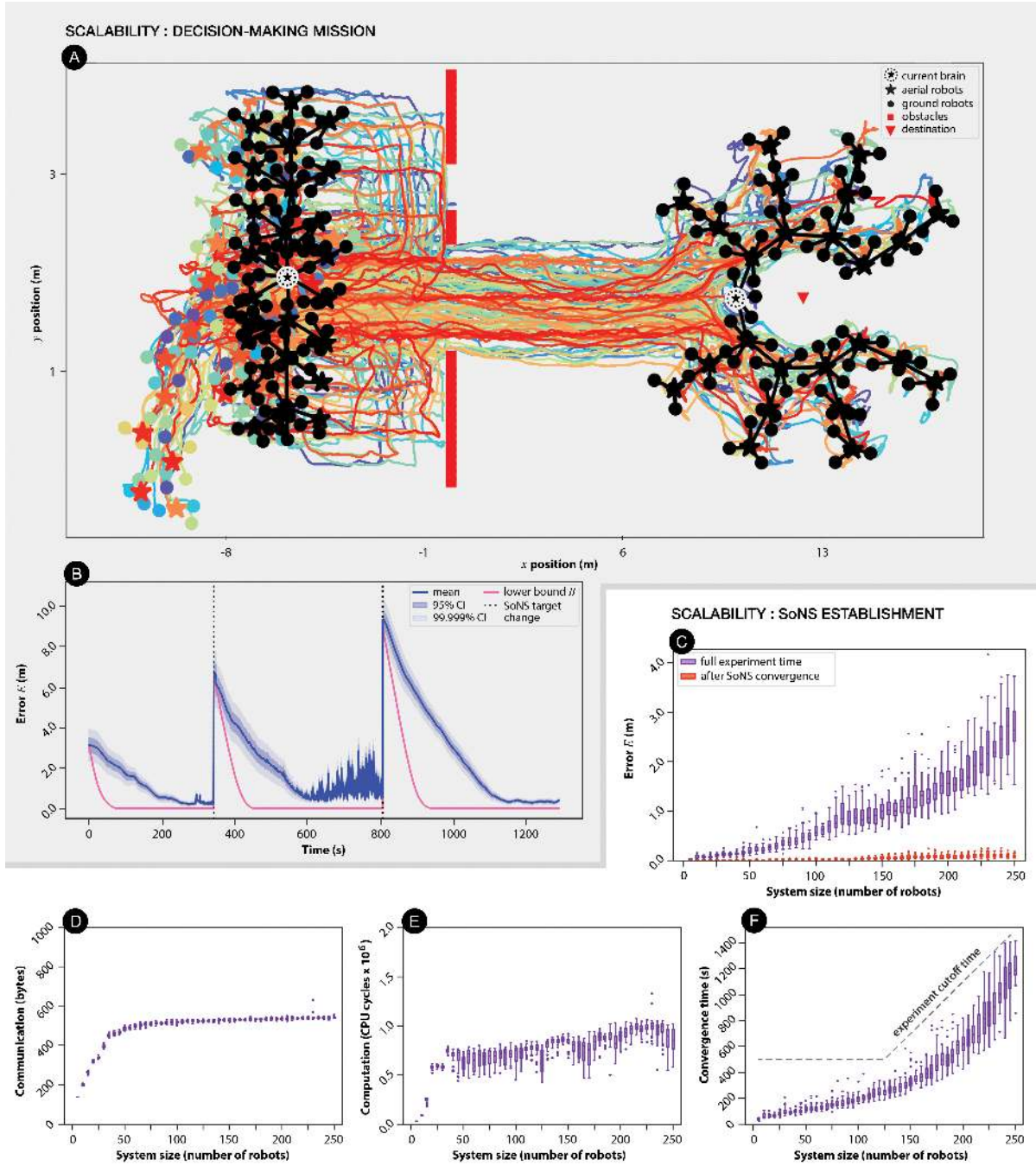


Fig. 7: Scalability study. Results in simulation with up to 250 robots (20% drones and 80% ground robots). **(A-B)** Demonstration of a 125-robot SoNS completing the binary decision-making mission shown in Fig. 5. **(A)** Trajectories of robots over time (with the initial and final SoNS indicated in black, on the left and on the right, respectively) in an example trial with 125 robots (shown in Movie S6 in the supplementary materials). **(B)** Actuation error E (mean and confidence interval per robot over time, see Eq. 1, with lower bound B , see Eq. 2, plotted for reference) in the example trial shown in (A). **(C-F)** Scalability measured by different performance metrics when the number of simulated robots increases from 5 to 250 in steps of 5, with 30 trials per system size (several example trials shown in Movie S7 in the supplementary materials). In this experiment setup, robots simply establish a SoNS, as in the mission shown in Fig. 2. All performance metrics are calculated per robot per step. **(C)** Actuation error E throughout the experiment (purple bars) and after the SoNS converges, thus reaching a steady state (red bars); **(D)** communication, measured in bytes of messages passed; **(E)** computation, measured in the maximum CPU clock cycles for any robot in the SoNS; and **(F)** convergence time. (For more detailed experiment results, see Sec. S1 of the supplementary materials.)

are shown in Fig. 7C-F. (Several example trials are shown in Movie S7 in the supplementary materials.) Fig. 7C (see purple bars) shows the mean and variance of the actuation error per robot. The error increases with the number of robots: slowly at first, and more substantially in systems larger than 100 robots. This increase in error according to system size can be mostly attributed to the rising Euclidean distance between the starting positions and the target positions in the eventual SoNS—i.e., the greater the number of robots, the higher the lower bound B of the error (see Eq. 2). Indeed, when we measure only the actuation error after the SoNS has finished converging, and therefore the starting positions are no longer relevant (see red bars in Fig. 7C), we see that the steady-state error increases only slightly with system size and always remains low (less than $E = 0.5$ m).

We measure the communication load in terms of mean bytes per robot (inclusive of inbound and outbound messages) and the computational work in terms of maximum CPU clock cycles for any robot in the SoNS, which would be the same on the real robots as in the simulation. (Our simulations run the same software modules as those run on the real robots, see Sec. S2 of the supplementary materials.) The results show that, in system sizes of 50 robots or more, the communication load nearly plateaus (see Fig. 7D) and the mean CPU cycles rise only slightly, with a moderate increase in variability (see Fig. 7E). The rise in communication load that occurs in small swarm sizes (see Fig. 7D) is in part a function of the maximum communication range and maximum robot density (due to minimum safety distances). For small swarm sizes, the average number of robots each robot has in its field of view (including both connected robots and unconnected robots that might be candidates for recruitment operations) grows with swarm size until the robot density reaches its upper limit, after which, the average number of robots in each robot’s field of view plateaus (at approx. 50 robots). The similar rise in computational work in small swarm sizes (see Fig. 7E) is in part a function of the maximum number of connections per robot in the target SoNS’s communication topology. For small swarm sizes, the average realized connections per robot rises until the swarm is large enough that most robots reach the maximum number of connections per robot allowed in the current topology design (max. 8 per robot), after which, the average realized connections per robot plateaus (at approx. 35 robots). We conclude that the fusion and compression techniques used for sensing information and actuation instructions in this implementation of the SoNS concept are sufficient for the robots and missions at hand, and that the SoNS architecture can be considered scalable in terms of overall communication and calculation loads.

2.3 Fault tolerance

We demonstrate several aspects of fault tolerance in SoNS, in real and simulated swarms. First, using real robots, we show replacement of a single robot that has failed permanently, including a

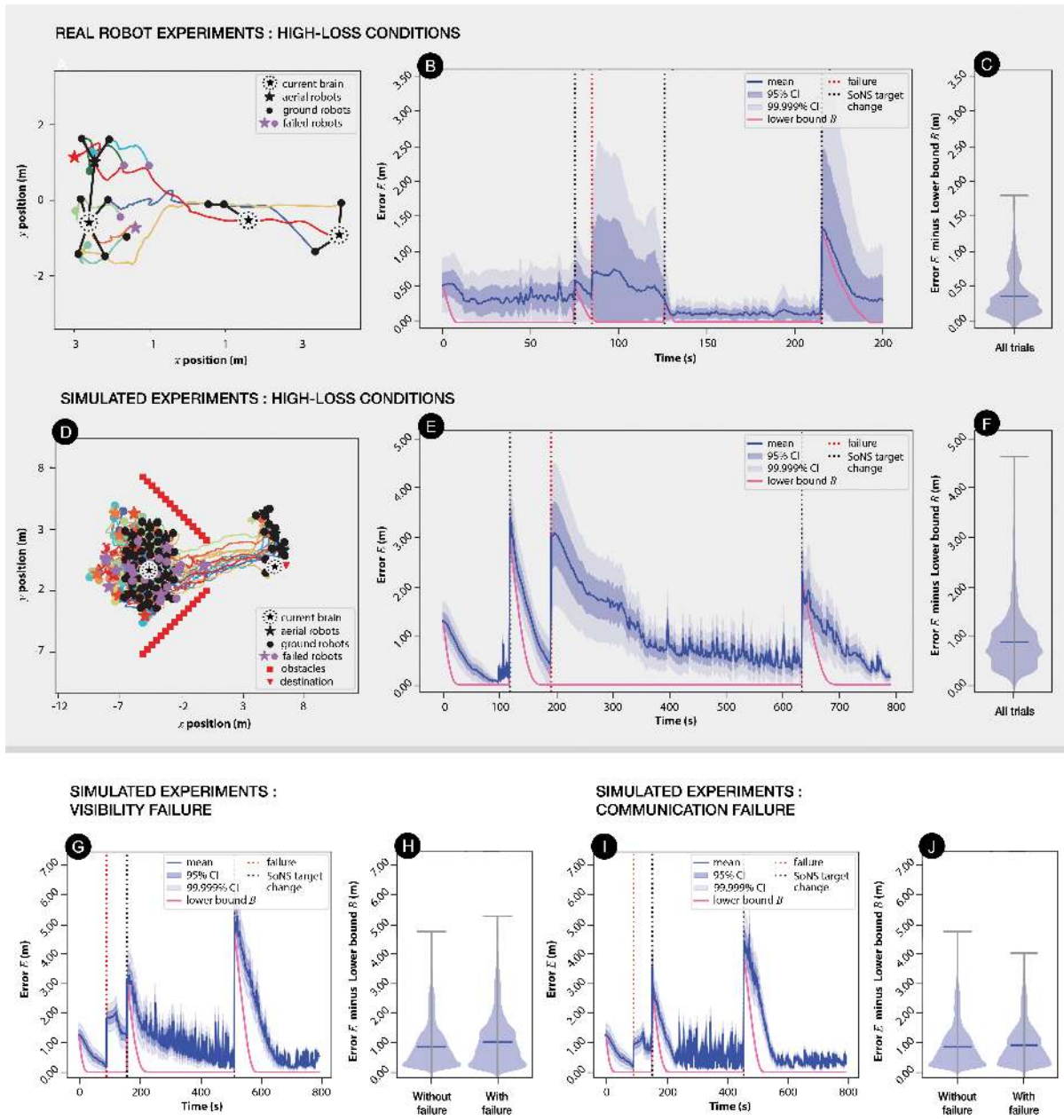


Fig. 8: Fault tolerance study. Results with real robots and in simulation, testing both permanent robot failures in part of the SoNS (up to two-thirds of the robots) and temporary system-wide failures of communication or vision. (A-C) High-loss conditions with real robots: arbitrary permanent failures of multiple robots. (A) Trajectories of all robots over time, with the initial, an example intermediate, and the final SoNS indicated in black (from left to right, respectively), and the failed robots indicated in purple at their shutdown positions, and (B) actuation error E (mean and confidence interval per robot over time, see Eq. 1, with lower bound B , see Eq. 2, plotted for reference) in an example trial (shown in Movie S9 in the supplementary materials); (C) violin plot of the actuation error minus the lower bound $E - B$ (mean and confidence interval per robot per second) in all five real robot experiment trials. (D-F) High-loss conditions in simulation, with either 33.3% or 66.6% probability for each robot to fail. (D) Trajectories of robots over time, with the initial and final SoNS indicated in black (on the left and the right, respectively), and the failed robots indicated in purple at their shutdown positions, and (E) actuation error E in an example trial with 66.6% probability for each robot to fail (shown in Movie S10 in the supplementary materials); (F) violin plot of the actuation error minus the lower bound $E - B$ in all high-loss experiment trials (including both 33.3% and 66.6% failure probabilities, 50 trials per probability). (G-H) System-wide failure of vision in simulation: (G) actuation error E in an example trial with 30 s failure (shown in Movie S11 in the supplementary materials) and (H) violin plots comparing the actuation error minus the lower bound $E - B$ in all 50 trials with 30 s vision failure to the 50 trials without failure. (I-J) System-wide failure of communication in simulation: (I) actuation error E in an example trial with 30 s failure (shown in Movie S12 in the supplementary materials) and (J) violin plots comparing the actuation error minus the lower bound $E - B$ in all 50 trials with 30 s communication failure to the 50 trials without failure. (For more detailed experiment results, see Sec. S1 of the supplementary materials.)

failed brain (see Movie S8 in the supplementary materials). In these demonstrations, one robot is remotely triggered to fail (for the aerial robots, this includes immediately landing in place). Then, a new robot of the same hardware type is manually placed in the arena and switched on, after which it is recruited by the SoNS. When a brain robot or a robot at an inner hierarchy level fails, it is immediately and automatically replaced by another robot already in the SoNS, and the SoNS reorganizes around the change. Then, when a new robot is recruited, it fills the leftover vacancy in the re-organized SoNS.

Using real robots, we also demonstrate SoNSs reorganizing in high-loss conditions, i.e., after arbitrary permanent failure(s) when the failed robot(s) cannot be replaced. The setup used is that of the collective sensing and actuation mission (see Sec. 2.1.3), with five trials run. The robots start the mission as one SoNS and, after failure occurs, the SoNS continues the mission with the robots available. The full set of results (see Sec. S1 in the supplementary materials) shows that when ground robots fail, the rest of the robots are able to stay connected in one SoNS and complete the mission, whereas when an aerial robot fails, some of the ground robots downstream from it will be disconnected from the primary SoNS, but the remaining connected robots are able to continue the mission. For example, in the trial shown in Figs. 8A,B, one of the aerial robots fails (see purple star in A) at approx. 85 s (see red dotted line in B). The only aerial robot that remains functional recruits the ground robots that remain functional and reachable, and continues with the mission, eventually reaching the object marking the final destination. In all trials in which at least one robot of each type remains functional, the SoNS is able to re-organize itself with the remaining robots and continue with the mission, resulting in a relatively low overall error rate for all trials (see Fig. 8C).

In simulation, we run the same setup with a larger swarm of 65 robots to test task performance under high-loss conditions. In this setup, each robot has probability p to fail regardless of hardware type. We test two variants by setting $p = 0.\overline{3}$ or $p = 0.\overline{6}$, with 50 trials per variant. In these simulated experiments, the brain can be one of the robots failing. For example, in the trial in Fig. 8D-F, after two-thirds of all robots fail (including the brain), the SoNS continues with the mission, eventually reaching the final destination object. In all trials, the SoNS is able to re-organize itself with available robots (i.e., robots that have not failed and are not stuck in place), returning to a low steady-state error (see Fig. 8E).

Also in simulation in a swarm of 65 robots, we test two types of temporary system-wide failures that would be likely to occur in practice: vision failure (for example, because of an obstruction in the environment) and wireless communication failure. We test visibility and wireless communication failures with durations of 0.5 s, 1.0 s, and 30 s, with 50 trials per duration for each failure type. The results (see Fig. 8G-J) show that, in all cases, the SoNS is able to re-establish itself after the system-wide failure, converging on the correct communication topology and relative positions. The results also show that in all trials the SoNS is able to

continue with the mission, leaving behind less than 5% of robots (i.e., three robots or less) in any trial, with the actuation error increasing temporarily after failure occurs (increasing by less than $E = 0.7$ m after the red dotted line in Fig. 8G,I) and then returning to a lower steady state.

2.4 Convergence and stability analysis

The convergence of the position errors and closed-loop stability of position tracking, with respect to the control inputs, are guaranteed in SoNS for both moving and motionless cases under ideal relative distance sensing. For all simulated or real setups studied in this paper, a SoNS of n robots using a baseline control law will stably track the target relative positions. The large-scale swarms in our scalability experiments are shown theoretically to converge and be stable. A full description of the analysis results is provided in Sec. S3 of the supplementary materials.

3 Discussion

The presented results demonstrate that the SoNS approach greatly expands the state of the art in swarm robotics. Its four novel features allow robot swarms, for the first time, to complete centrally-defined mission goals using a self-reconfigurable system architecture.

Firstly, the self-organized controllable hierarchy of the SoNS approach, which is novel for robot swarms, is demonstrated in all missions through the maintenance and reconfiguration of remote communication structures. The topology and relative positions are both controllable (sometimes separately), without any external intervention. For example, in the sweeping mission showing collective sensing and actuation (Sec. 2.1.3), the SoNS adapts to moderate changes in the environment by reconfiguring relative positions separately from topology, then adapts to greater changes by reconfiguring them both. The second novel feature of the SoNS approach—its self-organization of explicit and interchangeable leadership roles—enables robot systems to establish and reorganize themselves flexibly, without having to reinitialize the whole architecture if, for example, a brain loses connection or some of the robots at an inner hierarchy level experience environmental disturbances. This feature can be seen in the fault tolerance experiments (Sec. 2.3), in which SoNSs are able to automatically reorganize after a brain is lost or a majority of robots fail, continuing their mission without having to backtrack on past progress or synchronize with an external reference. Thirdly, inter-system reconfiguration is demonstrated during all SoNS establishment operations and splitting and merging operations, such as in the search-and-rescue mission (Sec. 2.1.5). As another example, during establishment operations in the scalability experiments (Sec. 2.2), many SoNSs of a few robots are first formed, then these SoNSs begin to merge with each other simultaneously, redistributing themselves retaining existing sub-structures when possible. The fourth novel feature of the SoNS

approach is the ability to reconfigure the system’s internal behavior structures, for instance by renegotiating the inter-level control distribution, as seen in the balancing of global and local goals during the obstacle-avoidance mission (Sec. 2.1.2). As another example, internal management of the SoNS-wide behaviors is demonstrated in the path-planning mission with binary decision-making (Sec. 2.1.4), during which a SoNS must multi-task, combining behaviors in an ad-hoc way. This on-demand multi-tasking might be straightforward in single robots or centralized systems, but is a novel advancement for robot swarms.

In short, we have shown the SoNS system architecture to provide a reconfigurable explicit control hierarchy for robot swarms, which is in itself novel. Beyond that, because the SoNS approach allows the system architecture to be multi-level and dynamic while still being self-organized, it enables qualitatively new behaviors and behavior combinations in robot swarms. The collective sensing and actuation (Sec. 2.1.3), binary decision-making (Sec. 2.1.4), and splitting and merging missions (Sec. 2.1.5) reflect the relative ease and quickness of behavior design and management as well as flexibility in behavior combinations. Using the SoNS approach, we have shown that robot swarms can handle increased mission complexity, self-reconfiguring their communication, control, and behavior structures on demand.

We have also shown that the SoNS approach retains the fault tolerance and scalability advantages for which robot swarms are often studied. Any robot, including the brain, is shown to be immediately and automatically replaceable in both reality and simulation, and SoNSs are able to recover and continue missions in high-loss conditions, as well as recover from temporary system-wide failures of detection or communication. In these fault tolerance experiments (Sec. 2.3), we specifically tested a SoNS’s ability to recover without adapting its overall mission, to assess baseline performance. If robots had instead been allowed to search for each other after experiencing disconnections (as shown in the search-and-rescue mission), then more robots could have been retained. Although the robot group sizes in the real missions are constrained by arena size, all missions were also successfully completed in swarms of up to 250 robots in simulation, using a simulator whose reliability has been verified against the behavior of the real robots (see Sec. S2 of the supplementary materials). Furthermore, in swarms of up to 250 robots, the simulation studies (Sec. 2.2) confirmed that the computation and communication metrics scale linearly or plateau, demonstrating the absence of problematic bottlenecks. Overall, the results show that, using the SoNS, sensing, actuation, and decision-making can be coordinated SoNS-wide, without sacrificing scalability, flexibility, and fault tolerance.

3.1 Limitations and future work

Although the actuation error in a SoNS remains relatively small in systems of less than 150 robots, we can see that it becomes much larger as the SoNS scales, especially in SoNSs of

220 robots and larger. This error is not only caused by the SoNS architecture: some of the error is unavoidably caused by using a purely reactive control law to manage relative positions when following a leader. In the current setup, robots calculate their new velocities after the velocities of the upstream robots have already been updated. Using this strategy, error can be guaranteed to be bounded (see Sec. S3 of the supplementary materials) but not guaranteed to be zero. The error could be reduced by incorporating feedforward terms into the motion control of a SoNS. Delays could be handled by, for example, sufficient preview of a reference signal: if the brain were to perform short-term online path planning, it could communicate a near-future reference trajectory downstream, instead of simply its current velocity. Similarly, in the current setup, robots calculate velocities based on distance measurements, which requires high sensing precision. The error could be reduced by instead using bearing (i.e., angle of arrival) measurements of two robots, which requires less sensing precision (for details, see our work on bearing-based frameworks [36]).

Closely related to this issue is the reaction time, which in SoNSs is based on communication speed and localization speed. In SoNSs with vision-based positioning, overall reaction time and stability will be increased as advances are made in visual tracking (for example, using ultraviolet LED markers [45] or faster fiducial marker tracking [46]). Advances made in other types of relative positioning that are less influenced by disturbances—such as vibration in powerful aerial robots or signal attenuation problems in underwater robots—will also be important for SoNS implementations with other types of robot platforms, for instance that can move at higher velocities than the laboratory ground robots used here. Certain approaches to relative positioning will require SoNSs with multi-layer networks (for instance, one layer for bearing-based relative positing and another layer for hierarchy of supervision [36]). If it is found useful in the future for some layers of the network to be cyclic, then consensus mechanisms will need to be studied for the management of information fusion at the occurrence of cycles.

Currently, our open-source SoNS software supports experiments with real robots that use vision-based relative positioning, with aerial robots that are able to detect the relative positions and relative heading orientations of their ground robot neighbors. Note that the restrictions we imposed on the real experiments due to practical constraints are not limitations of the SoNS approach in general—for instance, SoNS could be applied to other types of robot platforms. Additional versioning of our current software repository could also support other robot platforms and relative positioning techniques. Regardless of the approach used for positioning in a SoNS, a bidirectional connection can only be formed between two robots if at least one of them can detect the position of the other.

One of the biggest potentials moving forward is the possibility for more advanced SoNS brains and more advanced hierarchical computation, for instance by developing SoNSs with greater situational awareness (i.e., understanding of a situation and environment, especially for

detecting risks), online learning, or autonomous mission planning capabilities. Automatic design approaches such as neuroevolution might help SoNS brains handle ever increasing mission complexity, or artificial neural networks such as autoencoders might expand the ability of a SoNS to internally manage and react to large amounts of sensor data.

4 Materials and Methods

4.1 SoNS control

Full descriptions of the SoNS algorithms are provided in Sec. S4 of the supplementary materials and the SoNS software is available in an open-source repository. The primary operations are establishment, splitting, merging, node allocation, collective actuation via motion, and collective sensing and reaction, as described below.

Establishing a SoNS. The process by which a SoNS establishes, maintains, and reconfigures its dynamic hierarchical network is fully self-organized. At the start, the robots are all running identical SoNS software. Each robot starts as an independent single-robot SoNS, of which it is the brain by default. Each brain has a map of the communication structure it would like to build: the target topology represented by graph G and target relative positions and other conditions represented by attributes A associated to the nodes and links of G . The map can be, for example, calculated by a robot based on environmental features, predefined manually, or defined using a lookup table. Each brain then searches for robots to recruit in order to populate its map. When two SoNS that are searching for robots meet, each tries to recruit the other and, if they reach an agreement, the two merge under a single brain to become one SoNS. When a recruitment operation is completed and a new link $\{x_n, x_{n+1}\}$ is established, the robots have already reached an agreement about which robot will be at the higher level in the system architecture (i.e., x_n , the “parent” node for that connection) and which will be at the lower level (i.e., x_{n+1} , the “child” node). Once a brain x_1 with map G has successfully recruited some children $\{x_2, x_3, \dots, x_n\}$, it sends each child a map of the respective structure that should be built downstream from it: the subgraphs $\{G'_1, G'_2, \dots, G'_n\}$ and the associated subsets $\{A_1, A_2, \dots, A_n\}$. Each child then takes full responsibility for the sub-structure directly downstream from it. It tries to recruit robots to populate the partial map it has received, becoming the parent of those new robots if an agreement is reached. It then repeats the process of map subdivision, and a new level of children begins to recruit robots and become parents. Meanwhile, as a robot recruits children successfully, if it receives messages from its parent that the new children are needed more urgently elsewhere in the SoNS, it might choose to hand over some of its children to its parent, to be redistributed in the SoNS in a self-organized way. These operations continue until the SoNS is complete, according to the brain’s dynamic map G , or until no new robots can be

found for recruitment.

Splitting and merging. The simplest splitting operation is the departure of a single-robot SoNS, triggered by the former parent of the robot that departed. If, for instance, robot x_n has completed its map G'_n , but the parent of robot x_n then updates G'_n from $E = \{\{x_n, x_{n+1}\}, \{x_n, x_{n+2}\}\}$ to $E = \{x_n, x_{n+1}\}$, robot x_n might choose to expel one of its children by breaking the link to it. The expelled child will immediately and automatically return to being the brain of its own single-robot SoNS. A splitting operation that results in two multi-level SoNS is similar, but will be triggered at a higher level in the hierarchy. For example, a brain might update its own map G from having two children $\{x_2, x_3\}$ with their subgraphs G'_1 and G'_2 to having only one child x_2 . In that case, it will choose to expel x_3 , which after splitting will maintain all its downstream links according to its map G'_2 . The expelled robot x_3 will automatically become the brain of its own SoNS, and might choose to update its map, for instance from $G'_2 = (V = \{x_4, x_5, x_6, x_7\}, E = \{\{x_3, x_4\}, \{x_3, x_5\}, \{x_4, x_6\}, \{x_5, x_7\}\})$ to G with the same V set but $E = \{\{x_3, x_4\}, \{x_4, x_5\}, \{x_4, x_6\}, \{x_4, x_7\}\}$. In this case, x_3 will not reinitialize the whole structure, but will instead, for example, maintain links $\{x_3, x_4\}$ and $\{x_4, x_6\}$ and reorganize the other two links to match its new G . Unlike in splitting operations, the negotiations of merging operations always include the two brains of the respective SoNS. The simplest merging case is when one brain tries directly to recruit another brain. In this case the two brains will compare their internal assessments of their quality (for example, how many robots are in their SoNS), and the lower quality brain will agree to become the child of the higher quality brain. If the two brains find that they have equal quality, the allocation is chosen randomly. If a merging operation instead initiates downstream from one or both brains, the same quality comparison and agreement occurs, but some information propagation and node reallocation occurs to enable the comparison and the eventual merge.

Node allocation. All reconfiguration in a SoNS is done in a self-organized way using strictly local communication, and therefore multiple operations will often occur simultaneously. This makes it likely that a robot in the inner hierarchy levels will have to allocate more than one new child at the same time, due to recruitment or redistribution through handovers. For example, in the case of a local node allocation (i.e., a parent needs to allocate multiple candidates to its own children roles), the parent will compare the current relative position and current downstream order (i.e., number of vertices in the sub-graph) of each candidate with the target relative position and the target downstream order for its child nodes, as defined in its map G_n and A_n . Similar operations assessing robots according to position and downstream order are used for all node allocation problems, including a candidate matching multiple local roles, multiple candidates matching a single local role, and a candidate matching no local role and therefore being reallocated upstream. Importantly, a parent can also use these operations to reallocate children it already has. A parent can replace an existing child with a candidate that is a better match for

the role, then demote the former child to candidate status and enter it into the next round of local node allocation. Because of this replacement possibility, the robots in a SoNS continuously redistribute themselves within the SoNS. For instance, if a SoNS has one unoccupied node on its eastern-most side and recruits a new robot on its western-most side, the new robot will not be inefficiently handed over link-by-link until it reaches the unoccupied node. Rather, the robots in the SoNS will shift themselves in a self-organized way, so that the new robot is allocated to a nearby node, replacing a previous child, and a robot that was already near to the unoccupied node ends up being reallocated to it.

Collective actuation via motion. A connection can only be established and maintained in a SoNS if one robot is in the other’s field of view, so it is essential that the robots of a SoNS can move together collectively. In the simplest case, the brain defines and follows its motion trajectory independently of the robots downstream from it, and the rest of the robots act as followers of their respective parents, using strictly reactive control. In this case, each non-brain robot receives two controller inputs from its parent, which the parent calculates from the attribute set A_n of its map G_n : a target displacement vector $\mathbf{d}$ and target orientation in unit quaternion $\mathbf{q}$. The parent will update the controller inputs $\mathbf{d}$ and $\mathbf{q}$ for its child when there is a change in its A_n . Using the inputs $\mathbf{d}$ and $\mathbf{q}$ and a mass-spring-damper model, each child outputs its own target linear velocity vector $\mathbf{v}_t$ and target angular velocity vector $\boldsymbol{\omega}_t$ and uses them to calculate its motor inputs for time t . In cases where the inter-level control distribution of the SoNS is incorporating some decentralized motion control (such as in the obstacle avoidance mission), a child might add another control layer to its mass-spring-damper model, or its parent might override its default behavior to temporarily diverge the calculation of $\mathbf{d}$ and $\mathbf{q}$ from the goals specified in A_n .

Collective sensing and reaction. When a non-brain robot senses an external signal in the environment, it chooses both whether to respond to the signal directly and whether to send some compressed representation of this sensor information (for example, symbolic representations of identified objects). If a robot chooses to send sensor information to neighbors, the information is sent upstream. In exceptional cases, it can also choose to send the information downstream. Its choices might be based on a behavior model received from its parent or one defined by itself, depending on the inter-level control distribution of the SoNS. If it sends the sensor information upstream or downstream, the recipient robot will in turn make its own choices about response and propagation, based on elaborations of the compressed information. If it chooses to send the information onwards, it might first fuse it with other information from another child or its own onboard sensing, and send some compressed representation of the fused information. If robots continue to choose to send information upstream, the information will culminate at the brain. At any point along the hierarchy, a robot that chooses to respond to information it received might not only update its own behavior but might also update behavior models or actuation

instructions sent to its child(ren). If the brain chooses to respond in this way, it triggers a SoNS-wide response in the SoNS. It is important to note that SoNS-wide responses coming from the brain are not necessarily triggered by sensor information fused SoNS-wide. For instance, in the case of a very high-priority signal observed by only one robot, the signal would get sent to the brain and all other robots (at one time step per hop of the shortest path) and trigger an immediate SoNS-wide response.

4.2 Analysis metrics

To analyze the empirical results according to actuation error, we use Euclidean distance to calculate the position tracking error E at each timestep, as follows:

$$E = \frac{1}{n} \sum_{i=1}^n E_i, \quad E_i = |d(\mathbf{p}_i - \mathbf{p}_1) - d(\mathbf{f}_i - \mathbf{f}_1)|, \quad (1)$$

where n is the total number of robots, $\mathbf{p}_i$ is the current position of robot r_i , $\mathbf{f}_i$ is the target position of robot r_i , and $i = 1$ is the brain. The brain's error E_1 is always zero, because the brain's position is always the same as its target position.

The lower bound B of position tracking error E indicates the minimum error that would be present over time if the robots always moved directly to their target positions at maximum speed on the shortest Euclidean path, as if no obstacles, self-organized re-configuration, nor inter-robot collisions were present. It represents the total Euclidean distance between all robot target positions and their respective start positions when that target was set, such that

$$B = \begin{cases} \frac{1}{n} \sum_{i=1}^n B_i, & \text{if } B_i > 0 \\ 0 & \text{otherwise} \end{cases}, \quad B_i = |d(\mathbf{p}_{\epsilon i} - \mathbf{f}_{\epsilon i})| - \kappa_i(t - t_{\epsilon}), \quad (2)$$

where t is the current time, t_{ϵ} is the start time of the current target communication structure, $\mathbf{p}_{\epsilon i}$ is the position of robot r_i at t_{ϵ} , $\mathbf{f}_{\epsilon i}$ is the target position of r_i at t_{ϵ} , κ_i is a constant describing the maximum speed of r_i according to its type (ground robot or aerial robot).

4.3 Theoretical guarantees

In the experiments conducted in this paper, to verify our SoNS architecture, we consider a simple distance-based positioning approach and reactive control law to establish and maintain the target relative positions using onboard measurements. This simple approach can be understood as a performance baseline for the tracking of target positions within a SoNS. To provide theoretical analysis, we represent an n -robot SoNS as a system of $n - 1$ robot pairs, use a proportional

control law that is strictly reactive, and derive the leader–follower tracking kinematics assuming the robots cannot access any global information or external reference frame, instead using only local sensing of relative information that is available in our real setup. We provide theoretical analyses and mathematical proofs regarding the convergence of the position errors and closed-loop stability of position tracking in a SoNS with respect to the control inputs. A full description of the analysis method is provided in Sec. S3 of the supplementary materials.

4.4 Experiment setup

To be able to conduct real hardware experiments, we developed an open-source quadrotor platform with sensing and computational capabilities suited for swarm robotics experiments and an accompanying simulator model (see technical report [44], which includes links to open-source repositories, and Secs. S5, S6 in the supplementary materials). We use this quadrotor with standard e-puck ground robots [42, 43] mounted with fiducial tags (specifically AprilTags, see Sec. S7 of the supplementary materials). For the purpose of data logging and drone safety, we also built a drone arena equipped with an off-the-shelf motion capture system (see Sec. S8 in the supplementary materials) and an open-source software package for experiment management (see technical report [47] and open-source repository¹).

In the experiments presented here, the only actuation utilized is motion. The SoNS software produces kinematic control outputs for all robots regardless of type, and a second control layer is used to calculate motor inputs for the differential drive ground robots and the quadrotors. These two layers are used in both reality and simulation, as the simulator is equipped with models of the internal dynamics of the robots where needed (for details about the robot models and the control layers, see Secs. S4, S6, and S7 of the supplementary materials). The only sensors utilized in the SoNS experiments (i.e., excluding an optical flow camera and single-point LiDAR the quadrotor uses for flight stabilization) are the downward-facing visual cameras onboard the quadrotors. The ground robots do not use any onboard sensing and must rely on virtual sensor information they receive from the quadrotors through local communication over the SoNS architecture (in other words, a ground robot in a single-robot SoNS is blind). Due to the downward-facing field of view of the quadrotors’ cameras, the quadrotors cannot sense each other directly. Therefore, for relative positioning between two quadrotors, the quadrotors also must rely on information they receive through local communication over the SoNS architecture. Communication in the SoNS occurs over a wireless network, and two robots are only allowed to communicate with each other if one of them is in the other’s field of view (using onboard sensing). These sensing and communication constraints are maintained in the simulator, matching reality.

¹<https://github.com/iridia-ulb/supervisor>

The indoor arena for the real experiments is tightly constrained, so we have supplemented the real demonstrations with simulated experiments conducted in the multi-robot simulator AR-GoS [48, 49, 50]. We have conducted empirical cross-verification of the simulator and the real setup, to ensure that the results of the simulated experiments are a reliable approximation of the behavior of SoNS on the real robots (see Sec. S2 of the supplementary materials).

Acknowledgments

W.Z., S.O., and M.K.H. contributed equally to this work and share co-first authorship. All authors made substantial contributions to the conception of the work. The experiment design was led by W.Z., M.K.H., and M.D., and contributed to by all authors. The experiments were conducted and the data was collected by W.Z. and S.O., supervised by M.K.H. and M.D. The main algorithms used in the study were developed by W.Z. and supervised by M.K.H., M.A., and M.D., with contributions from S.O. The hardware, software, and infrastructure used to support the experiments were developed by W.Z., S.O., M.K.H., M.A. and M.W., led by M.A. The analysis of experimental results was conducted by W.Z. and M.K.H., supervised by M.D. The theoretical analysis was developed by S.O., supervised by M.K.H., E.G., and M.D. The presentation of the results, including figures and movies, was executed by W.Z., S.O., M.K.H., and M.W., supervised by M.D. The supplementary materials were prepared by W.Z., S.O., and M.K.H., supervised by M.D., with contributions from all authors. The writing of the manuscript was led by M.K.H., supervised by M.D., with contributions from all authors. The original idea was provided by M.D. and A.L.C. All authors read and approved the content of this manuscript.

Funding

This work was partially supported by the Program of Concerted Research Actions (ARC) of the Université libre de Bruxelles, by the Belgian F.R.S.-FNRS under Grant J.0064.20, by the Office of Naval Research Global (Award N62909-19-1-2024), by the European Union’s Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement No 846009, by the Independent Research Fund Denmark under grant 0136-00251B, and by the China Scholarship Council Award No 201706270186. Mary Katherine Heinrich and Marco Dorigo acknowledge support from the Belgian F.R.S.-FNRS, of which they are a Postdoctoral Researcher and a Research Director respectively.

Figures and Tables

Fig.1. The Self-organizing Nervous System (SoNS) concept: robots self-organize dynamic multi-level system architectures using exclusively local communication.

Fig.2. Establishing self-organized hierarchy.

Fig.3. Balancing global and local goals.

Fig.4. Collective sensing and actuation.

Fig.5. Binary decision making.

Fig.6. Splitting and merging systems.

Fig.7. Scalability study.

Fig.8. Fault tolerance study.

Supplementary Materials

Section S1. Full set of experiment results

Section S2. Cross-verification of the SoNS in simulation and on real hardware

Section S3. Theoretical analysis and mathematical proofs

Section S4. SoNS control algorithm details

Section S5. Simulator setup

Section S6. Aerial robot setup

Section S7. Ground robot setup

Section S8. Real indoor arena setup

Figures S1-S62 are included in Secs. S1-S8.

Movies S1-S12 each show an example trial, except for S8, which shows a demonstration.

Movie S1. Establishing self-organized hierarchy with real robots

Movie S2. Balancing global and local goals with real robots

Movie S3. Collective sensing and actuation with real robots

Movie S4. Binary decision making with real robots

Movie S5. Splitting and merging systems with real robots

Movie S6. Scalability in the binary decision-making mission, 125 robots in simulation

Movie S7. Scalability in the establishing self-organized hierarchy mission, several example system sizes in simulation

Movie S8. Fault tolerance demonstration showing interchangeability of a failed brain robot

Movie S9. Fault tolerance under multiple permanent failures, with real robots

Movie S10. Fault tolerance under high-loss conditions in simulation, 66.6% probability to fail

Movie S11. Fault tolerance under 30 s system-wide vision failure in simulation

Movie S12. Fault tolerance under 30 s system-wide communication failure in simulation

Supplementary Materials for SELF-ORGANIZING NERVOUS SYSTEMS FOR ROBOT SWARMS

Section S1. Full set of experiment results

In this section, for each of the five robot missions (see Sec. 2.1 in the main paper), we provide key frames of a simulation to illustrate the general setup. We also provide the results of all experiment trials with real robots as well as several example trials for each type of simulation experiment with a larger system size.

For each trial included in this section, we provide the following results: (on the left) trajectories of the robots over time, with the initial, final, and sometimes intermediary SoNS indicated in black; and (on the right) the mean and confidence interval per robot of the actuation error E over time (see Eq. 1 in Sec. 4.2 in the main paper), with the lower bound B (see Eq. 2 in Sec. 4.2 in the main paper) plotted for reference.

Note that, for each type of experiment (see the five missions, Sec. 2.1, scalability setups, Sec. 2.2, and the fault tolerance setups, Sec. 2.3, in the main paper), a video of an example trial is included within the supplementary materials (see Movies S1-S12). The code for each type of experiment is available in the online code repository. The experiment data for all trials, videos of all trials with real robots, and videos of example simulation trials are all available in the online data repository.

(Section continued on next page.)

Mission: Establishing self-organized hierarchy (see Sec. 2.1.1 in the main paper)

This mission includes two different variants, both run in experiments with real robots and in simulation.

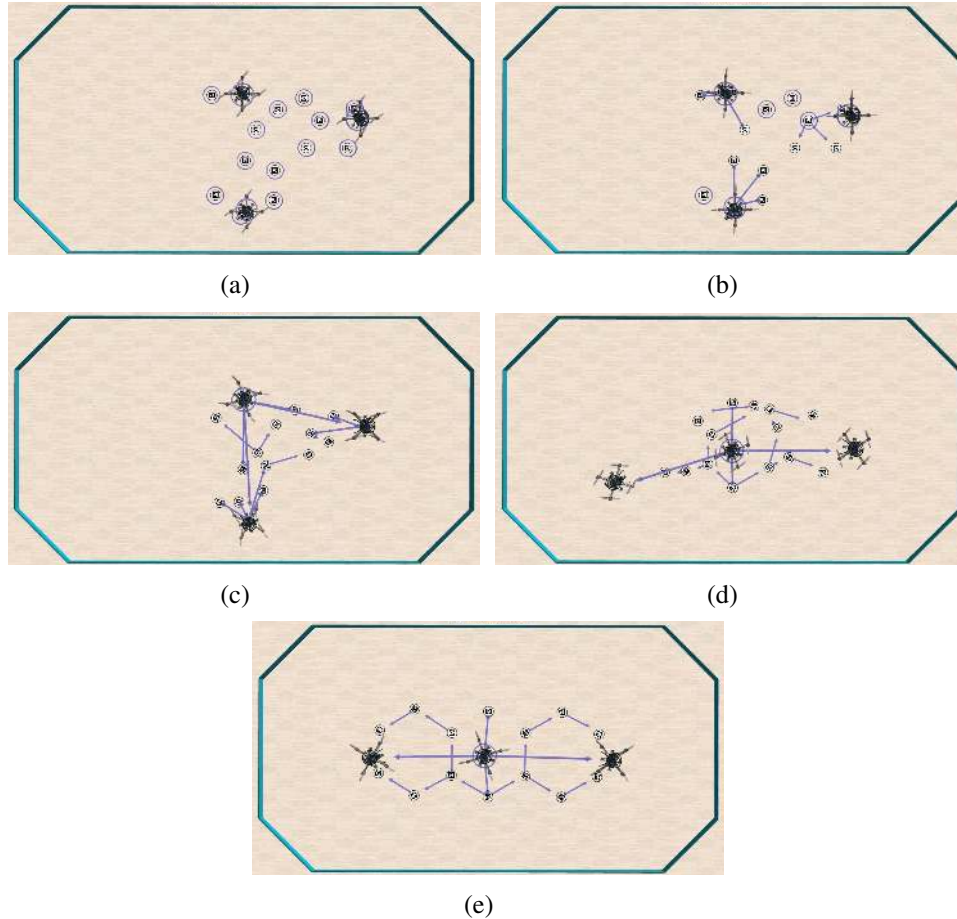


Fig. S1: Establishing self-organized hierarchy: Key frames. (a) Robots start in scattered or clustered positions on the ground and the aerial robots take off. (b) All robots begin searching for peers and some robots start forming connections. (c,d) Robots merge their respective SoNSs and reallocate themselves into positions that match the target SoNS, continually adjusting their relative positions while coordinating locally to avoid collisions. (e) The target SoNS is complete.

(Section continued on next page.)

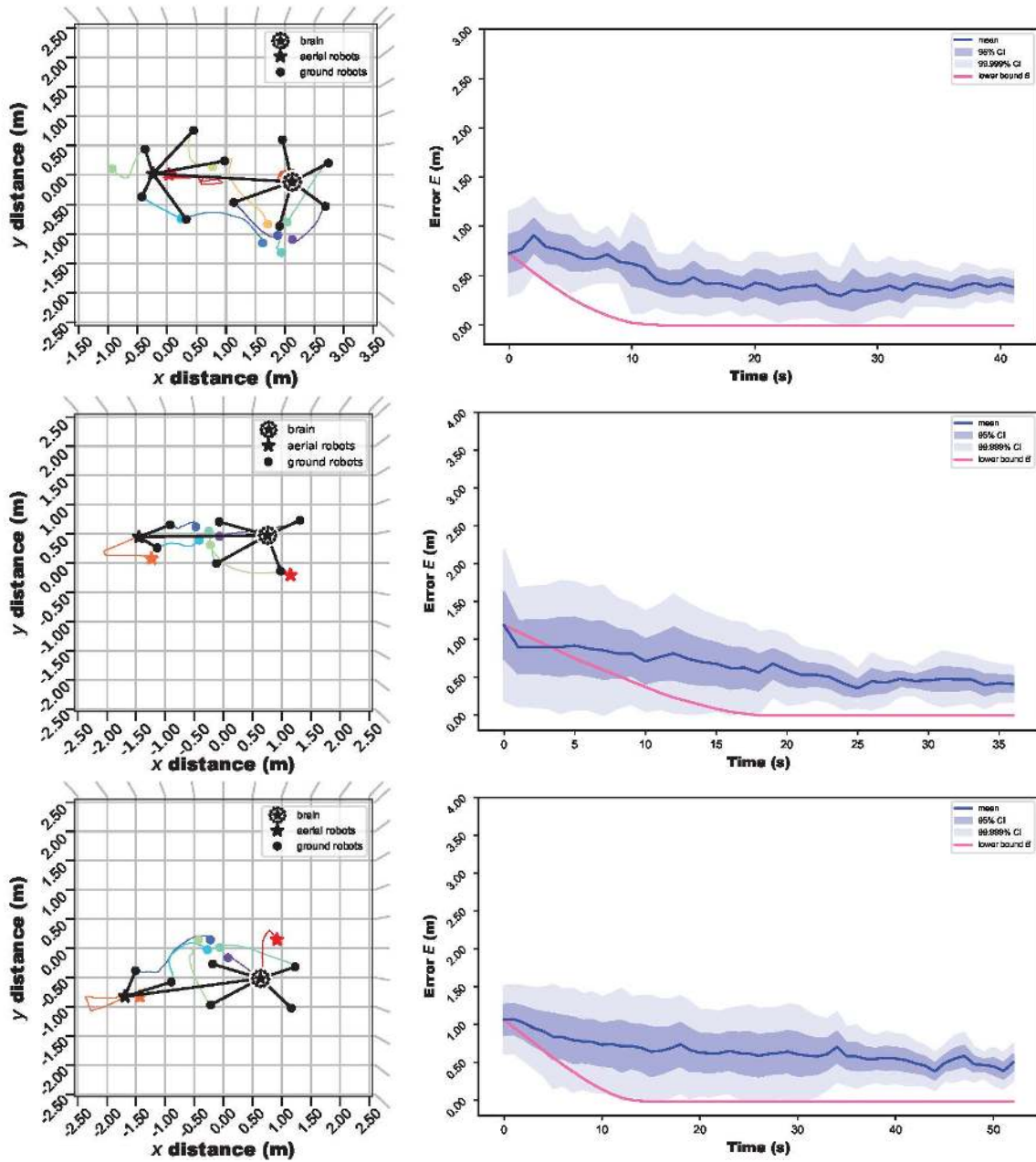
Variant: Clustered start

Fig. S2: Establishing self-organized hierarchy, clustered start: Real robot trials. Note that, 12 robots were used in the first trial shown, but this pushed the safety limits of the indoor arena for this experiment type, so eight robots were used in the remaining five trials. In total, six trials with real robots were conducted (*figure continued on next page*).

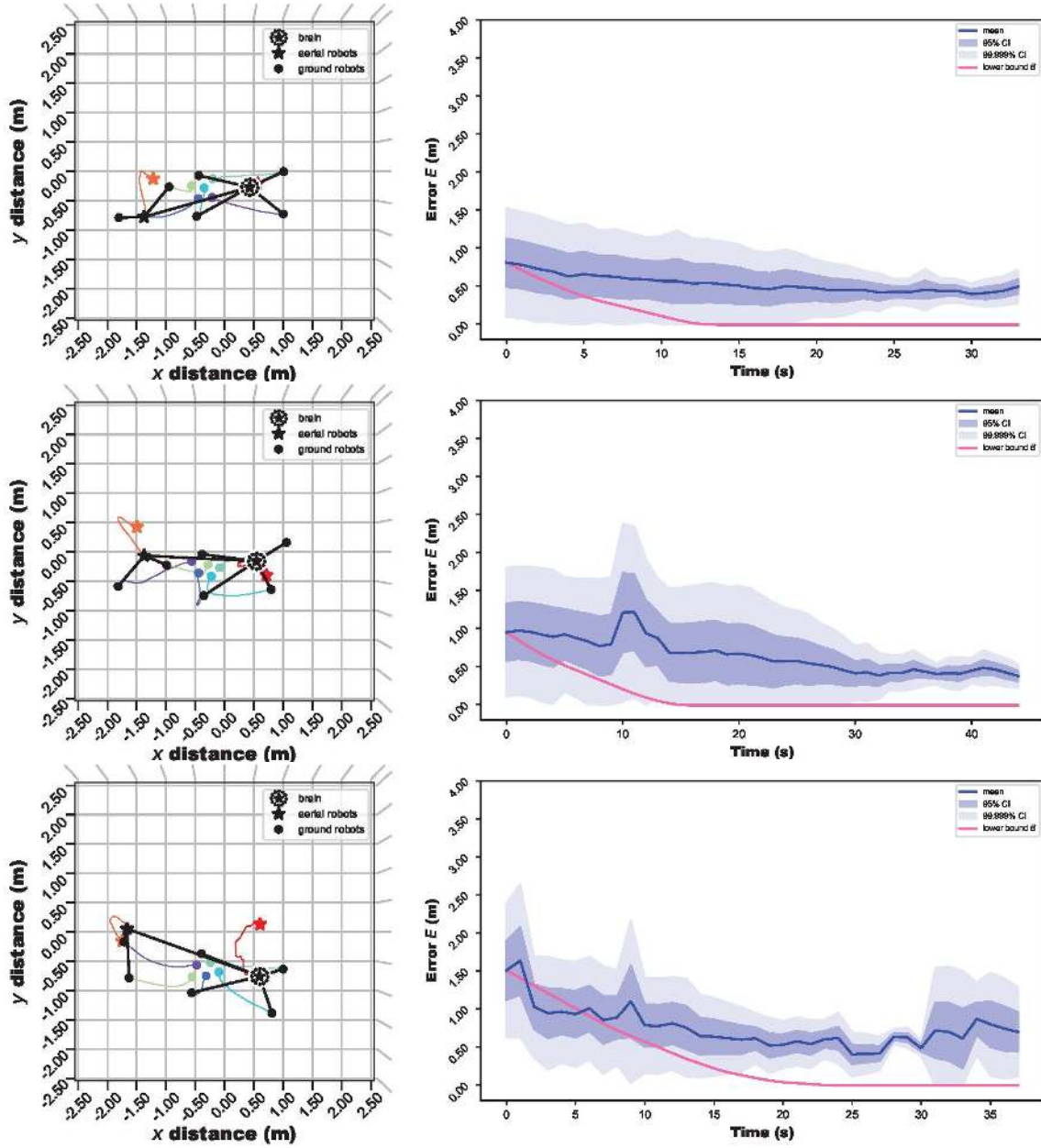


Fig. S2: (cont'd) Establishing self-organized hierarchy, clustered start: Real robot trials. Note that, 12 robots were used in the first trial shown, but this pushed the safety limits of the indoor arena for this experiment type, so eight robots were used in the remaining five trials. In total, six trials with real robots were conducted.

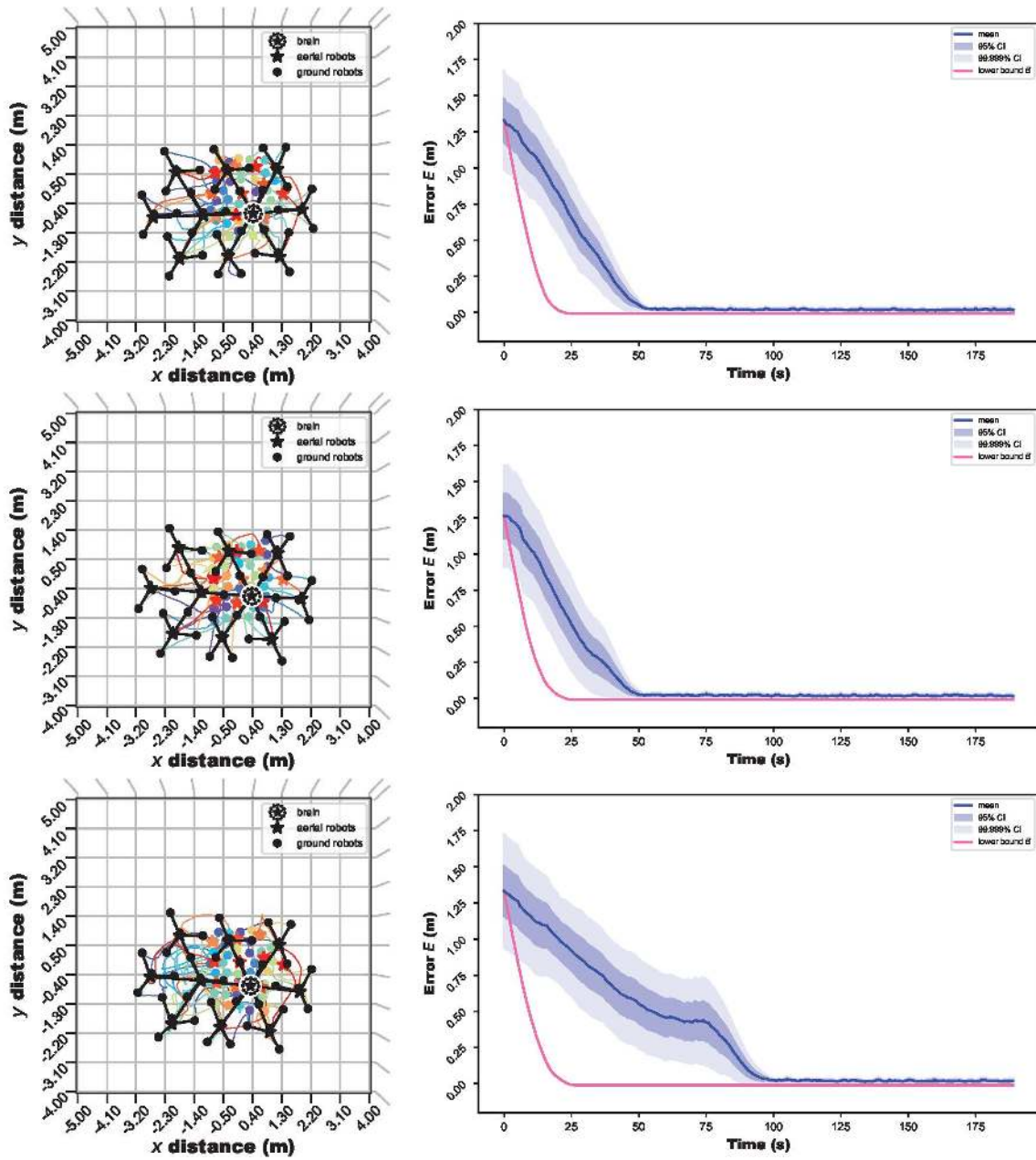
Variant: Clustered start

Fig. S3: Establishing self-organized hierarchy, clustered start: Example simulation trials. 50 trials were conducted in simulation, each with 50 robots.

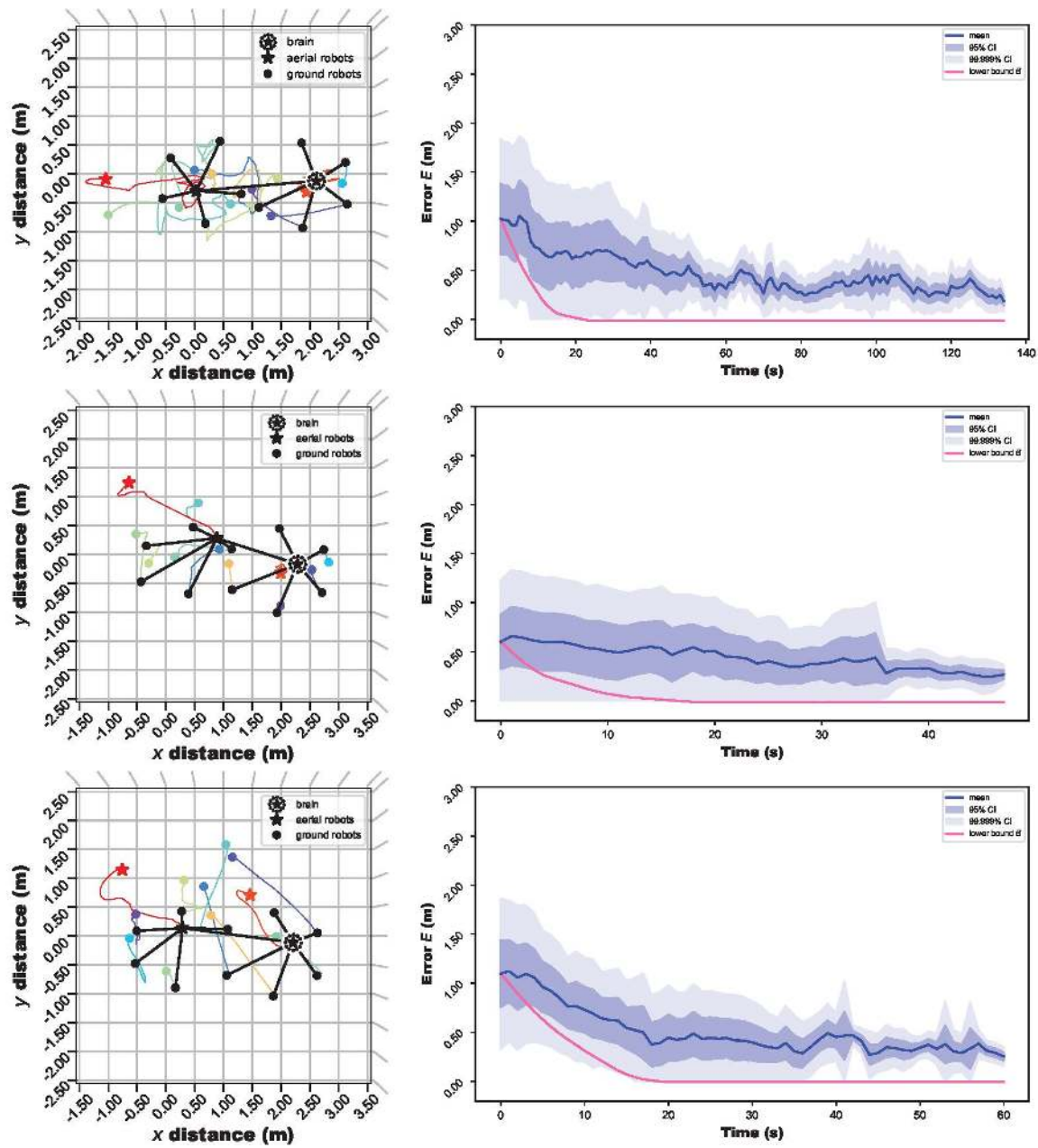
Variant: Scattered start

Fig. S4: Establishing self-organized hierarchy, scattered start: Real robot trials. Five trials with real robots were conducted, each with 12 robots (*figure continued on next page*).

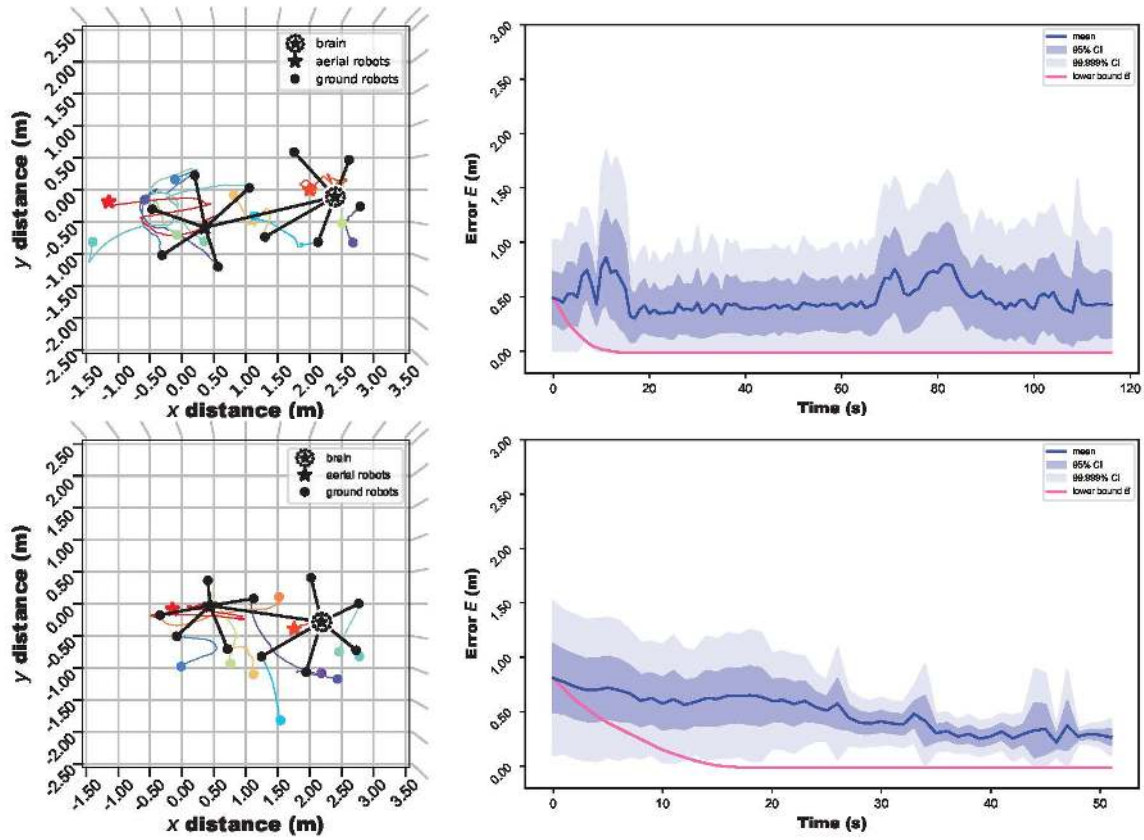


Fig. S4: (*cont'd*) Establishing self-organized hierarchy, scattered start: **Real robot trials.** Five trials with real robots were conducted, each with 12 robots.

(Section continued on next page.)

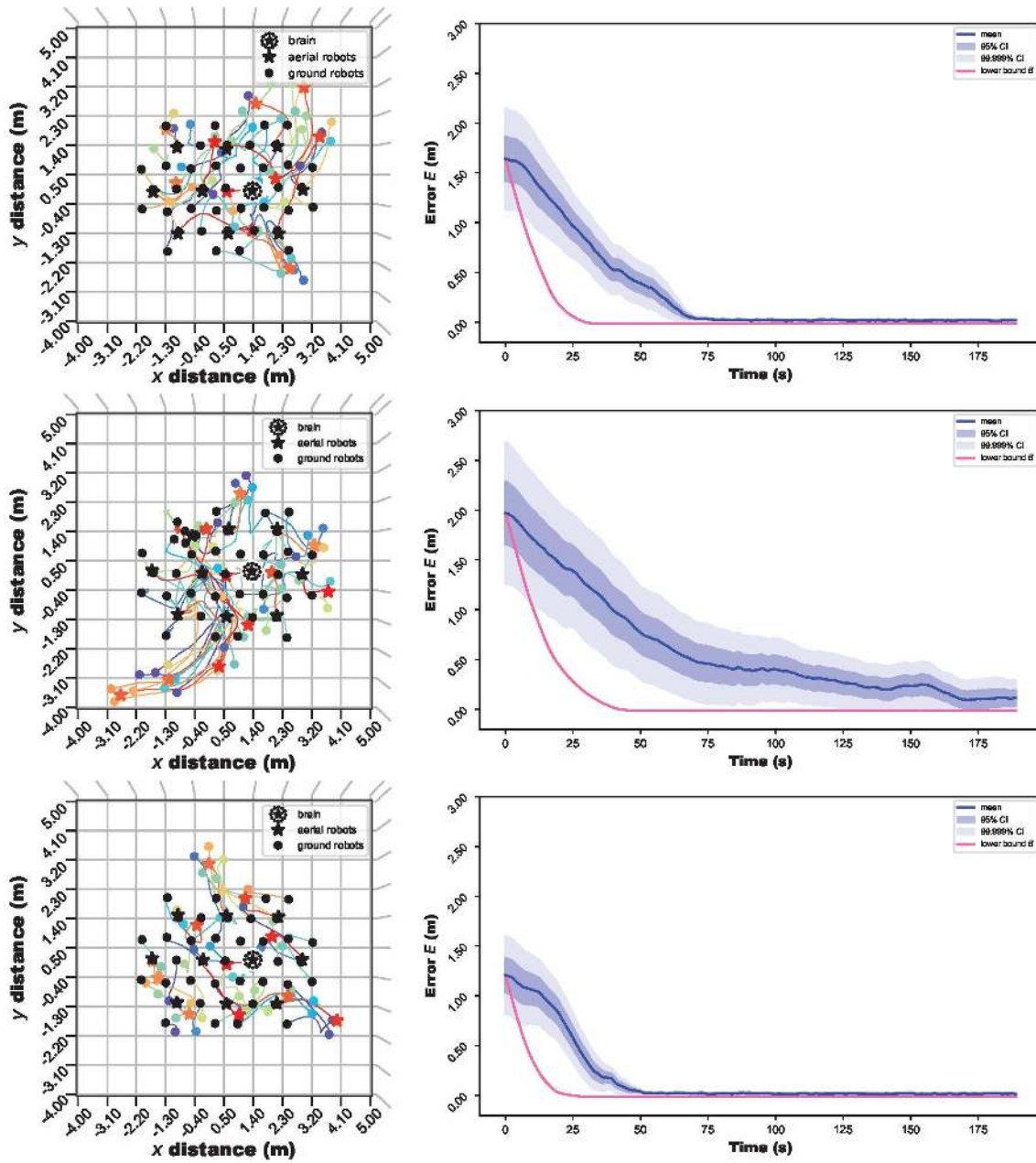
Variant: Scattered start

Fig. S5: Establishing self-organized hierarchy, scattered start: Example simulation trials. 50 trials were conducted in simulation, each with 50 robots.

Mission: Balancing global and local goals (see Sec. 2.1.2 in the main paper)

This mission includes two different variants, both run in experiments with real robots and in simulation.

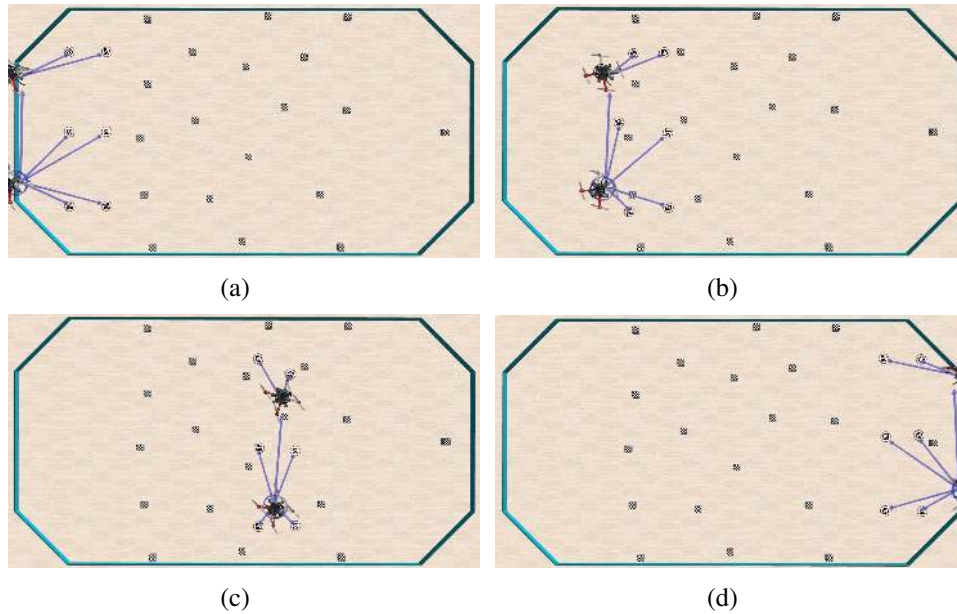


Fig. S6: Balancing global and local goals: Key frames. (a,b) Robots begin as members of a single SoNS and begin moving across an environment with an unknown field of small, dense obstacles, searching for an object that marks the final destination. (c) As the robots move through the obstacle field, they collaboratively balance global and local goals at each bidirectional link, to avoid obstacles while still keeping the SoNS together. (d) The SoNS surpasses the obstacle field and senses the final destination object, and the mission is complete.

(Section continued on next page.)

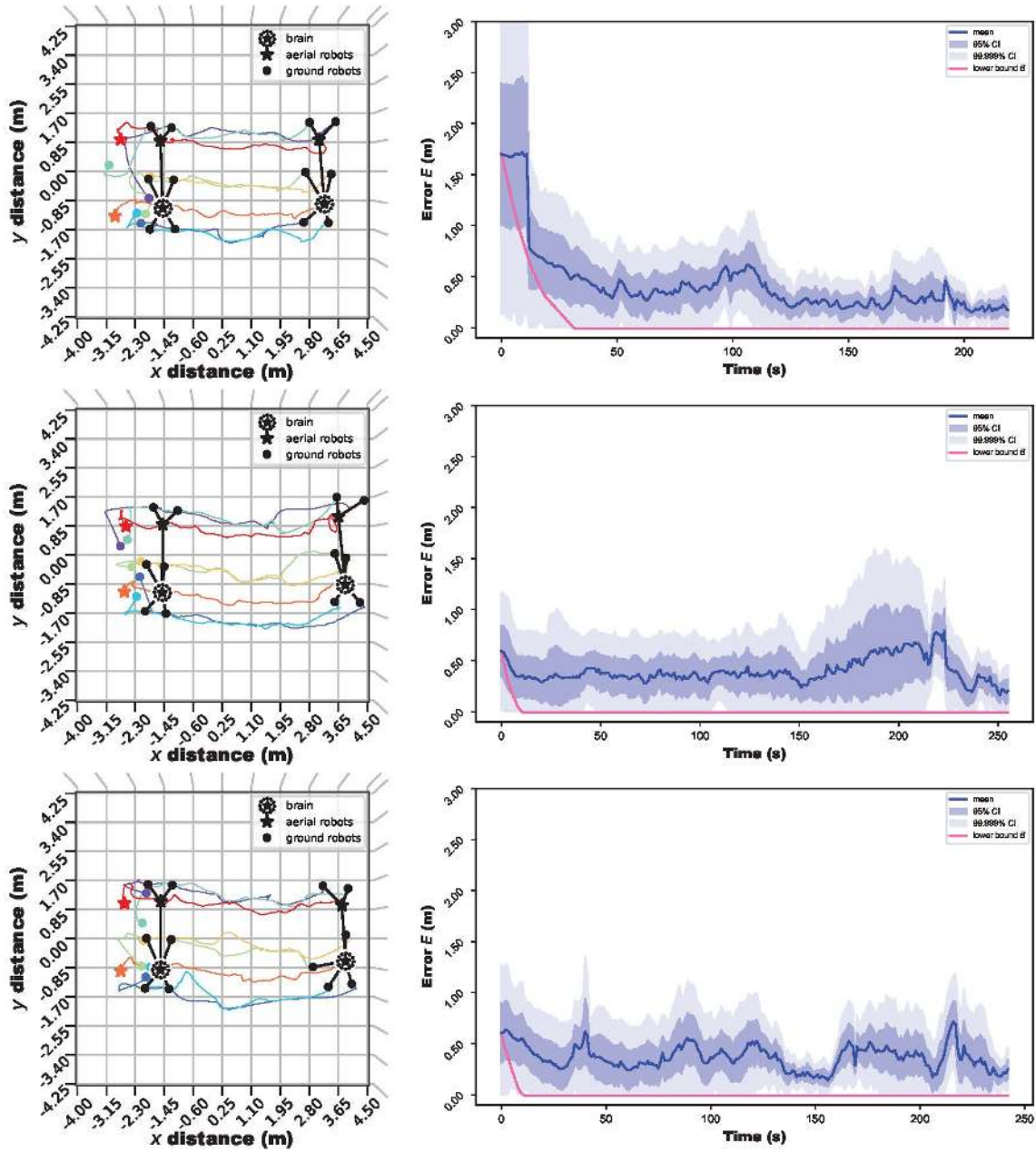
Variant: Smaller, denser obstacles

Fig. S7: Balancing global and local goals, with smaller, denser obstacles: Real robot trials. Five trials with real robots were conducted, each with eight robots (*figure continued on next page*).

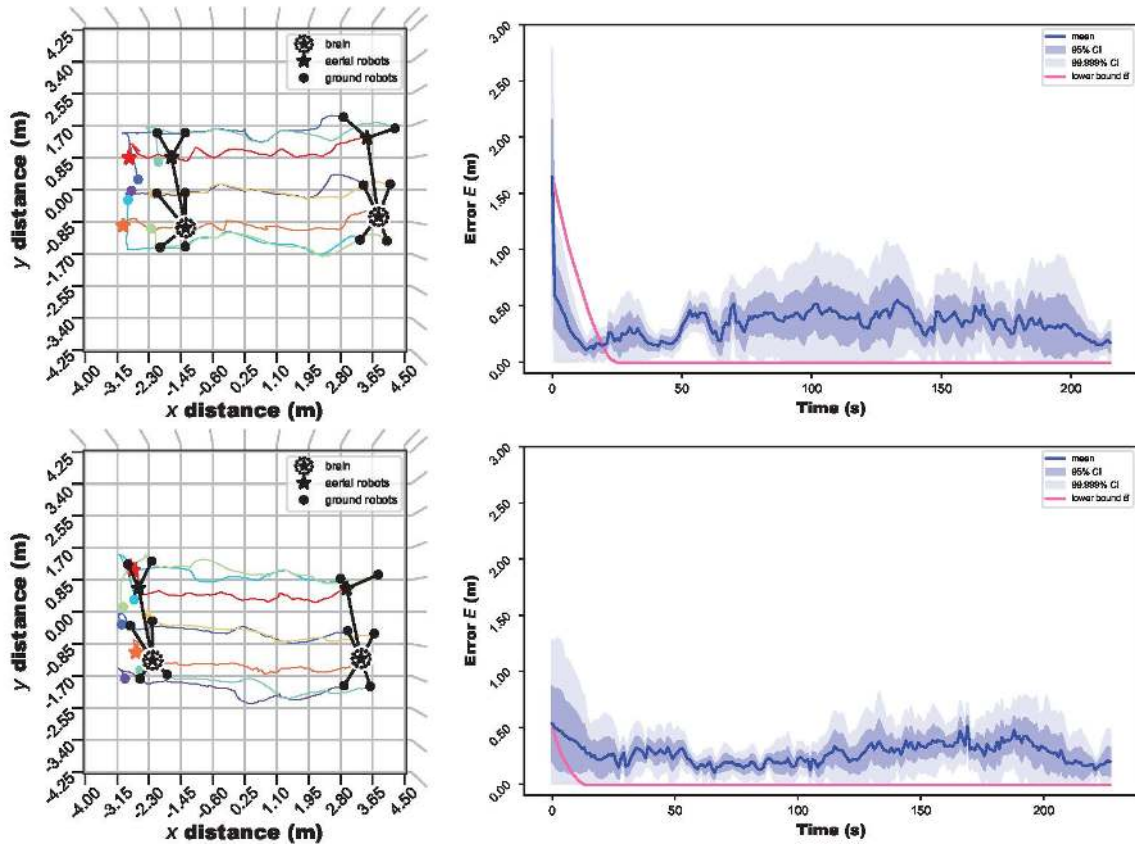


Fig. S7: (cont'd) Balancing global and local goals, with smaller, denser obstacles: Real robot trials. Five trials with real robots were conducted, each with eight robots.

(Section continued on next page.)

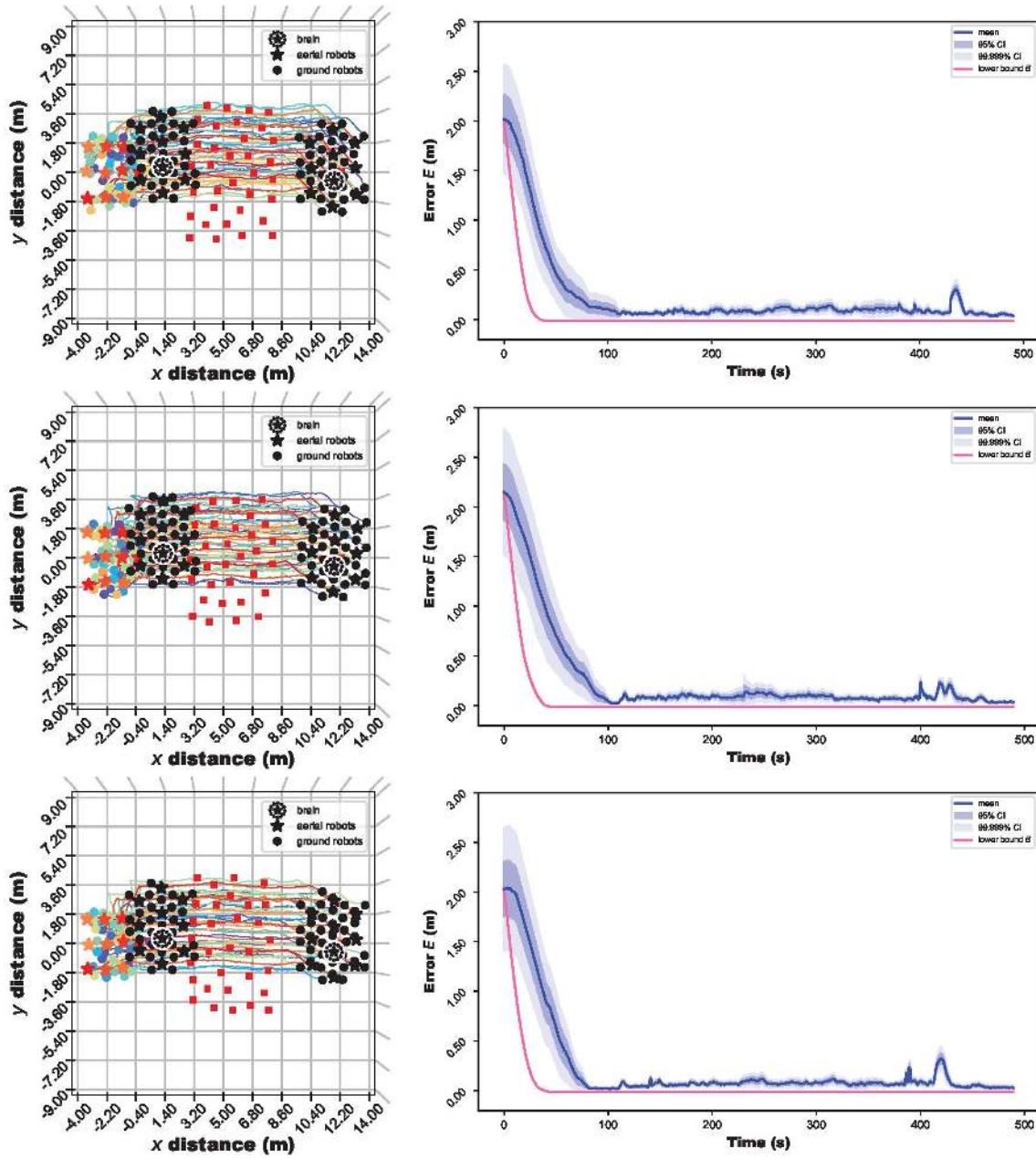
Variant: Smaller, denser obstacles

Fig. S8: Balancing global and local goals, with smaller, denser obstacles: Example simulation trials. 50 trials were conducted in simulation, each with 50 robots.

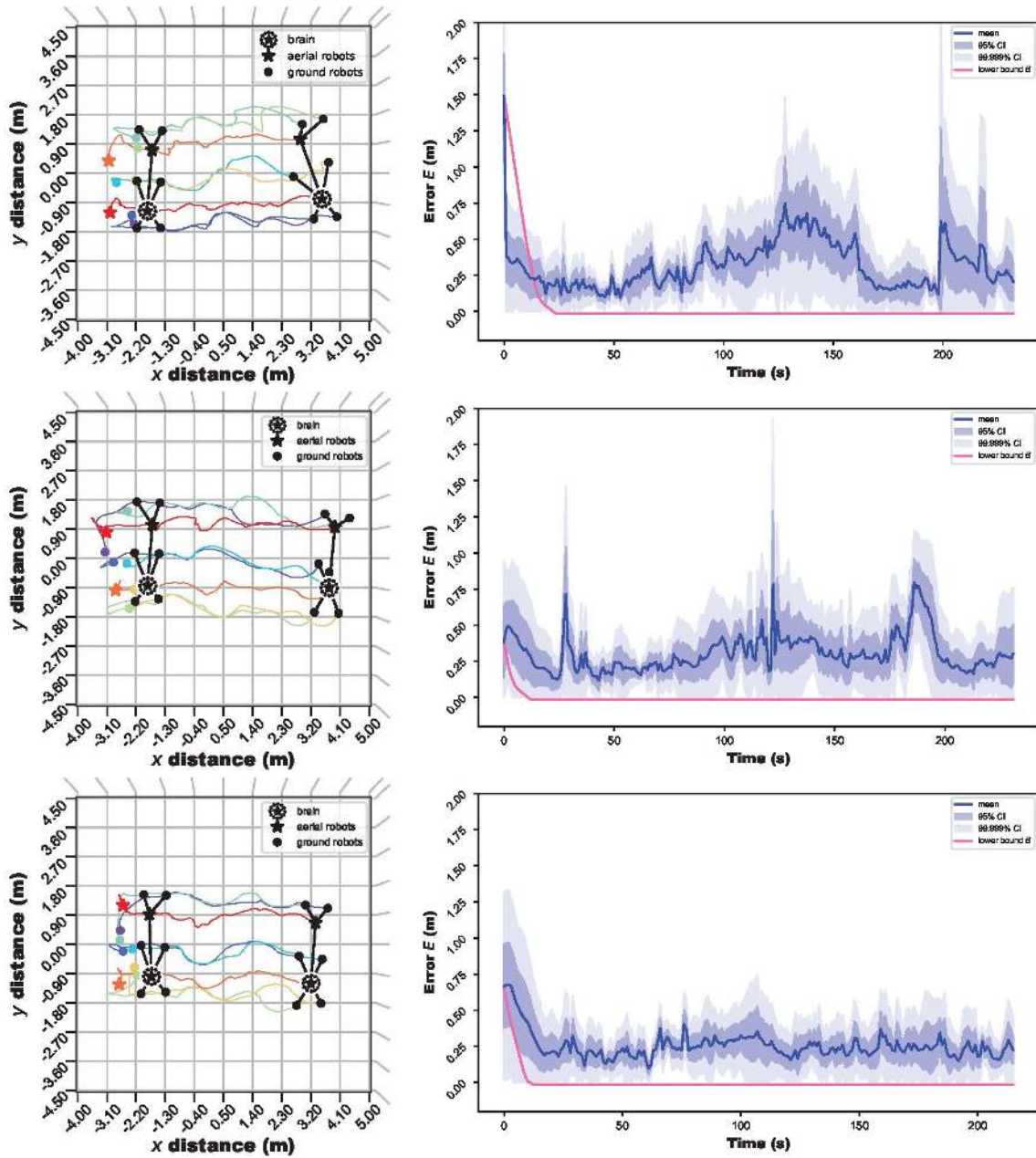
Variant: Larger, less dense obstacles

Fig. S9: Balancing global and local goals, with larger, less dense obstacles: Real robot trials. Five trials with real robots were conducted, each with eight robots (*figure continued on next page*).

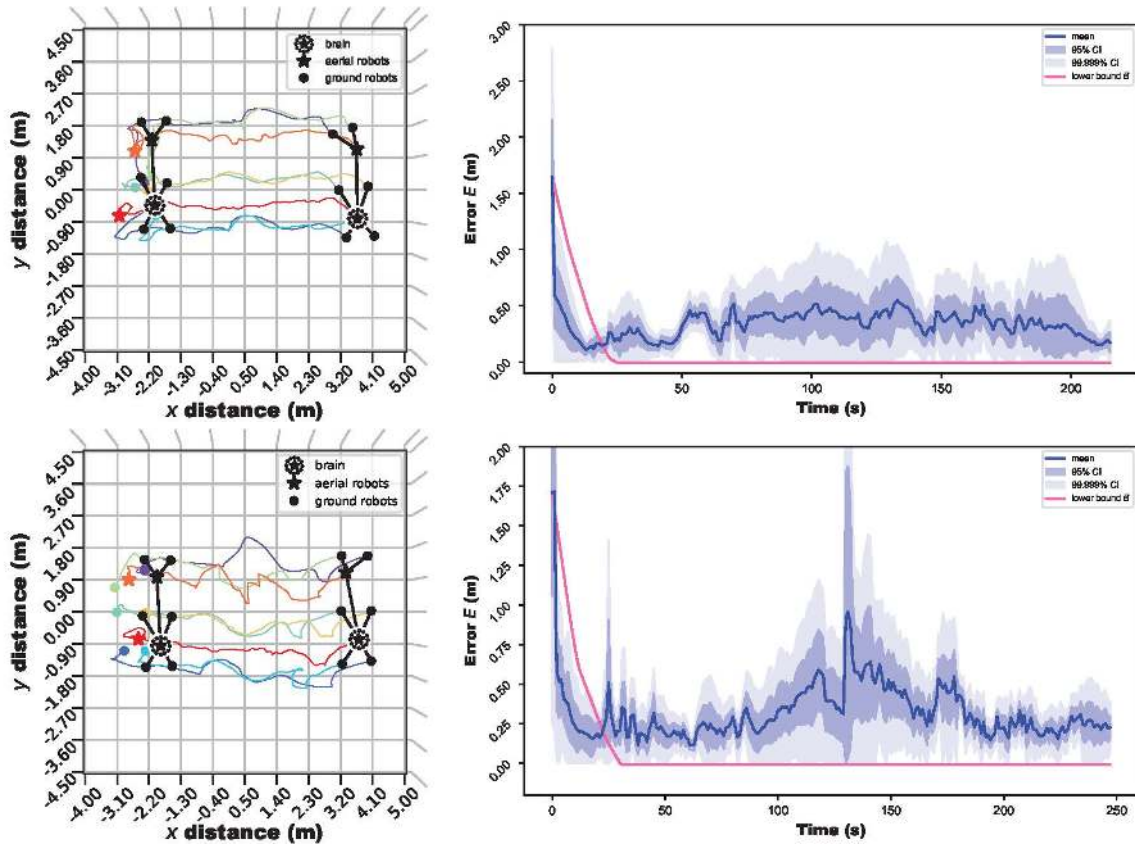


Fig. S9: (cont'd) Balancing global and local goals, with larger, less dense obstacles: Real robot trials. Five trials with real robots were conducted, each with eight robots.

(Section continued on next page.)

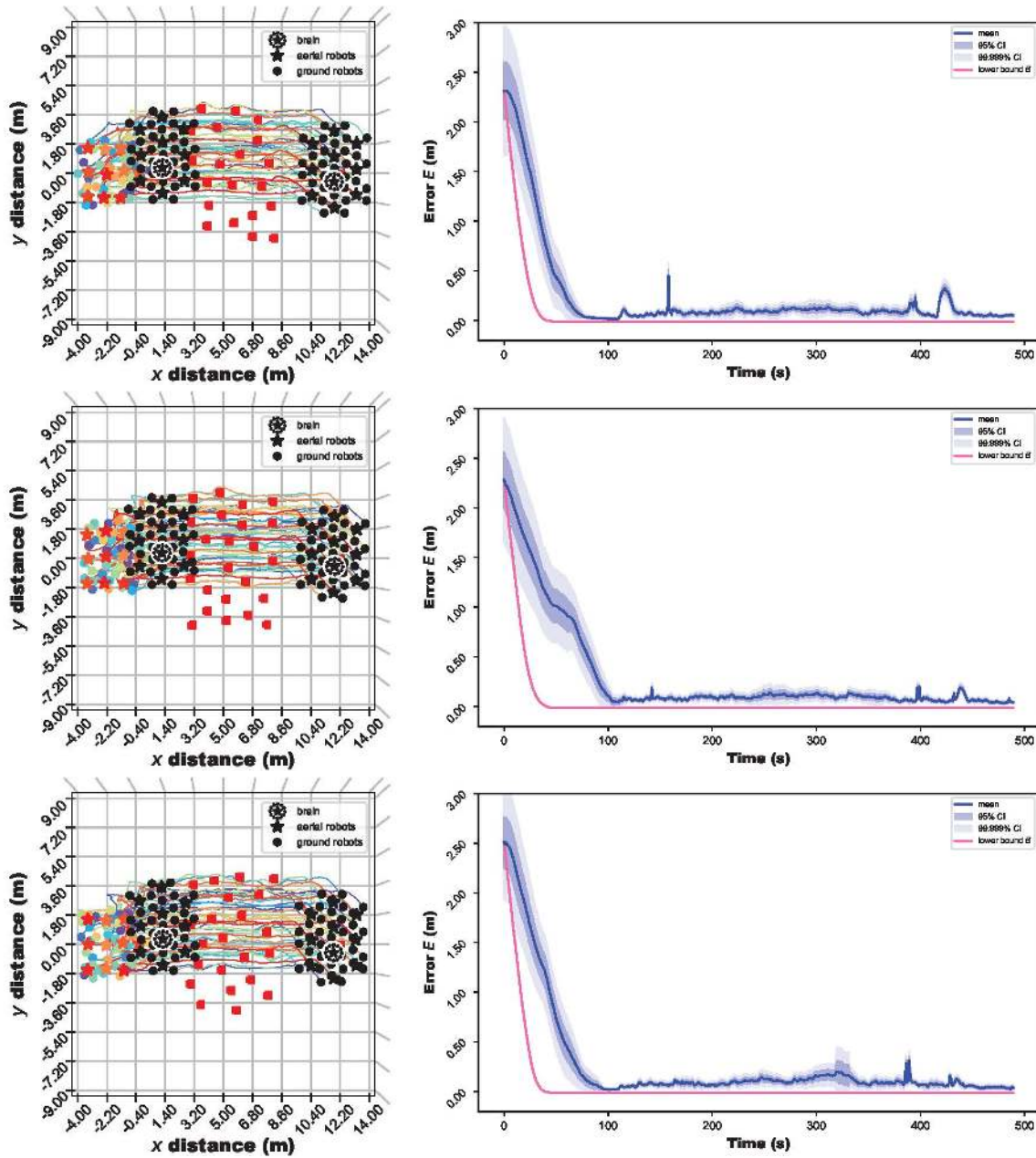
Variant: Larger, less dense obstacles

Fig. S10: Balancing global and local goals, with larger, less dense obstacles: Example simulation trials. 50 trials were conducted in simulation, each with 50 robots.

Mission: Collective sensing and actuation (see Sec. 2.1.3 in the main paper)

This mission includes only one variant, run in experiments with real robots and in simulation.

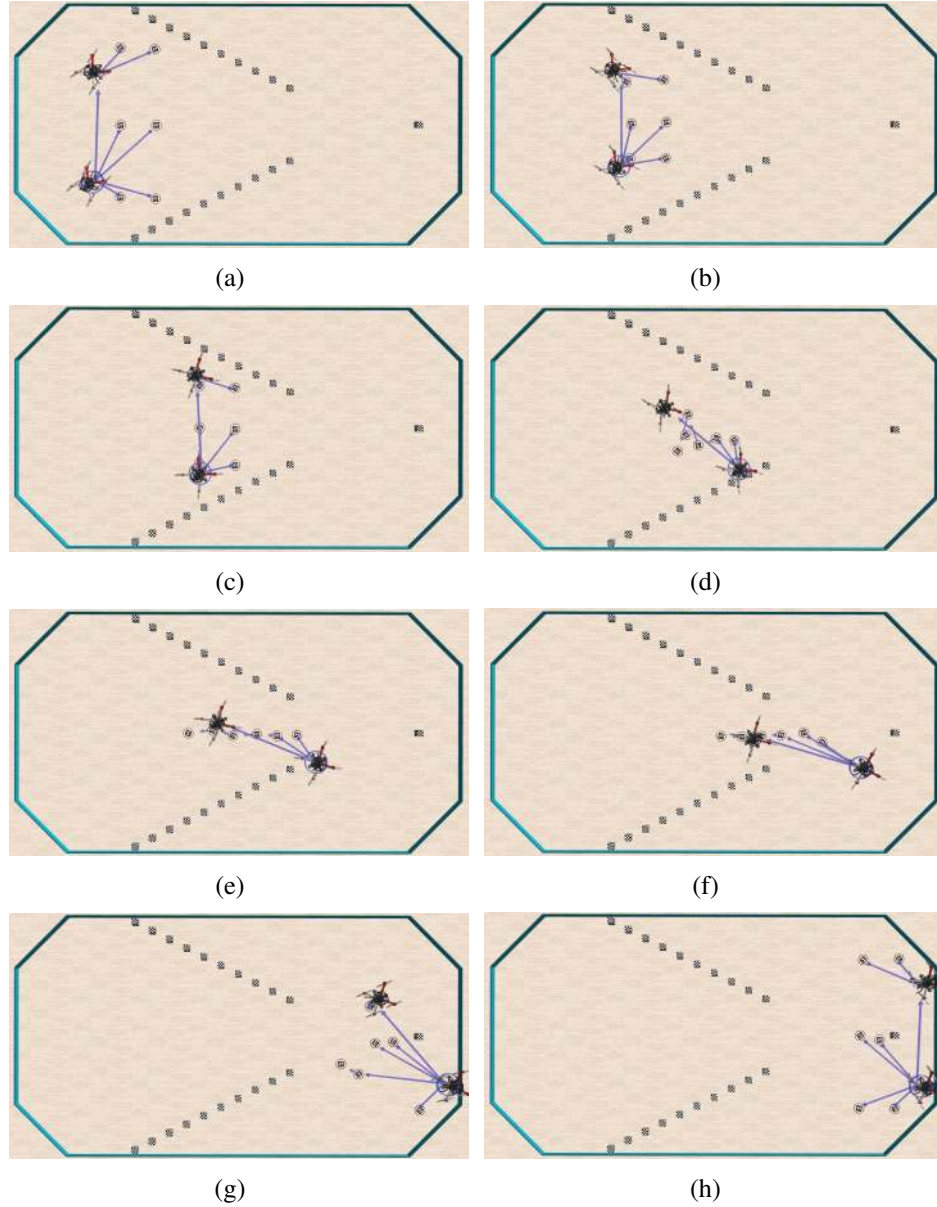


Fig. S11: Collective sensing and actuation: Key frames. (a) Robots that have established their target SoNS begin moving across an environment, searching for an object that marks the final destination. (b,c) Robots sense obstacles that form the walls of a passage and start to adapt by narrowing the shape of the target SoNS. (d,e) Robots sense the walls narrowing and then adapt further by reconfiguring into a different target SoNS that has an even narrower shape. (f,g) Robots sense that there are no longer walls constraining them and start to return to their original target SoNS. (h) Robots sense the final destination object and return to their original target SoNS, and the mission is complete.

Mission: Collective sensing and actuation

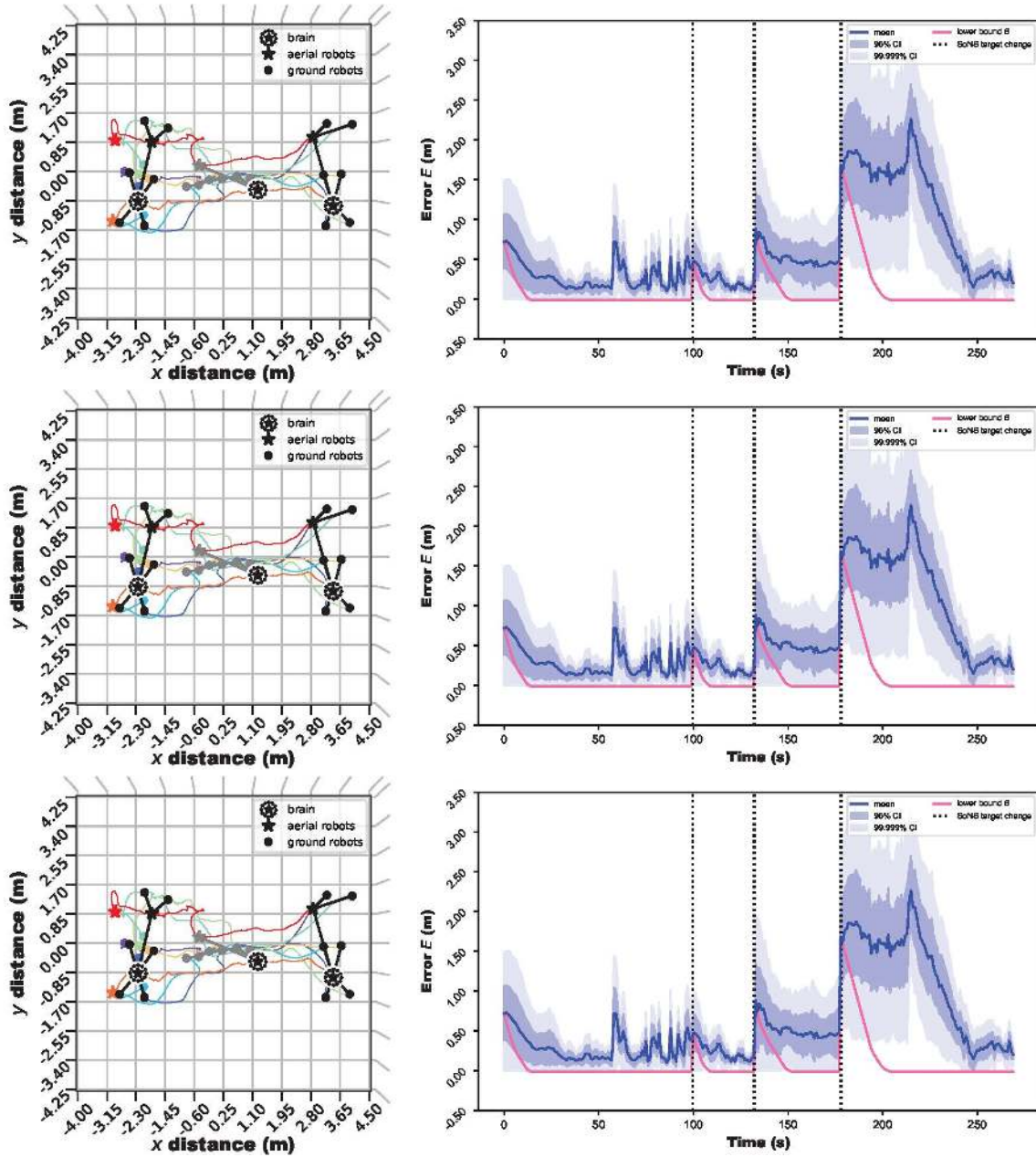


Fig. S12: Collective sensing and actuation: Real robot trials. Five trials with real robots were conducted, each with eight robots (*figure continued on next page*).

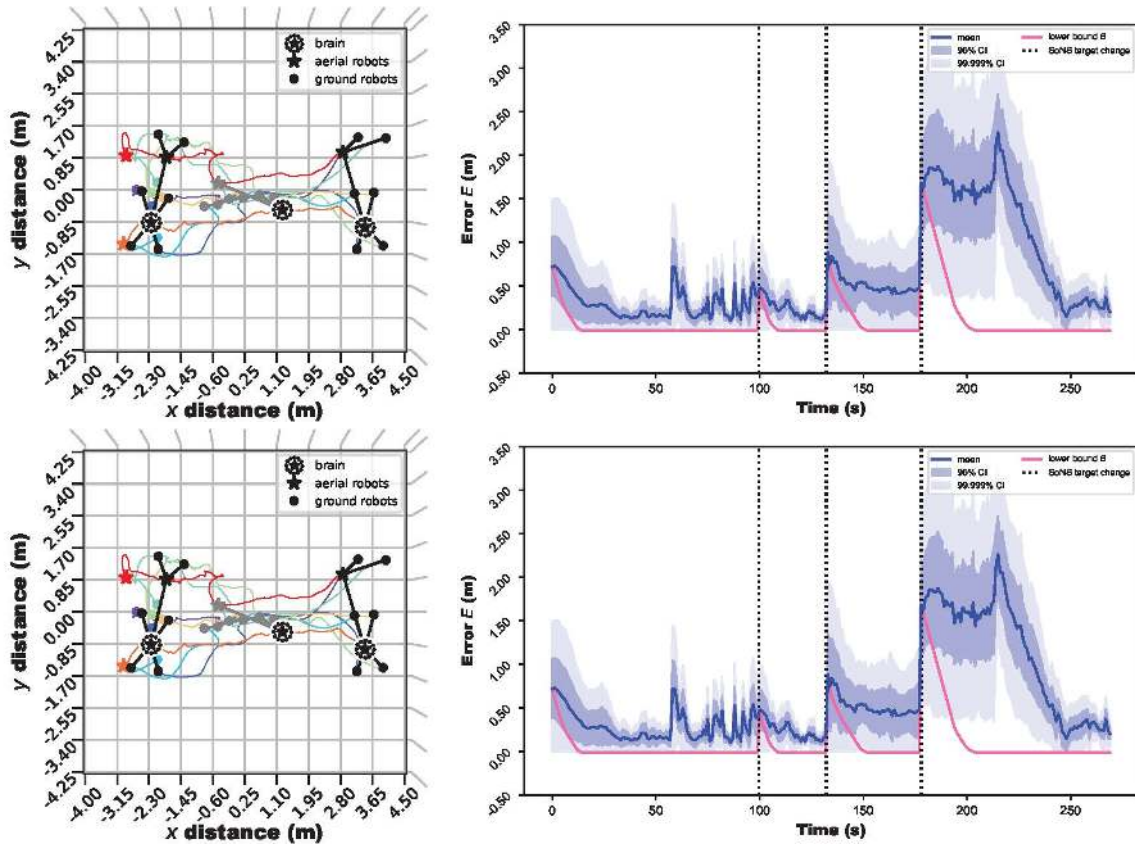


Fig. S12: (cont'd) Collective sensing and actuation: Real robot trials. Five trials with real robots were conducted, each with eight robots.

(Section continued on next page.)

Mission: Collective sensing and actuation

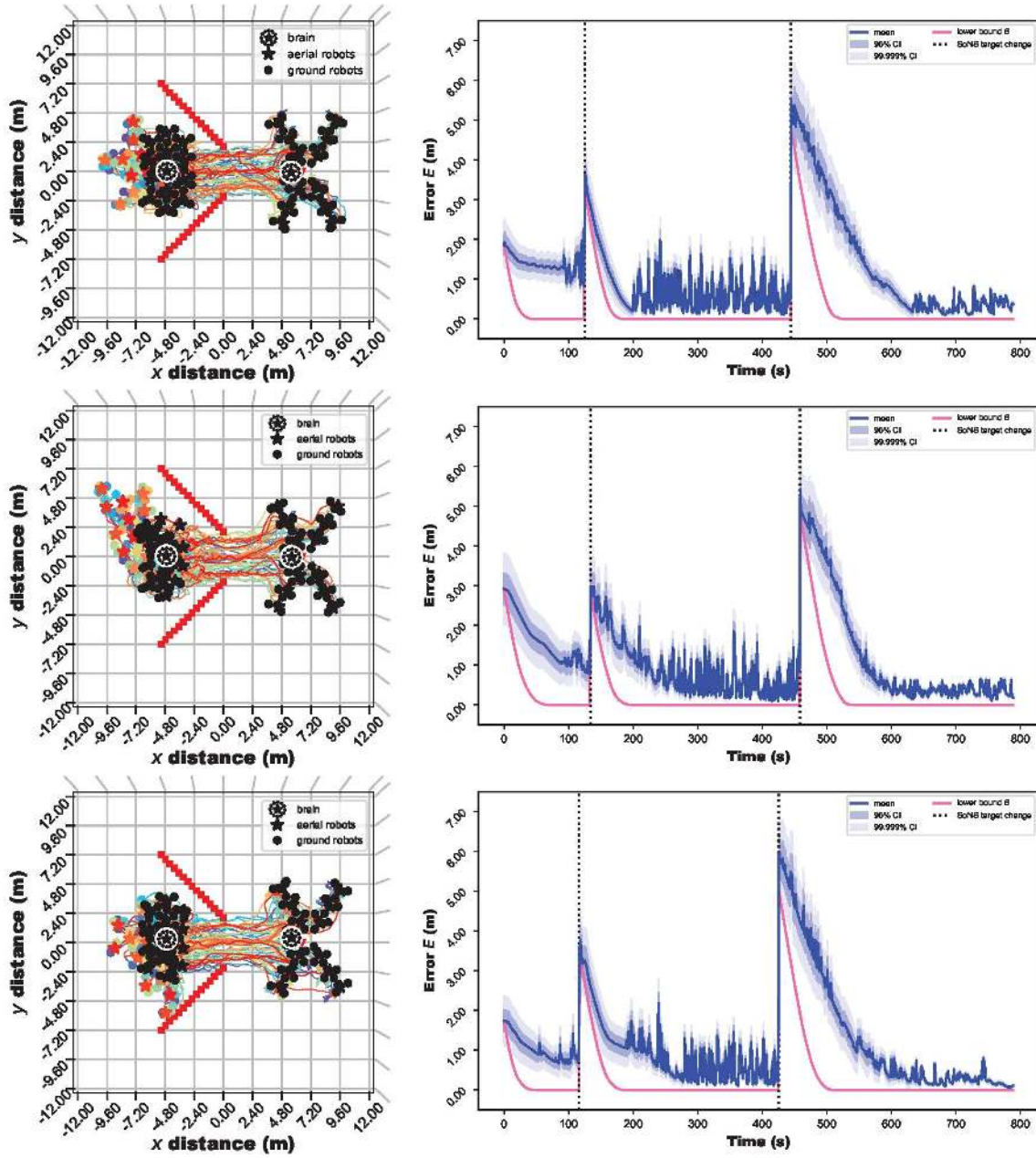


Fig. S13: Collective sensing and actuation: Example simulation trials. 50 trials were conducted in simulation, each with 50 robots.

Mission: Binary decision making (see Sec. 2.1.4 in the main paper)

This mission includes only one variant, run in experiments with real robots and in simulation. Note that simulation trials for this mission are provided in the “Scalability” subsection, below.

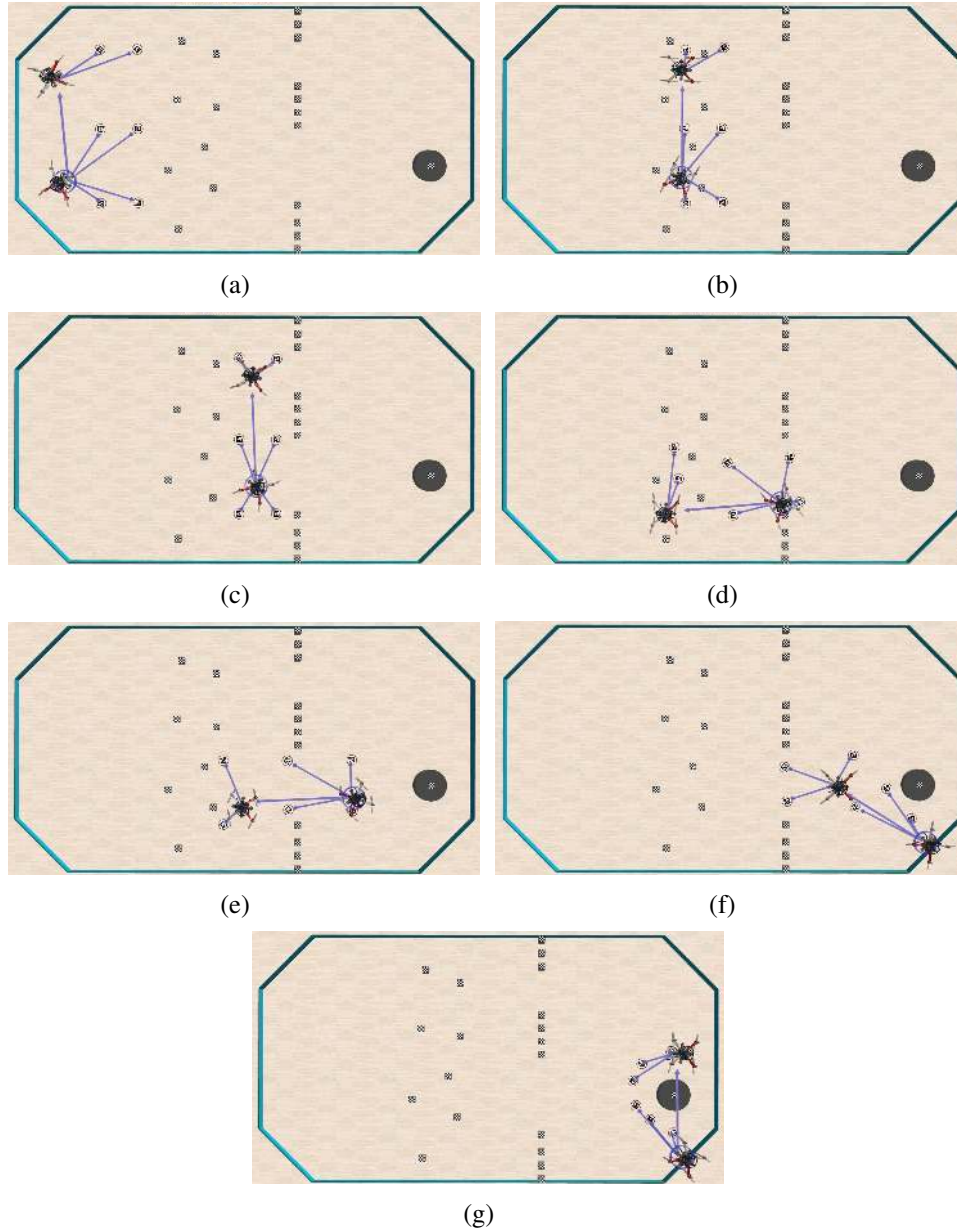


Fig. S14: Binary decision making: Key frames. (a,b) Robots that have established their target SoNS begin moving across an environment, searching for the final destination object, and encounter a field of small obstacles. (c) After surpassing the obstacle field, robots sense a wall and collaboratively choose the largest opening. (d,e) The robots adjust the position and path of the SoNS to reposition in front of and then pass through the selected opening. (f,g) Robots sense that there are no longer walls constraining them, then sense the final destination object and change their target SoNS to surround it, and then the mission is complete.

Mission: Binary decision making

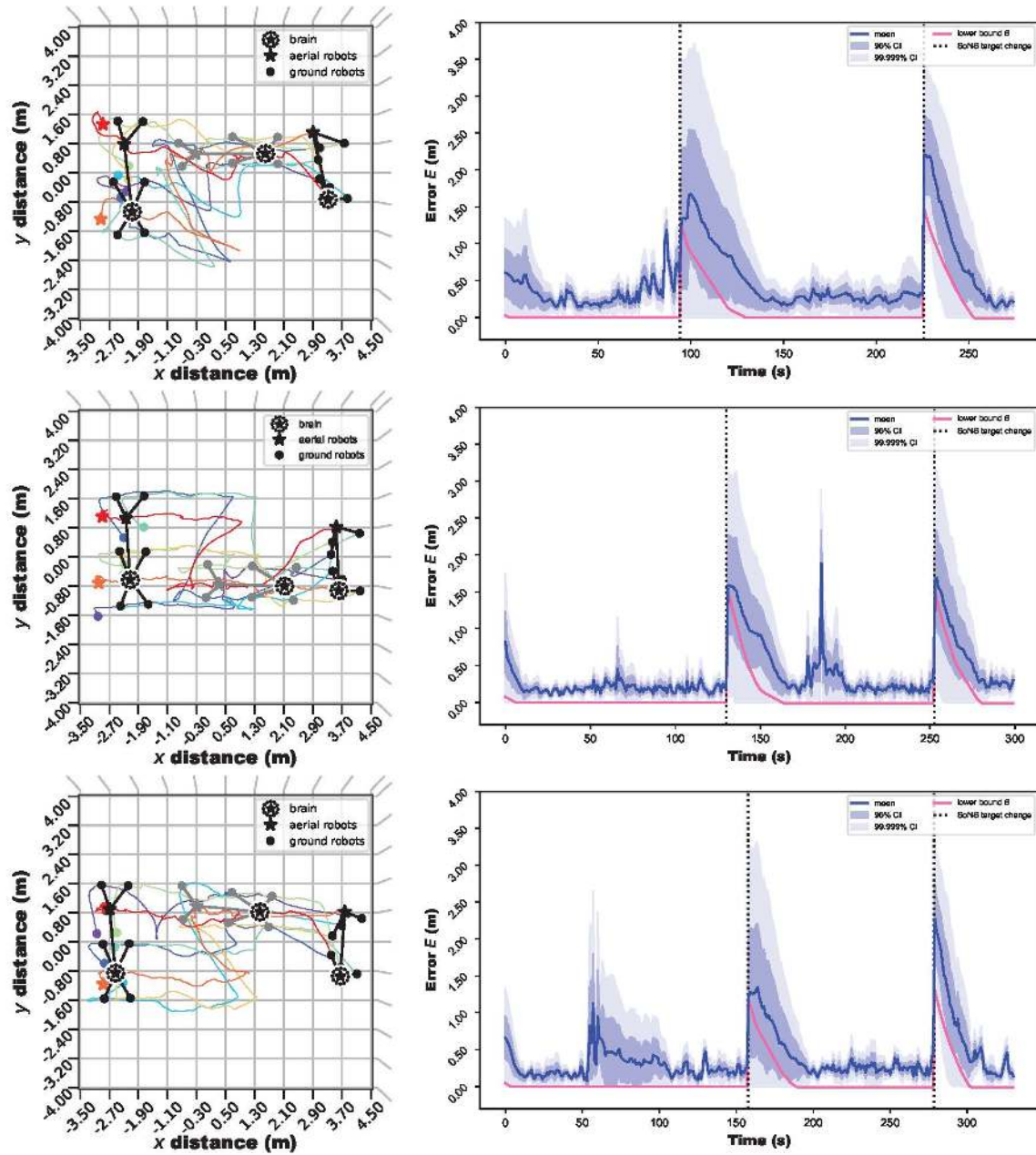


Fig. S15: Binary decision making: Real robot trials. Five trials with real robots were conducted, each with eight robots (*figure continued on next page*).

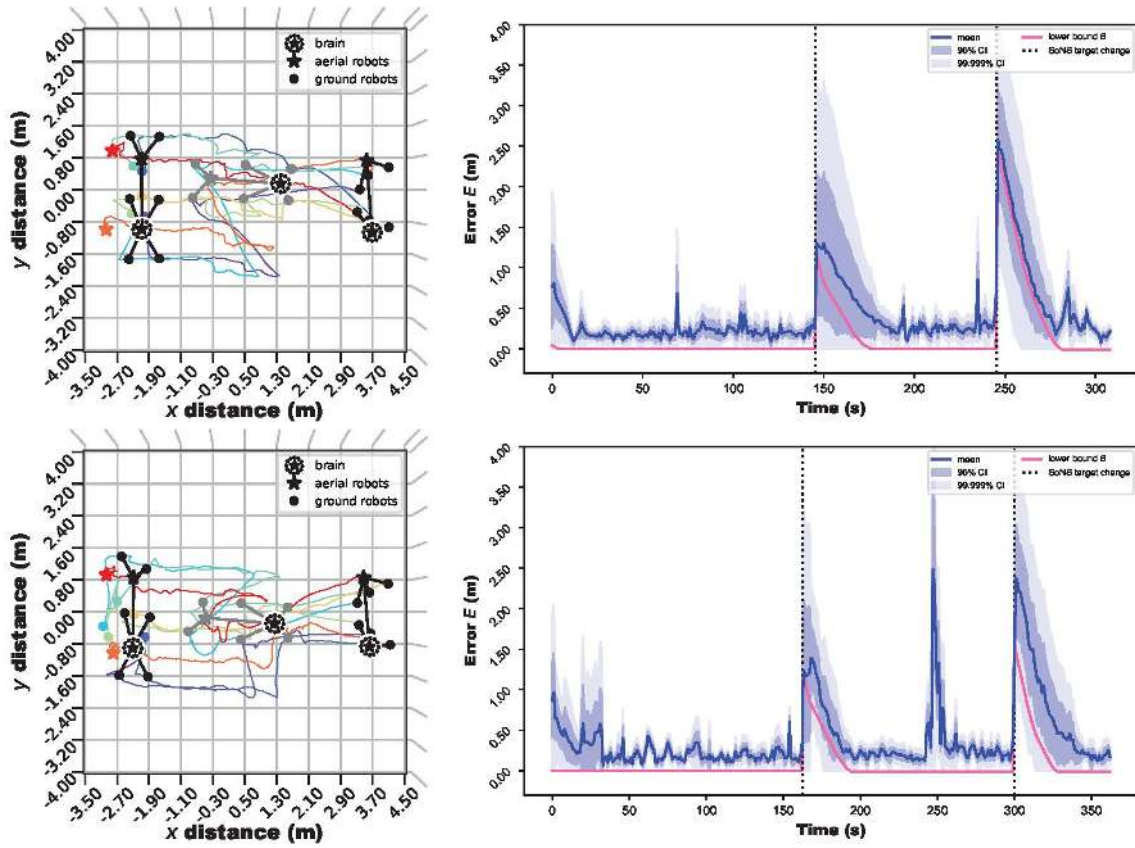


Fig. S15: (cont'd) Binary decision making: Real robot trials. Five trials with real robots were conducted, each with eight robots.

(Section continued on next page.)

Mission: Splitting and merging systems (see Sec. 2.1.5 in the main paper)

This mission includes three different variants, two run in experiments with real robots and one run in simulation.

Variant with real robots: Search and rescue

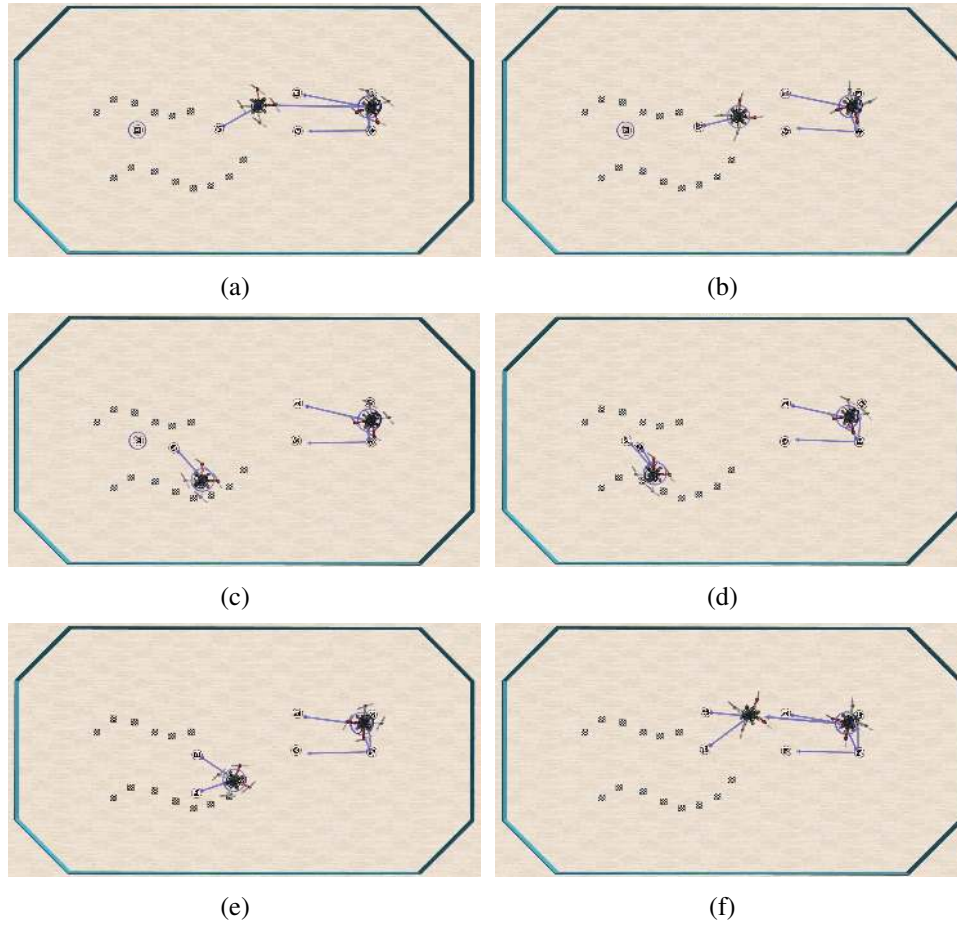


Fig. S16: Splitting and merging systems, search and rescue: Key frames. (a) When robots start, they are in a SoNS that is missing one or two of its members. (b) The SoNS-brain instructs one of the robots to split from it and temporarily form its own multi-robot SoNS as a rescue team. (c,d) The rescue team SoNS searches the environment until it finds the missing robot(s) and merges with it/them. (e,f) The merged rescue team SoNS then returns to the location where it initially split off and re-merges with the remaining SoNS, so that all robots are reunited, and the mission is complete. Note that, in the experiments, there are more obstacles in the environment, forming the walls of an arbitrary passage that the robots need to navigate to complete the search and rescue mission.

Variant with real robots: Search and rescue

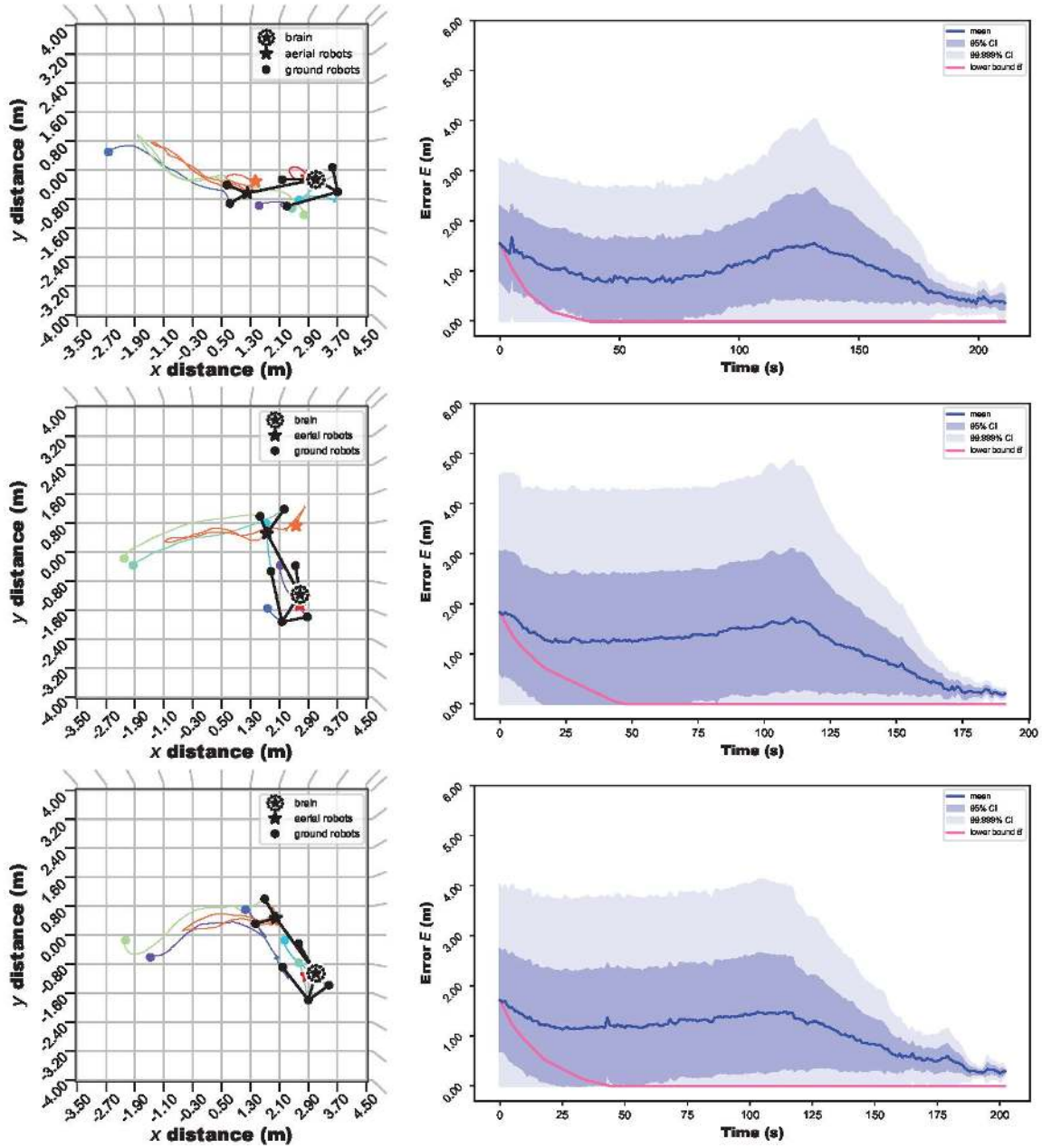


Fig. S17: Splitting and merging systems, search and rescue: Real robot trials. Five trials with real robots were conducted, each with eight robots (*figure continued on next page*).

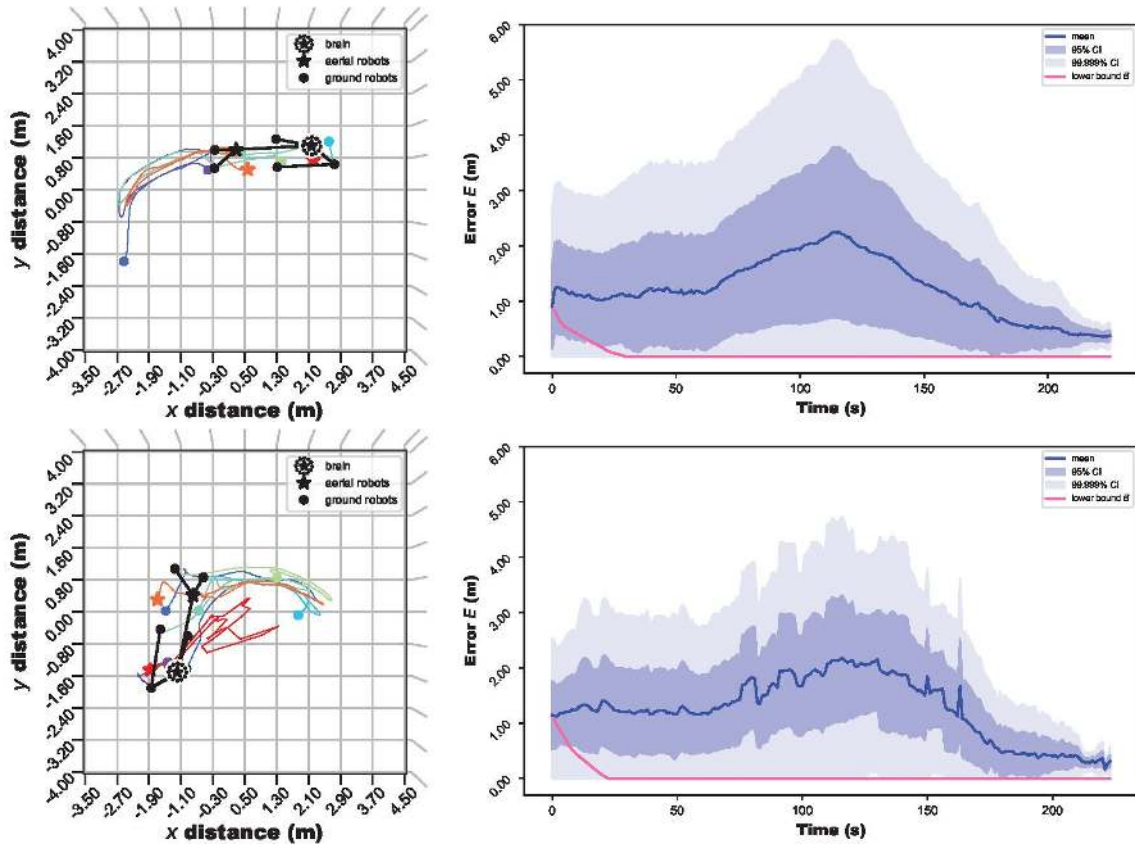


Fig. S17: (cont'd) Splitting and merging systems, search and rescue: Real robot trials. Five trials with real robots were conducted, each with eight robots.

(Section continued on next page.)

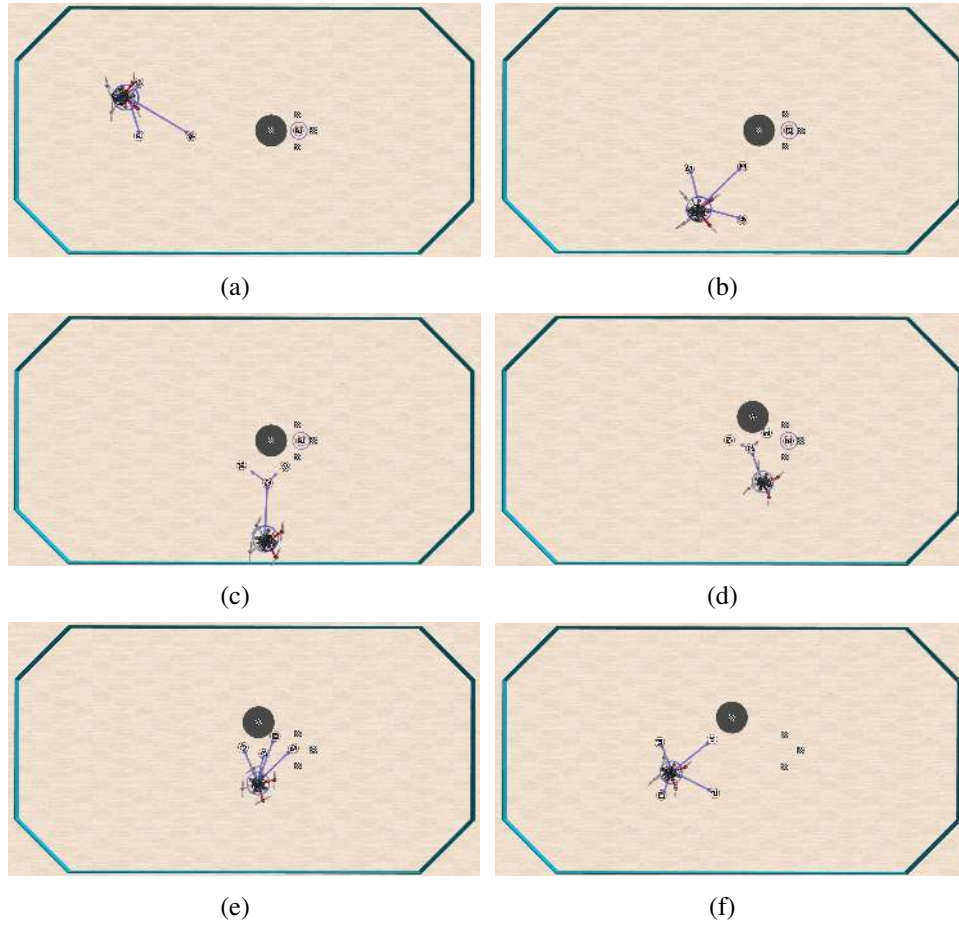
Variant with real robots: Push away an obstruction

Fig. S18: Push away an obstruction: Key frames. (a) When robots start, they are in a SoNS that is missing a member. (b) The SoNS searches the environment and finds the missing robot trapped by obstacles. (c) The SoNS reorganizes into a shape that allows its ground robots to collaboratively push a large obstruction. (d,e) The robots push the obstruction away and the SoNS merges with the missing robot, so that all robots are reunited. (f) The robots reorganize into the target SoNS and the mission is complete.

(Section continued on next page.)

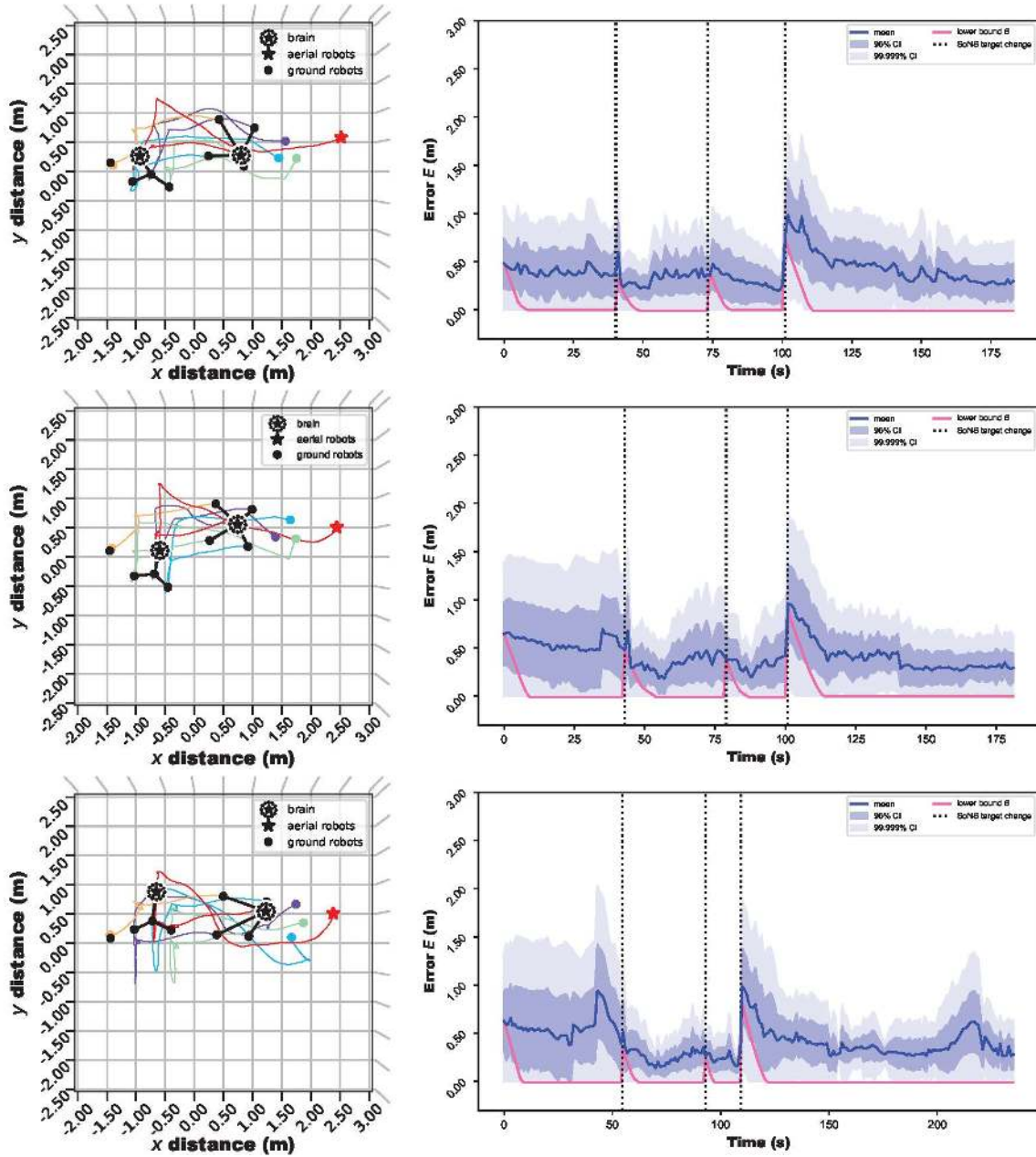
Variant with real robots: Push away an obstruction

Fig. S19: Splitting and merging systems, push away an obstruction: Real robot trials. Note that the highlighted keyframes shown here are from an intermediary time step, not the end of the trial. Five trials with real robots were conducted, each with eight robots (*figure continued on next page*).

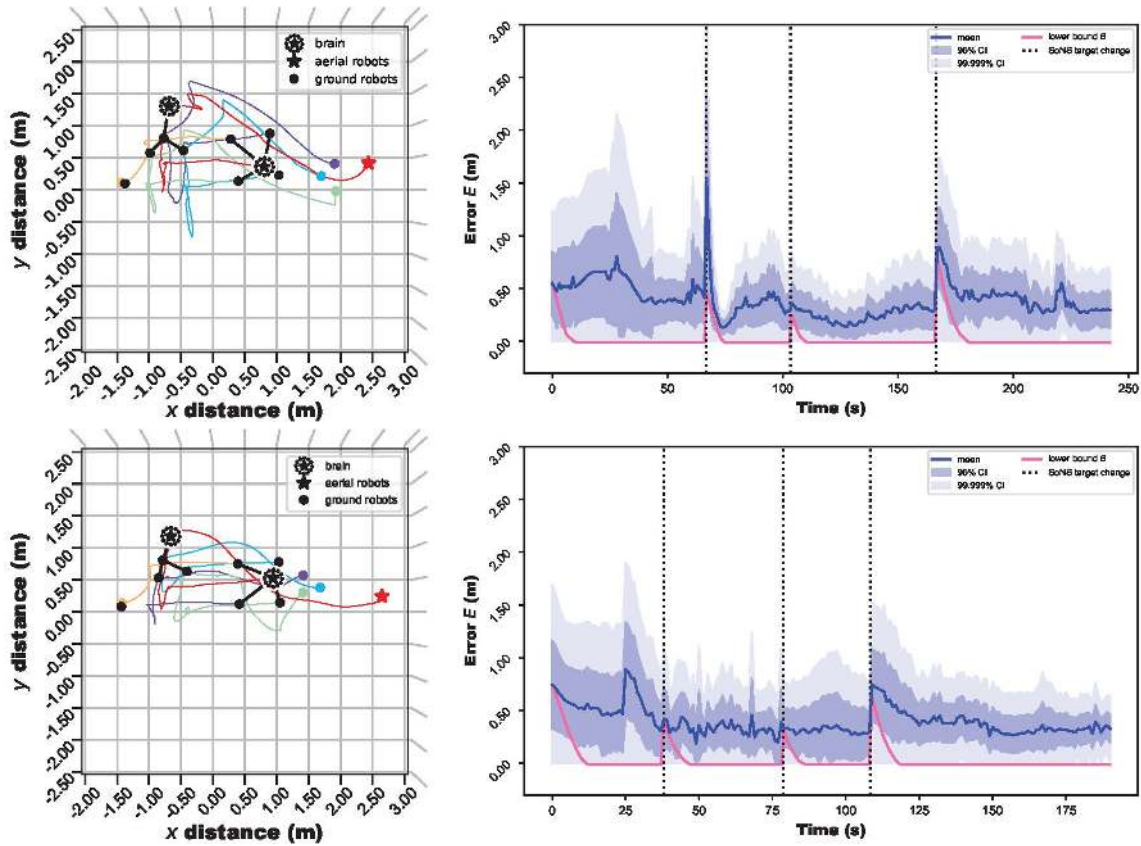


Fig. S19: (cont'd) Splitting and merging systems, push away an obstruction: Real robot trials. Note that the highlighted keyframes shown here are from an intermediary time step, not the end of the trial. Five trials with real robots were conducted, each with eight robots.

(Section continued on next page.)

Simulation variant: Simple split and merge

This variant consists simply of split and merge operations, without search-and-rescue or other constituent tasks.

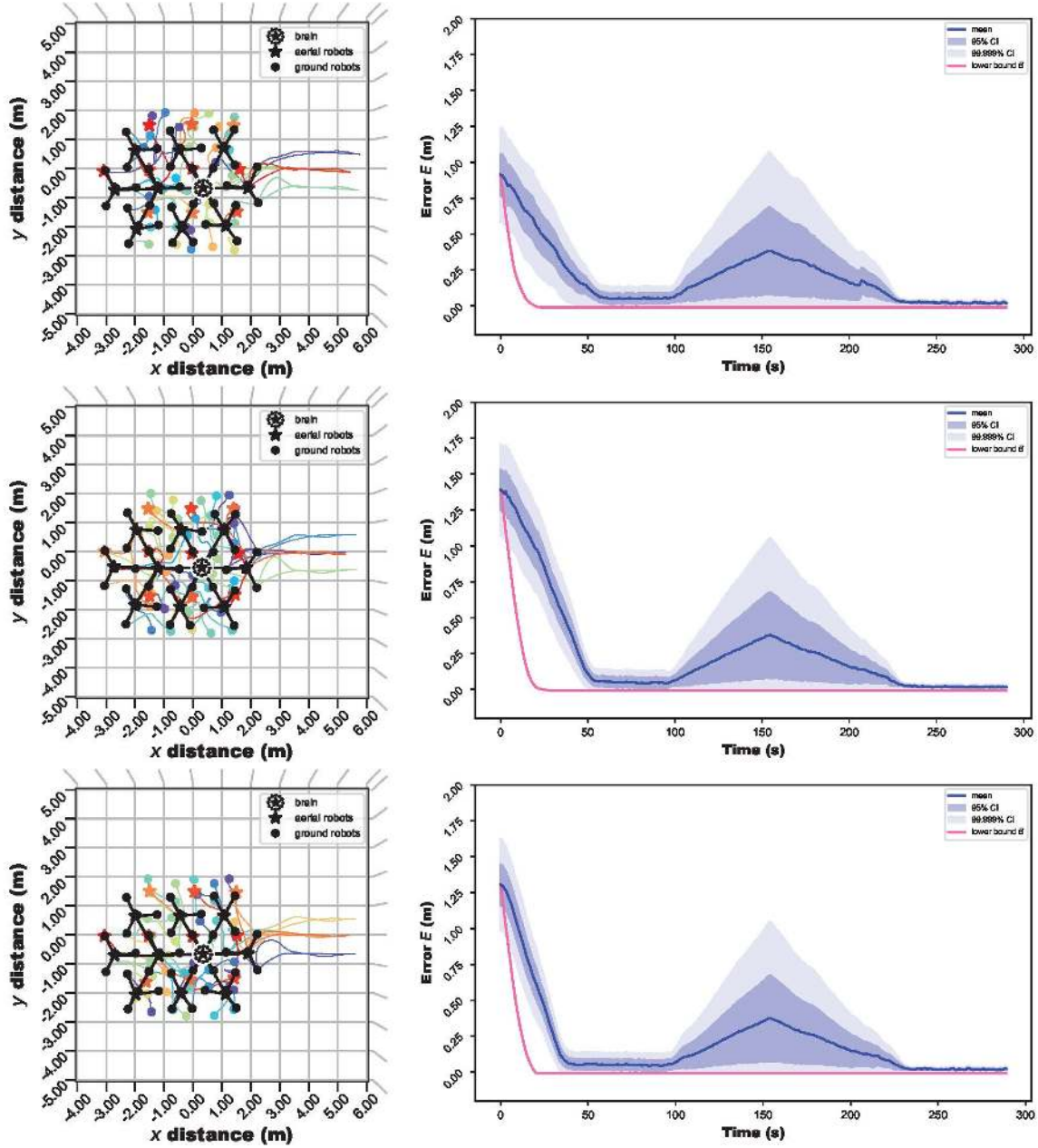


Fig. S20: Splitting and merging systems, simple split and merge: Example simulation trials. 50 trials were conducted in simulation, each with 50 robots.

Scalability setups (see Sec. 2.2 in the main paper)

The scalability setups include two variants based on two of the robot missions (those shown in Secs. 2.1.1 and 2.1.4 in the main paper), both run in simulation only.

Scalability in the binary decision-making mission

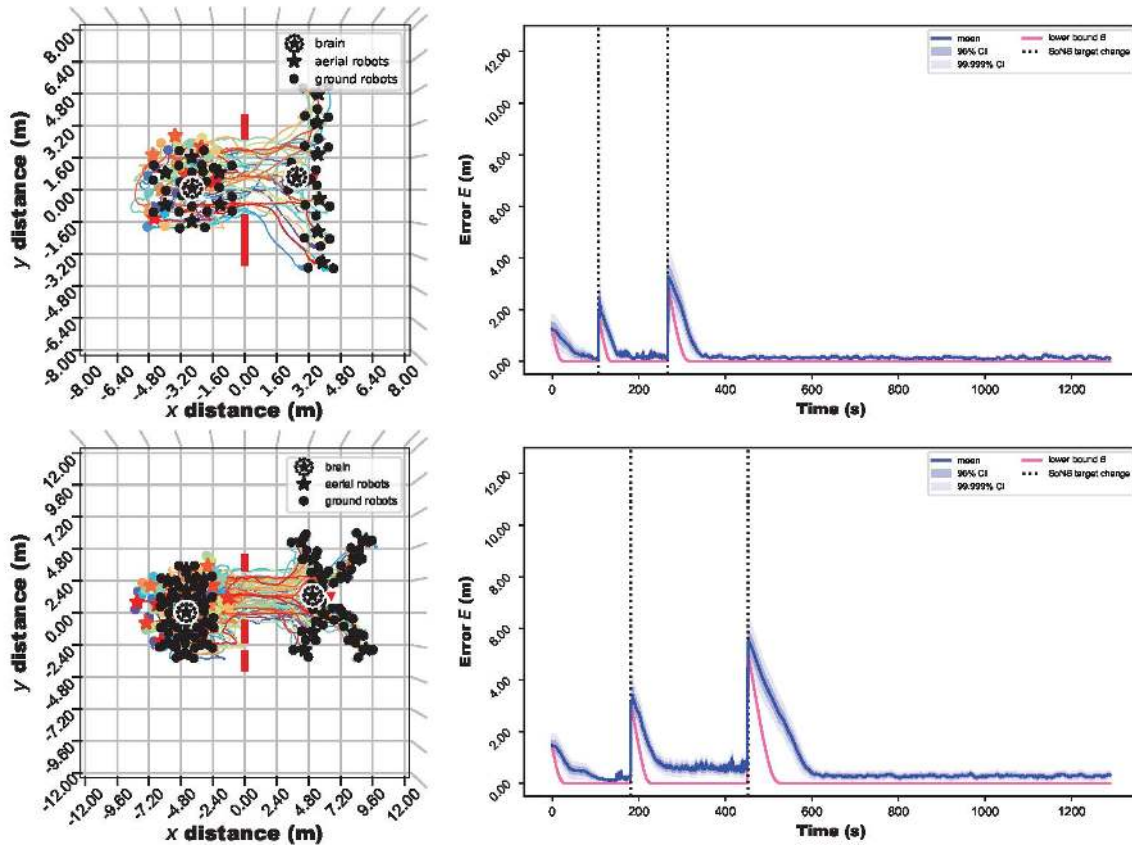


Fig. S21: Scalability in the binary decision-making mission: Example simulation trials of four different system sizes (from top to bottom, 35, 65, 95, and 125 robots). 50 trials per system size were conducted in simulation, for four different system sizes that are all shown here (*figure continued on next page*).

(Section continued on next page.)

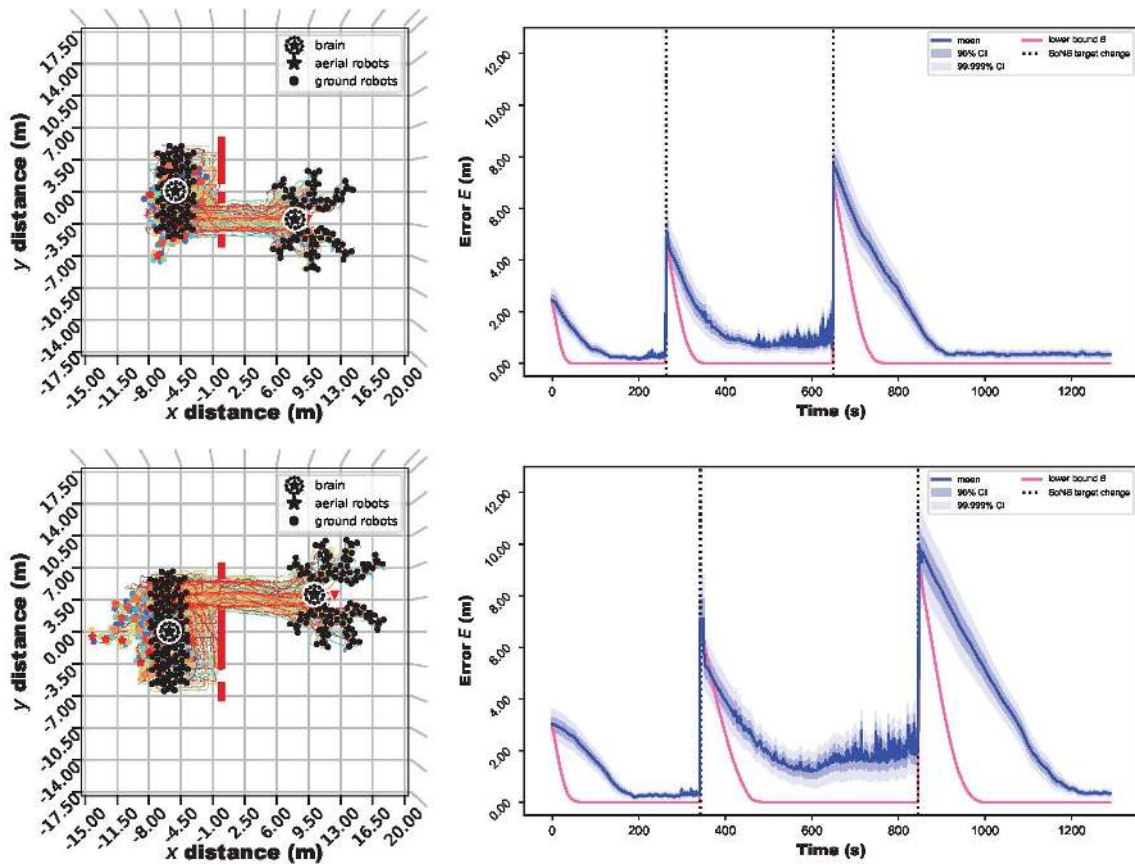


Fig. S21: (cont'd) Scalability in the binary decision-making mission: Example simulation trials of four different system sizes (from top to bottom, 35, 65, 95, and 125 robots). 50 trials per system size were conducted in simulation, for four different system sizes that are all shown here.

(Section continued on next page.)

Scalability in the establishing self-organized hierarchy mission

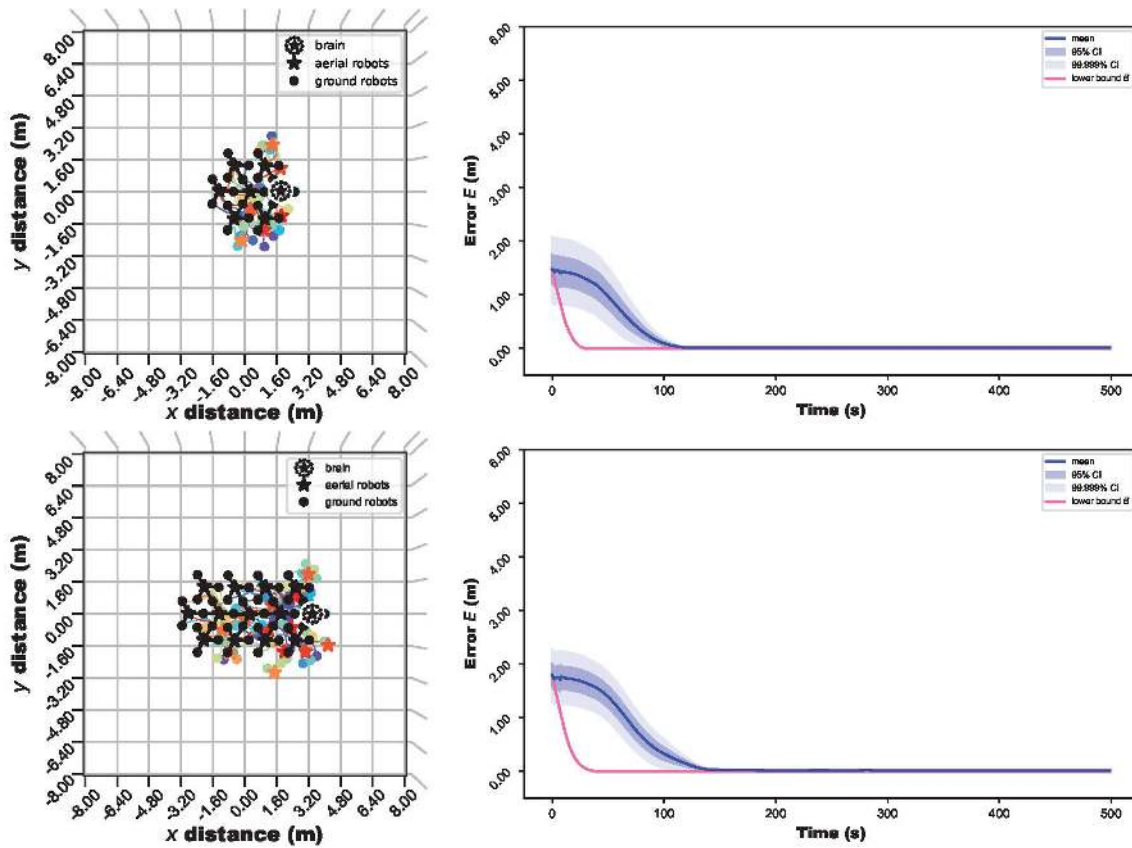


Fig. S22: Scalability in the establishing self-organized hierarchy mission: Example simulation trials of four different system sizes (from top to bottom, 35, 65, 95, and 125 robots). 30 trials per system size were conducted in simulation, for 50 different system sizes in total (*figure continued on next page*).

(Section continued on next page.)

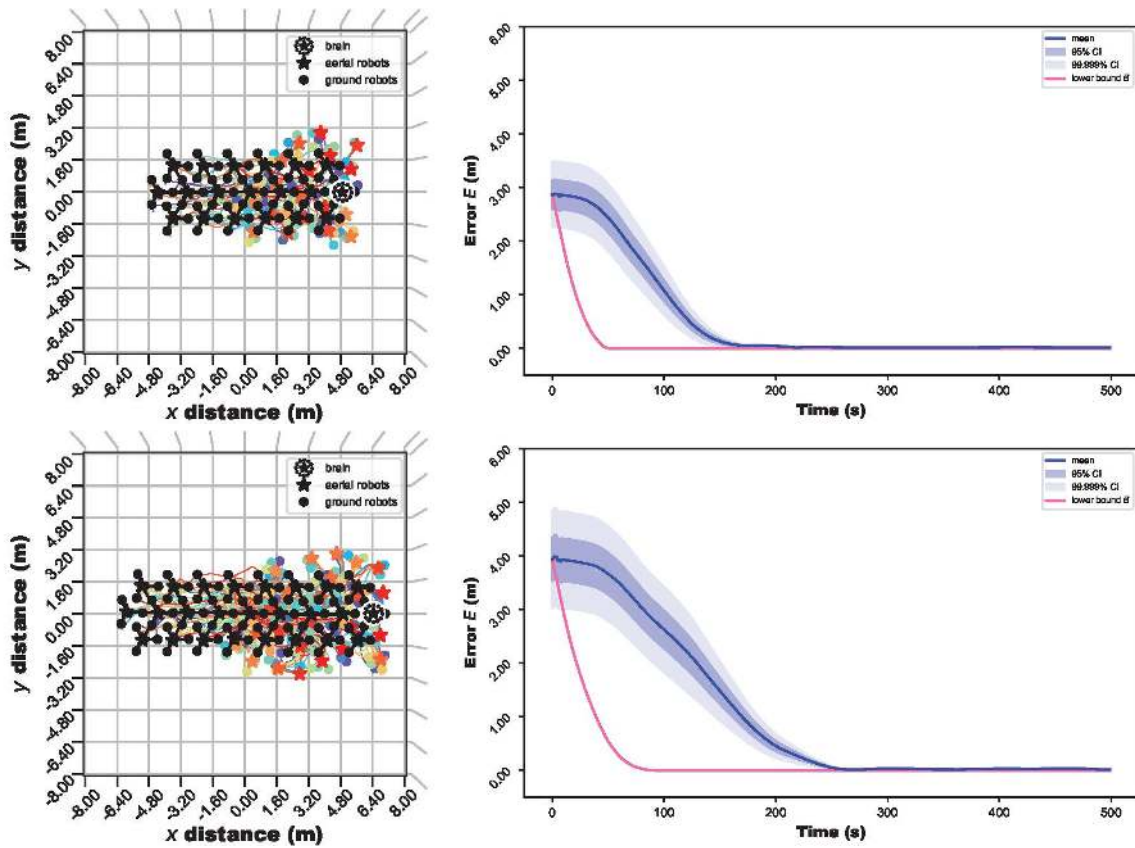


Fig. S22: (cont'd) Scalability in the establishing self-organized hierarchy mission: Example simulation trials of four different system sizes (from top to bottom, 35, 65, 95, and 125 robots). 30 trials per system size were conducted in simulation, for 50 different system sizes in total.

(Section continued on next page.)

Fault tolerance setups (see Sec. 2.3 in the main paper)

The fault tolerance setups include four variants that are all based on the same robot mission (shown in Sec. 2.1.4 in the main paper), with one variant run in experiments with real robots and the other three run in simulation.

Variant with real robots: Multiple permanent failures

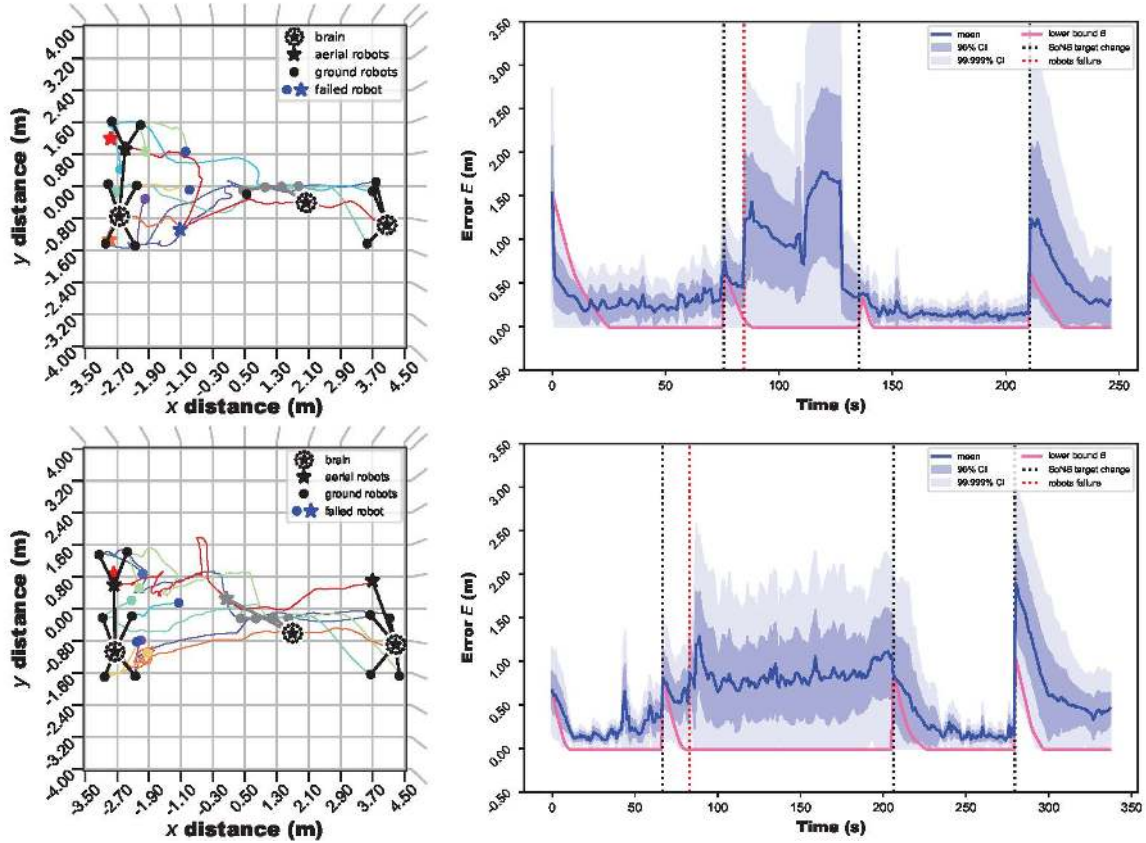


Fig. S23: Multiple permanent failures: Real robot trials. Five trials with real robots were conducted, each with eight robots. Failures are triggered uniformly randomly. Note that in the third trial shown here, both aerial robots failed, and therefore the remaining functional robots (all ground robots) did not fulfill the requirements of the mission and did not continue with the mission after the failures occurred (*figure continued on next page*).

(Section continued on next page.)

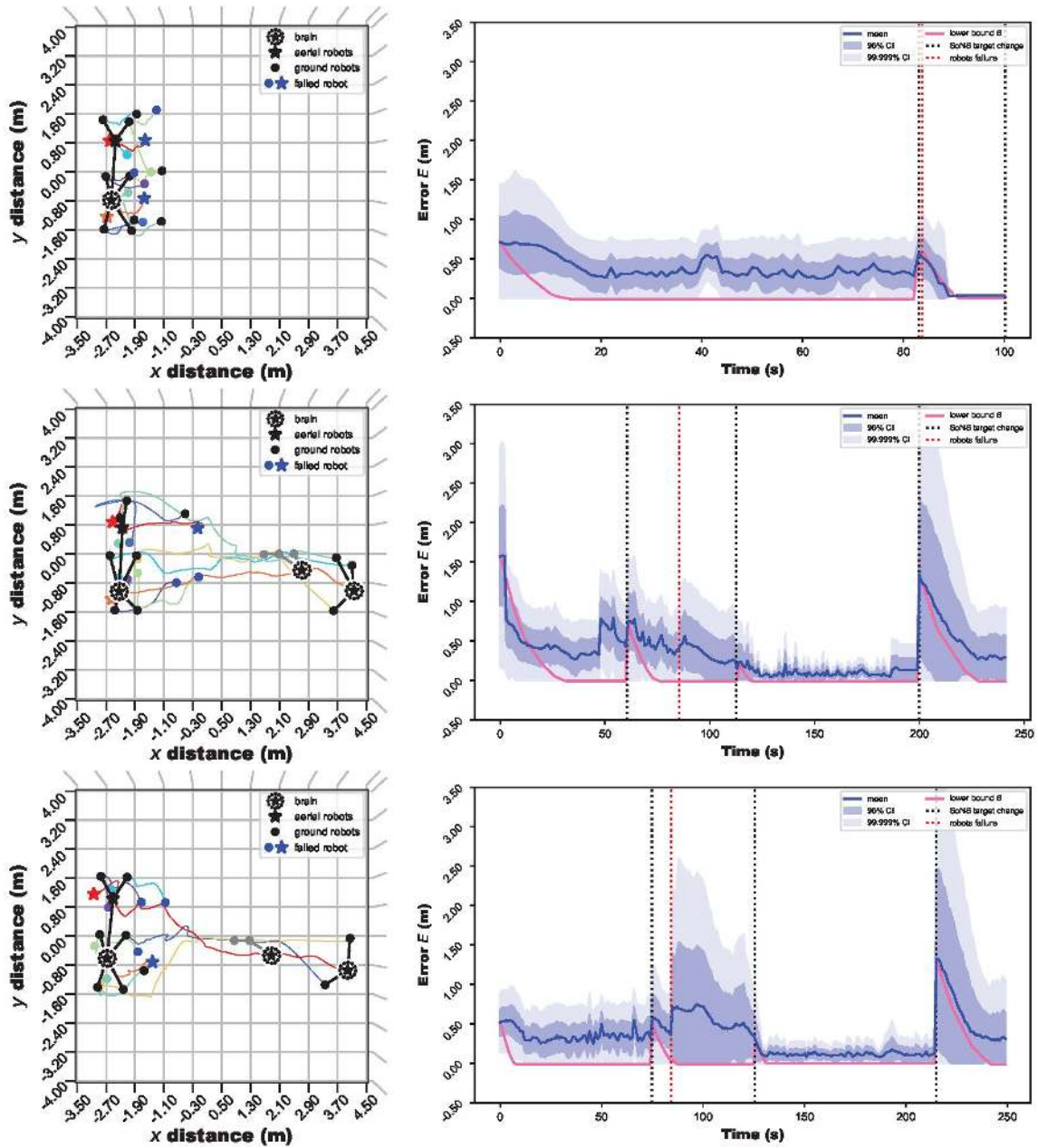


Fig. S23: (cont'd) Multiple permanent failures: Real robot trials. Five trials with real robots were conducted, each with eight robots. Failures are triggered uniformly randomly. Note that in the third trial shown here, both aerial robots failed, and therefore the remaining functional robots (all ground robots) did not fulfill the requirements of the mission and did not continue with the mission after the failures occurred.

Simulation variant: High-loss conditions, $33.\bar{3}\%$ probability to fail

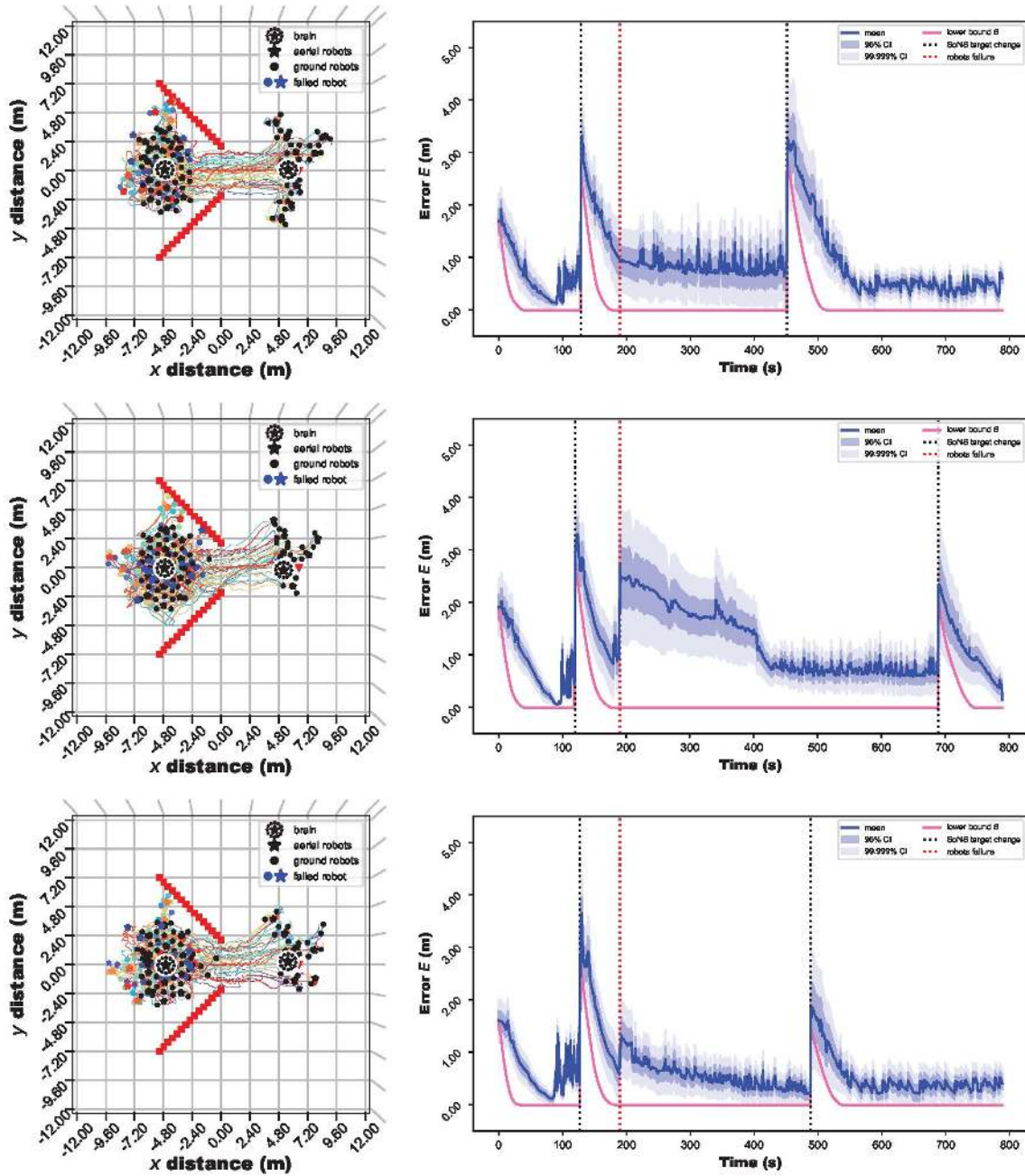


Fig. S24: High-loss conditions, $33.\bar{3}\%$ probability to fail: Example simulation trials. 50 trials were conducted in simulation, each with 65 robots.

Simulation variant: High-loss conditions, $66.\bar{6}\%$ probability to fail

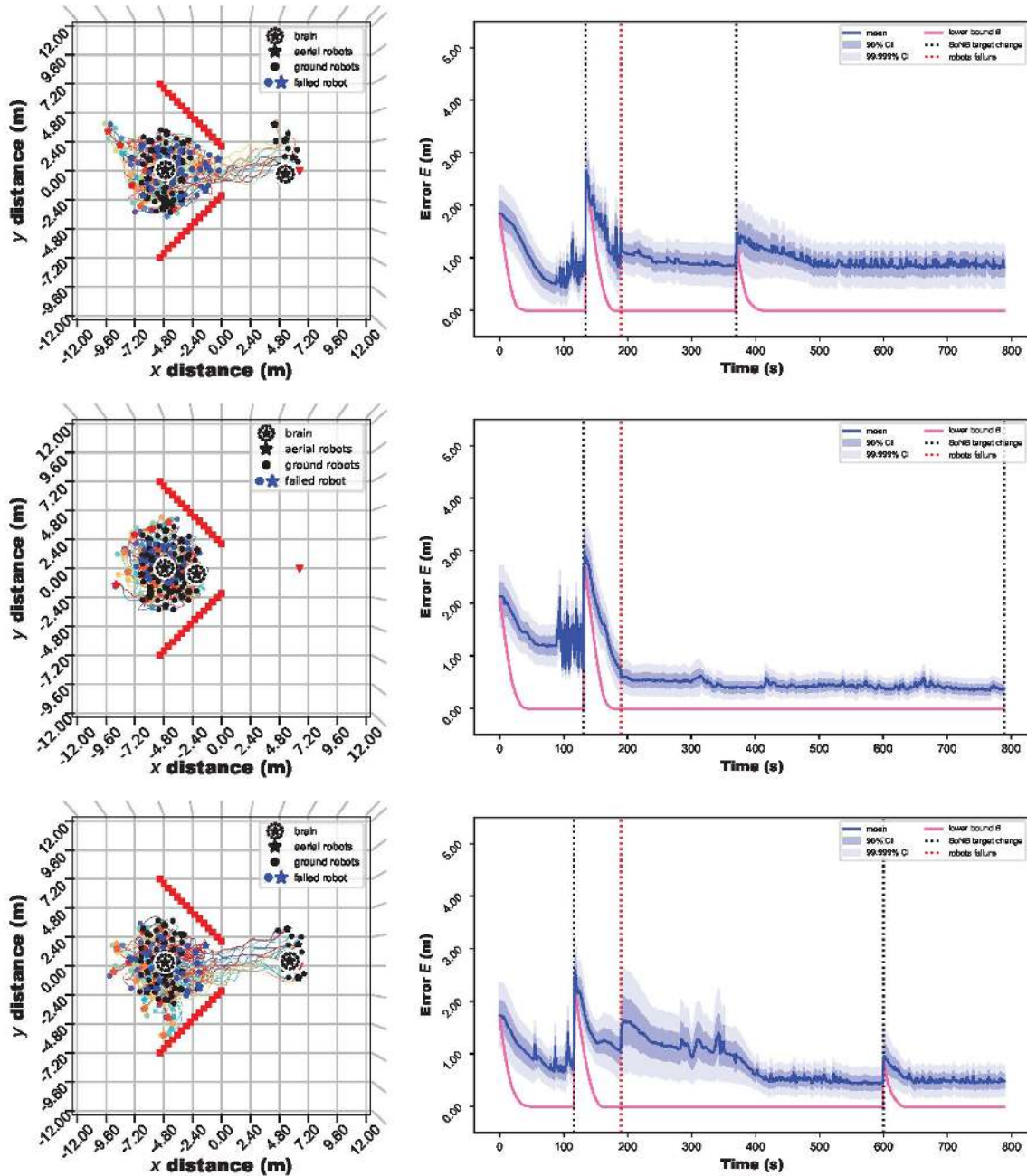


Fig. S25: High-loss conditions, $66.\bar{6}\%$ probability to fail: Example simulation trials. Note that in some trials under this condition, the SoNS was able to re-establish itself after the failures occurred, but was not able to complete other portions of the mission due to, for example, functional ground robots in the SoNS becoming trapped by the failed ground robots (as in the middle trial shown here). 50 trials were conducted in simulation, each with 65 robots.

Simulation variant: Temporary system-wide vision failure

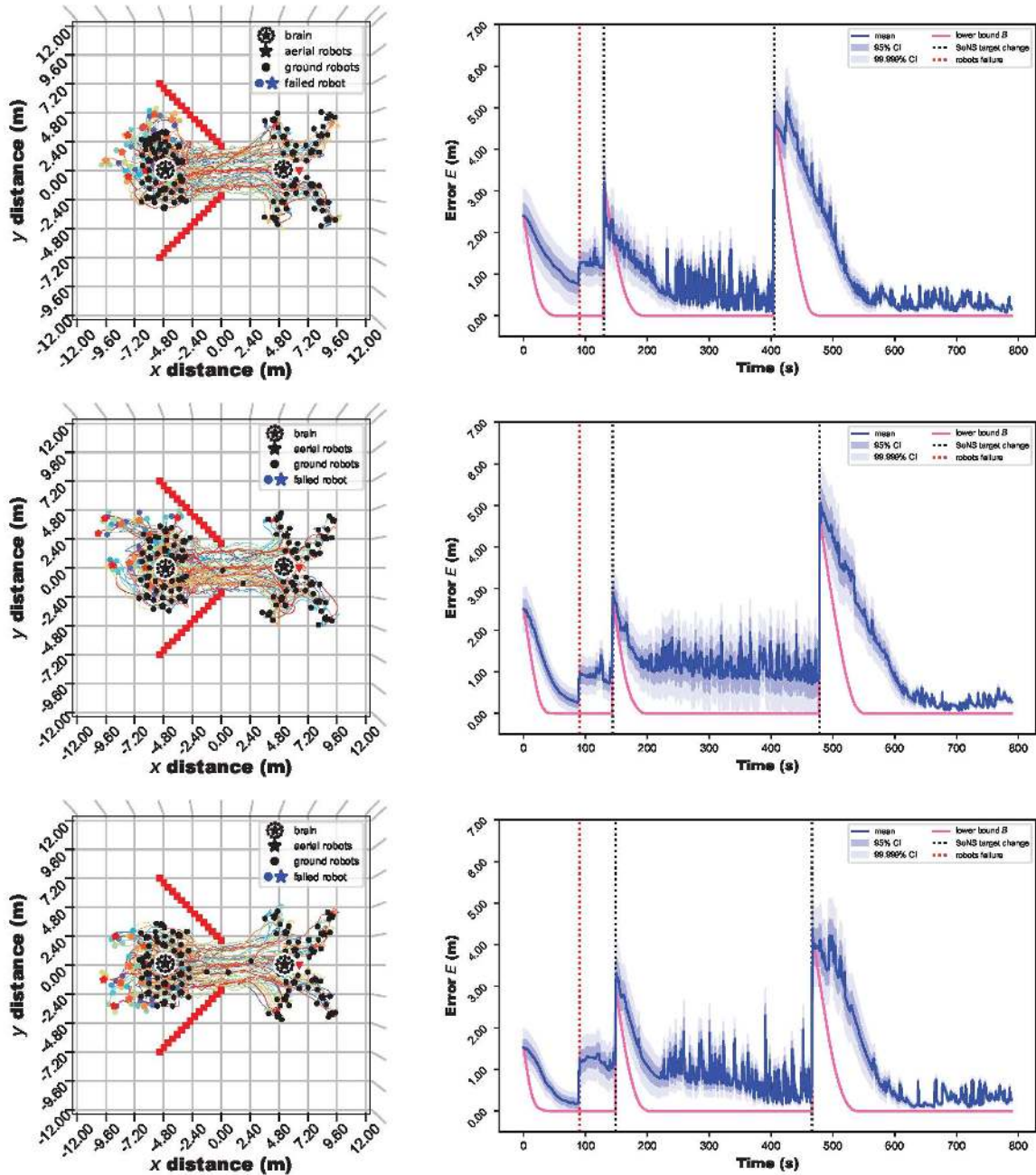


Fig. S26: Temporary system-wide vision failure: Example simulation trials of three different failure durations (from top to bottom, 0.5, 1.0, and 30 s). 50 trials per duration were conducted in simulation, each with 65 robots.

Simulation variant: Temporary system-wide communication failure

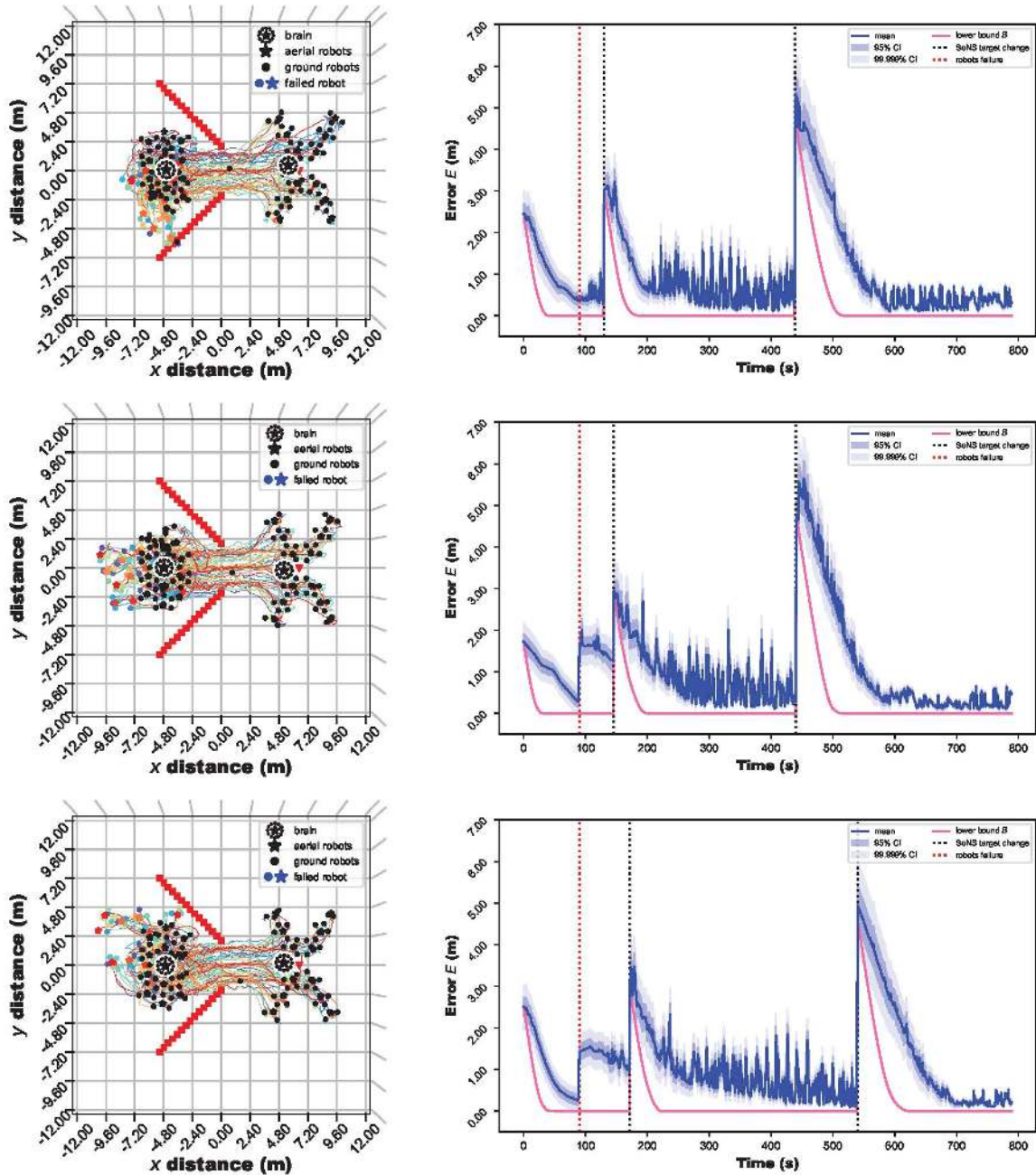


Fig. S27: Temporary system-wide communication failure: Example simulation trials of three different failure durations (from top to bottom, 0.5, 1.0, and 30 s). 50 trials per duration were conducted in simulation, each with 65 robots.

Section S2. Cross-verification of the SoNS in simulation and on real hardware

To cross-verify the SoNS behaviors in the simulator with those of the real robots, we ran simulation experiments (50 trials per setup) with the same setups as six of the experiments with real robots. This section provides results from an example simulation trial of each of the cross-verification setups as well as statistical comparisons of the 50 simulation trials and five real robot trials of the matching setup. The experiment data for all trials of all setups is available in the online data repository.

Example simulation trials that match real robot setups

For each example trial included in this section, we provide the following results: (on the left) trajectories of the robots over time, with the final (and sometimes the initial and/or an intermediary) SoNS indicated in black; and (on the right) the mean and confidence interval per robot of the actuation error E over time (see Eq. 1 in Sec. 4.2 in the main paper), with the lower bound B (see Eq. 2 in Sec. 4.2 in the main paper) plotted for reference.

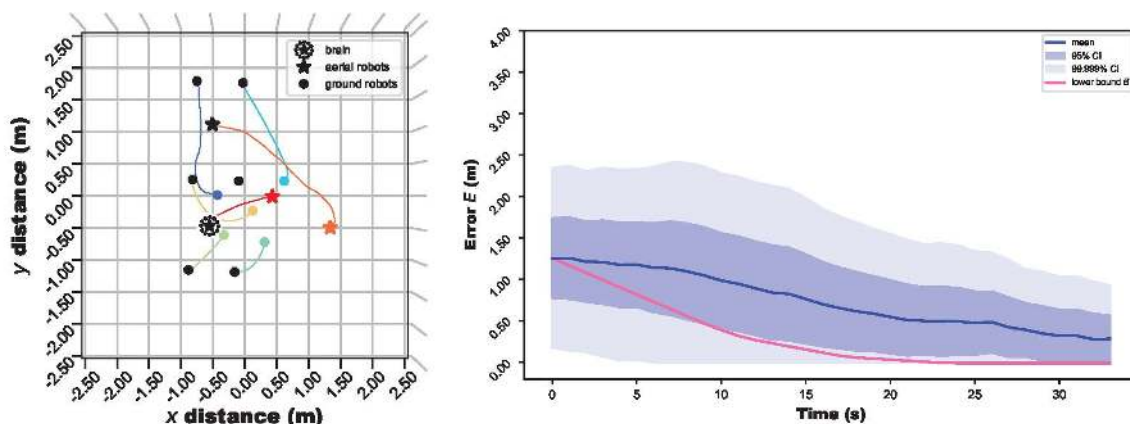


Fig. S28: Establishing self-organized hierarchy, clustered start: Example simulation trial with eight robots that matches the setup of the real robot trials shown in Fig. S2.

(Section continued on next page.)

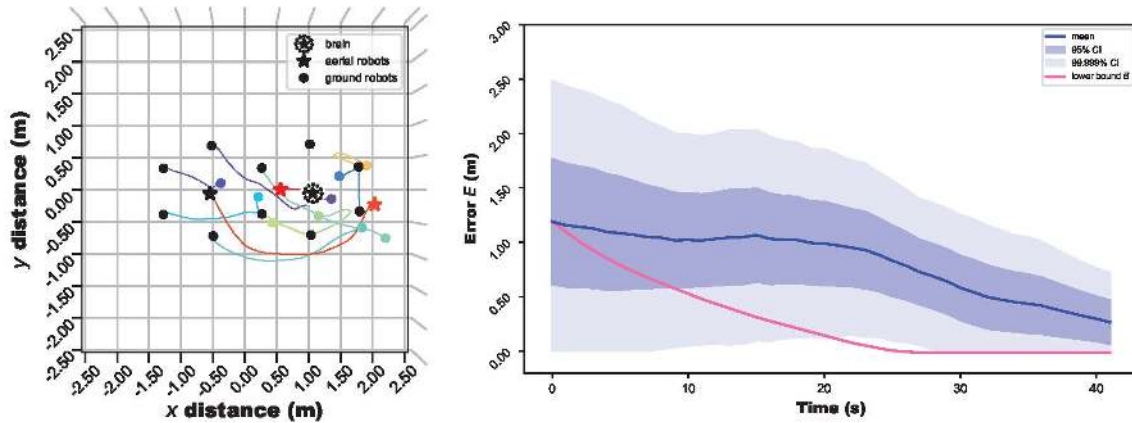


Fig. S29: Establishing self-organized hierarchy, scattered start: Example simulation trial with 12 robots that matches the setup of the real robot trials shown in Fig. S4.

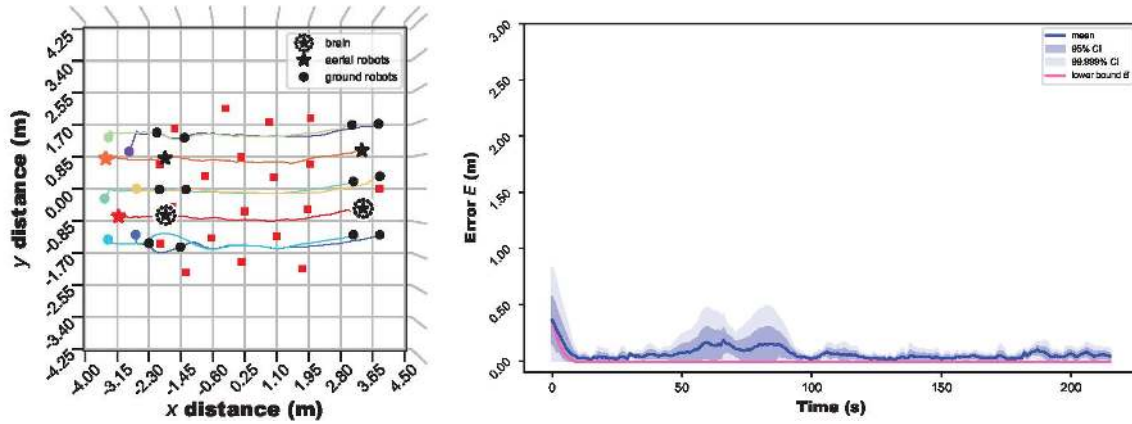


Fig. S30: Balancing global and local goals, with smaller, denser obstacles: Example simulation trial with eight robots that matches the setup of the real robot trials shown in Fig. S7.

(Section continued on next page.)

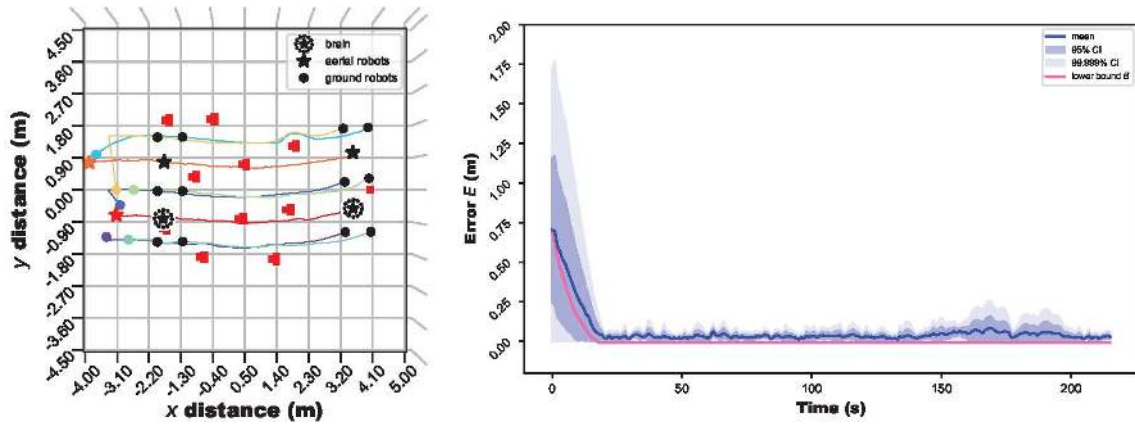


Fig. S31: Balancing global and local goals, with larger, less dense obstacles: Example simulation trial with eight robots that matches the setup of the real robot trials shown in Fig. S9.

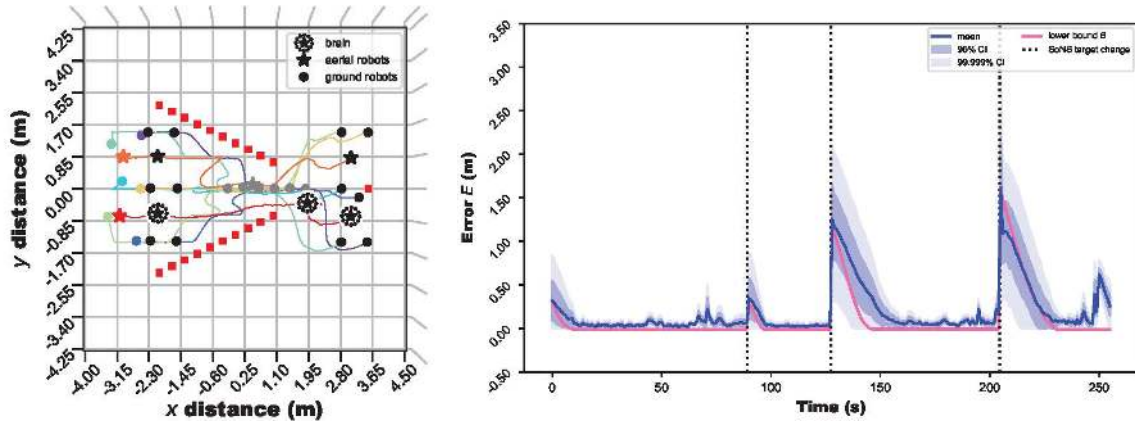


Fig. S32: Collective sensing and actuation: Example simulation trial with eight robots that matches the setup of the real robot trials shown in Fig. S12.

(Section continued on next page.)

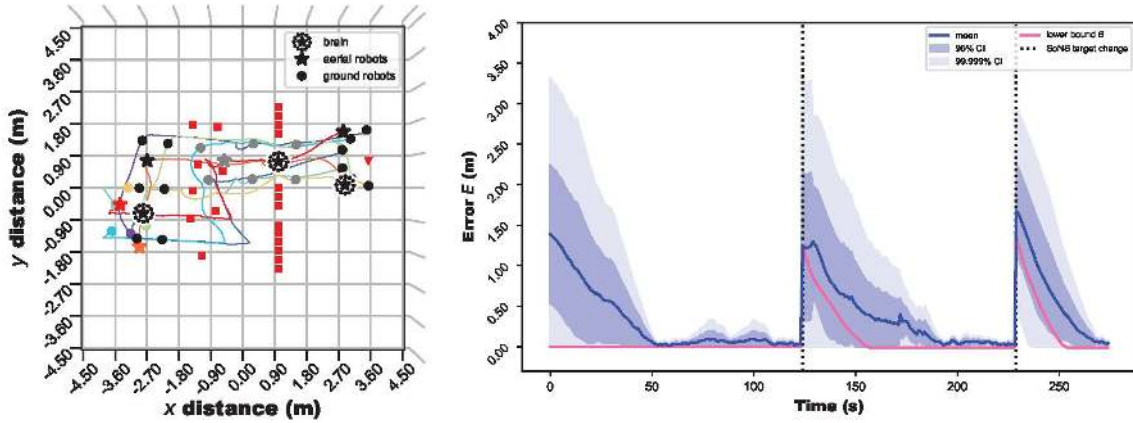


Fig. S33: Binary decision making: Example simulation trial that matches the setup of the real robot trials shown in Fig. S15.

Comparisons of trials with real robots and trials in simulation

The SoNS behaviors observed in the cross-verification simulation trials are very similar to those observed in their real-robot counterparts (see Figs S28–S33). However, there is noticeably higher error in the real robot experiments than in the simulations. To assess the overall difference in error, we provide violin plots and Q–Q plots (quantile–quantile plots) comparing the real robot trials to their matching simulations (see Figs. S34, S35).

It can be seen in the comparative violin plots that the difference in mean error between the real robots and simulation is very low (always less than $E = 0.5$ m, usually less than $E = 0.25$ m), but the error in the real robots is greater overall. It can also be seen that the biggest difference in mean error, between the real robots and simulation, occurs in the missions to establish self-organized hierarchy. Indeed, this is expected, as a large portion of experiment time in this mission is spent with the robots in a steady phase, and therefore with the aerial robots attempting to hover in place. Much of the difference in error here can be attributed to the simulation model of the aerial robot, rather than to the SoNS behaviors. These observations are further confirmed by the Q–Q plots, which show a great difference in the probability distributions of the error between the real robots and simulation in the missions to establish self-organized hierarchy. In the other mission types, the real robots indeed always show greater error, but are much more similar to the simulations overall.

(Section continued on next page.)

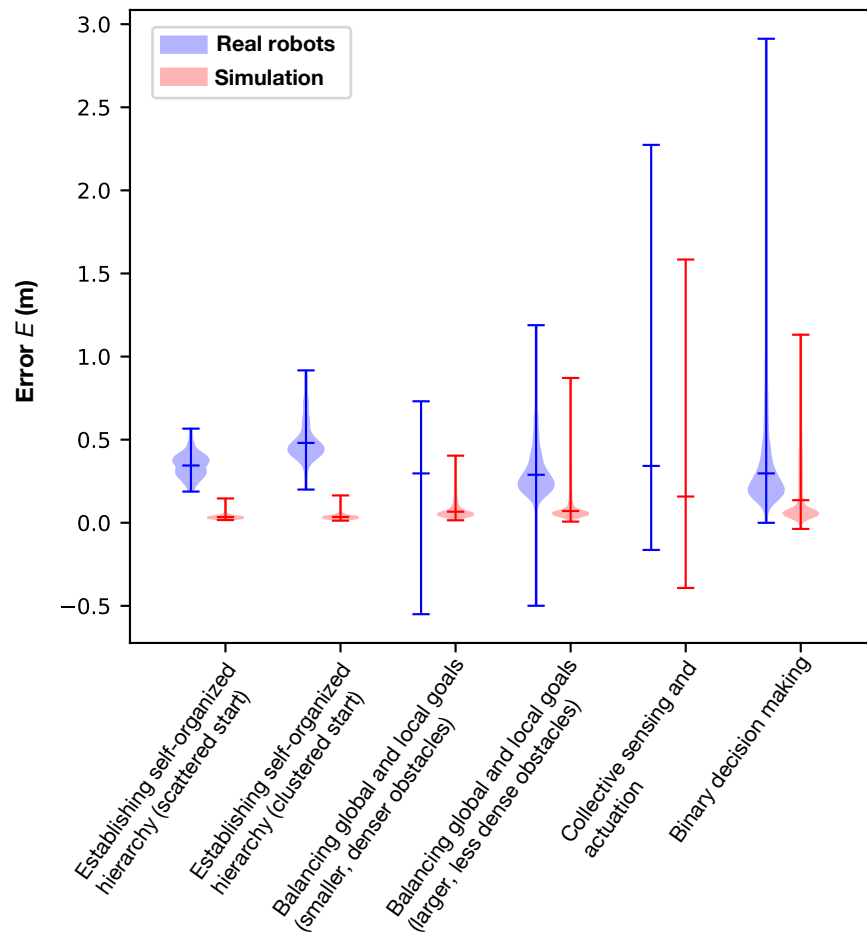
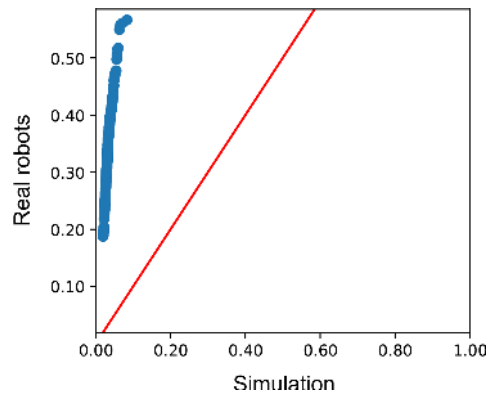
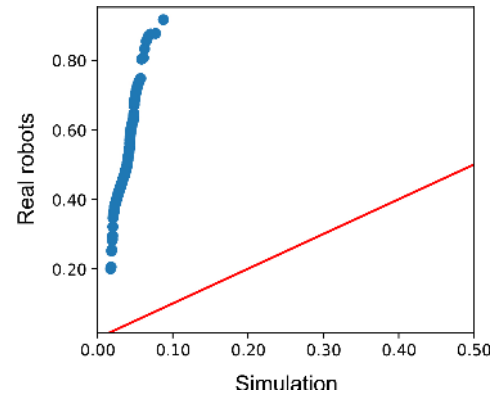


Fig. S34: Comparison of the real robot trials to their matching simulations. Violin plots of the actuation error E (mean and 95% confidence interval per robot per second) in all trials of each experiment type.

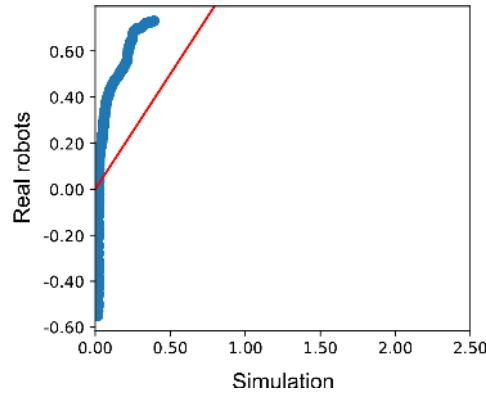
(Section continued on next page.)



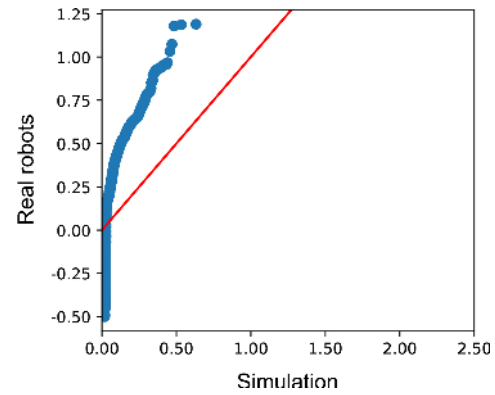
(a) Establishing self-organized hierarchy, clustered start



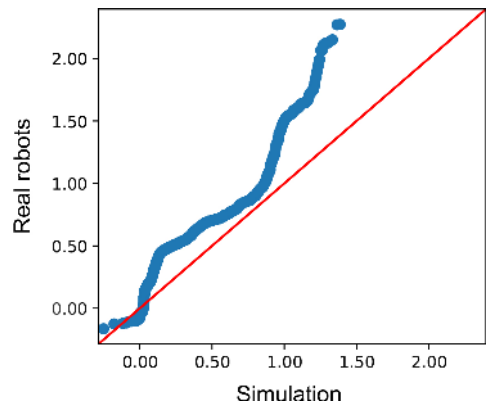
(b) Establishing self-organized hierarchy, scattered start



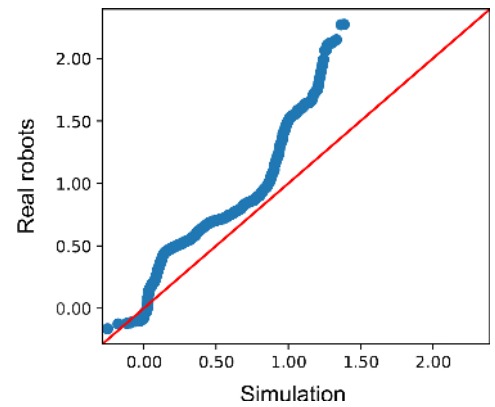
(c) Balancing global and local goals, with smaller, denser obstacles



(d) Balancing global and local goals, with larger, less dense obstacles



(e) Collective sensing and actuation



(f) Binary decision making

Fig. S35: Comparison of the real robot trials to their matching simulations. Q-Q plots of the probability distributions of actuation error E in all trials of the specified type.

Section S3. SoNS theoretical analysis and mathematical proofs

This section presents theoretical analyses and mathematical proofs related to the tracking of desired relative positions in a SoNS. We analyze convergence and stability with motionless and moving SoNS-brain robots.

For analysis, we represent a SoNS of n robots as a multi-robot formation and decompose it into $n - 1$ sub-formations of leader-follower pairs. To study the interactions and stability bounds of a SoNS, we analyze the local leader-follower formation tracking problems of these pairs. We use a standard reactive control law [51, 52] to generate appropriate inputs for the follower robot, in order to maintain the desired relative position with respect to its leader. This strictly reactive control law is the same approach we use in our real robot experiments; it establishes a performance baseline for the tracking of target positions within a SoNS. We assess the convergence and closed-loop stability of position tracking in the system, utilizing the *leader-to-formation stability* notion [53]. At the end of this section, we discuss other types of control laws that could be combined with the SoNS architecture to improve performance beyond the baseline established using a strictly reactive control law.

Modeling

Consider a SoNS with n robots (including both aerial robots and ground robots). The translational motion in $\mathbb{R}^2$ space of a robot R_i , $i \in \{1, 2, \dots, n\}$, is governed by [51]:

$$\dot{\vec{p}}_i = \vec{u}_i, \quad (\text{S1})$$

where $\vec{p}_i = [x_i \ y_i]^T \in \mathbb{R}^2$ is the absolute position in a global coordinate system $\mathcal{F}_{\mathcal{I}}$ used for analysis and $\vec{u}_i \in \mathbb{R}^2$ is the control input.

Control of a leader-follower pair

We first analyze the interactions of a single leader-follower pair, before extending to multiple leader-follower pairs.

Leader-follower kinematics

We derive the leader-follower kinematics for a formation consisting of a single robot pair. Consider the leader-follower setup in Fig. S36, where $\vec{p}_i = [x_i \ y_i]^T$ is the position of the leader robot R_i , $\vec{p}_j = [x_j \ y_j]^T$ is the position of the follower robot R_j , $\vec{d}_{ij} \in \mathbb{R}^2$ is the desired displacement of R_j w.r.t. R_i and is defined according to the control outputs of the SoNS software, all expressed in the global coordinate frame used for the analysis. From this setup, we can define $\vec{z}_{ij} = \vec{p}_i - \vec{p}_j$ as the displacement of the follower R_j with respect to the leader R_i , $\|\vec{z}_{ij}\| \in \mathbb{R}$ as the Euclidean norm of that displacement, $\|\vec{d}_{ij}\| = \|\vec{d}_{ji}\| \in \mathbb{R}$ as the Euclidean norm of the desired displacement $\vec{d}_{ij}$, $\vec{p}_j^d = \vec{p}_i - \vec{d}_{ij}$ as the desired position of R_j , and $\vec{e}_{ij} = \vec{z}_{ij} - \vec{d}_{ij} \in \mathbb{R}^2$ as the formation tracking error.

With this notation, the kinematics of a leader-follower pair can be defined as

$$\begin{aligned} \dot{\vec{e}}_{ij} &= \dot{\vec{p}}_i - \dot{\vec{p}}_j \\ &= \vec{u}_i - \vec{u}_j, \end{aligned} \quad (\text{S2})$$

Remark S1 Note that robots in a SoNS are assumed to not have access to any global position information or external reference frame. Each robot can only access the relative positions $\vec{z}_{ij}$ of its neighbors $j \in \mathcal{N}_i$, with respect to its own local reference frame. Global position information is used exclusively for analysis.

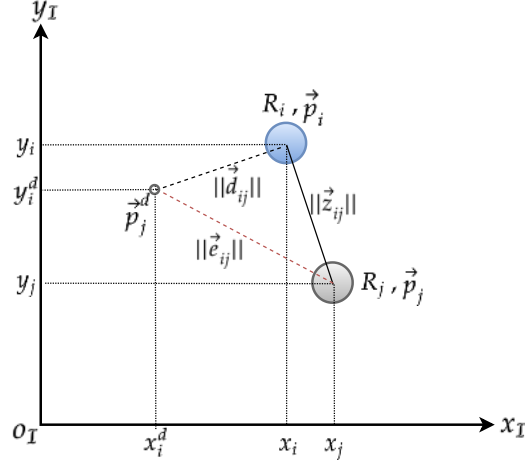


Fig. S36: An example leader-follower pair.

Problem formulation

The control law that generates inputs for the follower robot to move from its current position $\vec{p}_j$ to the desired position $\vec{p}_j^d$ can be expressed as follows.

Problem S1 The goal is to ensure that the formation tracking of the follower robot R_j adheres to the motion model given in Eq. (S1) and conforms to the leader-follower kinematics given in Eq. (S2), using the following control law $\vec{u}_j$ for the follower robot:

$$\vec{u}_j = f(t, \vec{z}_{ij}, \vec{d}_{ij}) \in \mathbb{R}^2, \quad (\text{S3})$$

where $\vec{z}_{ij}$ is the displacement with respect to the leader and $\vec{d}_{ij}$ is the desired displacement. The function $f(\cdot)$ generates the required control inputs to move the follower robot from its current position $\vec{p}_j$ to the desired position $\vec{p}_j^d$, such that the norm of the formation tracking error $\|\vec{e}_{ij}\| \rightarrow 0$ and $\|\vec{z}_{ij}\| \rightarrow \|\vec{d}_{ij}\|$.

Control law design and analysis

Based on the leader-follower kinematics given in Eq. (S2), we use the following standard reactive controller [51, 52] for the follower robot R_j :

$$\begin{aligned} \vec{u}_j &= \mathbf{K}^j (\vec{z}_{ij} - \vec{d}_{ij}) \\ &= \mathbf{K}^j \vec{e}_{ij} \end{aligned} \quad (\text{S4})$$

where $\mathbf{K}^j = (\mathbf{K}^j)^T = \begin{bmatrix} k_1^j & 0 \\ 0 & k_2^j \end{bmatrix}$ is a symmetric positive-definite matrix, the constants k_1^j and k_2^j are control gains, and $\vec{u}_j \in \mathbb{R}^2$ is the input for maintaining the desired displacement $\|\vec{d}_{ij}\|$ of the follower with respect to the leader. The control law is independent of any global position information; it uses only relative position information between the follower and the leader, making it independent of any global position information.

Input-to-state stability and bounding of formation tracking errors

We first analyze the stability properties of the leader-follower kinematics Eq. (S2) in the case of zero external input based on the following definition of Lyapunov stability.

Definition 1 (Exponential Stability [54]) For a time-invariant system $\dot{x} = f(x)$ where $f : \mathcal{D} \subset \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a locally Lipschitz function and $x = 0 \in \mathcal{D}$ is an equilibrium point of the system, exponential stability can be established through the use of a Lyapunov function $V(x)$ that satisfies the following conditions:

- $V(x)$ is $\mathcal{C}^1$ (i.e., continuously differentiable),
- $c_1\|x\|^a \leq V(x) \leq c_2\|x\|^a$,
- and $\dot{V}(x) \leq -c_3\|x\|^a$,

where $a > 0$, $c_1 > 0$, $c_2 > 0$, and $c_3 > 0$.

Theorem S1 For a leader-follower robot pair with the motion model Eq. (S1), an initial condition $\vec{p}_i(0) \neq \vec{p}_j(0)$, and control input $\vec{u}_i = 0$, the controller Eq. (S4) guarantees exponential stability of $\vec{e}_{ij} \rightarrow 0$ as $t \rightarrow \infty$, in the system given in Eq. (S2).

Proof S1 By substituting the control law Eq. (S4) into the motion model Eq. (S1), we obtain the following closed-loop system:

$$\dot{\vec{p}}_j = \mathbf{K}^j \vec{e}_{ij}. \quad (\text{S5})$$

The leader-follower kinematics Eq. (S2) can then be expressed as

$$\dot{\vec{e}}_{ij} = -\mathbf{K}^j \vec{e}_{ij} + \vec{u}_i. \quad (\text{S6})$$

We consider the Lyapunov function candidate $V_1 = \frac{1}{2} \vec{e}_{ij}^T \vec{e}_{ij}$ that satisfies the following condition:

$$c_1^j \|\vec{e}_{ij}\| \leq V_1 \leq c_2^j \|\vec{e}_{ij}\|, \quad (\text{S7})$$

where $a_j = 2$, $0 < c_1^j = \min(k_1^j, k_2^j)$, and $0 < c_2^j = \max(k_1^j, k_2^j)$. Note that the constants k_1^j and k_2^j are the gains of the matrix $\mathbf{K}^j$ defined in Eq. (S4). Then, differentiating Eq. (S7) with respect to time yields

$$\begin{aligned} \dot{V}_1 &= \vec{e}_{ij}^T \dot{\vec{e}}_{ij} \\ &= \vec{e}_{ij}^T (\vec{u}_i - \mathbf{K}^j \vec{e}_{ij}) \\ &= -\vec{e}_{ij}^T \mathbf{K}^j \vec{e}_{ij} + \vec{e}_{ij}^T \vec{u}_i \\ &\leq -2c_1^j \|\vec{e}_{ij}\|^2 + \|\vec{e}_{ij}\| \|\vec{u}_i\| \leq -c_3^j \|\vec{e}_{ij}\|^2 \leq 0, \quad \forall \|\vec{e}_{ij}\| \geq \frac{\|\vec{u}_i\|}{2c_1^j \theta}, \end{aligned} \quad (\text{S8})$$

where $c_3^j \triangleq 2c_1^j(1 - \theta)$, $\theta \in (0, 1)$. The proof is concluded by substituting $\vec{u}_i$.

With minor modification and following the lines of [55], the same proof can be used also to show that if the control input $\vec{u}_i \rightarrow 0$ then the formation tracking error $\|\vec{e}_{ij}\|$ converges to 0.

If the leader is moving with other velocity regimes, then there is a lower bound that the formation tracking error can attain, according to $\vec{u}_i$. In order to ensure that the quadrotors maintain stability within some flight safety requirements, it is crucial to bound the error amplitudes in the worst-case scenario. We use the input-to-state stability (ISS) notion [56, 57, 54] to establish an upper limit for the formation tracking error and an upper limit for the admissible leader input $\vec{u}_i$ that can maintain flight safety at all times. We then use the analysis of the formation's ISS to establish a link between the magnitude of the leader's input and the evolution of the formation tracking errors, i.e., the error dynamics given in Eq. (S6).

Definition 2 (Input-to-State-Stability [56]) *Let a leader-follower pair be input-to-state stable. Then, there is a class $\mathcal{KL}$ function β and a class $\mathcal{K}$ function γ such that, for any initial formation tracking error $\vec{e}_{ij}(0)$ and for any bounded input of the leader $\vec{u}_i(t)$, the solution $\vec{e}_{ij}(t)$ exists for all $0 \leq t$ and satisfies the following inequality [53]:*

$$\|\vec{e}_{ij}(t)\| \leq \beta_{ij}(q, t) + \gamma_{ij}(r), \quad (\text{S9})$$

where the functions $\beta_{ij}(q, t)$ and $\gamma_{ij}(r)$ are transient and asymptotic ISS gain functions, respectively. These functions help to measure the impact of initial conditions and the leader's input on the formation tracking errors [58].

In order to further analyze the ISS properties of the leader-follower pair, we treat the error dynamics given by Eq. (S6) as a perturbed system and derive an upper bound on the error norm $\|\vec{e}_{ij}(t)\|$. This bound provides a measure of the rate at which the formation tracking error converges and indicates that it is ISS with respect to the leader's velocity $\vec{u}_i$ [58, 54, 57]. Using the initial error norm $\|\vec{e}_{ij}(0)\|$ we can then rewrite the inequality Eq. (S9) as

$$\begin{aligned} \|\vec{e}_{ij}(t)\| &\leq \beta_{ij}(\|\vec{e}_{ij}(0)\|, t) + \gamma_{ij}\left(\sup_{0 \leq \tau \leq t} \|\vec{u}_i(\tau)\|\right) \\ &\leq \hat{\beta}_{ij} \|\vec{e}_{ij}(0)\| e^{-\frac{c_3^j}{c_2^{a_j}} t} + \hat{\gamma}_{ij} \sup_{\tau \leq t} \|\vec{u}_i(\tau)\| \end{aligned} \quad (\text{S10})$$

where the terms $\hat{\beta}_{ij}$ and $\hat{\gamma}_{ij}$ are gain estimates that provide insight into the relationship between the initial error, the leader's input, and the observed interconnection errors observed. They are defined as

$$\hat{\beta}_{ij} \triangleq \left(\frac{c_2^j}{c_1^j}\right)^{\frac{1}{a_j}}, \quad \hat{\gamma}_{ij} \triangleq \frac{c_2^j}{c_1^j \theta}. \quad (\text{S11})$$

By substituting the gain estimates from Eq. (S11) into Eq. (S10), we obtain

$$\|\vec{e}_{ij}(t)\| \leq \left(\frac{c_2^j}{c_1^j}\right)^{\frac{1}{a_j}} \|\vec{e}_{ij}(0)\| e^{-\frac{c_3^j}{c_2^{a_j}} t} + \frac{c_2^j}{c_1^j \theta} \sup_{\tau \leq t} \|\vec{u}_i(\tau)\|. \quad (\text{S12})$$

We use the formation ISS measure as a metric to help provide an upper bound on the leader's input and ensure that the formation remains within desired specifications. Additionally, we use it to compare the stability properties of different formation shapes and connection schemes.

Definition 3 (Formation ISS Measure [58]) Consider a leader-follower pair that is ISS with gain functions $\beta_{ij}(q, t)$ and $\gamma_{ij}(r)$. Assume that $\gamma_{ij}(r) \in C^1$ (i.e., it is continuously differentiable) and let $\mathcal{U} \subseteq \mathbb{R}^n$ be a compact neighborhood of the origin containing all $\vec{u}_i \in \mathcal{U}$ that are of interest. The formation tracking error $\vec{e}_{ij}$ always satisfies the following inequality if the leader-follower pair is ISS:

$$\lim_{t \rightarrow \infty} \|\vec{e}_{ij}(t)\| \leq \gamma_{ij}(r). \quad (\text{S13})$$

If we consider a specification such as the first leader's input bounded inside a unit sphere where $r = 1$, we can derive the formation's ISS performance measure $P_{ISS} \in [0, 1]$, based on the performance measure of leader-to-follower stability P_{LFS} in [53], as follows:

$$P_{ISS} \triangleq \frac{1}{\gamma_{ij}(1)}. \quad (\text{S14})$$

Numerical simulations: formation tracking error with bounded leader inputs

Consider a leader-follower pair as depicted in Fig. S36, where the initial positions are $\vec{p}_i(0) = [5, 10]^T$ and $\vec{p}_j(0) = [0, 0]^T$, the desired displacement vector is given as $\vec{d}_{ij} = [3, 4]^T$, and the controller gain matrix $\mathbf{K}^j$ has $k_1^j = 5$ and $k_2^j = 5$.

Remark S2 In the real-robot and simulated experiments in this study, we use a strictly reactive control law; there is no feedforward control nor preview of the reference signal. We use this reactive control with the aim of establishing a performance baseline for the tracking of target positions within a SoNS—in other words, to study the lower bounds of performance that the formation tracking error can attain. For a discussion of incorporating feedforward information in SoNS for improved performance, see the end of this section.

Under a stationary leader (i.e., $\vec{u}_i = [0, 0]$), the formation tracking error exponentially converges to zero (see Fig. S37a). The upper bound defined in Eq. (S10) decays with respect to the initial formation tracking error and the lower bound defined in Eq. (S8) is always 0 (the error norm converges to 0).

For cases of moving leaders, we consider the lower bound that the error can attain under velocity regimes that span the conditions occurring in our real robot experiments. We consider the example error norms $\|e_{ij}\|_1$, $\|e_{ij}\|_2$, and $\|e_{ij}\|_3$, which represent the formation tracking errors under leader velocities $\vec{u}_i^1 = [10, 10]^T$, $\vec{u}_i^2 = [5, 5]^T$, and $\vec{u}_i^3 = [2, 2]^T$, respectively. Under these velocity regimes, the error norms converge to a lower bound (see Fig. S37b). For instance, when the leader velocity is $\vec{u}_i^2$ and $\theta = 0.5$, the norm of the formation tracking error $\|e_{ij}\|_2$ converges to a lower bound around 1.5.

Control of multiple leader-follower pairs

We also assess how stability bounds are propagated in formations composed of multiple leader-follower pairs.

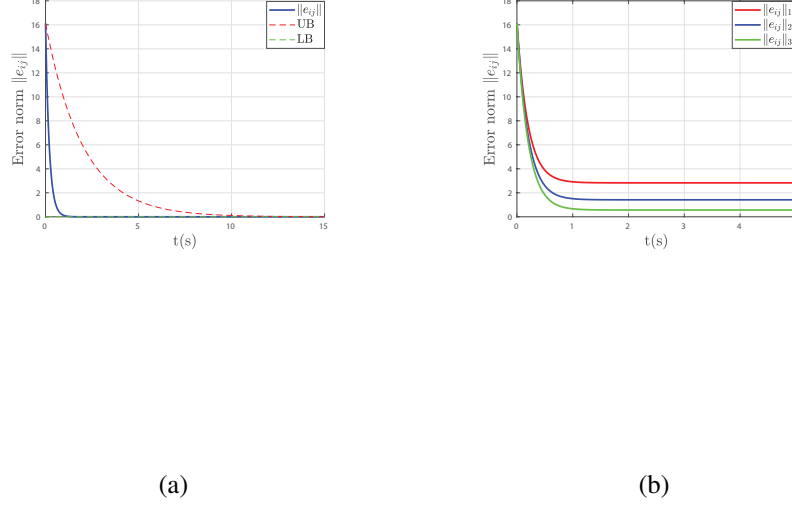


Fig. S37: Norm of the formation tracking error without feed-forward information. (a) Error norm under a stationary leader, with the upper bound (UB) and lower bound (LB). Note that the lower bound line (green) is along the bottom of the graph. (b) Error norm under a moving leader with different velocity regimes.

Graph theory preliminaries

For a SoNS, we define a target topology $G = (V, E, D)$, where $V = \{v_1, \dots, v_n\}$ is a set of vertices, $E = \{\xi_{ij} = (\overrightarrow{v_j}, \overrightarrow{v_i}) \mid v_j, v_i \in V, v_i \neq v_j\}$ is a set of edges, and $D = \vec{p}_j^d$ is a set of formation attributes that includes information such as the desired positions. Each vertex v_i has a set of neighbors $\mathcal{N}_i = \{v_j \in V \mid (i, j) \in E\}$. We also define an adjacency matrix $A_n = [a_{ij}] \in \mathbb{R}^{n \times n}$ to represent the connections between vertices. For example, if vertices v_i and v_j are connected, the corresponding entry a_{ij} in the adjacency matrix will be non-zero. An entry of the adjacency matrix is defined as

$$a_{ij} = \begin{cases} 0, & i = j \\ 0, & (i, j) \notin E \\ 1, & (i, j) \in E \end{cases} \quad (\text{S15})$$

The Laplacian matrix $L = [l_{ij}] \in \mathbb{R}^{n \times n}$ related to the adjacency matrix A for $i \in \{1, 2, \dots, n\}$ and $j \in \{1, 2, \dots, n\}$ is defined as

$$l_{ij} = \begin{cases} \sum_{k=1}^n a_{ik}, & i = j \\ -a_{ij}, & i \neq j \end{cases} \quad (\text{S16})$$

Remark S3 Note that, in the SoNS approach, a directed edge between vertices v_j and v_i represents a communication and control link between the corresponding follower robot R_j and the leader R_i , and the indegree of each vertex is 1 (i.e., each follower has only one leader). Therefore, an n -node

graph has $n - 1$ edges and can be considered an n -robot formation with $n - 1$ pairs of leaders and followers. For each pair, the control law Eq. (S4) drives the follower robot to its desired position. If Theorem S1 is satisfied for all pairs, the formation of n robots will be stable [59].

Propagation of the stability bounds

As we demonstrate in Eq. (S12), a leader-follower pair has input-to-state stability (ISS). Input-to-state stability is preserved in cascaded connections [57], such that ISS bounds are calculated from one robot to another, from the first leader to the last follower of the formation. However, the upper bound of the formation's ISS depends on the initial magnitude of the formation tracking error, $\|\vec{e}_{ij}(0)\|$, and this error term tends to increase as more leader-follower pairs form complex structures. Thus, the formation's ISS depends on the longest path length of information passed from leaders to their respective followers (i.e., the length of the path from the first leader to the last follower) in the formation. Any formation can be constructed from two types of 3-robot primitives: either with cascaded connections or parallel connections. We analyze the ISS properties of these formation primitives.

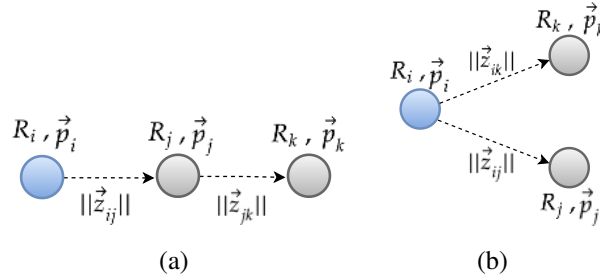


Fig. S38: Formation primitives. (a) Cascaded formation primitive. Robot R_j follows the first leader of the formation R_i , and robot R_k in turn follows robot R_j . Dashed arrows denote information passed from leader to follower. (b) Cascaded formation primitive. Robot R_j follows the first leader of the formation R_i , and robot R_k in turn follows robot R_j . Dashed arrows denote information passed from leader to follower.

In a formation primitive of three robots with cascaded connections (see Fig. S38a), the control laws for each follower robot R_j and R_k are defined as

$$\begin{aligned} \vec{u}_j &= \mathbf{K}^j(\vec{z}_{ij} - \vec{d}_{ij}) = \mathbf{K}^j \vec{e}_{ij} \\ \vec{u}_k &= \mathbf{K}^k(\vec{z}_{jk} - \vec{d}_{jk}) = \mathbf{K}^k \vec{e}_{jk} \end{aligned} \quad (\text{S17})$$

The ISS bounds of each pair can be expressed as follows:

$$\begin{cases} \|\vec{e}_{ij}(t)\| & \leq \beta_{ij}(\|\vec{e}_{ij}(0)\|, t) + \gamma_{ij} \left(\sup_{0 \leq \tau \leq t} \|\vec{u}_i(\tau)\| \right) \\ & \leq \left(\frac{c_2^j}{c_1^j} \right)^{\frac{1}{a_j}} \|\vec{e}_{ij}(0)\| e^{-\frac{c_3^j}{c_2^j a_j} t} + \frac{c_2^j}{c_1^j \theta} \sup_{\tau \leq t} \|\vec{u}_i(\tau)\| \\ \|\vec{e}_{jk}(t)\| & \leq \beta_{jk}(\|\vec{e}_{jk}(0)\|, t) + \gamma_{jk} \left(\sup_{0 \leq \tau \leq t} \|\vec{u}_j(\tau)\| \right) \\ & \leq \left(\frac{c_2^k}{c_1^k} \right)^{\frac{1}{a_k}} \|\vec{e}_{jk}(0)\| e^{-\frac{c_3^k}{c_2^k a_k} t} + \frac{c_2^k}{c_1^k \theta} \sup_{\tau \leq t} \|\vec{u}_j(\tau)\| \end{cases} \quad (\text{S18})$$

The proof of *Proposition III.1* in [53] shows that the error norm $\|\vec{e}_{jk}(t)\|$ between two followers can be expressed in terms of the first leader's velocity $\vec{u}_i$, as follows:

$$\begin{aligned} \|\vec{e}_{jk}(t)\| &\leq \beta_{jk}(\|\vec{e}_{jk}(0)\|, t) + \gamma_{jk} \left(\sup_{0 \leq \tau \leq t} \|\vec{u}_i(\tau)\| \right) \\ \beta_{jk}(\|\vec{e}_{jk}(0)\|, t) &\triangleq \left(\frac{c_2^k}{c_1^k} \right)^{\frac{1}{a_k}} \|\vec{e}_{jk}(0)\| e^{-\frac{c_3^k}{c_2^k a_k} t}, \quad \gamma_{jk} \left(\sup_{0 \leq \tau \leq t} \|\vec{u}_i(\tau)\| \right) \triangleq \frac{c_2^k c_2^j}{c_1^k \theta} \sup_{\tau \leq t} \|\vec{u}_i(\tau)\|. \end{aligned} \quad (\text{S19})$$

To demonstrate that the ISS of the first leader-follower pair (robots R_i and R_j) can be extended to the second pair (robots R_j and R_k) in the cascaded formation primitive in Fig. S38a, we can construct the following composite error vector:

$$\vec{e}_{ik} \triangleq [\vec{e}_{ij} \quad \vec{e}_{jk}]^T. \quad (\text{S20})$$

A system composed of two cascaded ISS systems is also ISS [57, 54]. Therefore, the composite formation tracking error of two systems $\vec{e}_{ik}$ satisfies the inequality

$$\|\vec{e}_{ik}(t)\| \leq \beta_{ik}(\|\vec{e}_{ik}(0)\|, t) + \gamma_{ik} \left(\sup_{0 \leq \tau \leq t} \|\vec{u}_i(\tau)\| \right), \quad (\text{S21})$$

where

$$\begin{aligned} \beta_{ik}(\|\vec{e}_{ik}(0)\|, t) &= \beta_{ik1}(\|\vec{e}_{ik}(0)\|, t) + \beta_{ik2}(\|\vec{e}_{ik}(0)\|, t) \\ \beta_{ik1}(\|\vec{e}_{ik}(0)\|, t) &= \beta_{jk} \left(\left(2\beta_{jk}(\|\vec{e}_{ik}(0)\|, \frac{t}{2}) + \gamma_{jk}(2\beta_{ij}(\|\vec{e}_{ik}(0)\|, \frac{t}{2})) + 2\gamma_{jk}(2\beta_{ij}(\|\vec{e}_{ik}(0)\|, 0)) \right), \frac{t}{2} \right) \\ \beta_{ik2}(\|\vec{e}_{ik}(0)\|, t) &= \beta_{ij}(\|\vec{e}_{ik}(0)\|, t), \end{aligned} \quad (\text{S22})$$

and

$$\begin{aligned} \gamma_{ik} \left(\sup_{0 \leq \tau \leq t} \|\vec{u}_i(\tau)\| \right) &= \gamma_{jk}(2\gamma_{ij}(\sup \|\vec{u}_i(\tau)\|) + 2\sup \|\vec{u}_i(\tau)\|) + \beta_{ik}(2\gamma_{jk}(2\gamma_{ij}(\sup \|\vec{u}_i(\tau)\|) \\ &\quad + 2\sup \|\vec{u}_i(\tau)\|), 0) + \gamma_{ij}(\sup \|\vec{u}_i(\tau)\|). \end{aligned} \quad (\text{S23})$$

Then, Eqs. (S22) and (S23) can be transformed into

$$\begin{aligned} \beta_{ik}(q, t) &= \beta_{jk} \left(2\beta_{jk}(q, \frac{t}{2}) + 2\gamma_{jk}(2\beta_{ij}(q, 0)), \frac{t}{2} \right) + \gamma_{jk} \left(2\beta_{ij}(q, \frac{t}{2}) \right) + \beta_{ij}(q, t) \\ \gamma_{ik}(r) &= \gamma_{jk}(2\gamma_{ij}(r) + 2r) + \beta_{ik}(2\gamma_{jk}(2\gamma_{ij}(r)) + 2r, 0) + \gamma_{ij}(r), \end{aligned} \quad (\text{S24})$$

where $q = \|\vec{e}_{ik}(0)\|$ and $r = \sup \|\vec{u}_i(\tau)\|$.

Because a system composed of two ISS systems is also ISS [57, 54], the composite formation tracking error satisfies the inequality

$$\|\vec{e}_{ijk}(t)\| \leq \beta_{ijk}(\|\vec{e}_{ijk}(0)\|, t) + \gamma_{ijk} \left(\sup_{0 \leq \tau \leq t} \|\vec{u}_i(\tau)\| \right), \quad (\text{S25})$$

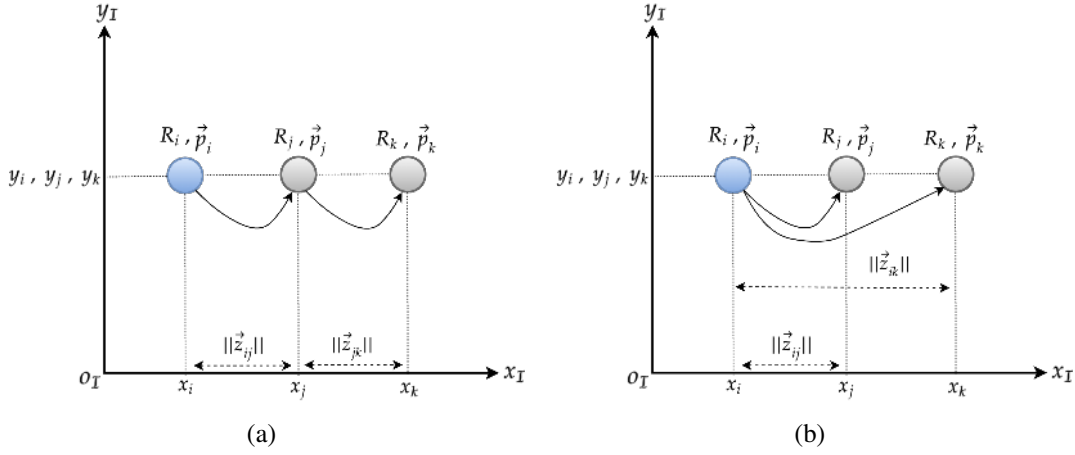


Fig. S39: Two example formations of three robots. The arrows denote graph connections, where information is passed from leader to follower. (a) Cascaded connections. (b) Parallel connections.

where

$$\begin{aligned} \beta_{ijk}(\|\vec{e}_{ijk}(0)\|, t) &= \beta_{ij}(\|\vec{e}_{ij}(0)\|, t) + \beta_{ik}(\|\vec{e}_{ik}(0)\|, t) \\ \gamma_{ijk} \left(\sup_{0 \leq \tau \leq t} \|\vec{u}_i(\tau)\| \right) &= \gamma_{ij}(\sup \|\vec{u}_i(\tau)\|) + \gamma_{ik}(\sup \|\vec{u}_i(\tau)\|). \end{aligned} \quad (\text{S26})$$

Then, Eq. (S26) can be rewritten as

$$\begin{aligned} \beta_{ijk}(q, t) &= \beta_{ij}(q, t) + \beta_{ik}(q, t) \\ \gamma_{ijk}(r) &= \gamma_{ij}(r) + \gamma_{ik}(r). \end{aligned} \quad (\text{S27})$$

In a formation primitive of three robots with parallel connections (see Fig. S38b), both follower robots R_j and R_k can be assumed to be equivalent to the first follower robot R_j in the cascaded formation primitive shown in Fig. S38a.

Numerical simulation of error norm between the first leader and the first follower

We assume that both formations move along the y -axis. The closed-loop kinematics for the connections can be given as

$$\begin{aligned} \vec{u}_j &= \mathbf{K}^j(\vec{z}_{ij} - \vec{d}_{ij}) \\ \vec{u}_k &= \mathbf{K}^k(\vec{z}_{jk} - \vec{d}_{jk}) \end{aligned} \quad (\text{S28})$$

where $\mathbf{K}^j$ and $\mathbf{K}^k$ are controller gain matrices and $k_1^j = k_2^j = k_1^k = k_2^k = 5$. Note that the initial positions of the robots are $\vec{p}_i(0) = [5, 10]^T$, $\vec{p}_j(0) = [1, 3]^T$, and $\vec{p}_k(0) = [3, -2]^T$. We set the constant reference velocity and desired displacement vectors as $\vec{u}_i = [0, 10]^T$, $\vec{d}_{ij} = [6, 0]^T$, $\vec{d}_{jk} = [6, 0]^T$, $\vec{d}_{ik} = [12, 0]^T$.

To calculate the formation ISS measure P_{ISS} for the cascaded connections (shown in Fig. S39a), we redefine the asymptotic ISS gain Eq. (S24) by setting $r = 1$, $q = 1$, and $t = 0$. This bounds the

inputs of the first leader inside the unit ball and ensures that P_{ISS} varies in the range of $[0, 1]$. This results in the following expression:

$$\begin{aligned}
 \gamma_{ik}(1) &= \gamma_{jk}(2\gamma_{ij}(1) + 2) + \beta_{ik}(2\gamma_{jk}(2\gamma_{ij}(1) + 2), 0) + \gamma_{ij}(1) \\
 &= \gamma_{jk}(2\hat{\gamma}_{ij} + 2) + \beta_{jk}(2\gamma_{jk}(2\hat{\gamma}_{ij} + 2), 0) + \hat{\gamma}_{ij} \\
 &= 2\hat{\gamma}_{jk}\hat{\gamma}_{ij} + 2\hat{\gamma}_{jk} + \beta_{jk}(4\hat{\gamma}_{jk}\hat{\gamma}_{ij} + 4\hat{\gamma}_{jk}), 0) + \hat{\gamma}_{ij} \quad , \quad (S29) \\
 &= 2\hat{\gamma}_{jk}\hat{\gamma}_{ij} + 2\hat{\gamma}_{jk} + (4\hat{\beta}_{jk}\hat{\gamma}_{jk}\hat{\gamma}_{ij} + 4\hat{\beta}_{jk}\hat{\gamma}_{jk})q + \hat{\gamma}_{ij} \\
 &= 2\hat{\gamma}_{jk}\hat{\gamma}_{ij} + 2\hat{\gamma}_{jk} + 4\hat{\beta}_{jk}\hat{\gamma}_{jk}\hat{\gamma}_{ij} + 4\hat{\beta}_{jk}\hat{\gamma}_{jk} + \hat{\gamma}_{ij}
 \end{aligned}$$

where $\hat{\beta}_{ij} \triangleq \left(\frac{c_2^j}{c_1^j}\right)^{\frac{1}{a_j}}$, $\hat{\beta}_{jk} \triangleq \left(\frac{c_2^k}{c_1^k}\right)^{\frac{1}{a_k}}$, $\hat{\gamma}_{ij} \triangleq \frac{c_2^j}{c_1^j\theta}$, $\hat{\gamma}_{jk} \triangleq \frac{c_2^k}{c_1^k\theta}$, $c_1^j \triangleq \min(k_1^j, k_2^j)$, $c_2^j \triangleq \max(k_1^j, k_2^j)$, $c_1^k \triangleq \min(k_1^k, k_2^k)$, $c_2^k \triangleq \max(k_1^k, k_2^k)$, and $a_j = a_k = 2$. This results in the following P_{ISS} value:

$$P_{ISS}^c = \frac{1}{1 + \gamma_{ik}(1)} = \frac{\theta^2}{\theta^2 + 7\theta + 6} \quad , \quad (S30)$$

where $\theta \in (0, 1)$.

The follower robots R_j and R_k with parallel connections (see Fig. S38b) can be assumed to be equivalent to the first follower robot R_j with cascaded connections (see Fig. S38a). A follower that is directly connected to the first leader has a lower magnitude of relative errors with respect to the first leader's velocity than a follower that is instead connected to another follower. Fig. S40a shows a comparison of the error norm for the first leader and its first follower in the two formations. In both formations, R_j is in the first hierarchy layer and therefore the error norms $\|e_{ij}\|$ for R_j are the same. However, in the cascaded connections, robot R_k follows R_j , while in the parallel connections, R_k directly follows R_i . As a result, the error norm $\|e_{ik}\|$ for R_k in the cascaded connections is higher than that for all other followers in both formations.

ISS gains calculation for n -robot formations

To obtain the total ISS gains for a formation of multiple leader-follower pairs, we start from the terminal nodes (i.e., vertices with outdegree of 0) and apply Eqs. (S22), (S23), and (S26) iteratively based on the algorithm given in [53], thus reducing the original graph to a depth of one.

To illustrate this calculation for a specific target topology, consider a graph $G = (V, E, D)$ with an $n \times n$ adjacency matrix A , where a_i represents the i -th row. Begin by defining the following vectors:

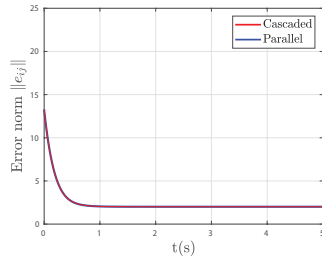
$$\hat{\beta}^0 \triangleq [\hat{\beta}_1^0 \dots \hat{\beta}_n^0]^T \quad (S31)$$

$$\hat{\gamma}^0 \triangleq [\hat{\gamma}_1^0 \dots \hat{\gamma}_n^0]^T \quad , \quad (S32)$$

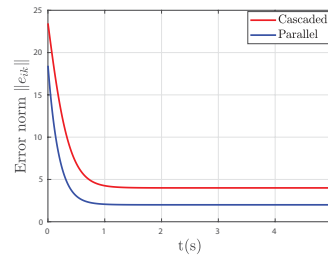
where $\hat{\beta}$ and $\hat{\gamma}$ are the gain estimates defined in Eq. (S11).

After $k + 1$ iterations, we obtain:

$$\begin{aligned}
 \hat{\beta}^{k+1} &\triangleq [\hat{\beta}_1^{k+1} \dots \hat{\beta}_n^{k+1}]^T \\
 \hat{\gamma}^{k+1} &\triangleq [\hat{\gamma}_1^{k+1} \dots \hat{\gamma}_n^{k+1}]^T \quad . \quad (S33)
 \end{aligned}$$



(a)



(b)

Fig. S40: Comparison of two formations' tracking error evolutions. (a) Error norm between the first leader R_i and first follower R_j . Note that the lines for the cascaded and parallel cases are the same (i.e., the red and blue lines of the graph are in the same position, and therefore not fully visible). (b) Error norm between the first leader R_i and last robot R_k .

Then, $\hat{\beta}^{k+1}$ and $\hat{\gamma}^{k+1}$ can be calculated as

$$\begin{aligned}
 \hat{\beta}^{k+1} &= \hat{\beta}^k + \eta_{n-k} c_{\beta}^k \\
 \hat{\gamma}^{k+1} &= \hat{\gamma}^k + \eta_{n-k} c_{\gamma}^k \\
 \eta_{n-k} &= \underbrace{[0 \ \dots \ 0]}_{n-k-1} [1 \ \dots \ 0]^T \\
 c_{\beta}^k &= a_{n-k} \hat{\beta}^k a_{n-k} \hat{\gamma}^k \hat{\beta}_{n-k}^k + (a_{n-k} \hat{\beta}^k)^2 \\
 c_{\gamma}^k &= a_{n-k} \hat{\beta}^k a_{n-k} \hat{\gamma}^k \hat{\gamma}_{n-k}^k + a_{n-k} \hat{\gamma}^k \hat{\gamma}_{n-k}^k
 \end{aligned} \tag{S34}$$

For any formation, the algorithm that is iteratively applied, Eqs. (S22), (S23), and (S26), will terminate in at most $n - 1$ steps (i.e., the maximum path length in a graph with n vertices).

It is important to note that the depth of a formation's graph will affect its stability: the higher the depth of the graph, the larger the ISS gains will be. Similarly, the depth of the graph will affect its robustness [60] in response to noisy local information (i.e., under random disturbances in the information transferred between robots).

Discussion: incorporating feed-forward information for improved performance

If a follower robot receives velocity information (feed-forward) from its leader without any time delay, the control law Eq. (S3) can be rewritten as follows:

$$\begin{aligned}
 \vec{u}_j &= f(t, \vec{z}_{ij}, \vec{d}_{ij}) + \vec{u}_i \\
 &= \mathbf{K}^j (\vec{z}_{ij} - \vec{d}_{ij}) + \vec{u}_i, \\
 &= \mathbf{K}^j \vec{e}_{ij} + \vec{u}_i
 \end{aligned} \tag{S35}$$

where the first term in the control input is used to maintain the desired displacement from the leader and the second term is used to follow the leader's reference velocity.

If we substitute the control law given by Eq. (S35) into the equation for the motion model given by Eq. (S1), we obtain

$$\dot{\vec{p}}_j = \mathbf{K}^j \vec{e}_{ij} + \vec{u}_i. \tag{S36}$$

Then, the leader-follower kinematics given by Eq. (S2) can be rewritten as

$$\dot{\vec{e}}_{ij} = -\mathbf{K}^j \vec{e}_{ij}. \tag{S37}$$

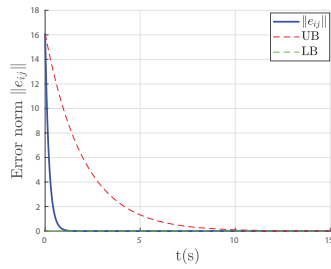
By solving the differential equation Eq. (S37), we then obtain

$$\vec{e}_{ij}(t) = \mathcal{C} e^{-(\mathbf{K}^j)^T t}, \tag{S38}$$

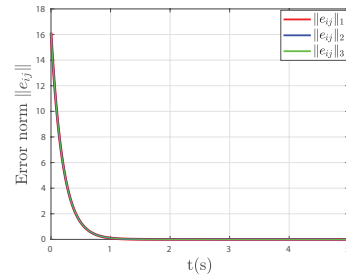
where $\mathcal{C}$ is any positive constant. Since $(\mathbf{K}^j)^T = \mathbf{K}^j$ is positive definite, then $\|\vec{e}_{ij}(t)\| \rightarrow 0$. As $t \rightarrow \infty$, $\dot{\vec{p}}_j$ (or $\vec{u}_j$) approaches $\vec{u}_i$, such that the leader can steer the follower with the velocity $\vec{u}_i$.

Remark S4 Unlike the control law given by Eq. (S4), the control law in Eq. (S35) is not influenced by the leader's velocity, due to the use of feed-forward information. This means that the stability of the leader-follower pair is not affected by whether the leader is stationary or moving.

To demonstrate the benefit of using feed-forward information, we conduct a simulation using the same parameters as the previous case, but with the follower robot receiving the leader's velocity information without any delay. The results in Fig. S41a,b, demonstrate that the error norm indeed approaches zero regardless of the leader's velocity.



(a)



(b)

Fig. S41: Norm of the relative formation tracking error with feed-forward information. (a) Stationary leader with the upper bound (UB) and lower bound (LB) of the error norm. Note that the lower bound line (green) is along the bottom of the graph. (b) Moving leader under different velocity regimes. Note that the lines for the three velocity regimes are the same (i.e., the red, blue, and green lines of the graph are all in the same position, and therefore not fully visible).

Section S4. SoNS control algorithm details

The open-source code for the SoNS control algorithm and all experiment setups is available in an online repository.

SoNS preliminaries

Graph preliminaries

A SoNS is a directed rooted tree, denoted by a directed graph $G = \{V, E\}$ with sets of attributes A_V and A_E associated to V and E respectively, where V represents the set of robots r_i ($|V| = n$), E represents the links e_{ij} between parent robots r_i and child robots r_j , and the robot at the root node r_1 is the SoNS-brain. In other words, a parent of robot r_j is the robot immediately upstream from r_j in the SoNS graph G , and similarly, a child of robot r_i is a robot immediately downstream from r_i in the SoNS graph G . The subgraph of graph G that includes robot r_i and all the robots downstream from it (i.e., all of its children, all of its children's children, and so on) is denoted as graph H_{r_i} .

The set of children of a robot r_i is denoted by C_{r_i} and the set of all robots that are directly connected (whether child or parent) to robot r_i is denoted by F_{r_i} . Note that, because the SoNS graph G is a tree (i.e., a connected acyclic graph), a robot cannot have more than one parent and no cycles are present in the SoNS.

The set of attributes A_V associated to V includes one categorical variable $\text{TYPE}_i = \tau_\ell$ for each robot r_i , where $\tau_\ell \in \{\tau_1, \tau_2, \dots, \tau_n\}$.² The set of attributes A_E associated to E includes two attributes for each link e_{ij} , which are: the relative position of r_j w.r.t. r_i , denoted by the displacement vector $\mathbf{d}_{r_i r_j}$, and the relative orientation of r_j w.r.t. r_i , denoted by the unit quaternion $\mathbf{q}_{r_i r_j}$.

The target subgraph that a robot r_i is attempting to have built downstream from it is denoted as $H_{r_i}^*$, and its components are denoted likewise (e.g., a target robot node is denoted as r_i^* , a target link as e_{ij}^* , and a target displacement as $\mathbf{d}_{r_i r_j}^*$). Note that the target subgraph $H_{r_1}^*$ of the SoNS-brain at root node r_1 is equivalent to G^* , which denotes the overall target graph of a SoNS.

Remark S5 *No robot in a SoNS has access to the graph $G = \{V, E\}$ and attributes A_V, A_E that would represent the current state of the SoNS (nor the subgraph H that would represent the current state of any branch of the SoNS) at any point in time. The current SoNS graph G and the overall target graph G^* are used only for analysis of the experiment results (see calculations of error and lower bound in Sec. 4.2 in the main paper). However, each robot r_i has access to the target subgraph $H_{r_i}^*$ that it is attempting to have built downstream from it. Each robot also tracks the attributes of some of its immediate neighbors, i.e., robots in F_{r_i} , using strictly local communication and local sensing.*

Robot initialization requirements

Independent of a SoNS, each robot is required to initialize with three static identifiers: a non-unique robot type $\text{ROBOTTYPE} = \tau_\ell$, where $\tau_\ell \in \{\tau_1, \tau_2, \dots, \tau_n\}$; a unique robot identifier $\text{ROBOTID} \in$

²In our current implementation, because we use aerial and ground robots and do not make other distinctions in robot type, TYPE is a binary condition, where $\tau_\ell \in \{\tau_1, \tau_2\}$.

$\{1, 2, \dots, n\}$; and a unique robot rank `ROBOTRANK` that can be assigned manually or can be assigned randomly according to the uniform distribution $U(0, 1)$.³ Note that `ROBOTTYPE`, `ROBOTID` and `ROBOTRANK` are all independent of a robot's vertex position r_i in the SoNS graph G .

The SoNS control algorithm considers all robots to have the same motion and robot-to-robot sensing capabilities. All robots are required to accept omnidirectional motion control inputs. All robots are also required to be capable of sensing the relative position, relative orientation, and robot identifier `ROBOTIDi` of robots in their sensing range (e.g., using computer vision and unique fiducial markers) and to be capable of mutual sensing (i.e., if robot A can sense robot B, then robot B can also sense robot A). If not all of the robot platforms being used have all of these features by default, then any gap in capability and/or heterogeneity of robot platforms is handled by external control layers that are specific to the individual robots (see Secs. S6 and S7 respectively for the aerial and ground robot control layers used in this study).

Each robot initializes as r_1 of a graph G with one node and no links. In other words, it initializes as the SoNS-brain of its own single-robot SoNS. Each robot also initializes with a default target $H_{r_1}^*$, which is equivalent to G^* , and with a certain configuration of the SoNS control algorithm to use as its local copy, detailed below.

Self-organized node attributes

Each robot updates its individual node attributes at each time step in a self-organized way, using strictly local communication and local sensing. Because the shared information used to inform these node attributes is updated asynchronously, the node attributes are usually updated with some delay. Using the SoNS control algorithm, the desired overall SoNS behaviors execute correctly without all (or even most) of the node attributes having to be fully up-to-date.

SoNS identifier and SoNS rank

The SoNS identifier of robot r_i is denoted as `SONSROOTIDi`. The SoNS identifier `SONSROOTIDi` of robot r_i is equivalent to the robot identifier of its respective SoNS-brain, i.e., of the root of graph G , denoted as `ROBOTID( $r_1$ )`. At each time step, each child robot r_j receives a SoNS identifier `SONSROOTIDi` from its parent and updates its own `SONSROOTIDj` $\leftarrow$ `SONSROOTIDi` accordingly, which it will then send to its own children, if it has any. If a robot r_i has no parent (i.e., it is the SoNS-brain), then it takes its own robot identifier as its SoNS identifier, such that `SONSROOTIDi` $\leftarrow$ `ROBOTID( $r_i$ )`.

Similarly to the SoNS identifier `SONSROOTID`, `SONSROOTRANKi` denotes the SoNS rank of robot r_i and is equivalent to the robot rank of its respective SoNS-brain `ROBOTRANK( $r_1$ )`. Like the SoNS identifier, each robot takes the SoNS rank received from its parent (Eq. S39) or takes its own robot rank as its SoNS rank if it has no parent (Eq. S40):

$$\text{SONSROOTRANK}_j \leftarrow \text{SONSROOTRANK}_i, \quad (\text{S39})$$

³In our current implementation, the robot ranks are normally generated randomly, with the generated number having 15 decimal digits of precision (double precision floating point) and the seed used for the pseudorandom generator being the microsecond of the UTC time at the moment of initialization. For safety considerations in the constrained indoor robot arena, we sometimes manually assign a robot the rank of 1, which helps ensure that the self-organized processes of the SoNS do not result in robots getting too close to the boundary of the indoor arena, see below for details.

$$\text{SONSROOTRANK}_i \leftarrow \text{ROBOTRANK}(r_i) . \quad (\text{S40})$$

In other words, every robot in a SoNS has the same SoNS identifier and SoNS rank, once all robots' attributes are up-to-date.

Note that, when the robot occupying the role of SoNS-brain changes, the SoNS identifier and SoNS rank of robot r_i will take μ_1 time steps to reflect the change, where μ_1 is equal to the path length between robot r_i and the new SoNS-brain.

Each robot r_i also stores its most recent former SoNS identifier, if it has any former ones, denoted by $\text{SONSROOTID}_i^{\text{OLD}}$, and the number of time steps t_i^{OLD} passed since it had that former SoNS identifier, according to its own internal clock.

Downstream vertex cardinality and vertex height

Recall that the subgraph H_{r_i} is the subgraph of robot r_i and all robots downstream from it. The vertex cardinality (i.e., the order) of the subgraph H_{r_i} of robot r_i is denoted by $\lambda^{\text{CARD}}(r_i)$.

The cardinality of vertices in subgraph H_{r_i} that have the vertex attribute $\text{TYPE}_i = \tau_\ell$ is denoted by $\lambda_{\tau_\ell}^{\text{CARD}}(r_i)$, and is defined as

$$\lambda_{\tau_\ell}^{\text{CARD}}(r_i) = |r_j \in V(H_{r_i}) : \text{TYPE}_j = \tau_\ell| , \quad (\text{S41})$$

where $V(H_{r_i})$ denotes the set of vertices of H_{r_i} . At each time step, each child robot r_j sends its parent r_i its respective $\lambda_{\tau_\ell}^{\text{CARD}}(r_j)$ values—one for each τ_ℓ represented in the range set $\{A_V(\text{TYPE}) : \text{TYPE} = \tau_1, \dots, \tau_n\}$ of its subgraph H_{r_j} . In other words, each robot r_i that has children receives at least one $\lambda_{\tau_\ell}^{\text{CARD}}(r_j)$ value from each child. Each robot r_i calculates its own $\lambda_{\tau_\ell}^{\text{CARD}}(r_i)$ value(s) at each time step as

$$\lambda_{\tau_\ell}^{\text{CARD}}(r_i) \leftarrow \begin{cases} \sum_{r_j \in C_{r_i}} \lambda_{\tau_\ell}^{\text{CARD}}(r_j) + 1 & \text{if } \text{TYPE}_i = \tau_\ell \\ \sum_{r_j \in C_{r_i}} \lambda_{\tau_\ell}^{\text{CARD}}(r_j) & \text{otherwise} \end{cases} , \quad (\text{S42})$$

which it will then send to its own parent, if it has one.

The vertex height (i.e., the longest path from the respective vertex to any downstream leaf node) of robot r_i is denoted as $\lambda^{\text{HIGH}}(r_i)$, defined as

$$\lambda^{\text{HIGH}}(r_i) = |[r_i, \dots, r_\alpha]| , \quad (\text{S43})$$

where r_α is the furthest leaf node from r_i in subgraph H_{r_i} . The vertex height $\lambda^{\text{HIGH}}(r_i)$ is updated similarly to the downstream vertex cardinality $\lambda_{\tau_\ell}^{\text{CARD}}(r_i)$. At each time step, each child robot r_j sends its vertex height $\lambda^{\text{HIGH}}(r_j)$ to its parent and each robot r_i calculates its own $\lambda^{\text{HIGH}}(r_i)$ value as

$$\lambda^{\text{HIGH}}(r_i) \leftarrow \max(\lambda^{\text{HIGH}}(r_j) | r_j \in C_{r_i}) + 1 , \quad (\text{S44})$$

which it will then send to its own parent, if it has one.

Thus, if a robot r_i with type τ_ℓ has no children, its $\lambda_{\tau_\ell}^{\text{CARD}}(r_i) = 1$ and its $\lambda^{\text{HIGH}}(r_i) = 1$. Note that, when a change occurs to the subgraph H_{r_i} , the downstream vertex cardinality and vertex height of robot r_i will take at least μ_2 time steps to reflect the change, where μ_2 is equal to the path length between robot r_i and the closest added or removed robot. In some cases, more time steps might be needed, if the closest added or removed robot also has experienced simultaneous changes to its own downstream subgraph.

SoNS targets

Each robot r_i always has a target subgraph $H_{r_i}^*$ that it is attempting to have built downstream from it. At each time step, each child robot r_j receives its target subgraph $H_{r_j}^*$ from its parent r_i . If a robot r_i has no parent (i.e., it is the SoNS-brain), then it uses its default target subgraph $H_{r_i}^*$. Also at each time step, each robot r_i that has at least one target child $r_j^* \in V(H_{r_i}^*)$ subdivides its $H_{r_i}^*$ into a new target subgraph for each target child r_j^* , as follows:

$$H_{r_j}^* = \left\{ V(H_{r_j}^*), E(H_{r_j}^*) \right\} = \left\{ R_{H_{r_i}^*}^+(r_j^*) \cup r_j^* \in V(H_{r_i}^*), e_{ij} | i \in V(H_{r_j}^*) \right\} \quad (\text{S45})$$

where $R_{H_{r_i}^*}^+(r_j^*)$ denotes all vertices in the directed graph $H_{r_i}^*$ reachable from vertex r_j^* , i.e., all downstream vertices. When robot r_i becomes connected to a new child r_j that matches its target $r_j^* \in V(H_{r_i}^*)$, it sends the child the respective target subgraph $H_{r_j}^*$.

From its own target subgraph $H_{r_i}^*$ and the target subgraphs $H_{r_j}^*$ it calculates for its children, each robot r_i also calculates the target downstream vertex cardinality and target vertex height for itself, $\lambda_{\tau_\ell}^{\text{CARD}}(r_i)^*$ and $\lambda^{\text{HIGH}}(r_i)^*$, and for each of its children, $\lambda_{\tau_\ell}^{\text{CARD}}(r_j)^*$ and $\lambda^{\text{HIGH}}(r_j)^*$.

Neighbor information

For each of its current children $r_j \in C_{r_i}$, robot r_i stores the robot identifier $\text{ROBOTID}(r_j)$, vertex position r_j in subgraph H_{r_i} , and the most recent downstream vertex cardinality $\lambda_{\tau_\ell}^{\text{CARD}}(r_j)$ and vertex height $\lambda^{\text{HIGH}}(r_j)$ values it has received.

At each time step, robot r_i also stores the most recent displacement $\mathbf{d}_{r_i r_j}$ and the relative orientation $\mathbf{q}_{r_i r_j}$ for each current child or parent robot $r_j \in F_{r_i}$, according to its own sensor information. Recall that the target subgraph $H_{r_i}^*$ of robot r_i includes the target displacement $\mathbf{d}_{r_i r_j}^*$ and the target relative orientation $\mathbf{q}_{r_i r_j}^*$ w.r.t. robot r_i for each child $r_j \in F_{r_i}$. Thus, the robot r_i also has access to this information for its children (but not for its parent).

Hierarchically-organized sensor information

When a child robot r_j senses an environmental feature a , it uses its own sensor information and local decisions to determine whether to respond to this feature itself (detailed in the next subsection), whether to send this information upstream to be considered by its parent r_i , or both. At each time step that r_j sends feature information to its parent r_i , it sends a displacement $\mathbf{d}_{r_j a}$ and relative orientation $\mathbf{q}_{r_j a}$ for each feature a w.r.t. r_j . At each time step that r_i receives information about a feature a from its child, it first converts the information into its own coordinate system, such that

$$\begin{aligned} \mathbf{d}_{r_i a} &= \mathbf{d}_{r_j a} + \text{RT}(\mathbf{d}_{r_j a}, \mathbf{q}_{r_i r_j}) \\ \mathbf{q}_{r_i a} &= \text{H}(\mathbf{q}_{r_i r_j}, \mathbf{q}_{r_j a}) \end{aligned} \quad (\text{S46})$$

where $\text{RT}(\mathbf{x}, \mathbf{y})$ is a function to rotate vector $\mathbf{y}$ by unit quaternion $\mathbf{x}$ using the Euler–Rodrigues formula, with the Euler parameters given by the coefficients of quaternions $\mathbf{y}^p = (0, \mathbf{y})$ and $\mathbf{x}$, and $\text{H}(\mathbf{x}, \mathbf{y})$ takes the Hamilton product of two quaternions $\mathbf{x}$ and $\mathbf{y}$. Then, robot r_i makes its own decision about whether to respond to this feature, forward it upstream to be considered by its respective parent, or both.

Hierarchically-organized actuation instructions

In the current SoNS implementation, all actuation is motion-based. Motion instructions are communicated asynchronously in a SoNS using six reference vectors, which each robot then uses to calculate its motion control outputs at each time step.

Each robot r_i has three linear velocity reference vectors $\mathbf{v}_{r_i}^{\text{LOCAL}}$, $\mathbf{v}_{r_i}^{\text{HIERARCHICAL}}$, and $\mathbf{v}_{r_i}^{\text{GLOBAL}}$; and three angular velocity reference vectors $\omega_{r_i}^{\text{LOCAL}}$, $\omega_{r_i}^{\text{HIERARCHICAL}}$, and $\omega_{r_i}^{\text{GLOBAL}}$.

The reference vectors $\mathbf{v}_{r_i}^{\text{LOCAL}}$ and $\omega_{r_i}^{\text{LOCAL}}$ are dedicated to local goals and can only be updated by r_i according to its own sensor information and local decisions. At each time step, robot r_i can either set its $\mathbf{v}_{r_i}^{\text{LOCAL}} = [0, 0, 0]$ or define its $\mathbf{v}_{r_i}^{\text{LOCAL}}$ according to the displacement of a target $\mathbf{d}_{r_i a}$ based on its own sensor information and its goal to either reach or avoid some position, object, or other robot in the environment, such that

$$\mathbf{v}_{r_i}^{\text{LOCAL}} = \begin{cases} \mathbf{v}^{\text{MAXIMUM}} \times \widehat{\mathbf{d}_{r_i a}} & \text{if } \delta < k_1 \\ \min \left(\mathbf{v}^{\text{MAXIMUM}}, \lim_{\delta \rightarrow k_1} -\log(\delta) \times \frac{\delta - k_1}{k_3} \times k_2 \right) \times \widehat{\mathbf{d}_{r_i a}} & \text{if } k_1 < \delta < k_3 \\ [0, 0, 0] & \text{otherwise} \end{cases} \quad (\text{S47})$$

where $\mathbf{v}^{\text{MAXIMUM}}$ is a vector constant, $\widehat{\mathbf{d}_{r_i a}}$ denotes the unitized vector $\mathbf{d}_{r_i a}$, δ is the Euclidean distance between the robot r_i and the position that it is targeting to be reached or avoided, and k_1 , k_2 , k_3 are scalar constants, with all constants to be defined in the SoNS control algorithm before robots are initialized. Then, the relative orientation of a target $\mathbf{q}_{r_i a}$ is translated into an angular vector and an equivalent of Eq. S47, where ω^{MAXIMUM} and $\widehat{\mathbf{q}_{r_i a}}$ are used instead of $\mathbf{v}^{\text{MAXIMUM}}$ and $\widehat{\mathbf{d}_{r_i a}}$ respectively, is used by r_i to calculate $\omega_{r_i}^{\text{LOCAL}}$.

The reference vectors $\mathbf{v}_{r_i}^{\text{HIERARCHICAL}}$ and $\omega_{r_i}^{\text{HIERARCHICAL}}$ are dedicated to global goals that are defined by the SoNS-brain and can only be updated by r_i according to information received from its parent, in the following way. At each time step, for each child robot r_j , a parent robot r_i first converts the target displacement $\mathbf{d}_{r_i r_j}^*$ and target relative orientation $\mathbf{q}_{r_i r_j}^*$ into the coordinate system of r_j , producing the new target displacement $\mathbf{d}_{r_j r_j'}^*$ and target relative orientation $\mathbf{q}_{r_j r_j'}^*$, which it then sends to its child r_j . The target displacement is then used by r_j to calculate $\mathbf{v}_{r_j}^{\text{HIERARCHICAL}}$, such that

$$\mathbf{v}_{r_j}^{\text{HIERARCHICAL}} = \begin{cases} \mathbf{v}^{\text{DEFAULT}} \times \widehat{\mathbf{d}_{r_j r_j'}^*} & \text{if } |\mathbf{d}_{r_j r_j'}^*| > k_4 \\ \mathbf{v}^{\text{DEFAULT}} \times \frac{(|\mathbf{d}_{r_j r_j'}^*| - k_5)}{k_4} \times \widehat{\mathbf{d}_{r_j r_j'}^*} & \text{if } k_5 < |\mathbf{d}_{r_j r_j'}^*| < k_4 \\ [0, 0, 0] & \text{otherwise} \end{cases}, \quad (\text{S48})$$

where $\mathbf{v}^{\text{DEFAULT}}$ is a vector constant, $\widehat{\mathbf{d}_{r_j r_j'}^*}$ denotes the unitized vector $\mathbf{d}_{r_j r_j'}^*$, and k_4 and k_5 are scalar constants, with all constants to be defined in the SoNS control algorithm before robots are initialized. Then, an equivalent of Eq. S48, where ω^{DEFAULT} and $\mathbf{q}_{r_j r_j'}^*$ are used instead of $\mathbf{v}^{\text{DEFAULT}}$ and $\mathbf{d}_{r_j r_j'}^*$ respectively, is used by r_j to calculate $\omega_{r_j}^{\text{HIERARCHICAL}}$.

The reference vectors $\mathbf{v}_{r_i}^{\text{GLOBAL}}$ and $\omega_{r_i}^{\text{GLOBAL}}$ are dedicated to global goals defined by any robot in the same SoNS as r_i and can be updated according to the sensor information of r_i or according to messages received from any robot that r_i is connected to (i.e., parent or child). Likewise, robot r_i can also send messages of this type to any robot it is connected to. At each time step, each robot r_i first converts any $\mathbf{v}_{r_j}^{\text{GLOBAL}}$ it has received from a robot r_j (whether parent or child) into its own coordinate system, using the current relative orientation $\mathbf{q}_{r_i r_j}$ it has stored for robot r_j , such that

$$\mathbf{v}_{r_i}^{\text{GLOBAL}} = \text{RT}(\mathbf{v}_{r_j}^{\text{GLOBAL}}, \mathbf{q}_{r_i r_j}). \quad (\text{S49})$$

Robot r_i then sends the produced $\mathbf{v}_{r_i}^{\text{GLOBAL}}$ to any robot $r_j \in F_{r_i}$ from which it did not originally receive the respective $\mathbf{v}_{r_j}^{\text{GLOBAL}}$. If a robot r_i receives $\mathbf{v}_{r_j}^{\text{GLOBAL}}$ vectors from multiple robots r_j in one time step, it sums all its $\mathbf{v}_{r_i}^{\text{GLOBAL}}$ from that time step, producing a new $\mathbf{v}_{r_i}^{\text{GLOBAL}'}$ for its own use. Likewise, the same series of operations is applied to $\boldsymbol{\omega}_{r_j}^{\text{GLOBAL}}$ to produce $\boldsymbol{\omega}_{r_i}^{\text{GLOBAL}'}$.

If a robot is currently a SoNS-brain, it only uses reference vectors $\mathbf{v}_{r_i}^{\text{GLOBAL}}$ and $\boldsymbol{\omega}_{r_i}^{\text{GLOBAL}}$ (all other reference vectors are set to $[0, 0, 0]$).

At each time step, the SoNS control algorithm implemented on robot r_i produces two motion control outputs. All robots, regardless of their type, receive the same style of omnidirectional motion control outputs from the SoNS control algorithm. These omnidirectional outputs are translated into the appropriate motor inputs by the motion control layer of the individual robot (see Secs. S6 and S7, respectively, for details of the motion control layer of the aerial robots and ground robots used in this study).

The motion control outputs that each robot r_i produces are a target linear velocity vector $\mathbf{v}_{r_i}^*$ and target angular velocity vector $\boldsymbol{\omega}_{r_i}^*$, updated at each time step according to the current reference vectors of robot r_i , as follows:

$$\begin{aligned}\mathbf{v}_{r_i}^* &= \mathbf{v}_{r_i}^{\text{HIERARCHICAL}} + \mathbf{v}_{r_i}^{\text{LOCAL}} + \mathbf{v}_{r_i}^{\text{GLOBAL}} \\ \boldsymbol{\omega}_{r_i}^* &= \boldsymbol{\omega}_{r_i}^{\text{HIERARCHICAL}} + \boldsymbol{\omega}_{r_i}^{\text{LOCAL}} + \boldsymbol{\omega}_{r_i}^{\text{GLOBAL}}.\end{aligned}\tag{S50}$$

Establishing and reconfiguring SoNS connections

A SoNS connection e_{ij} between a parent robot r_i and a child r_j is established when one robot successfully recruits another robot, in the following way.

When two robots mutually sense each other, both robots send recruitment messages to each other and reach a consensus about which one of them should become the parent. Depending on the configuration of the SoNS algorithm, the robot that becomes the parent r_i of the new link is either the robot with the larger SoNS rank SONSROOTRANK , larger downstream vertex cardinality λ^{CARD} , or larger combination of the two values. When the new link e_{ij} is formed, if the child robot r_j had a former parent r_k , it breaks its link e_{kj} with its former parent and also sets a timer to ignore any new recruitment messages from any robot with its former SoNS identifier $\text{SONSROOTID}_i^{\text{OLD}}$ until the time steps passed since it had that former SoNS identifier t_i^{OLD} is equal to $\lambda^{\text{HIGH}}(r_j)$. Note that, because the r_j and its former parent r_k are no longer connected and are likely to mutually sense each other, they will likely try to recruit each other, and in this case the robot r_j will become the parent of r_k if the SoNS rank SONSROOTRANK is the recruitment metric, because r_j would then have the new (larger) SoNS rank SONSROOTRANK that it inherited from its new parent r_i .

A SoNS connection e_{ij} can be broken for reasons other than a new recruitment (either because of a local decision by robot r_i or r_j , or because of a disturbance or other error). When such a break occurs, the former child r_j will no longer have a parent. It then becomes the SoNS-brain of its own SoNS, returns to its original defaults with which it initialized, and randomly generates a new ROBOTRANK according to the defined uniform distribution. If it has children, it sends them updated information accordingly.

At any SoNS connection e_{ij} , the parent robot r_i can choose to transfer its child r_j to another robot r_k , in an operation called a “handover.” When parent robot r_i hands over its child to r_k , a new link e_{kj} is established and then the former link e_{ij} is broken.

Node allocation

For each parent robot r_i , each of its children robots r_j need to be allocated to one of its target child positions $r_j^* \in C_{r_i}^*$. Each node allocation operation occurs in the following way. For all children $r_j \in C_{r_i}$ and all target child positions $r_j^* \in C_{r_i}^*$ of parent robot r_i , define

$$\begin{aligned}
 \text{Source displacements: } \mathcal{S}^D &= \mathbf{d}_{r_i r_j} \forall r_j \in C_{r_i} \\
 \text{Source downstream cardinalities: } \mathcal{S}^{\text{CARD}} &= \lambda^{\text{CARD}}(r_j) \forall r_j \in C_{r_i} \\
 \text{Target displacements: } \mathcal{T}^D &= \mathbf{d}_{r_i r_j}^* \forall r_j^* \in C_{r_i}^* \\
 \text{Target downstream cardinalities: } \mathcal{T}^{\text{CARD}} &= \lambda^{\text{CARD}}(r_j^*) \forall r_j^* \in C_{r_i}^* \\
 \text{Displacement costs: } \mathcal{W}^D &= \|\mathcal{S}_j^D - \mathcal{T}_j^D\| \forall \text{CB}(r_j \in C_{r_i}, r_j^* \in C_{r_i}^*) \\
 \text{Cardinalities costs: } \mathcal{W}^{\text{CARD}} &= \|\mathcal{S}_j^{\text{CARD}} - \mathcal{T}_j^{\text{CARD}}\| \forall \text{CB}(r_j \in C_{r_i}, r_j^* \in C_{r_i}^*)
 \end{aligned} \tag{S51}$$

where $\text{CB}(\mathbf{x}, \mathbf{y})$ denotes all unique combinations of entries in $\mathbf{x}$ and $\mathbf{y}$. Then, using the matrices defined by Eq. S51, use an algorithm to allocate nodes. In the current implementation, we use the following algorithm:

Algorithm 1 Node Allocation

- 1: **procedure** ALLOCATE($\mathcal{S}^D, \mathcal{S}^{\text{CARD}}, \mathcal{T}^D, \mathcal{T}^{\text{CARD}}, \mathcal{W}^D, \mathcal{W}^{\text{CARD}}$)
 - 2: Sort the values in $\mathcal{W}^D$ and apply the same sorting transformation to $\mathcal{W}^{\text{CARD}}$
 - 3: Construct a network flow graph G_{FLOW} using the defined matrices
 - 4: Apply the Network Flow Algorithm in [61, 62] to obtain the network with the maximum possible flow rate and select the source in $\mathcal{S}^D, \mathcal{S}^{\text{CARD}}$ that should be matched to each target in $\mathcal{T}^D, \mathcal{T}^{\text{CARD}}$ accordingly
 - 5: **end procedure**
-

Robots in a SoNS continually redistribute themselves by adjusting their node allocations based on currently sensed conditions. For example, when at least one new SoNS connection e_{ij} is being established and the parent robot r_i currently has more than one target child position $r_j^* \in C_{r_i}^*$, or vice versa (i.e., more than one connection being established and at least one target child position), each incoming child robot r_j needs to be allocated to a target child position r_j^* . At this time, any existing children r_j of robot r_i are likewise (re)allocated. In any SoNS containing more than two robots, the robots continually redistribute themselves at every time step (even when no new robots are being added), in the following way.

At each time step, each robot r_i that has a parent r_k and has children r_j runs the node allocation algorithm (see Algorithm 1) twice. First, robot r_i runs the node allocation algorithm considering itself and all its downstream robots as members of the source matrices, and likewise considering all target positions of its parent's downstream subgraph H_{r_k} as members of the target matrices. If a child robot r_j matches with a single target, robot r_i instructs it to move towards that target position, or hands it over to robot r_k if the target is on a branch that is not reachable by r_i . If a child r_j matches with multiple targets, robot r_i hands it over to robot r_k so that r_j and its respective downstream robots can be reallocated to the multiple targets. If robot r_i matches a target, it moves towards that target position, otherwise, it moves towards its parent r_k . Second, if robot r_i has any remaining robots r_j

that are still its children, it checks if any of its children should substitute it, based on the inequality

$$d_{r_k r_j} \times \frac{d_{r_k r_j}^*}{\|d_{r_k r_j}^*\|} < d_{r_k r_i} \times \frac{d_{r_k r_i}^*}{\|d_{r_k r_i}^*\|} . \quad (\text{S52})$$

If a child r_j is in a better position, robot r_i hands it over to r_k so that it can be reassigned. Third, robot r_i runs the node allocation algorithm for its remaining children r_j . If one child matches one target, robot r_i instructs it to move to the target position. If multiple children match the same target, robot r_i selects the nearest child, instructs it to move to the target position, and then hands over to it the other children that matched with its target. If a child matches with multiple targets, robot r_i sends it the list of targets so that it can reallocate its own downstream robots accordingly. If a child does not meet any of these conditions, robot r_i hands it over to its parent r_k . At any point during these reallocations, if a robot r_i senses that one of its links e_{ij} spatially intersects with another link $e_{k\ell}$ in its sensing range, it triggers hand over operations such that r_i and r_k will swap children, resulting in the new links $e_{i\ell}$ and e_{kj} .

Section S5. Simulator setup

We conduct our simulated experiments in the ARGoS multi-robot simulator [48], a widely used simulator for swarm robotics research, using simulation models—custom-developed for this study [63, 64]—of the aerial and ground robots used in our real experiments (see Fig. S42).



Fig. S42: The (a) real and (b) simulated aerial robot. The (c) real and (d) simulated ground robot. A (e) real and (f) simulated physical obstacle; not to size. (The spherical markers visible in (c) and (d) are not used by the SoNS software; they are used for data logging in the real experiments, see Sec. S8.)

In ARGoS, sensors and actuators are plug-ins that have either read-only access or the ability to modify specific entities in the simulated 3D space [48]. In our setup, to control the aerial and ground robots in a way that is replicable in simulation, we use executables based on the libraries of ARGoS that we have custom-developed for this study [63, 64]. These executables initialize the sensors and actuators of the robots in such a way that the provided high-level control interface matches that of the sensor and actuator plug-ins of our robot models in the ARGoS simulator. In other words,

each robot’s control interface provided by the ARGoS libraries is an abstraction layer on top of the physical hardware (similar to the layer in [65]) and we have used this same abstraction layer to create simulation models of the robots in ARGoS. Therefore, we can run exactly the same control software on both the real robots and the simulated robots, using C++ and the Lua scripting language.

The robot motion models used in simulation as well as the motion control layers used to execute outputs from the SoNS software in both simulation and reality are detailed in Sec. S6 for the aerial robots and Sec. S7 for the ground robots.

For the models of the robot sensors and actuators in simulation, we conducted a trial-and-error calibration process to tune the speed and noise parameters. The speed parameters of the robot actuators in simulation have been tuned so that the speeds of the real robots match those of the simulated robots when controlled by the same control scripts. This calibration is straightforward, thus the maximum and average speeds of the robots in our simulated and real experiments are equivalent when running the same controllers. By contrast, noise is influenced by many (often unknown) factors and therefore the noise parameters of simulated sensors and actuators are much more challenging to calibrate and the result often underestimates the noise present in reality. To help compensate for this shortcoming, we tuned the noise parameters in simulation so that the simulated robots displayed greater noise in simple behaviors than the real robots when running the same controllers. However, in more complex missions (i.e., those used in our experiments), the noise displayed in reality is still noticeably greater than the noise displayed in simulation. The difference in error in our simulator and in reality is detailed in Sec. S2. The difference is most noticeable in portions of the missions when robots are in a steady phase and trying to remain stationary: in these steady phases, the observed error can be primarily attributed to the noise present in the aerial robot’s hovering behavior, which is noticeably greater in reality than in simulation (see Sec. S2). (Note that in our cross-verification between the simulator and reality in Sec. S2, the observed differences are associated primarily with the noise of the robots, not with the SoNS behaviors.)

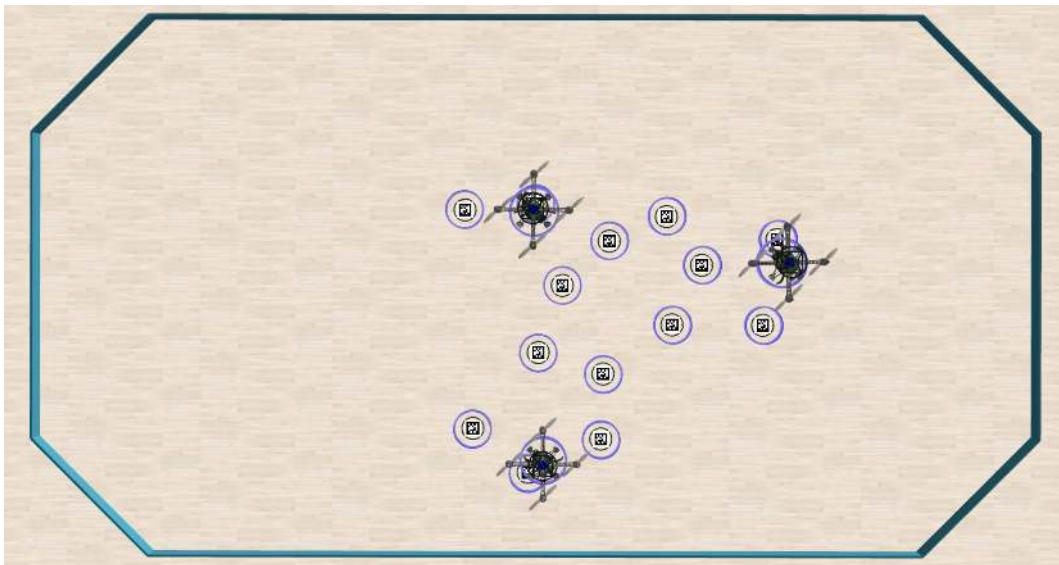
For the interactions between the robots and their environment, the ground robots are simulated using ARGoS’s 3D-dynamics engine based on the ODE library [48], because they can physically interact with objects on the ground, and the aerial robots are simulated using ARGoS’s more lightweight 3D particle engine [48]. Objects on the ground (such as obstacles to be avoided, see Fig S42e,f) are also simulated using ARGoS’s 3D-dynamics engine based on the ODE library, so that the ground robots can physically interact with them.

For the different experiment setups, customized environments in ARGoS are created using a python script to generate the .argos setup file. This generation can include arena walls, environmental features composed of obstacles that occur at certain positions relative to the size of the arena, and uniformly random initial positions of robots and positions of obstacles. In simulated experiment setups that are supposed to match a real experiment exactly (see Fig. S43), the generated arena walls in ARGoS match the border dimensions of the real arena floor. In other simulated experiments, in which more robots are used than in any real experiments, the arena size is generated to be large enough for the respective system size. We also display some objects (blue arrows to denote SoNS connections, blue circles to denote SoNS-brains) for visualization only, when recording videos directly from ARGoS (see key frames of missions in Sec. S1, for example in Fig. S1). All simulation models and experiment setups are available in the online code repository.

In all experiments, the time step in the simulator is equivalent to 0.2 seconds. Data logging is executed at each time step and records the global position and orientation of each robot as well as the



(a)



(b)

Fig. S43: The (a) real and (b) simulated arenas. Note that the blue circles around robots in (b) are for visualization purpose only and are not part of the robot models. Without these blue circles, the dimensions of the robots proportional to the arenas in the real and simulated environments can be seen to be equivalent.

SoNS information held locally by each robot. The experiment logs are available in the online data repository.

Depending on the system size, simulated experiments were run either on the experimenter's local machine or on our in-house IRIDIA computing cluster⁴, composed of 36 computational nodes for a total of 1536 CPU cores, in four logical racks. In analysis of our experiment results, computation work has only been assessed using CPU clock cycles, which is not affected by the machine on which the simulation is run.

⁴<https://majorana.ulb.ac.be/wordpress/cluster-composition/>

Section S6. Aerial robot setup

The aerial robot used in the experiments is the *S-drone* (Swarm-drone) quadrotor platform, which we custom-developed for this study.

The full hardware details of the *S-drone* quadrotor are available in an open-access technical report and open-source repositories [44]⁵, including the hardware description and specifications; design files and bill of materials; instructions for assembly, operation, tuning, and camera calibration for detection and tracking of fiducial markers; and example routines. As described in [44], a Linux operating system compiled by Yocto is installed on the UpCore single-board computer of the S-drone [66, 63]. The control software running in Linux is ARGoS [48]. The SoNS software is comprised of Lua scripts loaded and executed by ARGoS.

The *S-drone* quadrotor platform has several possible operation modes. In this study, we use its operation mode for fully autonomous flight control, based on autonomous vision-based navigation and relative positioning. These capabilities are primarily supported by the quadrotor’s single-board computer for onboard processing, its downward-facing optical flow smart camera module paired with single-point LiDAR for relative position estimation, and its four downward-facing camera modules for detection of fiducial markers and for relative position estimation.

In the remainder of Sec. S6, we present the modeling and flight control of the *S-drone* quadrotor. The section is organized in four subsections in which we give:

- the modeling preliminaries, including the reference frames, rotation matrix, quadrotor states, sensor modeling, and motor dynamics,
- the nonlinear model of the quadrotor system, including its kinematics and dynamics,
- a state space representation for system behavior analysis,
- the linearized model used for flight control, and
- the design of the position and attitude controllers.

The presented quadrotor modeling is used in the simulated experiments (see Sec. S5 of the supplementary materials for details about the simulator setup). The presented flight controllers and support for system behavior analysis (which are both based on the quadrotor modeling) are used for the simulated experiments and experiments with real robots. The flight controllers and simulation model are also available in open-source code repositories [44]⁶.

Modeling preliminaries

The preliminary information needed for the quadrotor modeling includes the reference frames, the rotation matrix for transformations between reference frames, the quadrotor states, and the sensor and motor information.

⁵The open-access technical report of the *S-drone* hardware, which includes URLs to the open-source repositories: <https://iridia.ulb.ac.be/IridiaTrSeries/link/IridiaTr2022-010.pdf>.

⁶The open-access technical report of the *S-drone* hardware, which includes URLs to the open-source repositories: <https://iridia.ulb.ac.be/IridiaTrSeries/link/IridiaTr2022-010.pdf>.

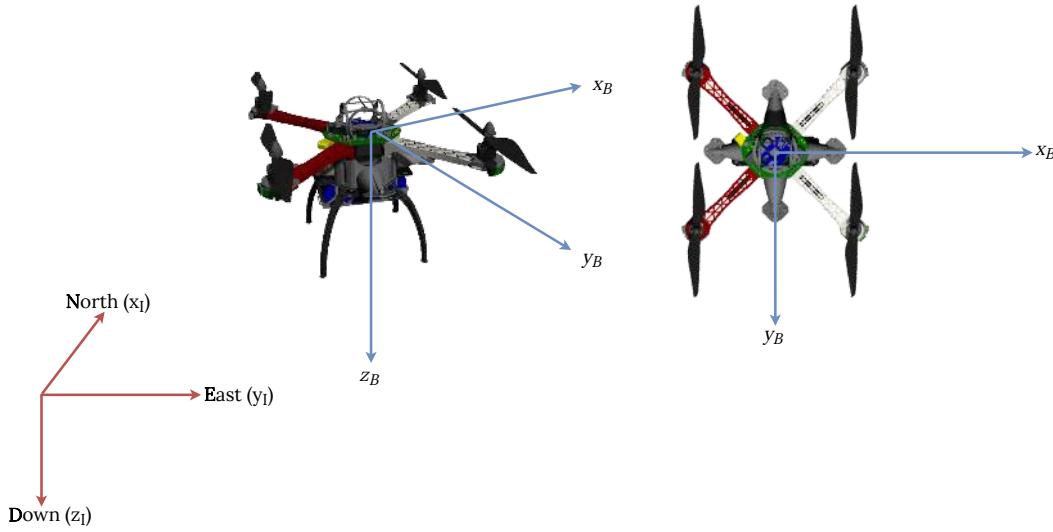


Fig. S44: Reference frames.

Remark S6 *In this study, there is no use of remote control, GPS, or other methods for off-board control or absolute positioning. All positioning is relative. The only absolute measurements used in the study are those of the quadrotor's on-board magnetometer, which gives measurements in a fixed inertial frame, but these measurements are not shared between quadrotors. The flight control inputs also make partial use of the fixed inertial frame (during the calculation of waypoints). However, the quadrotor only has access to its own calculations in its fixed inertial frame, not the measurements and calculations of other quadrotors, and only uses them to control its flight based on autonomous navigation commands it has generated onboard using relative positioning. Hence, there is no absolute reference synchronized between quadrotors or otherwise used to coordinate navigation. Navigation is strictly self-organized, using exclusively local communication and relative positioning.*

Reference frames

The quadrotor is modeled using the body frame (denoted B) and the inertial frame (denoted $\mathcal{I}$), as shown in Fig. S44.

The body frame is a relative coordinate system that represents the body of the quadrotor. The origin of the frame is the quadrotor center of mass, the x -axis of the frame is the quadrotor roll axis (i.e., longitudinal axis, directed to the front), the y -axis is the pitch axis (i.e., transverse axis, directed to the right), and the z -axis is the yaw axis (i.e., vertical axis, directed to the bottom).

The inertial frame is a fixed coordinate system defined at an arbitrary point on the Earth's surface and can be defined at any point on the surface. The inertial frame uses a North-East-Down (NED) configuration, in which the x -axis is directed northward, the y -axis is directed eastward, and the z -axis is directed downward.

Rotation matrix

To transform vectors defined in the body frame B into the inertial frame $\mathcal{I}$, we construct a zyx rotation matrix using Euler angles (roll, pitch, yaw). A zyx rotation involves three rotations (see Fig. S45):

the frame is rotated around the z -axis (yaw rotation), then around the y -axis (pitch rotation), and lastly around the x -axis (roll rotation). The zyx rotation matrix ${}^I\mathbf{R}_B(\Phi, \Theta, \Psi) = \mathbf{R}_z(\Psi)\mathbf{R}_y(\Theta)\mathbf{R}_x(\Phi)$ is defined as:

$${}^I\mathbf{R}_B(\Phi, \Theta, \Psi) = \begin{bmatrix} \cos \Psi \cos \Theta & \cos \Psi \sin \Theta \sin \Phi - \sin \Psi \cos \Phi & \cos \Phi \sin \Theta \cos \Psi + \sin \Phi \sin \Psi \\ \sin \Psi \cos \Theta & \sin \Psi \sin \Theta \sin \Phi + \cos \Psi \cos \Phi & \cos \Phi \sin \Theta \sin \Psi - \sin \Phi \cos \Psi \\ -\sin \Theta & \sin \Phi \cos \Theta & \cos \Phi \cos \Theta \end{bmatrix}. \quad (\text{S53})$$

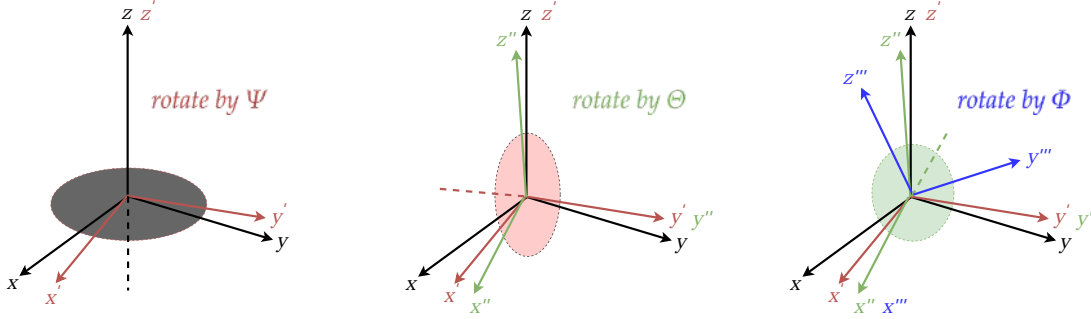


Fig. S45: The zyx rotation configuration.

Quadrotor states

The quadrotor has both inertial and body frame states, which can be used to describe its position, attitude, and velocity (see Table 1). The inertial frame states, which are defined relative to a fixed reference point (the reference point is fixed locally and used only for onboard calculations, not shared or synchronized between quadrotors), include the quadrotor's position coordinates (x, y, z) , its rotational angles (ϕ, θ, ψ) , and its linear and angular velocities $(\dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi})$. The body frame states, which are defined relative to the quadrotor itself, include the body frame linear velocities (u, v, w) and body frame angular velocities (p, q, r) . These states are important for understanding the quadrotor's motion and internal dynamics.

Sensor modeling

To control the quadrotor, it is necessary to observe the states. The simulation model of the drone uses the following measurement models for the inertial sensors: a three-axis accelerometer, a three-axis gyroscope, and a three-axis magnetometer.

Remark S7 *The S-drone does not rely on external localization information, such as GPS signals, so we do not consider those types of sensors. Instead, we use inertial sensors in our quadrotor to measure the platform's orientation, velocity, and acceleration. These sensors provide independent and non-jammable measurements of the platform, as described in [67].*

The gyroscope measures angular rotation around the body frame axis by measuring the Coriolis Force, which acts on objects in a rotating reference frame [67]. In our measurement model, we assume that the gyroscope measures body frame angular velocities (p, q, r) directly, which can be

Vector	Description
$\zeta_I = [x \ y \ z]$	Positions in the inertial frame
$\eta_I = [\phi \ \theta \ \psi]$	Euler angles (roll, pitch, yaw) in the inertial frame
$V_I = [\dot{x} \ \dot{y} \ \dot{z}]$	Linear velocities in the inertial frame
$\omega_I = [\dot{\phi} \ \dot{\theta} \ \dot{\psi}]$	Angular velocities in the inertial frame
$V_B = [u \ v \ w]$	Linear velocities in the body frame
$\omega_B = [p \ q \ r]$	Angular velocities in the body frame

Table 1: State vectors.

integrated over time to compute the orientation of the sensor (ϕ, θ, ψ) . The measurement model for the gyroscope is as follows:

$$\begin{aligned}\omega_B^m &= \omega_B + \mathbf{b}(t) + \boldsymbol{\mu} \\ \dot{\mathbf{b}}(t) &= \boldsymbol{\nu}_g \mathbf{1}_{3 \times 1}\end{aligned}, \quad (\text{S54})$$

where $\omega_B^m \in \mathbb{R}^3$ represents the measurement value of the angular velocities (p, q, r) in the body frame, the slowly time-varying bias term $\mathbf{b}(t) \in \mathbb{R}^3$ changes with white Gaussian noise $\boldsymbol{\nu}_g$, $\boldsymbol{\mu} \in \mathbb{R}^3$ is the measurement noise term, and $\mathbf{1}_{3 \times 1}$ is a matrix of 1s in the shape 3 by 1.

The accelerometer detects the forces present and uses them to calculate acceleration, according to the mass of the object and D'Alembert's force principle [67]. Our measurement model for the accelerometer determines the forces acting on the quadrotor and calculates its acceleration as follows:

$$\begin{aligned}\mathbf{a}_B^m &= \mathbf{a}_B^l + {}^B\mathbf{R}_I \mathbf{a}_I^g \mathbf{b}(t) + \boldsymbol{\mu} \\ \dot{\mathbf{b}}(t) &= \boldsymbol{\nu}_a \mathbf{1}_{3 \times 1}\end{aligned}, \quad (\text{S55})$$

where $\mathbf{a}_B^l \in \mathbb{R}^3$ is the linear acceleration vector in the body frame, $\mathbf{a}_I^g \in \mathbb{R}^3$ is the gravitational acceleration vector in the inertial frame, the slowly time-varying bias term $\mathbf{b}(t) \in \mathbb{R}^3$ changes with the white Gaussian noise $\boldsymbol{\nu}_a$, and $\boldsymbol{\mu} \in \mathbb{R}^3$ is the measurement noise term.

The magnetometer measures the strength and direction of the earth's magnetic field, is used to help the quadrotor maintain a stable hover, and is used in the autonomous navigation and positioning processes happening onboard. (It is not used as a reference for coordination between quadrotors.) Our measurement model for the magnetometer is:

$$\begin{aligned}\mathbf{H}_B^m &= {}^B\mathbf{R}_I \mathbf{H}_I + \boldsymbol{\mu} \\ \mathbf{H}_I &= \begin{bmatrix} H_x \\ H_y \\ H_z \end{bmatrix} = \|\mathbf{H}_I\| \cdot \begin{bmatrix} \cos \beta \\ 0 \\ \sin \beta \end{bmatrix},\end{aligned} \quad (\text{S56})$$

where $\mathbf{H}_I \in \mathbb{R}^3$ is the magnetic field vector and β is the magnetic inclination. If the measurements

of the roll and pitch angles are already known, the yaw angle can then be calculated as:

$$\psi = \arctan \frac{H_x^m \cos \theta + (H_y^m \sin \phi + H_z^m \cos \phi \sin \theta)}{H_x^m \sin \phi - H_y^m \cos \phi} . \quad (\text{S57})$$

Motor dynamics

We represent the dynamics of the motors using a first-order transfer function [68], which allows for modeling the time-varying behavior of the motors and thereby accurately predicting the response of the quadrotor to control inputs. The transfer function describes the relationship between the input and output of the motor and can be used to design control algorithms that accurately regulate the speed and torque of the motors. The model is given as

$$\frac{\Omega}{\Omega_d} = \frac{1}{T_{rot}s + 1} , \quad (\text{S58})$$

where T_{rot} is a time constant, Ω_d is the desired motor speed, and Ω is the calculated motor speed.

Nonlinear model of the quadrotor

For our nonlinear model of the quadrotor's dynamic behavior, we make the following assumptions, as in [68], to simplify some of the modeling calculations:

- the quadrotor body structure is rigid,
- the quadrotor is symmetrical in all axes,
- the propeller structure is rigid and the oscillation on the quadrotor body does not show motion in the vertical direction,
- the body frame coincides with the quadrotor's center of gravity,
- motors are identical,
- motors are positioned perpendicular to the body frame,
- propeller thrust and drag moment are directly proportional to the square of the motor speed,
- the ground effect is neglected,
- the gyroscopic effect of the motors is neglected.

Quadrotor kinematics

To study the motion of the quadrotor using kinematics, we use the rotation matrix (S53) to transform linear velocities measured in the body frame to the inertial frame, as follows:

$$\begin{aligned} \dot{\zeta}_I &= \mathbf{V}_I = {}^I\mathbf{R}_B \mathbf{V}_B \\ {}^B\mathbf{R}_I &= ({}^I\mathbf{R}_B)^T . \end{aligned} \quad (\text{S59})$$

Then, to transform the angular velocities from the body frame to the inertial frame, we use the angular transformation matrix ${}^{\mathcal{I}}\mathbf{T}_{\mathcal{B}}$, defined as

$${}^{\mathcal{I}}\mathbf{T}_{\mathcal{B}} = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi \sec \theta & \cos \phi \sec \theta \end{bmatrix} \text{ and } {}^{\mathcal{B}}\mathbf{T}_{\mathcal{I}} = ({}^{\mathcal{I}}\mathbf{T}_{\mathcal{B}})^T. \quad (\text{S60})$$

We can then transform the angular velocities from the body frame to the inertial frame, as

$$\dot{\boldsymbol{\eta}}_{\mathcal{I}} = {}^{\mathcal{I}}\mathbf{T}_{\mathcal{B}} \boldsymbol{\omega}_{\mathcal{B}}, \quad (\text{S61})$$

and write the quadrotor kinematics as

$$\begin{bmatrix} \dot{\boldsymbol{\zeta}}_{\mathcal{I}} \\ \dot{\boldsymbol{\eta}}_{\mathcal{I}} \end{bmatrix} = \begin{bmatrix} {}^{\mathcal{I}}\mathbf{R}_{\mathcal{B}} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & {}^{\mathcal{I}}\mathbf{T}_{\mathcal{B}} \end{bmatrix} \begin{bmatrix} \mathbf{V}_{\mathcal{B}} \\ \boldsymbol{\omega}_{\mathcal{B}} \end{bmatrix}, \quad (\text{S62})$$

where $\mathbf{0}_{3 \times 3}$ is a matrix of 3s in the shape 3 by 3 and

$$\dot{\boldsymbol{\zeta}}_{\mathcal{I}} = \begin{cases} \dot{x} &= w[\sin \phi \sin \psi + \cos \phi \cos \psi \sin \theta] - v[\cos \phi \sin \psi + \cos \psi \sin \phi \sin \theta] + u[\cos \psi \cos \theta] \\ \dot{y} &= -w[\cos \psi \sin \phi - \cos \phi \sin \psi \sin \theta] + v[\cos \phi \cos \psi + \sin \phi \sin \psi \sin \theta] + u[\cos \theta \sin \psi] \\ \dot{z} &= w[\cos \phi \cos \theta] + v[\cos \theta \sin \phi] - u \sin \theta \end{cases} \quad (\text{S63})$$

and

$$\dot{\boldsymbol{\eta}}_{\mathcal{I}} = \begin{cases} \dot{\phi} &= p + r[\cos \phi \tan \theta] + q[\sin \phi \tan \theta] \\ \dot{\theta} &= q \cos \phi - r \sin \phi \\ \dot{\psi} &= r \frac{\cos \phi}{\cos \theta} + q \frac{\sin \phi}{\cos \theta} \end{cases}. \quad (\text{S64})$$

Quadrotor dynamics

For the dynamics of the quadrotor, we need to consider the mass and inertia, the forces, and the torques acting on the body.

To derive the differential equations describing the quadrotor dynamics using the Newton-Euler method [69], we start with the following equalities:

$$\begin{bmatrix} \mathbf{F}_{\mathcal{B}} \\ \boldsymbol{\tau}_{\mathcal{B}} \end{bmatrix} = \begin{bmatrix} m\mathbf{I}_3 & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{J} \end{bmatrix} \begin{bmatrix} \dot{\mathbf{V}}_{\mathcal{B}} \\ \dot{\boldsymbol{\omega}}_{\mathcal{B}} \end{bmatrix} + \begin{bmatrix} \boldsymbol{\omega}_{\mathcal{B}} \times m\mathbf{V}_{\mathcal{B}} \\ \boldsymbol{\omega}_{\mathcal{B}} \times \mathbf{J}\mathbf{V}_{\mathcal{B}} \end{bmatrix} \quad (\text{S65})$$

$$\mathbf{F}_{\mathcal{B}} = \begin{cases} F_x &= m(\dot{u} + qw - rv) \\ F_y &= m(\dot{v} - pw - ru) \\ F_z &= m(\dot{w} + pv - qu) \end{cases}, \quad \boldsymbol{\tau}_{\mathcal{B}} = \begin{cases} \tau_x &= \dot{p}J_x - qrJ_y + prJ_z \\ \tau_y &= \dot{q}J_y + prJ_x - prJ_z \\ \tau_z &= \dot{r}J_z + pqJ_x + pqJ_y \end{cases},$$

where $\mathbf{I}_3$ is a 3 by 3 identity matrix, $\mathbf{J}$ is a 3 by 3 diagonal inertia matrix, m is the mass of the quadrotor, and $\mathbf{F}_{\mathcal{B}} = [F_x \ F_y \ F_z]^T \in \mathbb{R}^3$, $\boldsymbol{\tau}_{\mathcal{B}} = [\tau_x \ \tau_y \ \tau_z]^T \in \mathbb{R}^3$ are force and torque vectors that act on the body. The force vector ($\mathbf{F}_{\mathcal{B}}$) is composed of the gravitational force and the thrust force generated by the propulsion system. Similarly, the torque vector ($\boldsymbol{\tau}_{\mathcal{B}}$) is composed of the roll, pitch, and yaw moments as well as the gyroscopic moments generated by the propulsion system. They can be expressed as

$$\begin{aligned} \mathbf{F}_{\mathcal{B}} &= \overbrace{mg^{\mathcal{B}}\mathbf{R}_{\mathcal{I}}\hat{\mathbf{e}}_z}^{\text{the gravity effect}} - \overbrace{U_1\hat{\mathbf{e}}_3}^{\text{the thrust force}}, \\ \boldsymbol{\tau}_{\mathcal{B}} &= \overbrace{U_{\tau}}^{\text{the moment vector}} - \overbrace{\boldsymbol{\tau}_g}^{\text{the gyroscopic moment}}, \end{aligned} \quad (\text{S66})$$

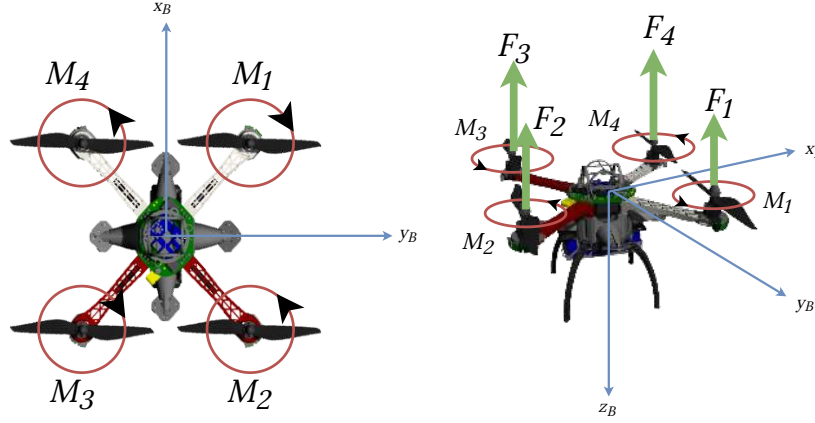


Fig. S46: The moments and forces acting on the quadrotor body.

where g is the gravitational acceleration, $\hat{e}_z$ is the z -axis unit vector in the inertial frame, $\hat{e}_3$ is the z -axis unit vector in the body frame, and the total thrust force U_1 (i.e., the combination of forces F_1 , F_2 , F_3 , and F_4) provides the lift necessary for the quadrotor to ascend or descend (see Fig. S46). The moment vector $\mathbf{U}_\tau = [U_2 \ U_3 \ U_4]^T$ includes moments U_2 , U_3 , and U_4 . U_2 and U_3 are generated through the arms of the quadrotor and correspond to roll and pitch moments, respectively, whereas U_4 is the total yaw moment, which is the combination of moments M_1 , M_2 , M_3 , and M_4 (see Fig. S46). The gravitational force mg acting on the quadrotor is defined in the inertial frame and transformed to the body frame using the rotation matrix ${}^B\mathbf{R}_I$. In addition to these forces, the quadrotor's propulsion system generates a gyroscopic moment $\boldsymbol{\tau}_g$, which arises due to the change in the direction of the angular momentum vector of the motors as the quadrotor rolls and pitches, that can be expressed as

$$\boldsymbol{\tau}_g = \sum_{i=1}^4 \mathbf{J}(\boldsymbol{\omega}_B \times \hat{e}_3)(-1)^{i+1}\Omega_i, \quad (\text{S67})$$

where $\mathbf{J}$ is the inertia of the motors and is one of the moments acting on the quadrotor, and Ω_i is the angular velocity of the i -th motor. Because two of the motors on the quadrotor rotate clockwise and the other two rotate counterclockwise, the total angular velocity of the motors can be expressed as $\Omega_T = \Omega_1 - \Omega_2 + \Omega_3 - \Omega_4$. Then, we can rewrite the gyroscopic moment (S67) as

$$\boldsymbol{\tau}_g = \mathbf{J} \left(\begin{bmatrix} p \\ q \\ r \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ \Omega_T \end{bmatrix} \right). \quad (\text{S68})$$

The motor inertia $\mathbf{J}$ is minimal compared to the other moments acting on the quadrotor [68]. Therefore, when building the model of the quadrotor, it is generally safe to neglect the gyroscopic effect of the motors, the ground effect during takeoff or landing, the dynamic effects of the propellers' airflow and blade flapping, or other aerodynamic effects that might be observed during flight. Using these simplifications, and under the assumptions previously mentioned, we can construct the

dynamics model of the quadrotor as follows:

$$\begin{aligned}
 -mg \sin \theta &= m(\dot{u} + qw - rv) \\
 mg \cos \theta \sin \phi &= m(\dot{v} - pw - ru) \\
 mg \cos \theta \cos \phi - U_1 &= m(\dot{w} + pv - qu) \\
 U_2 &= \dot{p}J_x - qrJ_y + prJ_z \\
 U_3 &= \dot{q}J_y + prJ_x - prJ_z \\
 U_4 &= \dot{r}J_z + pqJ_x + pqJ_y
 \end{aligned} \tag{S69}$$

In this model, U_1 is the total thrust force and only acts in the z -axis direction of the body frame. Assuming K_T is a constant thrust coefficient, U_1 can be defined as

$$U_1 = \begin{bmatrix} 0 \\ 0 \\ K_T(\Omega_1^2 + \Omega_2^2 + \Omega_3^2 + \Omega_4^2) \end{bmatrix}, \tag{S70}$$

such that the total thrust force is a function of the square of the rotational speed of the motors. U_2 is the roll angular moment, which is generated about the x -axis of the body frame, and is defined as

$$U_2 = l_x K_T (-\Omega_1^2 - \Omega_2^2 + \Omega_3^2 + \Omega_4^2) \sin 45^\circ, \tag{S71}$$

where l_x is the moment arm distance of the motor frame from the x -axis of the body frame.

U_3 is the pitch angular moment, which is generated about the y -axis of the body frame, and is defined as

$$U_3 = l_y K_T (\Omega_1^2 - \Omega_2^2 - \Omega_3^2 + \Omega_4^2) \sin 45^\circ, \tag{S72}$$

where l_y is the moment arm distance of the motor frame from the y -axis of the body frame.

U_4 is the yaw angular momentum, which is generated about the z axis of the body frame, and is defined as

$$U_4 = K_M (\Omega_1^2 - \Omega_2^2 + \Omega_3^2 - \Omega_4^2), \tag{S73}$$

where K_M is a constant moment coefficient that reflects the inherent moment-generating capabilities of the motor-propeller system. Note that there is no moment arm coefficient included here because, in our quadrotor, there is no distance between the point at which the moment is applied and the yaw axis.

By combining equations (S70), (S71), (S72), and (S73), the forces and moments generated by the propulsion system of the quadrotor can be obtained as

$$\begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \end{bmatrix} = \begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \end{bmatrix} = \begin{bmatrix} K_T & K_T & K_T & K_T \\ -K_T l_x \frac{\sqrt{2}}{2} & -K_T l_x \frac{\sqrt{2}}{2} & -K_T l_x \frac{\sqrt{2}}{2} & -K_T l_x \frac{\sqrt{2}}{2} \\ K_T l_y \frac{\sqrt{2}}{2} & -K_T l_y \frac{\sqrt{2}}{2} & -K_T l_y \frac{\sqrt{2}}{2} & K_T l_y \frac{\sqrt{2}}{2} \\ K_M & -K_M & K_M & -K_M \end{bmatrix} \begin{bmatrix} \Omega_1^2 \\ \Omega_2^2 \\ \Omega_3^2 \\ \Omega_4^2 \end{bmatrix}. \tag{S74}$$

The inverse of the transformation matrix in Eq. (S74) can then be used to determine the motor angular velocities needed to produce the desired forces and moments, as follows:

$$\begin{bmatrix} \Omega_1^2 \\ \Omega_2^2 \\ \Omega_3^2 \\ \Omega_4^2 \end{bmatrix} = \begin{bmatrix} \frac{1}{4K_T} & -\frac{1}{4K_T l_x} & \frac{1}{4K_T l_y} & \frac{1}{4K_M} \\ \frac{1}{4K_T} & -\frac{1}{4K_T l_x} & -\frac{1}{4K_T l_y} & -\frac{1}{4K_M} \\ \frac{1}{4K_T} & \frac{1}{4K_T l_x} & -\frac{1}{4K_T l_y} & \frac{1}{4K_M} \\ \frac{1}{4K_T} & \frac{1}{4K_T l_x} & \frac{1}{4K_T l_y} & -\frac{1}{4K_M} \end{bmatrix} \begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \end{bmatrix}. \tag{S75}$$

Remark S8 *The motor angular velocities serve as inputs that determine the thrust and torque generated by the motors, which in turn affect the motion of the quadrotor. These inputs are determined by an onboard controller, which receives input from sensors and processes it to compute the desired motor angular velocities. The controller uses these velocities to drive the motors, which produce the necessary thrust and torque to achieve the desired behavior of the quadrotor. In other words, the motor angular velocities are an important factor in the control and operation of the quadrotor.*

State space modeling

To construct the quadrotor model we use for control and simulation, we now express the model in state space form, which combines the kinematic and dynamic equations. To create the state space representation of the quadrotor's model, we define the following state vector:

$$\mathbf{X} = [x \ y \ z \ u \ v \ w \ \phi \ \theta \ \psi \ p \ q \ r]^T. \quad (\text{S76})$$

Then, the complete model of the quadrotor can be written using the equation sets in Eq. (S63) (S64) and (S69), as

$$\dot{\mathbf{X}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{u} \\ \dot{v} \\ \dot{w} \\ \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \\ \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} w(\sin \phi \sin \psi + \cos \phi \cos \psi \sin \theta) - v(\cos \phi \sin \psi + \cos \psi \sin \phi \sin \theta) + u \cos \psi \cos \theta \\ -w(\cos \psi \sin \phi - \cos \phi \sin \psi \sin \theta) + v(\cos \phi \cos \psi + \sin \phi \sin \psi \sin \theta) + u \cos \theta \sin \psi \\ w \cos \phi \cos \theta + v \cos \theta \sin \phi - u \sin \theta \\ rv - qw - g \sin \theta \\ pw - ru + g \cos \theta \sin \phi \\ qu - pv + g \cos \theta \cos \phi - \frac{U_1}{m} \\ p + r \cos \phi \tan \theta + q \sin \phi \tan \theta \\ q \cos \phi - r \sin \phi \\ r \frac{\cos \phi}{\cos \theta} + q \frac{\sin \phi}{\cos \theta} \\ qr \frac{I_y - I_z}{I_x} + \frac{U_2}{I_x} \\ pr \frac{I_z - I_x}{I_y} + \frac{U_3}{I_y} \\ pq \frac{I_x - I_y}{I_z} + \frac{U_4}{I_z} \end{bmatrix}. \quad (\text{S77})$$

The model given in Eq. (S77) includes states that are defined in the body frame, namely translational acceleration states ($\dot{u}$, $\dot{v}$, $\dot{w}$) and rotational acceleration states ($\dot{p}$, $\dot{q}$, $\dot{r}$). Because it is more convenient to work with calculations in the inertial frame for control studies [68], we redefine the body frame states of the state vector in the inertial frame. To express the translational states ($\dot{u}$, $\dot{v}$, $\dot{w}$) in the inertial frame, we apply the rotation matrix ${}^{\mathcal{I}}\mathbf{R}_{\mathcal{B}}$ as follows:

$$\begin{aligned} \mathbf{F}_B &= m [\dot{u} \ \dot{v} \ \dot{w}]^T \\ \mathbf{F}_B &= mg^{\mathcal{B}} \mathbf{R}_{\mathcal{I}} \hat{\mathbf{e}}_z - U_1 \hat{\mathbf{e}}_3 \\ {}^{\mathcal{I}}\mathbf{R}_{\mathcal{B}} \mathbf{F}_B &= mg \hat{\mathbf{e}}_z - U_1 {}^{\mathcal{I}}\mathbf{R}_{\mathcal{B}} \hat{\mathbf{e}}_3 \end{aligned} \quad (\text{S78})$$

Then we can obtain the translational states in the inertial frame, denoted as $(\ddot{x}, \ddot{y}, \ddot{z})$:

$$\begin{aligned}
 {}^{\mathcal{I}}\mathbf{R}_{\mathcal{B}}\mathbf{F}_B &= mg\hat{\mathbf{e}}_z - U_1 {}^{\mathcal{I}}\mathbf{R}_{\mathcal{B}}\hat{\mathbf{e}}_3 \\
 m\dot{\mathbf{V}}_I &= mg\hat{\mathbf{e}}_z - U_1 {}^{\mathcal{I}}\mathbf{R}_{\mathcal{B}}\hat{\mathbf{e}}_3 \\
 m \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} &= mg\hat{\mathbf{e}}_z - U_1 {}^{\mathcal{I}}\mathbf{R}_{\mathcal{B}}\hat{\mathbf{e}}_3 \\
 \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} &= g\hat{\mathbf{e}}_z - \frac{U_1 {}^{\mathcal{I}}\mathbf{R}_{\mathcal{B}}\hat{\mathbf{e}}_3}{m} \\
 &= \begin{bmatrix} -\frac{U_1}{m}(\sin\phi\sin\psi + \cos\phi\cos\psi\sin\theta) \\ -\frac{U_1}{m}(-\cos\psi\sin\phi + \cos\phi\sin\psi\sin\theta) \\ g - \frac{U_1}{m}\cos\phi\cos\theta \end{bmatrix}
 \end{aligned} \tag{S79}$$

To express the rotational states $(\dot{p}, \dot{q}, \dot{r})$ in the inertial frame, we establish the relationship between the body frame and the inertial frame, using equation (S61), as

$$\begin{aligned}
 \begin{bmatrix} p \\ q \\ r \end{bmatrix} &= {}^{\mathcal{I}}\mathbf{T}_{\mathcal{B}} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}, \quad {}^{\mathcal{I}}\mathbf{T}_{\mathcal{B}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
 \begin{bmatrix} p \\ q \\ r \end{bmatrix} &= \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}
 \end{aligned} \tag{S80}$$

Remark S9 In the equation, (S80), we make the assumption that the quadrotor primarily hovers or moves with small angles, meaning that the roll and pitch angles are approximately zero. This assumption, known as the small-angle approximation [68], allows us to simplify the angular transformation matrix given ${}^{\mathcal{I}}\mathbf{T}_{\mathcal{B}}$ to a 3 by 3 identity matrix.

With the relationship established and the corresponding equalities for $(\dot{p}, \dot{q}, \dot{r})$ in Eq. (S77), we can obtain the transformed rotational states $(\ddot{\phi}, \ddot{\theta}, \ddot{\psi})$ as

$$\begin{bmatrix} \ddot{\phi} \\ \ddot{\theta} \\ \ddot{\psi} \end{bmatrix} = \begin{bmatrix} \dot{\psi}\dot{\theta}\frac{I_y - I_z}{I_x} + \frac{U_2}{I_x} \\ \dot{\phi}\dot{\psi}\frac{I_z - I_x}{I_y} + \frac{U_3}{I_y} \\ \dot{\phi}\dot{\theta}\frac{I_x - I_y}{I_z} + \frac{U_4}{I_z} \end{bmatrix}. \tag{S81}$$

Then, the model in Eq. (S77) becomes

$$\begin{aligned}
 \dot{\mathbf{X}} &= g(\mathbf{X}, \mathbf{U}) \\
 &= f(\mathbf{X}) + \sum_{i=1}^4 \mathbf{a}_i(\mathbf{X})U_i,
 \end{aligned} \tag{S82}$$

where

$$\begin{aligned}
 \mathbf{X} &= [x \ y \ z \ \phi \ \theta \ \psi \ \dot{x} \ \dot{y} \ \dot{z} \ \dot{\phi} \ \dot{\theta} \ \dot{\psi}]^T \\
 \dot{\mathbf{X}} &= [\dot{x} \ \dot{y} \ \dot{z} \ \dot{\phi} \ \dot{\theta} \ \dot{\psi} \ \ddot{x} \ \ddot{y} \ \ddot{z} \ \ddot{\phi} \ \ddot{\theta} \ \ddot{\psi}]^T, \\
 f(\mathbf{X}) &= [\dot{x} \ \dot{y} \ \dot{z} \ \dot{\phi} \ \dot{\theta} \ \dot{\psi} \ 0 \ 0 \ g \ (\dot{\psi}\dot{\theta})\frac{I_y - I_z}{I_x} \ (\dot{\phi}\dot{\psi})\frac{I_z - I_x}{I_y} \ (\dot{\phi}\dot{\theta})\frac{I_x - I_y}{I_z}]^T
 \end{aligned} \tag{S83}$$

and

$$\begin{aligned}
\mathbf{a}_1 &= [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ a_1^7 \ a_1^8 \ a_1^9 \ 0 \ 0 \ 0]^T, \\
a_1^7 &= -\frac{1}{m}(\sin \phi \sin \psi + \cos \phi \cos \psi \sin \theta), \\
a_1^8 &= -\frac{1}{m}(-\cos \psi \sin \phi + \cos \phi \sin \psi \sin \theta), \\
a_1^9 &= -\frac{1}{m} \cos \phi \cos \theta, \\
\mathbf{a}_2 &= [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ \frac{1}{I_x} \ 0 \ 0]^T, \\
\mathbf{a}_3 &= [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ \frac{1}{I_y} \ 0]^T, \\
\mathbf{a}_4 &= [0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ \frac{1}{I_z}]^T,
\end{aligned} \tag{S84}$$

Linearized modeling for flight control

In order to design a linear flight controller, for instance, a proportional–integral–derivative (PID) controller, it is necessary to linearize the model (S82) around an operating point that brings the system to an equilibrium state. This process is described, following [70], as

$$\dot{\mathbf{X}} = g(\bar{\mathbf{X}}, \bar{\mathbf{U}}) \rightarrow \mathbf{0}, \tag{S85}$$

where the control input vector $\mathbf{U} = [U_1 \ U_2 \ U_3 \ U_4]^T$ consists of the total thrust U_1 and control moments. $\bar{\mathbf{X}}$ represents an operating point with a constant input $\bar{\mathbf{U}}$, which is known as the trim condition and is defined as

$$\bar{\mathbf{U}} = [mg \ 0 \ 0 \ 0]^T, \tag{S86}$$

where mg represents the total thrust required to counteract the inertial force along the $+z$ -axis (due to gravity and to the weight of the quadrotor) and maintain the hover state of the quadrotor.

To get the linearized model of Eq. (S82) around the operating point $\bar{\mathbf{X}} = [\bar{x} \ \bar{y} \ \bar{z} \ 0 \ 0 \ 0 \ 0 \ 0 \ 0]^T$ with the constant input $\bar{\mathbf{U}}$, we use Taylor expansion as follows:

$$\dot{\mathbf{X}} = g(\bar{\mathbf{X}}, \bar{\mathbf{U}}) + \frac{\delta g}{\delta \mathbf{X}}|_{\bar{\mathbf{X}}} \delta \mathbf{X} + \frac{\delta g}{\delta \mathbf{U}}|_{\bar{\mathbf{U}}} \delta \mathbf{U} + \frac{1}{2} \frac{\delta^2 g}{\delta^2 \mathbf{X}}|_{\bar{\mathbf{X}}} \delta^2 \mathbf{X} + \frac{1}{2} \frac{\delta^2 g}{\delta^2 \mathbf{U}}|_{\bar{\mathbf{U}}} \delta^2 \mathbf{U} + \dots, \tag{S87}$$

where $\delta \mathbf{X}$ and $\delta \mathbf{U}$ represent deviations from the trim condition, with $\delta \mathbf{X} = \mathbf{X} - \bar{\mathbf{X}}$ and $\delta \mathbf{U} = \mathbf{U} - \bar{\mathbf{U}}$.

To simplify the Taylor expansion, we can ignore higher-order terms and consider only the first-order terms. Since $g(\bar{\mathbf{X}}, \bar{\mathbf{U}}) \rightarrow 0$, Eq. (S87) becomes

$$\begin{aligned}
\dot{\mathbf{X}} &\approx \frac{\delta g}{\delta \mathbf{X}}|_{\bar{\mathbf{X}}} \delta \mathbf{X} + \frac{\delta g}{\delta \mathbf{U}}|_{\bar{\mathbf{U}}} \delta \mathbf{U}, \\
&\approx \mathbf{A} \mathbf{X} + \mathbf{B} \mathbf{U}
\end{aligned} \tag{S88}$$

where

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & -g & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & g & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ \frac{1}{m} & 0 & 0 & 0 \\ 0 & \frac{1}{I_x} & 0 & 0 \\ 0 & 0 & \frac{1}{I_y} & 0 \\ 0 & 0 & 0 & \frac{1}{I_z} \end{bmatrix}.$$

Remark S10 The differential equation given in Eq. (S88) drives the deviation variables $\delta \mathbf{X}$ and $\delta \mathbf{U}$ towards zero as long as they remain small. This linearization procedure is known as *Jacobian linearization* [57].

Design of position and attitude controllers

The overall structure of the flight control system for the simulation studies is shown in Fig. S47. This system is based on the controllers used in the real quadrotor. We use a cascaded control structure that effectively decouples the position (position and linear velocity) and attitude (angle and angular velocity) control loops.

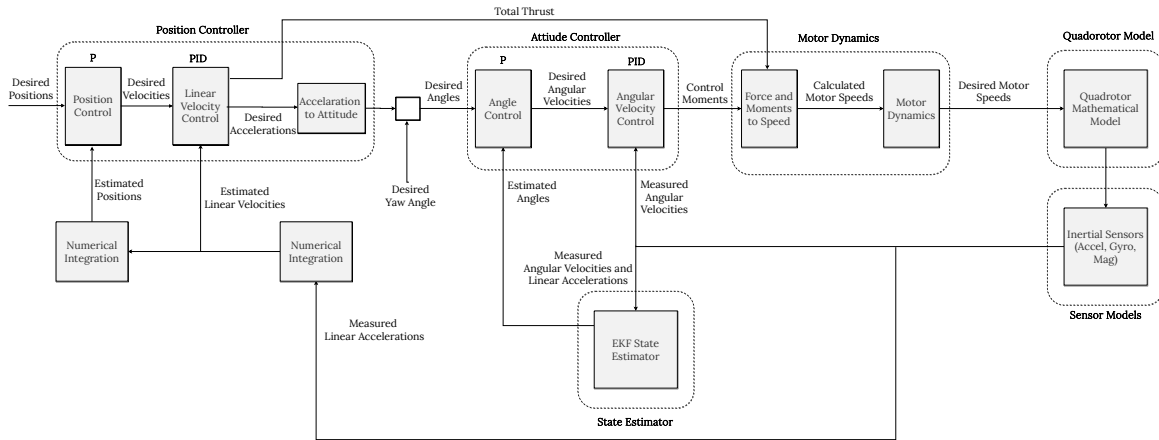


Fig. S47: The flight control system.

Position control

We use a position controller with a cascaded P/PID structure, with the first stage being a 3D position controller and the second stage being a linear velocity controller. Note that the quadrotor position

control system is underactuated [71], therefore, the position control for the xy -axes can be achieved through attitude control. The position controller uses a P controller and its outputs for the xy axes produce the desired acceleration values $\ddot{x}^d, \ddot{y}^d$ in the inertial frame. The outputs of the position controller for the xy axes produce the desired acceleration values $\ddot{x}^d, \ddot{y}^d$ in the inertial frame. These values are then used to generate the desired roll ϕ^d and pitch θ^d angles for the attitude controller to use. The overall structure of the position controller (see Fig. S48) is given as

$$\begin{aligned}
 \text{Altitude control : } U_1 &= K_{Pz}e_{Vz} + K_{Iz} \int e_{Vz} - K_{Dz}\dot{z} + mg \\
 \text{Desired roll and pitch angles : } \phi^d &= \frac{m}{U_1}(-\ddot{x}^d \sin \hat{\psi} + \ddot{y}^d \cos \hat{\psi}) \\
 \theta^d &= \frac{m}{U_1}(-\ddot{x}^d \cos \hat{\psi} - \ddot{y}^d \sin \hat{\psi}) . \quad (\text{S89})
 \end{aligned}$$

where $\ddot{x}^d = K_{Px}e_{Vx} + K_{Ix} \int e_{Vx} - K_{Dx}\dot{x}$

$$\ddot{y}^d = K_{Py}e_{Vy} + K_{Iy} \int e_{Vy} - K_{Dy}\dot{y}$$

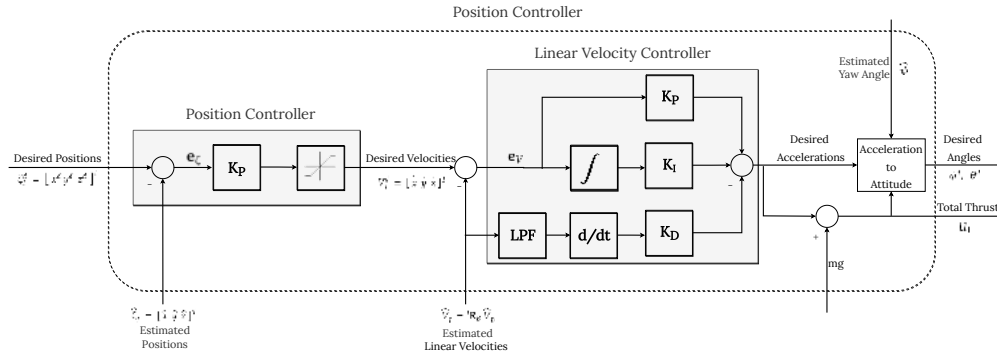


Fig. S48: The cascaded position controller.

Attitude Control

We use an attitude controller with a cascaded P/PID (proportional/proportional-integral-derivative) structure, with the first stage being an angle controller and the second stage being an angular velocity controller.

The angle controller uses a P controller that produces an output signal proportional to the error between the desired and actual values. The output of the angle controller becomes the desired value for the angular velocity controller, which uses a PID controller.

The outputs of the angular velocity controller, U_2 , U_3 , and U_4 , are the control moments that are used to manipulate the quadrotor's attitude. The overall structure of the attitude controller (see

Fig. S49) is given as

$$\begin{aligned}
 \text{Roll control : } U_2 &= K_{P\phi} e_{\omega\phi} + K_{I\phi} \int e_{\omega\phi} - K_{D\phi} \dot{\hat{\phi}} \\
 \text{Pitch control : } U_3 &= K_{P\theta} e_{\omega\theta} + K_{I\theta} \int e_{\omega\theta} - K_{D\theta} \dot{\hat{\theta}} \\
 \text{Yaw control : } U_4 &= K_{P\psi} e_{\omega\psi} + K_{I\psi} \int e_{\omega\psi} - K_{D\psi} \dot{\hat{\psi}}
 \end{aligned} \tag{S90}$$

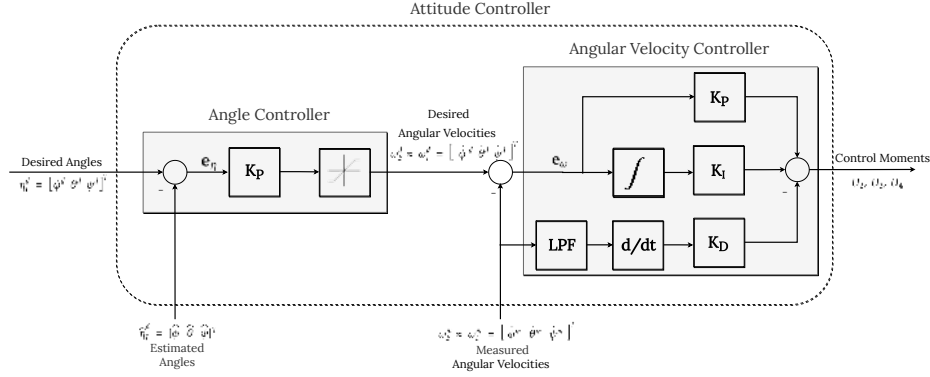


Fig. S49: The cascaded attitude controller

These controllers are used for the quadrotors in all real and simulated experiments included in this study.

Sensing robots and objects in the environment

In our setup, each robot or object has an identifier denoted as ρ . The identifier of a robot is unique and the identifier of an object is non-unique. Each robot has access to its own identifier ρ and all ground robots and objects are mounted with an AprilTag fiducial marker (i.e., a 2D barcode that contains a numeric ID code and supports 3D position and pose tracking [72, 73]) that encodes its identifier ρ . Using its four downward-facing camera modules, each quadrotor can sense the relative positions, relative orientations, and encoded identifiers ρ of AprilTags present in its field of view. Each quadrotor has a lookup table of AprilTag identifiers ρ that allows it to classify them as belonging either to a robot or to an object (see Fig. S42).

When a quadrotor ρ_i senses an AprilTag, it stores: its encoded identifier ρ_j ; its relative position, denoted by the displacement vector $\mathbf{d}_{\rho_i\rho_j}$ in the body frame of the quadrotor B_{ρ_i} ; and its relative orientation, denoted by the unit quaternion $\mathbf{q}_{\rho_i\rho_j}$, where the Euler axis portion of the quaternion is the z -axis of body frame B_{ρ_i} and the angle portion is relative to the x -axis of B_{ρ_i} . It stores these values either in a matrix of robot information $\mathcal{P}^{\text{ROBOT}}$ or a matrix of object information $\mathcal{P}^{\text{OBJECT}}$, according to its AprilTag lookup table.

Virtual sensing

The SoNS control algorithm (see Sec. S4) is meant to be general to different robot hardware platforms and assumes that all robots are capable of sensing the relative position, relative orientation, and ID of

robots in their fields of view and also capable of mutual sensing (i.e., if robot A can sense robot B, then robot B can also sense robot A), regardless of the robot type. However, no robot in our implementation has a sensor that can detect the orientation and ID of a quadrotor, because the ground robot platform has no sensor pointing upwards (see Sec. S7) and the quadrotor has no camera modules pointing laterally. To translate the onboard capabilities of the robots into the sensing capabilities expected by the SoNS control algorithm, we implement a virtual sensing control layer that is informed strictly by local communication with nearby robots.

The virtual sensing layer implemented on each quadrotor effectively gives two quadrotors access to each other's relative position, relative orientation, and ID, under the condition that they both have the same ground robot in their respective fields of view, in the following way.

At each time step, for any AprilTag with the ground robot identifier ρ_j in its field of view, each quadrotor ρ_i sends a message containing its information $\mathcal{P}_{ij}^{\text{ROBOT}} = (\rho_i, \rho_j, \mathbf{d}_{\rho_i\rho_j}, \mathbf{q}_{\rho_i\rho_j})$ to ground robot ρ_j . If a ground robot has received such a message from a quadrotor in the previous time step, it considers that quadrotor to be in its virtual sensing range. At each time step, each ground robot forwards the most recent message $\mathcal{P}^{\text{ROBOT}}$ it has received from any quadrotor in its virtual sensing range to all other quadrotors in its virtual sensing range. In other words, if two quadrotors ρ_i and ρ_k respectively send messages $\mathcal{P}_{ij}^{\text{ROBOT}}$ and $\mathcal{P}_{kj}^{\text{ROBOT}}$ to a ground robot ρ_j in time step $t - 1$, then ground robot ρ_j forwards $\mathcal{P}_{ij}^{\text{ROBOT}}$ to quadrotor ρ_k and $\mathcal{P}_{kj}^{\text{ROBOT}}$ to quadrotor ρ_i in time step t .

Each quadrotor ρ_i that senses a ground robot ρ_j and receives a message $\mathcal{P}_{kj}^{\text{ROBOT}}$ forwarded from quadrotor ρ_k has the required information to calculate the relative position $\mathbf{d}_{\rho_i\rho_k}$ and relative orientation $\mathbf{q}_{\rho_i\rho_k}$ of quadrotor ρ_k w.r.t. its own body frame B_{ρ_i} , as follows:

$$\begin{aligned} \mathbf{d}_{\rho_i\rho_k} &= \mathbf{d}_{\rho_k\rho_j} + \text{RT}\left(\mathbf{q}_{\rho_i\rho_j}^{-1}, \text{RT}(\mathbf{q}_{\rho_i\rho_j}^{-1}, -\mathbf{d}_{\rho_i\rho_j})\right), \\ \mathbf{q}_{\rho_i\rho_k} &= \text{H}(\mathbf{q}_{\rho_k\rho_j}, \mathbf{q}_{\rho_i\rho_j}^{-1}) \end{aligned} \quad (\text{S91})$$

where $\text{RT}(\mathbf{x}, \mathbf{y})$ is a function to rotate vector $\mathbf{y}$ by unit quaternion $\mathbf{x}$ using the Euler–Rodrigues formula, with the Euler parameters given by the coefficients of quaternions $\mathbf{y}^p = (0, \mathbf{y})$ and $\mathbf{x}$, and $\text{H}(\mathbf{x}, \mathbf{y})$ takes the Hamilton product of two quaternions $\mathbf{x}$ and $\mathbf{y}$. In this way, quadrotor ρ_i virtually senses quadrotor ρ_k , using strictly local communication.

Tracking objects for flight stabilization

As can be expected, the quadrotor drifts during flight, even when the control inputs produce desired xy acceleration values of 0, 0. We therefore introduce a flight stabilization control layer on each quadrotor, which can be used optionally, to adjust the target linear velocity vector $\mathbf{v}^*$ and target angular velocity vector $\boldsymbol{\omega}^*$ output by the SoNS control algorithm (see Sec. S4) on the respective quadrotor. The flight stabilization control layer effectively allows a quadrotor to adjust its target velocity vectors $\mathbf{v}^*$ and $\boldsymbol{\omega}^*$ by using a sensed object or ground robot as a reference landmark, whenever it has one in its field of view, as follows.

For a quadrotor r_i on which the flight stabilization control layer is active, when an object or ground robot r_j enters the field of view of quadrotor r_i , quadrotor r_i saves its first respective entry $\mathcal{P}_{ij}^{\text{OBJECT}}$ or $\mathcal{P}_{ij}^{\text{ROBOT}}$ to the matrix of initial references $\mathcal{I}^{\text{ROBOT}}$ or $\mathcal{I}^{\text{OBJECT}}$. Then, if the quadrotor r_i has any objects in its field of view, it uses the objects as reference landmarks. Otherwise, if it has at least one ground robot in its field of view, it uses a ground robot as a reference landmark.

At each time step that quadrotor r_i has at least one object in its field of view, it calculates the linear and angular displacements Δ of the relative position and orientation, respectively, of each object r_j , according to its current entry $\mathbf{d}_{\rho_i\rho_j}, \mathbf{q}_{\rho_i\rho_j} \in \mathcal{P}_{ij}^{\text{OBJECT}}$ and its initial entry $\mathbf{d}_{\rho_i\rho_j}^{\mathcal{I}}, \mathbf{q}_{\rho_i\rho_j}^{\mathcal{I}} \in \mathcal{I}_{ij}^{\text{OBJECT}}$, as follows:

$$\begin{aligned}\Delta(\mathbf{d}_{\rho_i\rho_j}) &= \mathbf{d}_{\rho_i\rho_j} + \text{RT}(\mathbf{q}_{\rho_i\rho_j}, \mathbf{d}_{\rho_i\rho_j}^{\mathcal{I}}) \\ \Delta(\mathbf{q}_{\rho_i\rho_j}) &= \text{H}(\mathbf{q}_{\rho_i\rho_j}, \mathbf{q}_{\rho_i\rho_j}^{\mathcal{I}})\end{aligned}\quad (\text{S92})$$

Quadrotor r_i then estimates its own linear displacement $\mathbf{d}_{\rho_i\rho'_i}$ and angular displacement $\mathbf{q}_{\rho_i\rho'_i}$ per time step, according to its current target velocity vectors $\mathbf{v}^*$ and $\boldsymbol{\omega}^*$. Also at each time step, quadrotor r_i adjusts $\Delta(\mathbf{d}_{\rho_i\rho_j})$ and $\Delta(\mathbf{q}_{\rho_i\rho_j})$ according to its estimated $\mathbf{d}_{\rho_i\rho'_i}$ and $\mathbf{q}_{\rho_i\rho'_i}$, such that they are incrementally updated to reflect the motion of quadrotor r_i according to $\mathbf{v}^*$ and $\boldsymbol{\omega}^*$. Then, quadrotor r_i takes the average of the linear and angular displacements of all sensed objects, such that

$$\begin{aligned}\Delta_{\mathbf{d}} &= \text{AVG}(\Delta(\mathbf{d}_{\rho_i\rho_j}) \forall \rho_j \in \mathcal{P}^{\text{OBJECT}}) \\ \Delta_{\mathbf{q}} &= \text{AVG}(\Delta(\mathbf{q}_{\rho_i\rho_j}) \forall \rho_j \in \mathcal{P}^{\text{OBJECT}})\end{aligned}\quad (\text{S93})$$

Quadrotor r_i then adjusts its target velocity vector $\mathbf{v}^*$ and target angular velocity $\boldsymbol{\omega}^*$ according to the linear and angular displacements of itself and of the objects, as follows:

$$\begin{aligned}\mathbf{v}^{*'} &= \left(\mathbf{d}_{\rho_i\rho'_i} + \text{RT}(\mathbf{q}_{\rho_i\rho'_i}, \mathbf{d}_{\rho_i\rho_j}^{\mathcal{I}}) \right) - \Delta_{\mathbf{d}} \\ \boldsymbol{\omega}^{*'} &= \text{H} \left(\text{H}(\mathbf{q}_{\rho_i\rho'_i}, \mathbf{q}_{\rho_i\rho_j}^{\mathcal{I}}), \Delta_{\mathbf{q}}^{-1} \right)\end{aligned}\quad (\text{S94})$$

At each time step that quadrotor r_i has at least one ground robot but no objects in its field of view, it takes the first entry in its matrix $\mathcal{P}^{\text{ROBOT}}$ and uses it as a reference landmark. For as long as quadrotor r_i chooses ground robot r_j as its reference landmark, it sends an override message to ground robot r_j to ignore any of its motion control inputs except for the target velocity vectors $\mathbf{v}^*$ and $\boldsymbol{\omega}^*$ that it receives from quadrotor r_i . In this way, quadrotor r_i and ground robot r_j move according to the same target velocities for as long as r_j is the reference landmark of r_i . Quadrotor r_i then calculates the linear and angular displacements $\Delta(\mathbf{d}_{\rho_i\rho_j})$ and $\Delta(\mathbf{q}_{\rho_i\rho_j})$ of ground robot r_j using Eq. S92. Quadrotor r_i then estimates its own linear displacement $\mathbf{d}_{\rho_i\rho'_i}$ and angular displacement $\mathbf{q}_{\rho_i\rho'_i}$ per time step, according to its current target velocity vectors $\mathbf{v}^*$ and $\boldsymbol{\omega}^*$. Finally, quadrotor r_i adjusts its target velocity vector $\mathbf{v}^*$ and target angular velocity $\boldsymbol{\omega}^*$ according to Eq. S94, using $\Delta(\mathbf{d}_{\rho_i\rho_j})$ and $\Delta(\mathbf{q}_{\rho_i\rho_j})$ instead of $\Delta_{\mathbf{d}}$ and $\Delta_{\mathbf{q}}$.

Section S7. Ground robot setup

The ground robot used in the experiments (see Fig. S50) is the small differential-drive *e-puck* robot [42] with the *Pi-puck* extension board [43], which provides an interface between the e-puck robot and a Raspberry Pi single-board computer. Each e-puck is also mounted with a unique AprilTag fiducial marker in the tag25h9 family [72, 73], which can be detected by the aerial robots using onboard computer vision (for details, see the technical report of our quadrotor platform [44]). Each AprilTag is $10.75\text{ cm} \times 10.75\text{ cm}$ and its x -axis is directed towards its e-puck's heading.

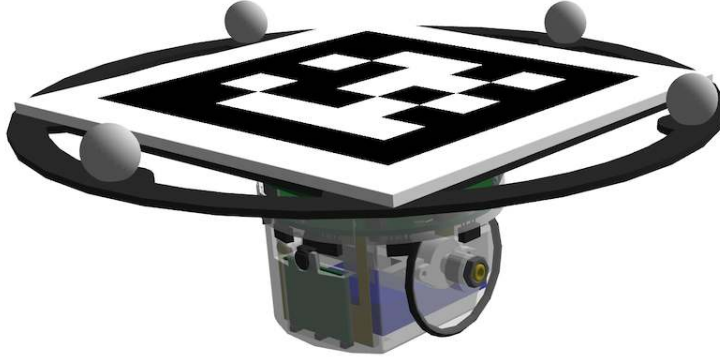


Fig. S50: E-puck robot with Pi-puck extension board and AprilTag fiducial marker (the spherical markers are not used by the SoNS software; they are used for data logging in the real experiments, see Sec. S8 on the real arena). The e-puck's heading bisects the directions of the two wheels and is directed towards the left of the image.

A Linux operating system compiled by Yocto is installed on the Raspberry-pi of the Pi-puck [66, 64]. The control software running in Linux is ARGoS [48]. The SoNS software is comprised of Lua scripts loaded and executed by ARGoS.

Modeling and motion control

Because the e-puck is a differential-drive robot, its motion control inputs (both on the real robots and in simulation) are the target velocities of the left and right wheels. The SoNS control algorithm (see Sec. S4) is meant to be general and usable with different robot hardware platforms and therefore provides omnidirectional motion control outputs regardless of the robot type. To translate the omnidirectional outputs of the SoNS algorithm to the left and right wheel inputs for the differential-drive e-pucks, we introduce a reference frame for intermediary motion management.

The e-puck is modeled using the body frame (denoted B_2) and the intermediary motion frame (denoted F). The body frame B_2 is a relative coordinate system that represents the body of the e-puck. The origin of the frame is the center of the e-puck fiducial marker, the x -axis of the frame is the longitudinal axis (i.e., directed to the front), the y -axis is directed to the left, and the z -axis is directed

upwards. The intermediary motion frame F has the same origin as B_2 but its 3D rotation is fixed and is equivalent to the original rotation of the body frame B_2 at initialization. The rotation of F (i.e., the difference between the rotation of the initial B_2 and the current B_2) is defined with respect to the current B_2 by quaternion q_{B_2} .

Remark S11 *In this study, all positioning is relative. Although the rotation of the intermediary motion frame F is fixed, it is defined and maintained locally using onboard measurements and calculations. The e-puck also only has access to its own calculations in its fixed-rotation intermediary motion frame, not the measurements and calculations of other robots. Hence, there is no absolute reference synchronized between robots or otherwise used to coordinate navigation. Navigation is strictly self-organized, using exclusively local communication and relative positioning.*

The SoNS control algorithm outputs a target linear velocity vector v^* and target angular velocity ω^* . The vector v^* is transformed from the intermediary motion frame F to the body frame B_2 , using $q_{B_2}^{-1}$, as follows:

$$v_{B_2} = \text{RT}(q_{B_2}^{-1}, v^*) \quad (\text{S95})$$

where $\text{RT}(x, y)$ is a function to rotate vector y by unit quaternion x using the Euler–Rodrigues formula, with the Euler parameters given by the coefficients of quaternions x and $y^p = (0, y)$. Vector ω^* is likewise transformed from the intermediary motion frame F to the body frame B_2 using Eq. S95, which provides ω_{B_2} .

Then, $v_{B_2} = (v_x^{B_2}, v_y^{B_2}, v_z^{B_2})$ is translated into left v_{left} and right v_{right} wheel inputs as follows:

$$\begin{aligned} v_{\text{left}} &= v_x^{B_2} - v_y^{B_2} \sin \theta \\ v_{\text{right}} &= v_x^{B_2} + v_y^{B_2} \sin \theta \end{aligned} \quad (\text{S96})$$

where $\theta = \arctan \frac{v_y^{B_2}}{v_x^{B_2}}$. Note that v_z is not used because the e-puck altitude does not change. The angle portion of q_{B_2} is meanwhile updated according to $-\theta$, because F remains fixed, and the intermediary motion frame F is rotated according to ω_{B_2} , which will effect the left and right wheel inputs calculated in the next time step.

When an e-puck sends information about relative positions or relative orientations to other robots, it always sends vectors in the body frame B_2 , as the origin and rotation of the e-puck’s body frame relative to another robot can be sensed directly by that robot via the e-puck’s fiducial marker (if within the field of view).

Virtual sensing of robots and objects in the environment

The SoNS control algorithm (see Sec. S4) is meant to be general to different robot hardware platforms and assumes that all robots are capable of sensing the relative position, relative orientation, and ID of robots in their fields of view and that they are also capable of mutual sensing (i.e., if robot A can sense robot B, then robot B can also sense robot A), regardless of the robot type. However, the e-puck robot does not have any sensor with a field of view pointed upwards, towards the aerial robots, and also does not have any sensor capable of sensing another robot’s orientation. To translate the onboard capabilities of the e-puck robot into the sensing capabilities expected by the SoNS control algorithm,

we implement a virtual sensing control layer that is informed strictly by local communication with nearby robots.

The virtual sensing layer of the e-puck robots is based on the assumption that each aerial robot can sense the relative positions, relative orientations, and encoded identifiers of the AprilTag fiducial markers in its field of view (see Sec. S6). The virtual sensing layer implemented on each e-puck robot effectively gives it access to the relative position, relative orientation, and ID of any aerial robot it is sensed by; as well as the relative position, relative orientation, and ID of any AprilTag fiducial markers in that aerial robot's field of view.

At each time step, each aerial robot ρ_i that senses an e-puck robot ρ_j sends it a message $\mathcal{P}_{ij}^{\text{ROBOT}} = (\rho_i, \rho_j, \mathbf{d}_{\rho_i\rho_j}, \mathbf{q}_{\rho_i\rho_j})$, which includes the relative position and orientation of ρ_j w.r.t. ρ_i , as well as both of their identifiers (see Sec. S6). From this message, e-puck robot ρ_j has the required information to calculate the relative position $\mathbf{d}_{\rho_j\rho_i}$ and relative orientation $\mathbf{q}_{\rho_j\rho_i}$ of aerial robot ρ_i w.r.t. its own body frame $B_2^{\rho_j}$, as follows:

$$\begin{aligned} \mathbf{d}_{\rho_j\rho_i} &= \text{RT}(\mathbf{q}_{\rho_i\rho_j}^{-1}, -\mathbf{d}_{\rho_i\rho_j}) \\ \mathbf{q}_{\rho_j\rho_i} &= \mathbf{q}_{\rho_i\rho_j}^{-1} \end{aligned} \quad (\text{S97})$$

In this way, e-puck robot ρ_j virtually senses aerial robot ρ_i , using strictly local communication.

At each time step, each aerial robot ρ_i that senses an e-puck robot ρ_j also sends ρ_j a $\mathcal{P}_{ik}^{\text{ROBOT}}$ or $\mathcal{P}_{ik}^{\text{OBJECT}}$ message for each e-puck robot or object ρ_k in the field of view of ρ_i . From these messages, e-puck robot ρ_j has the required information to calculate the relative position $\mathbf{d}_{\rho_j\rho_k}$ and relative orientation $\mathbf{q}_{\rho_j\rho_k}$ of the robot/object ρ_k w.r.t. its own body frame $B_2^{\rho_j}$, as follows:

$$\begin{aligned} \mathbf{d}_{\rho_j\rho_k} &= \mathbf{d}_{\rho_j\rho_i} + \text{RT}(\mathbf{q}_{\rho_j\rho_i}, \mathbf{d}_{\rho_i\rho_k}) \\ \mathbf{q}_{\rho_j\rho_k} &= \text{H}(\mathbf{q}_{\rho_j\rho_i}, \mathbf{q}_{\rho_i\rho_k}) \end{aligned} \quad (\text{S98})$$

In this way, e-puck robot ρ_j virtually senses the e-puck robot or object ρ_k , using strictly local communication.

Section S8. Real indoor arena setup

We conduct our experiments with real robots in an indoor arena custom-built for this study inside a multi-use room (see Fig. S51). The arena consists of an unobstructed flight zone and floor area surrounded by a safety buffer and removable security netting, with mounting positions above the center of the flight zone and around the perimeter. The arena is mounted with visual cameras and an optical motion capture system used for recording experiment data. Note that information from the motion capture system is not accessible by the robots and is only used for data logging.



Fig. S51: Wide-angle photograph of the indoor arena.

The arena perimeter consists of a truss system built on top of permanent metal fixtures (see Fig. S52) from modular 2-point trussing segments (Global Truss F32 Ladder Truss family). This results in a rigid truss perimeter around the arena that allows the visual and motion capture cameras to consistently record from the same fixed physical locations and orientations over experimentation periods of several months.

We use an infrared motion capture system with 10 cameras that each have a 56° field of view (OptiTrack Flex 13 family, 56° FOV, 5.5 mm, 800 nm long-pass IR, see Fig. S53).

The flight zone is 6 m x 10 m with a height of 3 m above the floor. The motion capture cameras are mounted in an 8 m x 12 m rectangle centered around the flight zone, at a height of 3.4 m above the floor. The cameras are mounted 4 m apart from each other (one in each corner, one in the center point of each of the shorter perimeter sides, and two at the one-third midway points of each of the longer perimeter sides, see cameras marked in red in Fig S52). Note that the truss system is larger than the rectangle of the camera mounting positions. With the cameras mounted in these positions, motion capture of the ground robots at the floor level and of the aerial robots at the altitude of 1.5 m above the floor is reliable everywhere in the 6 m x 10 m flight zone, except in the corners. Therefore,

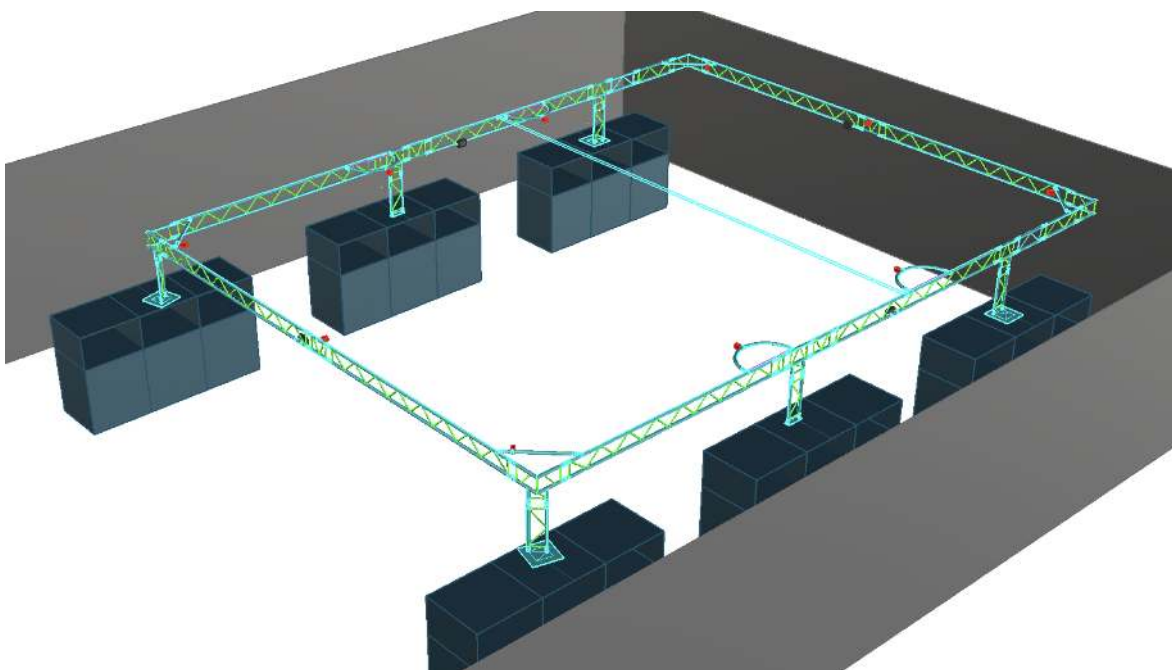


Fig. S52: Perspective view of the truss system of the arena. There are 10 motion capture cameras (red) mounted on the truss around the perimeter of the arena.



Fig. S53: Motion capture cameras (red) at a side location (left) and corner location (right) in the arena.

the corners of the arena are cut at a 45° angle, 0.5 m away from the original corner point, to form the final octagonal arena shape used in the simulated and real experiments (see Fig. S43).

The 10 cameras are connected via cable to four USB hubs (OptiHub 2) that manage camera syncing. The hubs are each connected via cable to one PC station with motion capture software (OptiTrack Motive) that outputs the position and orientation data of the tracked rigid bodies.

In the experiments, we track each aerial or ground robot as a rigid body, using four passive markers (OptiTrack precision spheres with 3M 7610 reflective tape) affixed to the robot (see real robots in Fig. S42). In order for the robot positions and orientations to be tracked correctly, the four markers of one robot need to be mounted in a 2D planar configuration that is asymmetrical along both axes formed between a marker and its opposite (i.e., between two opposite vertices of the quadrilateral formed by the four markers). The marker configurations of the robots also need to be geometrically unique, including when they are rotated in 2D or 3D (i.e., each marker configuration must be unique in any rotated position). The unique marker configurations are provided by custom mounting plates made for this study (see example mounting plate designs in Fig. S54) and are associated to the correct robot IDs using the motion capture software (OptiTrack Motive) at the beginning of each experiment session.

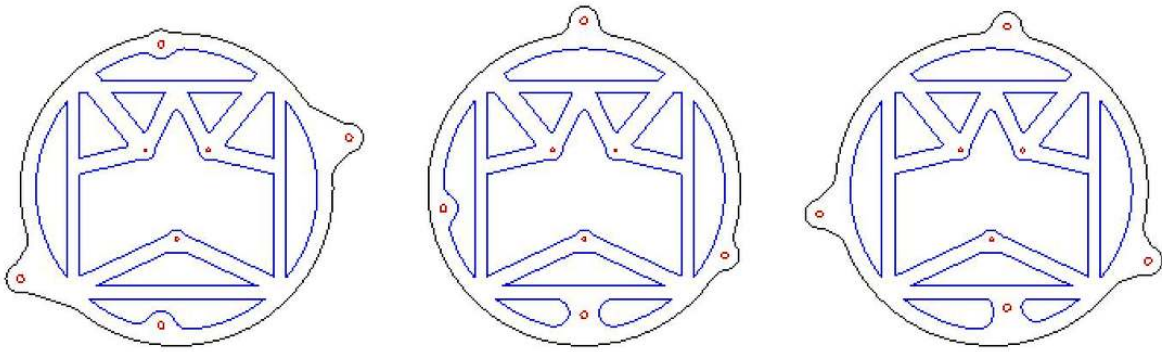


Fig. S54: 2D mounting plate designs for unique and asymmetrical passive marker configurations for each robot, for use by the motion capture system. Three example designs for ground robots are shown. Large red circles near the outer perimeter of each plate indicate the mounting positions of the four passive markers.

Besides recording the motion capture data, we also record experiment videos. We use two visual cameras: an HD action camera (GoPro) mounted to the ceiling, above the center of the arena, and a DSLR camera (Canon EOS 5D Mark IV) mounted to the truss system along the perimeter.

To assist with experiment management, we use an “experiment supervisor” software custom-developed for this study [47]⁷. The experiment supervisor software is used: 1) before the experiment, to confirm all robots are active, correctly configured, seen by the motion capture system, and in the correct state; 2) to send a signal to all robots to start the experiment at the same time; 3) to record experiment data during the experiment; and 4) to send a signal to all robots to end the experiment at the same time. During each experiment, the experiment supervisor software records the positions and orientations output from the motion capture system as well as all SoNS information output from the robots.

⁷<https://github.com/iridia-ulb/supervisor>

During the experiments (i.e., between the signals sent to start the experiment and end the experiment), the robots do not receive any centralized commands or global information. The robots are allowed to communicate only with each other, and only under certain conditions. Communication in the SoNS occurs over a wireless network, and two robots are only allowed to communicate with each other if they are connected in the SoNS or if one of them is in the other's field of view. In our indoor arena, the robots communicate using a wireless LAN. Messages between robots are routed by the experiment supervisor software, which has access to the MAC address associated to each robot ID. Note that the SoNS layer is constructed in such a way that messages between robots could instead be passed using a wireless ad-hoc network, with no changes to the SoNS software.

During an experiment, the experiment supervisor software records the messages passed between robots, for the purpose of data logging. Note that, during an experiment, the robots do not receive any commands or information originating from the experiment supervisor software; robots only communicate with each other.

Supplementary Movies S1–S12

Movie S1 : Establishing self-organized hierarchy with real robots.

Movie S2 : Balancing global and local goals with real robots.

Movie S3 : Collective sensing and actuation with real robots.

Movie S4 : Binary decision making with real robots.

Movie S5 : Splitting and merging systems with real robots.

Movie S6 : Scalability in the binary decision-making mission, 125 robots in simulation.

(Note: To play embedded videos, open the PDF in Adobe Acrobat.)

Movie S7 : Scalability in the establishing self-organized hierarchy mission, several example system sizes in simulation.

Movie S8 : Fault tolerance demonstration showing interchangeability of a failed brain robot.

Movie S9 : Fault tolerance under multiple permanent failures, with real robots.

Movie S10 : Fault tolerance under high-loss conditions in simulation, 66.6% probability to fail.

Movie S11 : Fault tolerance under 30 s system-wide vision failure in simulation.

Movie S12 : Fault tolerance under 30 s system-wide communication failure in simulation.

(Note: To play embedded videos, open the PDF in Adobe Acrobat.)

References

- [1] G. Valentini, E. Ferrante, H. Hamann, and M. Dorigo, “Collective decision with 100 kilobots: Speed versus accuracy in binary discrimination problems,” *Autonomous Agents and Multi-agent Systems*, vol. 30, no. 3, pp. 553–580, 2016.
- [2] S. Nouyan, R. Groß, M. Bonani, F. Mondada, and M. Dorigo, “Teamwork in self-organized robot colonies,” *IEEE Transactions on Evolutionary Computation*, vol. 13, no. 4, pp. 695–711, 2009.
- [3] M. Dorigo, D. Floreano, L. M. Gambardella, F. Mondada, S. Nolfi, T. Baaboura, M. Birattari, M. Bonani, M. Brambilla, A. Brutschy *et al.*, “Swarmanoid: a novel concept for the study of heterogeneous robotic swarms,” *IEEE Robotics & Automation Magazine*, vol. 20, no. 4, pp. 60–71, 2013.
- [4] J. Werfel, K. Petersen, and R. Nagpal, “Designing collective behavior in a termite-inspired robot construction team,” *Science*, vol. 343, no. 6172, pp. 754–758, 2014.
- [5] M. Wahby, M. K. Heinrich, D. N. Hofstadler, E. Neufeld, I. Kuksin, P. Zahadat, T. Schmickl, P. Ayres, and H. Hamann, “Autonomously shaping natural climbing plants: a bio-hybrid approach,” *Royal Society Open Science*, vol. 5, no. 10, p. 180296, 2018.
- [6] J. Halloy, G. Sempo, G. Caprari, C. Rivault, M. Asadpour, F. Tâche, I. Saïd, V. Durier, S. Canonge, J. M. Amé *et al.*, “Social integration of robots into groups of cockroaches to control self-organized choices,” *Science*, vol. 318, no. 5853, pp. 1155–1158, 2007.
- [7] M. Dorigo, G. Theraulaz, and V. Trianni, “Reflections on the future of swarm robotics,” *Science Robotics*, vol. 5, no. 49, p. eabe4385, 2020.
- [8] —, “Swarm robotics: past, present, and future [point of view],” *Proceedings of the IEEE*, vol. 109, no. 7, pp. 1152–1165, 2021.
- [9] A. Howard, “Multi-robot simultaneous localization and mapping using particle filters,” *The International Journal of Robotics Research*, vol. 25, no. 12, pp. 1243–1256, 2006.
- [10] H. N. Psaraftis, M. Wen, and C. A. Kontovas, “Dynamic vehicle routing problems: Three decades and counting,” *Networks*, vol. 67, no. 1, pp. 3–31, 2016.
- [11] G. Vásárhelyi, C. Virágh, G. Somorjai, T. Nepusz, A. E. Eiben, and T. Vicsek, “Optimized flocking of autonomous drones in confined environments,” *Science Robotics*, vol. 3, no. 20, p. eaat3536, 2018.
- [12] J. Buhl, D. J. Sumpter, I. D. Couzin, J. J. Hale, E. Despland, E. R. Miller, and S. J. Simpson, “From disorder to order in marching locusts,” *Science*, vol. 312, no. 5778, pp. 1402–1406, 2006.
- [13] C. Detrain and J.-L. Deneubourg, “Collective decision-making and foraging patterns in ants and honeybees,” *Advances in Insect Physiology*, vol. 35, pp. 123–173, 2008.

-
- [14] G. Theraulaz, E. Bonabeau, and J.-L. Deneubourg, “The origin of nest complexity in social insects,” *Complexity*, vol. 3, no. 6, pp. 15–25, 1998.
- [15] E. Bonabeau, M. Dorigo, and G. Theraulaz, *Swarm intelligence: from natural to artificial systems*. Oxford university press, 1999, no. 1.
- [16] G. Francesca and M. Birattari, “Automatic design of robot swarms: achievements and challenges,” *Frontiers in Robotics and AI*, vol. 3, p. 29, 2016.
- [17] D. Kengyel, H. Hamann, P. Zahadat, G. Radspieler, F. Wotawa, and T. Schmickl, “Potential of heterogeneity in collective behaviors: A case study on heterogeneous swarms,” in *PRIMA 2015: Principles and Practice of Multi-Agent Systems*, Q. Chen, P. Torroni, S. Villata, J. Hsu, and A. Omicini, Eds. Springer International Publishing, 2015, pp. 201–217.
- [18] E. Ferrante, A. E. Turgut, C. Huepe, A. Stranieri, C. Pinciroli, and M. Dorigo, “Self-organized flocking with a mobile robot swarm: a novel motion control method,” *Adaptive Behavior*, vol. 20, no. 6, pp. 460–477, 2012.
- [19] Z. Firat, E. Ferrante, Y. Gillet, and E. Tuci, “On self-organised aggregation dynamics in swarms of robots with informed robots,” *Neural Computing and Applications*, vol. 32, no. 17, pp. 13 825–13 841, 2020.
- [20] G. Valentini, H. Hamann, and M. Dorigo, “Efficient decision-making in a self-organizing robot swarm: On the speed versus accuracy trade-off,” in *Proceedings of 14th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2015)*, R. Bordini, E. Elkind, G. Weiss, and P. Yolum, Eds. International Foundation for Autonomous Agents and Multiagent Systems (IFAAMAS), 2015, pp. 1305–1314.
- [21] B. Balázs, G. Vásárhelyi, and T. Vicsek, “Adaptive leadership overcomes persistence–responsivity trade-off in flocking,” *Journal of the Royal Society Interface*, vol. 17, no. 167, p. 20190853, 2020.
- [22] P. Walker, S. A. Amraii, N. Chakraborty, M. Lewis, and K. Sycara, “Human control of robot swarms with dynamic leaders,” in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2014, pp. 1108–1113.
- [23] P. Walker, S. A. Amraii, M. Lewis, N. Chakraborty, and K. Sycara, “Control of swarms with multiple leader agents,” in *2014 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE, 2014, pp. 3567–3572.
- [24] T. K. Kaiser and H. Hamann, “Innate motivation for robot swarms by minimizing surprise: From simple simulations to real-world experiments,” *IEEE Transactions on Robotics*, 2022.
- [25] Q. Shan and S. Mostaghim, “Collective decision making in swarm robotics with distributed bayesian hypothesis testing,” in *International Conference on Swarm Intelligence*. Springer, 2020, pp. 55–67.

-
- [26] D. Gu and Z. Wang, “Leader–follower flocking: algorithms and experiments,” *IEEE Transactions on Control Systems Technology*, vol. 17, no. 5, pp. 1211–1219, 2009.
 - [27] S. A. Amraii, P. Walker, M. Lewis, N. Chakraborty, and K. Sycara, “Explicit vs. tacit leadership in influencing the behavior of swarms,” in *2014 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2014, pp. 2209–2214.
 - [28] H. Zheng, J. Panerati, G. Beltrame, and A. Prorok, “An adversarial approach to private flocking in mobile robot teams,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1009–1016, 2020.
 - [29] F. Dalmao and E. Mordecki, “Cucker–smale flocking under hierarchical leadership and random interactions,” *SIAM Journal on Applied Mathematics*, vol. 71, no. 4, pp. 1307–1316, 2011.
 - [30] C. Pignotti and I. R. Vallejo, “Flocking estimates for the cucker–smale model with time lag and hierarchical leadership,” *Journal of Mathematical Analysis and Applications*, vol. 464, no. 2, pp. 1313–1332, 2018.
 - [31] Y. Jia and T. Vicsek, “Modelling hierarchical flocking,” *New Journal of Physics*, vol. 21, no. 9, p. 093048, 2019.
 - [32] M. D. Soorati, M. K. Heinrich, J. Ghofrani, P. Zahadat, and H. Hamann, “Photomorphogenesis for robot self-assembly: adaptivity, collective decision-making, and self-repair,” *Bioinspiration & Biomimetics*, vol. 14, no. 5, p. 056006, 2019.
 - [33] X. Zhou, X. Wen, Z. Wang, Y. Gao, H. Li, Q. Wang, T. Yang, H. Lu, Y. Cao, C. Xu *et al.*, “Swarm of micro flying robots in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm5954, 2022.
 - [34] N. Mathews, A. L. Christensen, R. O’Grady, F. Mondada, and M. Dorigo, “Mergeable nervous systems for robots,” *Nature Communications*, vol. 8, no. 1, pp. 1–7, 2017.
 - [35] W. Zhu, M. Allwright, M. K. Heinrich, S. Oğuz, A. L. Christensen, and M. Dorigo, “Formation control of uavs and mobile robots using self-organized communication topologies,” in *International Conference on Swarm Intelligence*. Springer, 2020, pp. 306–314.
 - [36] Y. Zhang, S. Oğuz, S. Wang, E. Garone, X. Wang, M. Dorigo, and M. K. Heinrich, “Self-reconfigurable hierarchical frameworks for formation control of robot swarms,” *IEEE Transactions on Cybernetics*, 2023.
 - [37] A. Jamshidpey, W. Zhu, M. Wahby, M. Allwright, M. K. Heinrich, and M. Dorigo, “Multi-robot coverage using self-organized networks for central coordination,” in *International Conference on Swarm Intelligence*. Springer, 2020, pp. 216–228.
 - [38] A. Jamshidpey, M. Wahby, M. K. Heinrich, M. Allwright, W. Zhu, and M. Dorigo, “Centralization vs. decentralization in multi-robot coverage: Ground robots under uav supervision,” IRIDIA, Université Libre de Bruxelles, Brussels, Belgium, Tech. Rep. TR/IRIDIA/2021-008, May 2021.

-
- [39] A. Jamshidpey, M. Dorigo, and M. K. Heinrich, “Reducing uncertainty in collective perception using self-organized hierarchy,” *Intelligent Computing*, 2022.
- [40] F. Ducatelle, A. Förster, G. A. Di Caro, L. M. Gambardella *et al.*, “New task allocation methods for robotic swarms,” in *9th IEEE/RAS conference on autonomous robot systems and competitions*, vol. 5, 2009.
- [41] F. Ducatelle, G. A. Di Caro, C. Pinciroli, and L. M. Gambardella, “Self-organized cooperation between robotic swarms,” *Swarm Intelligence*, vol. 5, pp. 73–96, 2011.
- [42] F. Mondada, M. Bonani, X. Raemy, J. Pugh, C. Cianci, A. Klapacz, S. Magnenat, J.-C. Zufferey, D. Floreano, and A. Martinoli, “The e-puck, a robot designed for education in engineering,” in *Proceedings of the 9th Conference on Autonomous Robot Systems and Competitions*, vol. 1, no. CONF. IPCB: Instituto Politécnico de Castelo Branco, 2009, pp. 59–65.
- [43] A. G. Millard, R. Joyce, J. A. Hilder, C. Fleşeriu, L. Newbrook, W. Li, L. J. McDaid, and D. M. Halliday, “The pi-puck extension board: a raspberry pi interface for the e-puck robot platform,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 741–748.
- [44] S. Oguz, M. K. Heinrich, M. Allwright, W. Zhu, M. Wahby, E. Garone, and M. Dorigo, “S-drone: An open-source quadrotor for experimentation in swarm robotics,” IRIDIA, Université Libre de Bruxelles, Brussels, Belgium, Tech. Rep. TR/IRIDIA/2022-010, October 2022.
- [45] V. Walter, M. Saska, and A. Franchi, “Fast mutual relative localization of uavs using ultraviolet led markers,” in *2018 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2018, pp. 1217–1226.
- [46] J. Ulrich, A. Alsayed, F. Arvin, and T. Krajník, “Towards fast fiducial marker with full 6 dof pose estimation,” in *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*, 2022, pp. 723–730.
- [47] M. Allwright, “Supervisor: Software for starting, monitoring, and stopping experiments with swarms of robots,” IRIDIA, Université Libre de Bruxelles, Brussels, Belgium, Tech. Rep. TR/IRIDIA/2022-004, March 2022.
- [48] C. Pinciroli, V. Trianni, R. O’Grady, G. Pini, A. Brutschy, M. Brambilla, N. Mathews, E. Ferante, G. Di Caro, F. Ducatelle, M. Birattari, L. M. Gambardella, and M. Dorigo, “ARGoS: a modular, parallel, multi-engine simulator for multi-robot systems,” *Swarm Intelligence*, vol. 6, no. 4, pp. 271–295, 2012.
- [49] M. Allwright, N. Bhalla, C. Pinciroli, and M. Dorigo, “ARGoS plug-ins for experiments in autonomous construction,” IRIDIA, Université Libre de Bruxelles, Brussels, Belgium, Tech. Rep. TR/IRIDIA/2018-007, 2018.
- [50] —, “Simulating multi-robot construction in ARGoS,” in *Swarm Intelligence – Proceedings of ANTS 2018 – Eleventh International Conference*, ser. Lecture Notes in Computer Science, vol. 11172. Berlin, Germany: Springer, 2018, pp. 188–200.

-
- [51] K.-K. Oh and H.-S. Ahn, “Formation control of mobile agents based on inter-agent distance dynamics,” *Automatica*, vol. 47, no. 10, pp. 2306–2312, 2011.
- [52] L. Krick, M. E. Broucke, and B. A. Francis, “Stabilisation of infinitesimally rigid formations of multi-robot networks,” *International Journal of control*, vol. 82, no. 3, pp. 423–439, 2009.
- [53] H. G. Tanner, G. J. Pappas, and V. Kumar, “Leader-to-formation stability,” *IEEE Transactions on Robotics and Automation*, vol. 20, no. 3, pp. 443–455, 2004.
- [54] H. K. Khalil, *Nonlinear systems*, 3rd ed. Prentice Hall, 2002.
- [55] P. Seibert and R. Suarez, “Global stabilization of nonlinear cascade systems,” *Systems & Control Letters*, vol. 14, no. 4, pp. 347–352, 1990.
- [56] E. D. Sontag *et al.*, “On the input-to-state stability property,” *European Journal of Control*, vol. 1, no. 1, pp. 24–36, 1995.
- [57] A. Isidori, *Nonlinear control systems: an introduction*. Springer, 1985.
- [58] H. G. Tanner, V. Kumar, and G. J. Pappas, “The effect of feedback and feedforward on formation ISS,” in *Proceedings 2002 IEEE International Conference on Robotics and Automation*, vol. 4. IEEE Press, 2002, pp. 3448–3453.
- [59] A. K. Das, R. Fierro, V. Kumar, J. P. Ostrowski, J. Spletzer, and C. J. Taylor, “A vision-based formation control framework,” *IEEE Transactions on Robotics and Automation*, vol. 18, no. 5, pp. 813–825, 2002.
- [60] J. Wang, Y. Tan, and I. Mareels, “Robustness analysis of leader-follower consensus,” *Journal of Systems Science and Complexity*, vol. 22, no. 2, pp. 186–206, 2009.
- [61] K. D. Wayne, *Generalized maximum flow algorithms*. Cornell University, 1999.
- [62] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, “Section 26.2: The ford-fulkerson method,” *Introduction to algorithms*, pp. 651–664, 2001.
- [63] M. Allwright, “Software support for the iridia drone,” IRIDIA, Université Libre de Bruxelles, Brussels, Belgium, Tech. Rep. TR/IRIDIA/2022-002, October 2022.
- [64] —, “Software support for the pi-puck mobile robot,” IRIDIA, Université Libre de Bruxelles, Brussels, Belgium, Tech. Rep. TR/IRIDIA/2022-001, October 2022.
- [65] M. Allwright, W. Zhu, and M. Dorigo, “An open-source multi-robot construction system,” *HardwareX*, vol. 5, p. e00050, 2019.
- [66] O. Salvador and D. Angolini, *Embedded Linux development with Yocto project*. Packt Publishing Ltd, 2014.
- [67] J. Farrell, *Aided navigation: GPS with high rate sensors*. McGraw-Hill, Inc., 2008.

-
- [68] S. Bouabdallah and R. Siegwart, “Full control of a quadrotor,” in *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE Press, 2007, pp. 153–158.
- [69] T. I. Fossen, *Guidance and control of ocean vehicles*. John Wiley & Sons, 1994.
- [70] T. J. Koo and S. Sastry, “Output tracking control design of a helicopter model based on approximate linearization,” in *Proceedings of the 37th IEEE Conference on Decision and Control*, vol. 4. IEEE Press, 1998, pp. 3635–3640.
- [71] M.-D. Hua, T. Hamel, P. Morin, and C. Samson, “Introduction to feedback control of underactuated vehicles: A review of basic control design ideas and principles,” *IEEE Control Systems Magazine*, vol. 33, no. 1, pp. 61–75, 2013.
- [72] E. Olson, “Apriltag: A robust and flexible visual fiducial system,” in *2011 IEEE International Conference on Robotics and Automation*. IEEE Press, 2011, pp. 3400–3407.
- [73] J. Wang and E. Olson, “Apriltag 2: Efficient and robust fiducial detection,” in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE Press, 2016, pp. 4193–4198.